

Understanding Communication Performance on Emerging Heterogeneous Architectures

IEEE Hot Interconnects Aug 19-21, 2026

Amanda Bienz, Patrick Bridges

CS Dept, University of New Mexico



Sameer Shende

PRL, OACISS, University of Oregon



**Shelby Lockhart, Bob Robey,
Giacomo Capodaglio**

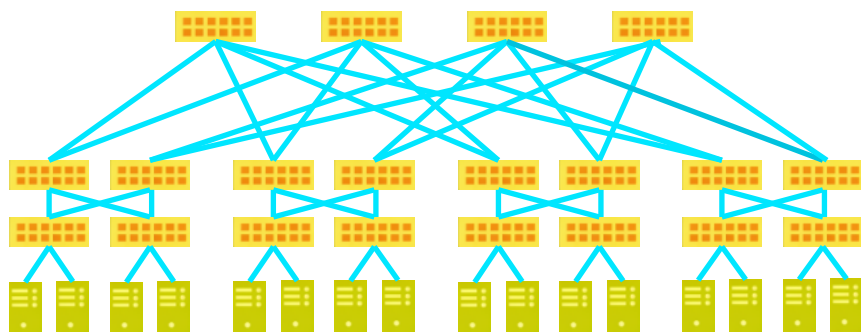
HPC & Sovereign AI Customer Enablement, AMD



Agenda

01	Introduction • Introduction to Hands-On Examples and AAC6 system
02	Functionality of Communication Libraries • MPI, RCCL, rocSHMEM
03	Performance of Communication Libraries • Microbenchmarks
04	Profiling with TAU • Introduction to TAU
05	Profiling Applications • Conjugate Gradient solver and PyTorch imagenet
06	Summary & Look Ahead

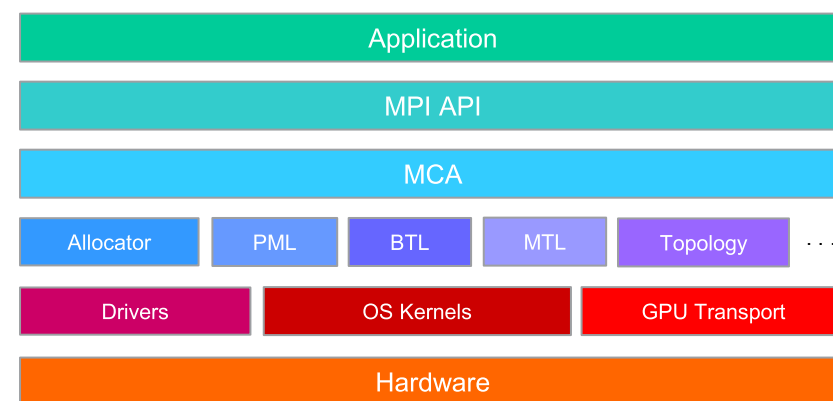
Heterogeneous systems are complex



Complex Topologies



Complex Hardware



Complex Software Stacks

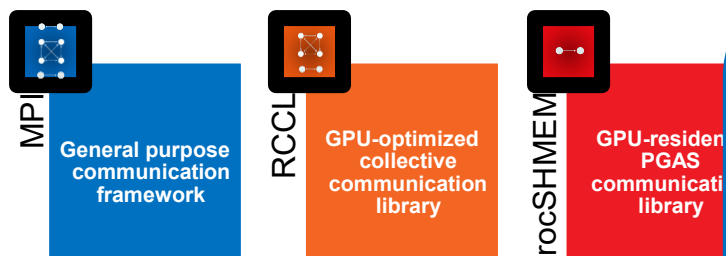
Data movement is complex

Tutorial goals

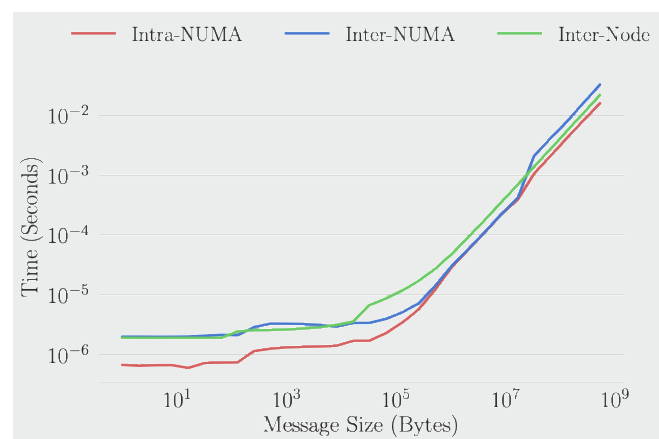
```
Initial residual: 195.472
181 iterations to converge
2-norm of residual: 0.000191484
CG solve time:      0.0614 s (0.0003 s/iter)
```

Is this good performance?

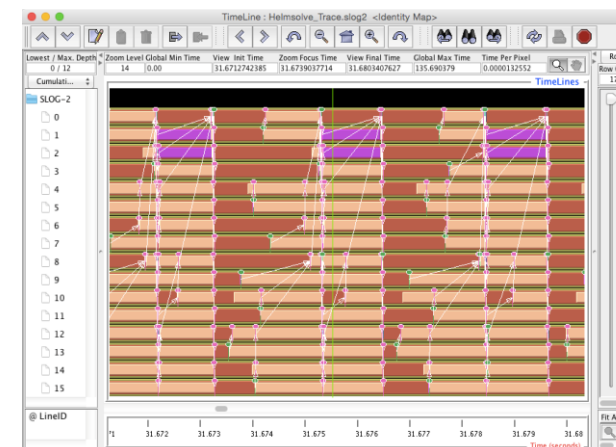
System Stack



Microbenchmarking



Profiling and Tracing



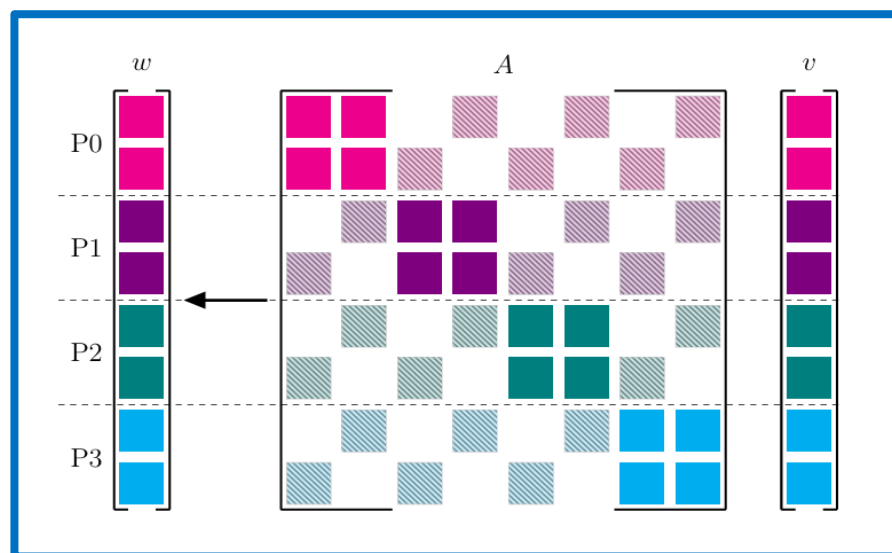
Intro to examples

Conjugate Gradient Solver

```

 $r_0 = b - A x_0$ 
 $p_0 = r_0$ 
for  $i = 0, 1, 2, \dots$ :
     $\alpha = (r_i, r_i) / (A p_i, p_i)$ 
     $x += \alpha p_i$ 
     $r_{i+1} = r_i - \alpha A p_i$ 
     $\beta = (r_{i+1}, r_{i+1}) / (r_i, r_i)$ 
     $p_{i+1} = r_{i+1} + \beta p_i$ 

```



```

Spmv_Code()
{
    // Post receives
    // Gather off-process send values
    // Post sends

    // Perform SpMV portion with on-process values

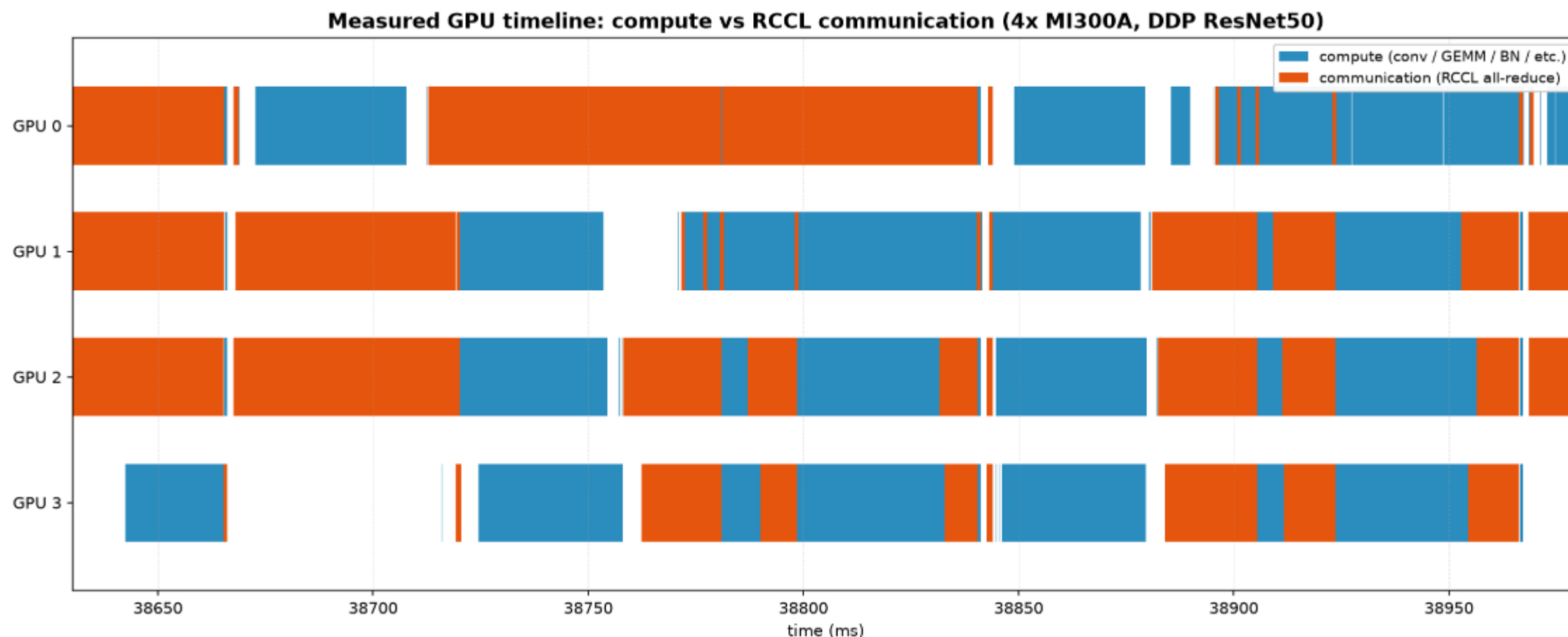
    // Wait for receives to complete

    // Perform SpMV portion with received values

    // Wait for sends to complete
}

```

Pytorch examples: ImageNet



Full code and documentation for this example at:

<https://github.com/amd/HPCTrainingExamples/tree/main/MLExamples/Pytorch/imagenet>

Intro to AMD Accelerator Cloud (AAC6) system for hands-on exercises

Getting an AAC6 account

- See form with heading
 - Hot Interconnects: Understanding Communication Performance on Emerging Heterogeneous Architectures
 - On your LAPTOP, open this URL:
 - <https://forms.gle/bhsFwKcP8BjgQbieA>
 - Your SSH public key is on your laptop, so please fill the form from your laptop browser. The form asks for your name, email and SSH public key.
 - Please take care in entering this information correctly. Bad entries will gum up the works.
 - Or scan this code to fill the form on your phone: (first copy your SSH public key to your phone so you can paste it)



- Please contact Bob Robey Bob.Robey@amd.com and Giacomo Capodaglio Giacomo.Capodaglio@amd.com about AAC6 access requests.

Logging into AAC6

Usually \$HOME/.ssh/id_* or wherever you stored the key you sent to register for AAC access

- Accessing AAC6

```
ssh <username>@aac6.amd.com -i <path>/<private ssh key file>
```

- To simplify the login to AAC6, you can add the following to your .ssh/config file (for each system)

```
Host aac6
  User <username>
  Hostname aac6.amd.com
  #IdentityFile <path>/<private key file> # customize location of key (typically in ~/.ssh/)
  ServerAliveInterval 600
  ServerAliveCountMax 30
```

- The ServerAlive* lines in the config file may be added to avoid timeouts when idle.
- For AAC6: Set up two-factor authentication during your first login as explained in the email you should have received
- Please read the message of the day when you login, there are also extensive man pages

File copying

You should be able to copy files in or out with the **scp** command.

```
In:  scp -i <path>/<keyfile> <file> <username>@aac6.amd.com:~/<path>/<file>
```

```
Out: scp -i <path>/<keyfile> <username>@aac6.amd.com:~/path/to/your/file ./
```

You can also use the **rsync** command:

```
rsync -avz -e "ssh -i <path>/<keyfile>" <file> uusername@aac6.amd.com:~/path/to/your/files
```

- Back-up your files regularly if you think you'll need them after this event
- Access to the system will persist after the event indefinitely*

*or until we have to revoke access due to irresponsible usage of the resources

Notes on the AAC6 system

AAC6 system overview

- Based on AMD [HPCTrainingDock](https://github.com/AMD/HPCTrainingDock) (<https://github.com/AMD/HPCTrainingDock>)
- Ubuntu® 24.04
- ROCm versions: 6.3.0 to 7.2.4, 7.13, 7.14
- Notable packages:
 - Communication packages like GPU-Aware OpenMPI
 - AI frameworks (PyTorch, JAX, Tensorflow) preinstalled
 - Many profiling tools preinstalled

```
ssh <username>@aac6.amd.com
```

Environment – modules

Modules using Lua modules:

AAC6: Compiler modules (amdc1ang) set the CC, CXX, FC flags. Only one compiler module can be loaded at a time. hipcc is in the path when the rocm module is loaded. These modules also set the OMPI_{CC/CXX/FC} environment variables so that the MPI compiler wrappers use the currently set compiler.

Module commands

- `module avail`
- `module list`
- `module load <package>`
- `module unload <package>`

AAC6 environment

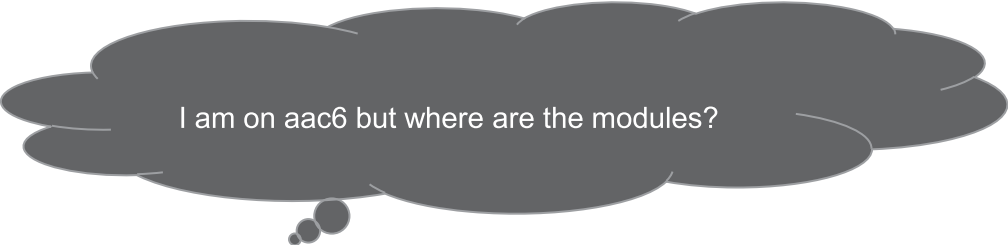
- Model Installation Environment:
<https://github.com/amd/HPCTrainingDock>

- Hierarchical set of Lua modules:

- `module avail` on initial login:
- Default: **rocm/7.2.4** - ROCm software stack including HIP and HIP libraries
 - AFAR versions are previews of future compiler releases. Versions greater than 7.12.0 are a new release process.

- After `module load rocm`

→ `module avail` shows full list of software built and dependent on it:



```
----- /nfsapps/ubuntu-24.04/modules/base -----
emacs/30.1      rocm/6.3.0  rocm/6.4.1  rocm/7.1.0  rocm/7.2.4 (D)
miniconda3/25.3.1 rocm/6.3.1  rocm/6.4.2  rocm/7.1.1  rocm/7.12.0
miniforge3/24.9.0 rocm/6.3.2  rocm/6.4.3  rocm/7.2.0  rocm/7.13.0
rocbudai/dev     rocm/6.3.3  rocm/7.0.0  rocm/7.2.1  roofline-extractor/dev
rocm/afar-23.1.0-7.12.0 rocm/6.3.4  rocm/7.0.1  rocm/7.2.2  tracelens/dev
rocm/afar-23.2.1-7.13.0 rocm/6.4.0  rocm/7.0.2  rocm/7.2.3  turbovnc/3.0.3
```

```
----- /nfsapps/ubuntu-24.04/modules/rocmplus-7.2.4 -----
cupy/13.5.1      hipifly/dev      mdb/1.0.6      pnetcdf/1.14.1  pytorch/2.10.0  scorep/9.4
elpa/master-548c64b hpctoolkit/2025.1.2 mpi4py/4.1.0    pytorch/2.7.1_tunableop_enabled pytorch/2.11.0_tunableop_enabled scorep/11.0-dev (D)
fftw/3.3.10      hypre/3.1.0      mpitrace/7.2.4  pytorch/2.7.1    pytorch/2.11.0  tau/dev
ftorch/2.12.0    intellikit/main  netcdf-c/4.10.0  pytorch/2.8.0_tunableop_enabled pytorch/2.12.0_tunableop_enabled tensorflow/2.20.0
ftorch_amdflang/2.12.0 jax/0.8.0        openblas/0.3.33  pytorch/2.8.0    pytorch/2.12.0  umpire/2025.12.0
hdf5/2.1.1       kokkos/4.7.04    openmpi/5.0.10-uccl1.6.0-ucx1.19.1-xpmem-2.7.4 pytorch/2.9.1_tunableop_enabled rccl-tests/7.2.4 (D) vllm/0.24.0
hip-python/7.2.4 likwid/5.5.1     petsc/3.24.1    pytorch/2.9.1    roc-optiq/0.5.0
hipblaslt/patched (L) magma/2.10.0     rocm/7.2.4 (L,D) roofline-extractor/dev
tracelens/dev
turbovnc/3.0.3

----- /nfsapps/ubuntu-24.04/modules/rocm-7.2.4 -----
amdclang/22.0.0-7.2.4 hipfort/7.2.4  opencl/7.2.4

----- /nfsapps/ubuntu-24.04/modules/base -----
emacs/30.1      rocbudai/dev      rocm/6.3.0  rocm/6.3.3  rocm/6.4.1  rocm/7.0.0  rocm/7.1.0  rocm/7.2.1  rocm/7.2.4 (L,D)  roofline-extractor/dev
miniconda3/25.3.1 rocm/afar-23.1.0-7.12.0 rocm/6.3.1  rocm/6.3.4  rocm/6.4.2  rocm/7.0.1  rocm/7.1.1  rocm/7.2.2  rocm/7.12.0  tracelens/dev
miniforge3/24.9.0 rocm/afar-23.2.1-7.13.0 rocm/6.3.2  rocm/6.4.0  rocm/6.4.3  rocm/7.0.2  rocm/7.2.0  rocm/7.2.3  rocm/7.13.0  turbovnc/3.0.3
```

Environment – ROCm modules

Additional modules that are available for each ROCm version

```
----- /nfsapps/ubuntu-24.04/modules/rocm-7.2.4 -----  
amdclang/22.0.0-7.2.4    hipfort/7.2.4
```

The ROCm module sets paths to hipcc and the rocm libraries.

AMDClang module sets the compiler variables (CC, CXX, FC)

Hipfort module adds paths and environment variables for using hipfort

If present, the rocprofiler-compute and rocprofiler-systems modules set any extra environment settings. If not present, no additions are needed

Environment – ROCmPlus modules

```
--- /nfsapps/ubuntu-24.04/modules/rocmplus-7.2.4 ---
```

```
cupy/13.5.1  
elpa/master-548c64b  
fftw/3.3.10  
ftorch/2.12.0  
ftorch_amdflang/2.12.0  
hdf5/2.1.1  
hip-python/7.2.4  
hipblaslt/patched  
hipifly/dev  
hpctoolkit/2025.1.2  
hypre/3.1.0  
intellikit/main  
jax/0.8.0  
kokkos/4.7.04  
likwid/5.5.1  
magma/2.10.0  
mdb/1.0.6  
mpi4py/4.1.0  
mpitrace/7.2.4  
netcdf-c/4.10.0  
netcdf-fortran/4.6.2  
openblas/0.3.33  
openmpi/5.0.10-ucc1.6.0-ucx1.19.1-xpmem-2.7.4
```

```
petsc/3.24.1  
pnetcdf/1.14.1  
pytorch/2.7.1_tunableop_enabled  
pytorch/2.7.1  
pytorch/2.8.0_tunableop_enabled  
pytorch/2.8.0  
pytorch/2.9.1_tunableop_enabled  
pytorch/2.9.1  
pytorch/2.10.0_tunableop_enabled  
pytorch/2.10.0  
pytorch/2.11.0_tunableop_enabled  
pytorch/2.11.0  
pytorch/2.12.0_tunableop_enabled  
pytorch/2.12.0  
rccl-tests/7.2.4  
roc-optiq/0.5.0  
rocshmem/3.2.0-ro_ipc  
scorep/9.4  
scorep/11.0-dev  
tau/dev  
tensorflow/2.20.0  
umpire/2025.12.0  
vllm/0.24.0
```

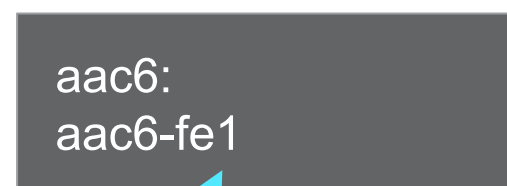
Environment – Slurm

- AAC6 uses SLURM to manage the GPU nodes
- You have to allocate resources via SLURM to get a GPU:
 - `salloc` can be used to schedule a long-term interactive session
 - `sbatch` is used to submit a job to the batch queue
 - `srun` is used to run a job in an already allocated a new session
- Some more useful commands:
 - `squeue` will show the jobs running in the batch queue
 - `squeue -u <username>` shows all of your current jobs
 - `scancel <jobid>` cancels a running job and releases the resources
 - `sinfo` show the available partitions
- **Please allocate only the GPUs that you need and release them after you are done!**

AAC6 nodes

AAC6 is all pre-production parts

Login or Control Node



You are here
after logging in

No GPUs on
front end node

SLURM

srun
salloc
sbatch

CPX:

Queue PPAC_MI300A_CPX (Default)

ppac-pl1-s25-40

4xMI300A (CPX*)
24 logical GPUs
48 CPUs
128 GB memory

Queue PPAC_MI300A_SPX

ppac-pl1-s24-16

ppac-pl1-s24-26

ppac-pl1-s24-30

ppac-pl1-s24-35

4xMI300A (SPX)
4 logical GPUs
192 CPUs
512 GB memory

Queue SH5_MI300A_SPX (and CPX)

sh5-pl1-s12-33

SPX

1xMI300A (SPX*)
1 logical GPU
48 CPUs
128 GB memory

sh5-pl1-s12-09

sh5-pl1-s12-12

sh5-pl1-s12-15

1xMI300A (CPX*)
6 logical GPUs
8 CPUs
21 GB memory

*CPX compute partitioning exposes each XCD of the MI300A as single GPU device allowing to make more GPUs available for training purposes.
See: [Deep dive into the MI300 compute and memory partition modes — ROCm Blogs](#)

Sample SLURM commands

- Review the “Message of the Day” when logging in to learn about partition names, SLURM commands, ...
- Some examples on how to use SLURM to get compute nodes on AAC6:

```
srun -p PPAC_MI300A_CPX --exclusive my_batch_script.batch
```

```
srun -p PPAC_MI300A_SPX --gpus=1 --ntasks=4 my_batch_script.batch
```

```
salloc --ntasks=8 --gpus=1
```

```
salloc -p PPAC_MI300A_SPX --nodelist=ppac-pl1-s24-30 --gpus=1 --time=01:00:00
```

```
salloc -N 1 --gpus=4 --cpus-per-gpu=12 --time=02:00:00
```

```
sbatch my_batch_script.batch
```

Sample SLURM batch file

- Add the requested resources at the top of the batch file
- The bash script commands follow this #SBATCH section

AAC6

```
#!/bin/bash
#SBATCH --partition=PPAC_MI300A_CPX
#SBATCH --gpus=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --time=01:00:00
#SBATCH --output=my_job.out
#SBATCH --error=my_job.out
```

Job goes to default
queue if not specified



Compute Modes

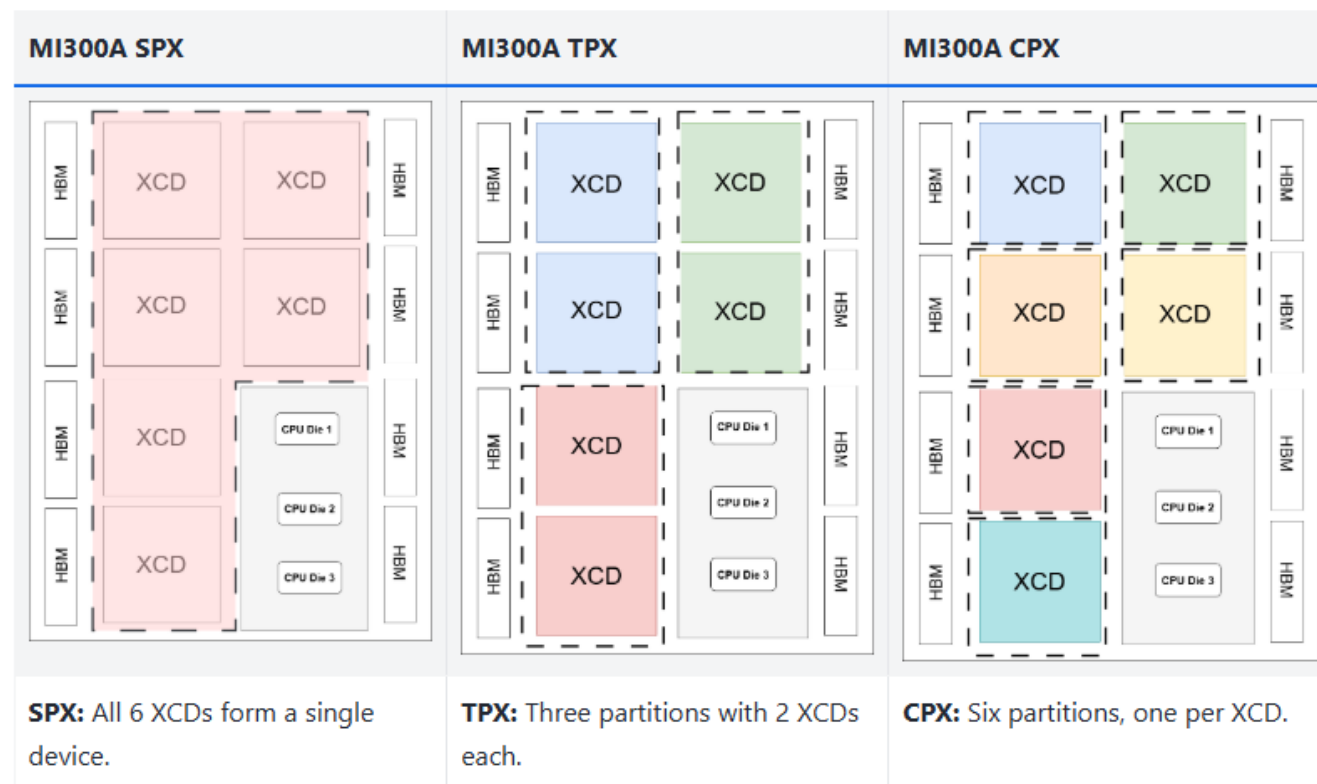
SPX – Single compute configuration
for GPU

CPX – Core compute configuration
splits MI300A into six GPUs

Always specify --gpus=X! Otherwise, you won't get a GPU!

AMD Instinct™ MI300A compute partitioning

- SPX (Single Partition X-celerator)
 - 1 logical GPU, 228 CUs, 128GB, 24 CPU cores / GPU
- TPX (Triple Partition X-celerator)
 - 3 logical GPUs, 76 CUs, 42GB, 8 CPU cores / GPU
- CPX (Core Partitioned X-celerator)
 - 6 logical GPUs, 38 CUs, 21GB, 4 CPU cores / GPU
- Logical GPU (APU) segmentation
 - Allows fine-grained scheduling, and resource optimization
 - Partitioning hardware-aware, operates at the kernel driver level
 - HSA software defined resources created for each logical GPU
- Detailed [official documentation](#)
 - Or search online for “MI300A GPU partitioning”
 - Ignore memory partitioning modes (NPS1)
 - Don’t mix up with slightly different MI300X



Disclaimer

The information presented in this document is for informational purposes only and may contain technical inaccuracies, omissions, and typographical errors. The information contained herein is subject to change and may be rendered inaccurate for many reasons, including but not limited to product and roadmap changes, component and motherboard version changes, new model and/or product releases, product differences between differing manufacturers, software changes, BIOS flashes, firmware upgrades, or the like. Any computer system has risks of security vulnerabilities that cannot be completely prevented or mitigated. AMD assumes no obligation to update or otherwise correct or revise this information. However, AMD reserves the right to revise this information and to make changes from time to time to the content hereof without obligation of AMD to notify any person of such revisions or changes.

THIS INFORMATION IS PROVIDED 'AS IS.' AMD MAKES NO REPRESENTATIONS OR WARRANTIES WITH RESPECT TO THE CONTENTS HEREOF AND ASSUMES NO RESPONSIBILITY FOR ANY INACCURACIES, ERRORS, OR OMISSIONS THAT MAY APPEAR IN THIS INFORMATION. AMD SPECIFICALLY DISCLAIMS ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, OR FITNESS FOR ANY PARTICULAR PURPOSE. IN NO EVENT WILL AMD BE LIABLE TO ANY PERSON FOR ANY RELIANCE, DIRECT, INDIRECT, SPECIAL, OR OTHER CONSEQUENTIAL DAMAGES ARISING FROM THE USE OF ANY INFORMATION CONTAINED HEREIN, EVEN IF AMD IS EXPRESSLY ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

© 2026 Advanced Micro Devices, Inc. All rights reserved. AMD, the AMD Arrow logo, ROCm, Radeon, Instinct and combinations thereof are trademarks of Advanced Micro Devices, Inc. Other product names used in this publication are for identification purposes only and may be trademarks of their respective companies.

Ubuntu and the Ubuntu logo are registered trademarks of Canonical® Ltd.
OpenMPI is distributed by The Open MPI Project under the New BSD license and copyright 2004 - 2012.





Functionality: Communication Frameworks and Libraries

IEEE Hot Interconnects Aug 19-21, 2026

Presenter: Shelby Lockhart, AMD

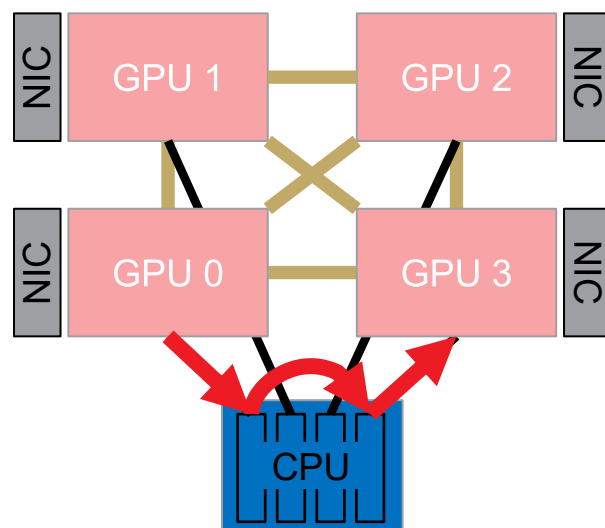
AMD 
together we advance_

Agenda

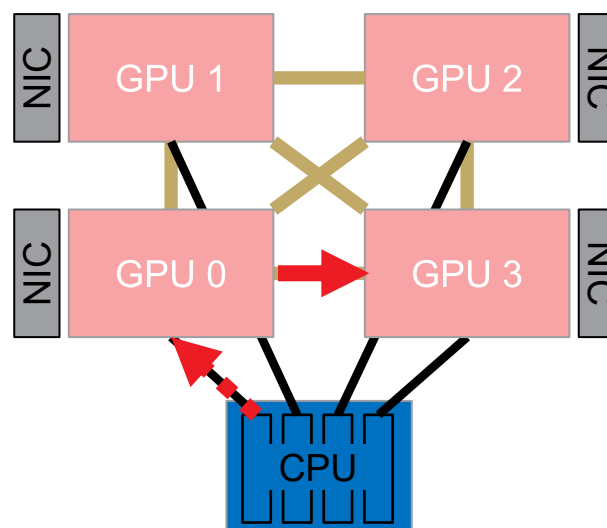
01	Architecture and Data Movement
02	MPI
03	RCCL
04	rocSHMEM
05	Functional Overlap
06	What's on Your System

Different Ways to Send Data between GPUs

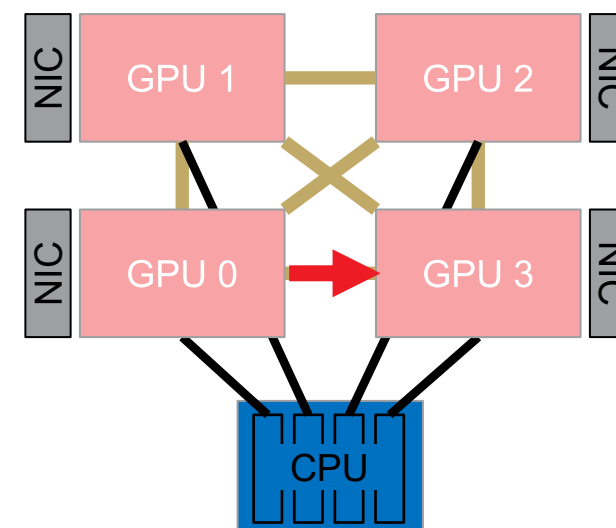
Copy to Host



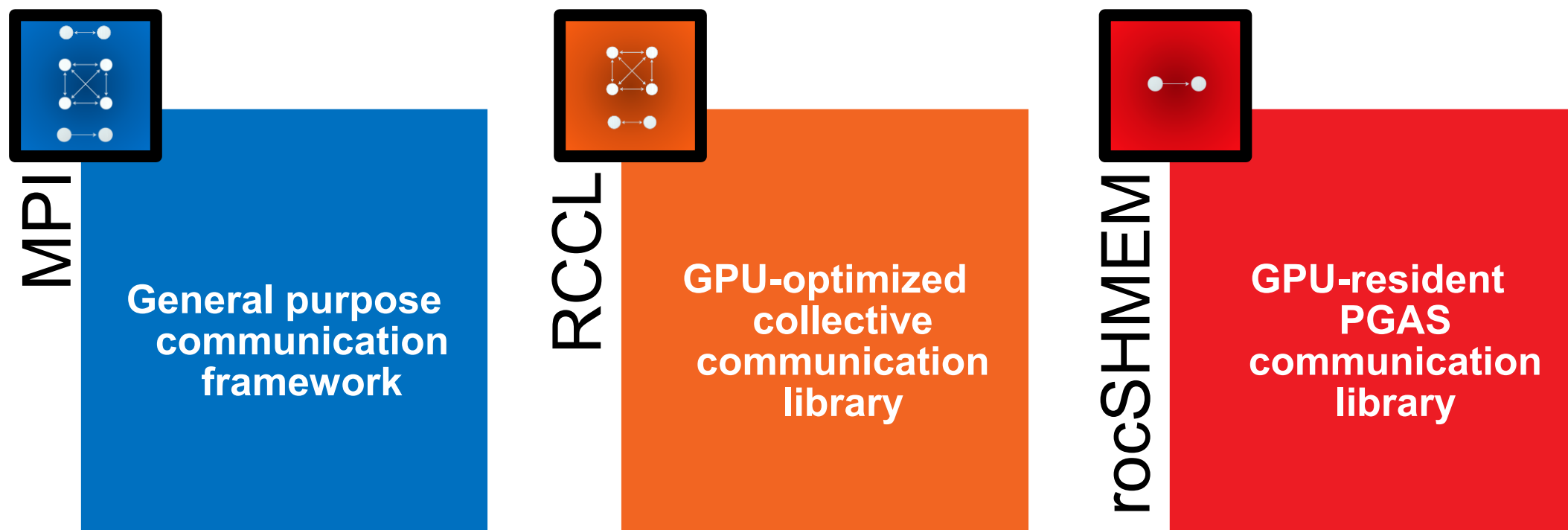
Host-Initiated



Device-Initiated



Communication Library* Landscape



All move data

*MPI is technically an API standard, which could have any number of underlying library implementations. Here, we are discussing the standard in general.

Aug 19-21, 2026

Hot Interconnects: Understanding Communication
Performance on Emerging Heterogeneous Architectures

AMD
together we advance_

MPI Is the Standard

Core Philosophy

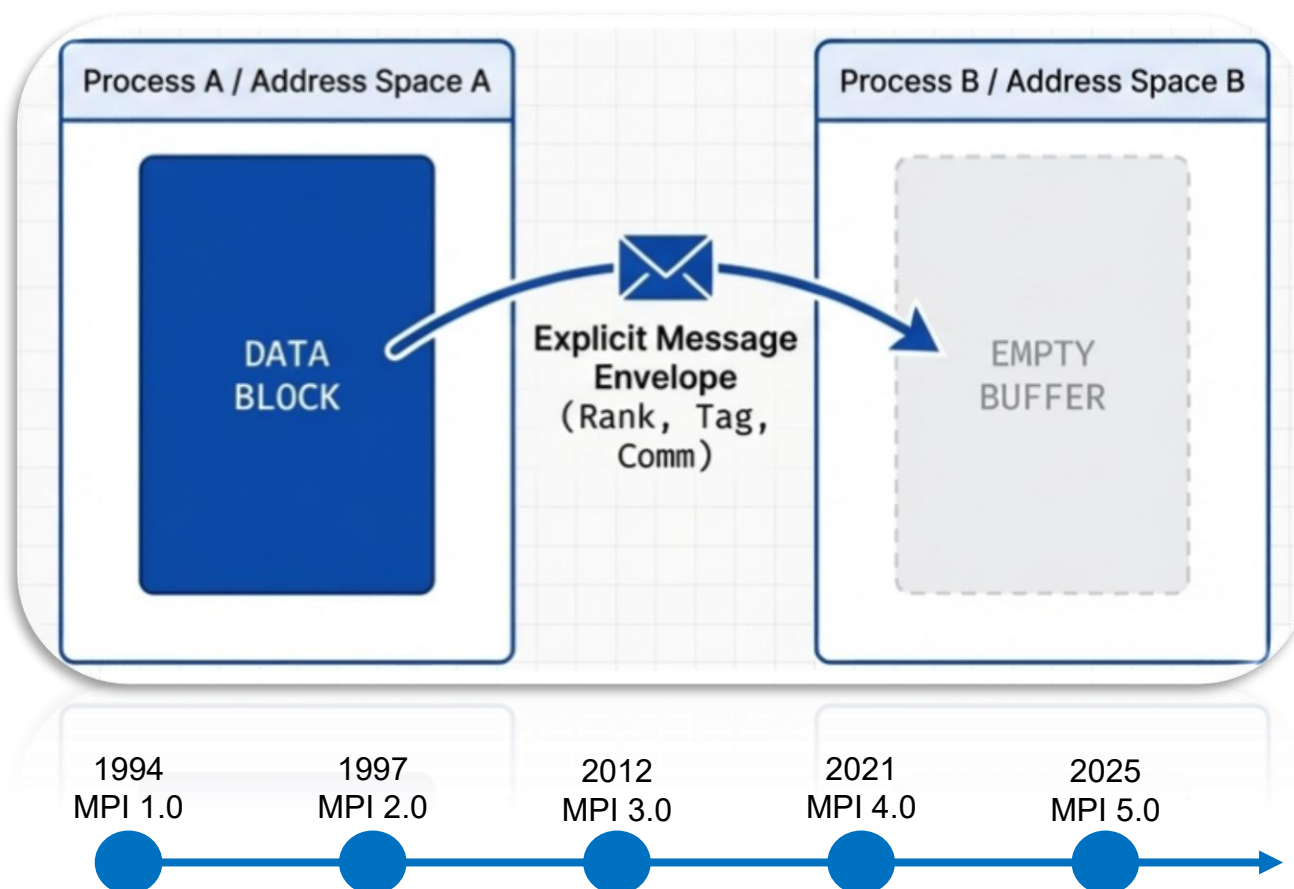
- 30+ years of defining message-passing
- Shared-nothing principle
 - Force explicit data movement between address spaces

Evolution

- CPU-centric → GPU-Aware → GPU-Initiated

Primary Strengths

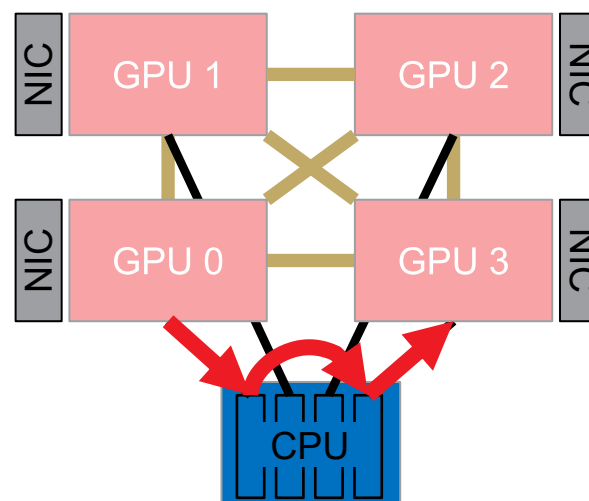
- Breadth
- Portability across heterogeneous systems
- Provides point-to-point, collective, and one-sided communication
- Focused on balancing latency and bandwidth
- Complex communication patterns
- Wildcards
- Complex data types



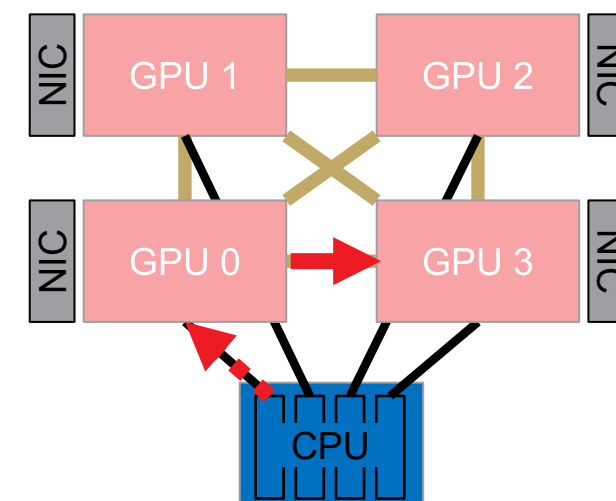
GPU-Aware MPI

- Traditionally, only pointers of the host buffers could be passed to MPI calls
- GPU-Aware MPI provides the opportunity to pass GPU buffers to MPI calls
- Without GPU-Aware MPI, GPU buffers have to be staged through host memory with `hipMemcpy`
- Many MPI implementations including CRAY-MPICH, MVAPICH and OpenMPI support GPU-Aware Communication

Copy to Host



Host-Initiated



What is GPUDirect RDMA?

- GPUDirect RDMA is a technology that provides the opportunity for network adapters to directly access GPU device memory and completely bypass the host
- Note that GPU-Aware MPI refers to support for passing GPU buffers to MPI calls in MPI implementations while GPUDirect RDMA is a technology that enables direct access to GPU memory
- A GPU-Aware MPI may or may not use GPUDirect RDMA for communications between GPUs
 - This is dependent on the HW and SW on your system, but helps determine achievable performance

GPU-Aware MPI Point-to-Point Communication Example

Allocate memory on host

Allocate
memory on
device

```
//allocate memory
h_buf=(int*) malloc(sizeof(int)*bufsize);
hipMalloc(&d_buf,bufsize*sizeof(int));

//initialize
if (rank == 0)
{
    for (i=0; i<bufsize; i++)
        h_buf[i] = i;
    hipMemcpy(d_buf, h_buf, (bufsize) * sizeof(int), hipMemcpyHostToDevice);
}

if (rank == 1)
{
    for (i=0; i<bufsize; i++)
        h_buf[i] = -1;
    hipMemcpy(d_buf, h_buf, (bufsize) * sizeof(int), hipMemcpyHostToDevice);
}

// communication
if (rank == 0) {
    MPI_Send(d_buf, bufsize, MPI_INT, 1, 123, MPI_COMM_WORLD); }

if (rank == 1) {
    MPI_Recv(d_buf, bufsize, MPI_INT, 0, 123, MPI_COMM_WORLD, &status); }

// validate results
if (rank == 1)
{
    hipMemcpy(h_buf, d_buf, (bufsize) * sizeof(int), hipMemcpyDeviceToHost);
    for (i=0; i<bufsize; i++)
    {
        if (h_buf[i] != i)
            printf("Error: buffer[%d] = %d but is expected to be %d\n", i, h_buf[i], i);
    }
    fflush(stdout);
}

free(h_buf);
hipFree(d_buf);
MPI_Finalize();
```

Initialize device buffer

GPU-Aware point-to-point communication

Validate results

Free memory

What if we don't have GPU-Aware MPI?

Stage GPU buffers through host memory with hipMemcpy

```
if (rank == 0) {  
    //copy send buffer from device to host  
    hipMemcpy(h_buf, d_buf, (bufsize) * sizeof(int), hipMemcpyDeviceToHost);  
  
    MPI_Send(h_buf, bufsize, MPI_INT, 1, 123, MPI_COMM_WORLD);  
}  
  
if (rank == 1) {  
    MPI_Recv(h_buf, bufsize, MPI_INT, 0, 123, MPI_COMM_WORLD, &status);  
  
    //copy receive buffer from host to device  
    hipMemcpy(d_buf, h_buf, (bufsize) * sizeof(int), hipMemcpyHostToDevice);  
}
```

GPU-Aware Collective Communication Example

Set device

```
//set device
hipSetDevice(rank%8);

//check device ID
hipGetDevice(&deviceID);
printf("rank%d running on device %d\n", rank, deviceID);
```

```
//allocate memory on host
h_buffer = (int *)malloc( count * sizeof(int) );
```

```
//allocate memory on device
hipMalloc(&d_sendbuf, count*sizeof(int));
hipMalloc(&d_recvbuf, count*sizeof(int));
```

Allocate send/recv buffers on device

```
//initialize send and receive buffers
for (i=0; i<count; i++) h_buffer[i] = i;
hipMemcpy(d_sendbuf, h_buffer, (count) * sizeof(int), hipMemcpyHostToDevice);

hipMemset(d_recvbuf, 0, count*sizeof(int));
```

Initialize send/recv buffers

```
//GPU-Aware Reduce
MPI_Reduce( d_sendbuf, d_recvbuf, count, MPI_INT, MPI_SUM, root, comm );
```

GPU-Aware Collective Communication

```
//validate results
if (rank == root) {
    for (i=0; i<count; i++) h_buffer[i] = 0;
    hipMemcpy(h_buffer, d_recvbuf, (count) * sizeof(int), hipMemcpyDeviceToHost);
    for (i=0; i<count; i++) {
        if (h_buffer[i] != i * size) {
            errs++;
        }
    }
    if(errs!=0) printf("errors=%d\n", errs);
}
```

Validate results

Free
memory

```
hipFree(d_sendbuf);
hipFree(d_recvbuf);
free( h_buffer );
```

RCCL Provides High-Throughput Collectives for GPUs

Core Philosophy

- Development in ~2015 to meet demands of deep learning training requirements
- Highly optimized, GPU-centric collective communication

Evolution

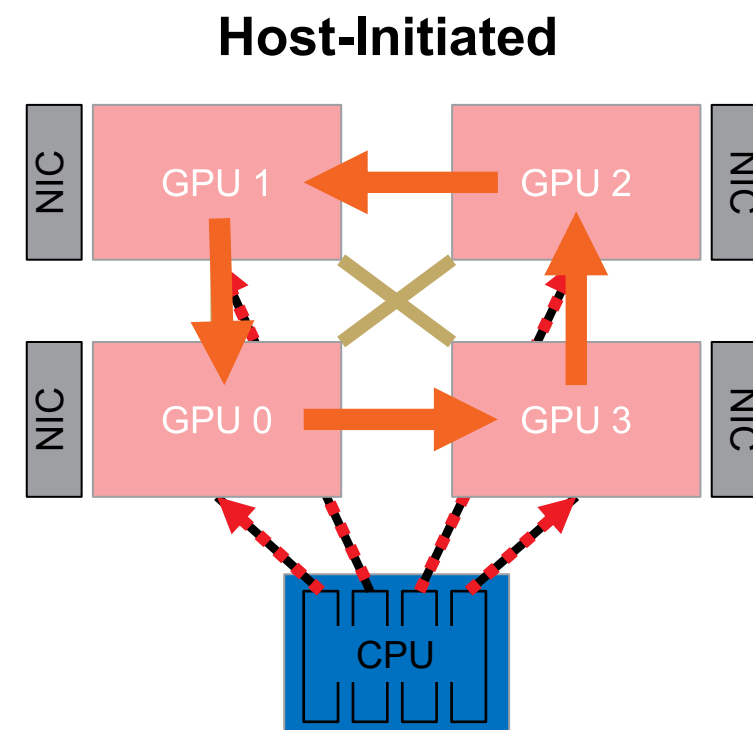
- Ring-based GPU collectives → more sophisticated collective algorithms and point-to-point communication
- Began as a straight port of NCCL, so uses NCCL API for compatability

Advantage

- Bypasses CPU, auto-detects optimal hardware topology for maximizing throughput

Primary Strengths

- Achieving high throughput in collective communications required by massive AI workloads



RCCL Point-to-Point Example

Allocate
memory on
device

Initialize device
buffer

RCCL
point-to-point
communication

```
//set device
HIPCHECK(hipSetDevice(rank));

//check device ID
HIPCHECK(hipGetDevice(&deviceID));
printf("rank%d running on device %d\n", rank, deviceID);

//rank 0 creates the RCCL id and broadcasts it to the other ranks over MPI
if (rank == 0) NCCLCHECK(ncclGetUniqueId(&id));
MPI_Bcast((void *)&id, sizeof(id), MPI_BYTE, 0, MPI_COMM_WORLD);

//create the RCCL communicator and a stream to run RCCL ops on
NCCLCHECK(ncclCommInitRank(&comm, size, id, rank));
HIPCHECK(hipStreamCreate(&stream));

//allocate memory
h_buf=(int*) malloc(sizeof(int)*bufsize);
HIPCHECK(hipMalloc(&d_buf,bufsize*sizeof(int)));

//initialize
if (rank == 0) { for (i=0; i<bufsize; i++) h_buf[i] = i; }
if (rank == 1) { for (i=0; i<bufsize; i++) h_buf[i] = -1; }
HIPCHECK(hipMemcpy(d_buf, h_buf, (bufsize) * sizeof(int), hipMemcpyHostToDevice));

// communication
if (rank == 0) {
    NCCLCHECK(ncclSend(d_buf, bufsize, ncclInt, 1, comm, stream)); }

if (rank == 1) {
    NCCLCHECK(ncclRecv(d_buf, bufsize, ncclInt, 0, comm, stream)); }

HIPCHECK(hipStreamSynchronize(stream));

// validate results
if (rank == 1)
{
    int flag=0;
    HIPCHECK(hipMemcpy(h_buf, d_buf, (bufsize) * sizeof(int), hipMemcpyDeviceToHost));
    for (i=0; i<bufsize; i++) { if (h_buf[i] != i) flag++; }
    if(flag==0) printf("Run successful: received buffer has the right value\n");
    fflush(stdout);
}

free(h_buf);
HIPCHECK(hipFree(d_buf));
HIPCHECK(hipStreamDestroy(stream));
ncclCommDestroy(comm);
```

Looks very similar to the GPU-Aware MPI point-to-point communication.

Except now we have a stream parameter.

And we have a stream synchronize.

Validate
results

Free memory

Hot Interconnects: Understanding Communication
Performance on Emerging Heterogeneous Architectures

RCCL

Collective Example

Allocate
memory on
device

Initialize device
buffer

RCCL
collective
communication

```
//set device
HIPCHECK(hipSetDevice(rank%8));

//check device ID
HIPCHECK(hipGetDevice(&deviceID));
printf("rank%d running on device %d\n", rank, deviceID);

//rank 0 creates the RCCL id and broadcasts it to the other ranks over MPI
if (rank == 0) NCCLCHECK(ncclGetUniqueId(&id));
MPI_Bcast((void *)&id, sizeof(id), MPI_BYTE, 0, MPI_COMM_WORLD);

//create the RCCL communicator and a stream to run RCCL ops on
NCCLCHECK(ncclCommInitRank(&comm, size, id, rank));
HIPCHECK(hipStreamCreate(&stream));

//allocate memory on host
h_buffer = (int *)malloc( count * sizeof(int) );

//allocate memory on device
HIPCHECK(hipMalloc(&d_sendbuf, count*sizeof(int)));
HIPCHECK(hipMalloc(&d_recvbuf, count*sizeof(int)));

//initialize send and receive buffers
for (i=0; i<count; i++) h_buffer[i] = i;
HIPCHECK(hipMemcpy(d_sendbuf, h_buffer, (count) * sizeof(int), hipMemcpyHostToDevice));

HIPCHECK(hipMemset(d_recvbuf, 0, count*sizeof(int)));

//GPU-to-GPU Reduce over RCCL
NCCLCHECK(ncclReduce(d_sendbuf, d_recvbuf, count, ncclInt, ncclSum, root, comm, stream));

HIPCHECK(hipStreamSynchronize(stream));

//validate results
if (rank == root) {
    for (i=0; i<count; i++) h_buffer[i] = 0;
    HIPCHECK(hipMemcpy(h_buffer, d_recvbuf, (count) * sizeof(int), hipMemcpyDeviceToHost));
    for (i=0; i<count; i++) {
        if (h_buffer[i] != i * size) {
            errs++;
        }
    }
    if(errs!=0) printf("errors=%d\n", errs);
    if(errs==0) printf("Run successful: Reduced buffer has the right value\n");
}

HIPCHECK(hipFree(d_sendbuf));
HIPCHECK(hipFree(d_recvbuf));
free( h_buffer );
HIPCHECK(hipStreamDestroy(stream));
ncclCommDestroy(comm);
```

Looks very similar to the GPU-Aware MPI collective communication.

Except now we have a stream parameter.

And we have a stream synchronize.

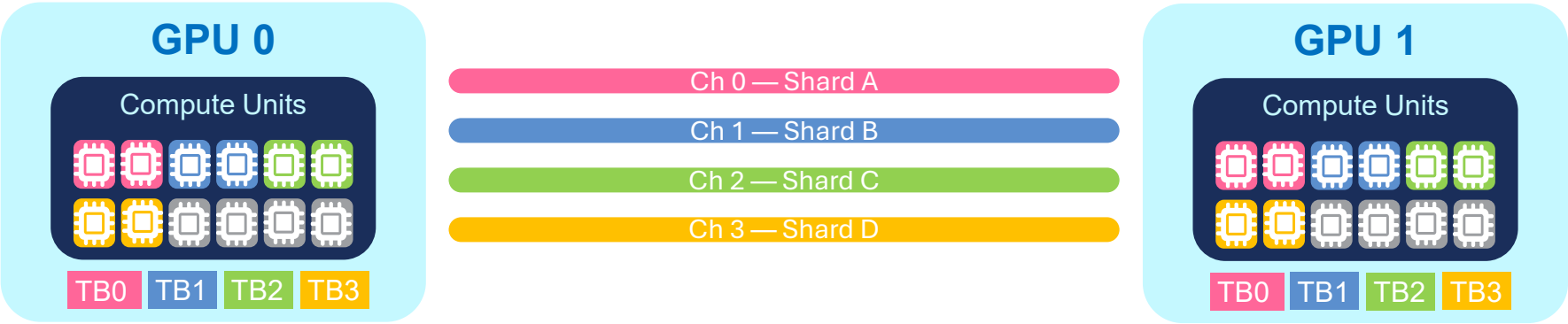
Validate
results

Hot Interconnects: Understanding Communication
Performance on Emerging Heterogeneous Architectures

Free memory

How does RCCL move data?

RCCL distributes data across multiple independent channels, each uses a dedicated thread block of Compute Units (CUs) launched via a stream



 **Remaining CUs can be used for compute**

Diagram Key

Thread Blocks

TB0 TB1 TB2 TB3

CUs

Each thread block uses multiple CUs for communication or computation















Shards

The data buffer being sent is divided into chunks, each sent with a distinct channel

Ch 0

Ch 1

Ch 2

Ch 3

Common RCCL Tuning Environment Variables

Environment Variable	Options	What It Does
NCCL_ALGO	RING, TREE	Determines the algorithm used for collective communication. AllReduce is the only collective supporting an option b/t these 2.
NCCL_PROTO	LL, LL128, SIMPLE	The protocol with which to send the data, LL protocols are lower latency and inherently designed for smaller chunks of data
NCCL_NCHANNELS	1..64	Determines the number of channels to be used (and as a consequence, the number of CUs)

rocSHMEM Maps a PGAS across GPUs

PGAS = Partitioned Global Address Space

Core Philosophy

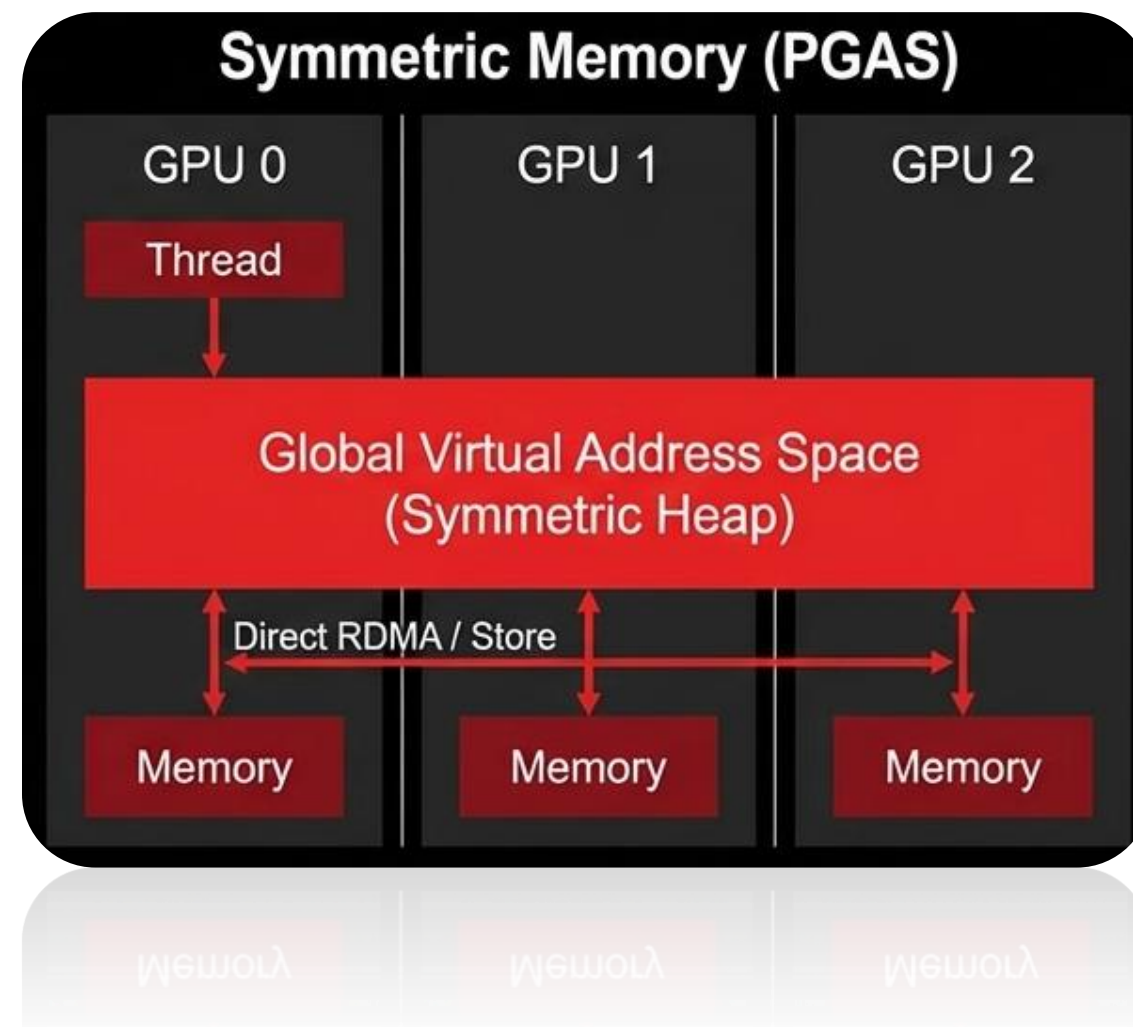
- Implements OpenSHMEM standard on GPUs
- Creates a symmetric heap where threads on GPU 0 can natively read/ write to a memory address on GPU 1

Advantage

- One-sided, fine-grained communication
- Completely device-initiated without leaving the GPU kernel

Primary Strengths

- Irregular data access patterns
- Graph analytics
- Scenarios where traditional message passing creates too high of overheads



Initialize
rocshmem

Allocate buffer from
the symmetric heap for
communication

Aug 19-21, 2026

Free memory and
finalize rocshmem

```
{ rocshmem_init();

int my_pe = rocshmem_my_pe();
int npes = rocshmem_n_pes();

if (npes < 2) {
    printf("Please run with two PEs.\n"); fflush(stdout);
    rocshmem_finalize();
    return 0;
}

//set device
HIPCHECK(hipSetDevice(get_launcher_local_rank()));
printf("pe%d running on device %d\n", my_pe, get_launcher_local_rank());

// d_buf is allocated from the symmetric heap: every PE calls rocshmem_malloc for the same size,
// so d_buf on PE 0 and d_buf on PE 1 refer to the same symmetric offset.
int *d_buf = (int *)rocshmem_malloc(bufsize * sizeof(int));
if (d_buf == NULL) {
    printf("Error allocating memory from symmetric heap\n");
    rocshmem_global_exit(1);
}

//initialize
if (my_pe == 0) { for (i = 0; i < bufsize; i++) d_buf[i] = i; }
if (my_pe == 1) { for (i = 0; i < bufsize; i++) d_buf[i] = -1; }
HIPCHECK(hipDeviceSynchronize());

// communication: PE 0 pushes its buffer directly into PE 1's symmetric
// d_buf; PE 1 makes no call at all -- this is the one-sided put.
if (my_pe == 0) {
    put_to_peer<<<dim3(1), dim3(32), 0, 0>>>(d_buf, d_buf, bufsize, 1);
}
rocshmem_barrier_all();
HIPCHECK(hipDeviceSynchronize());

// validate results
if (my_pe == 1) {
    int flag = 0;
    for (i = 0; i < bufsize; i++) {
        if (d_buf[i] != i) {
            flag++;
            printf("Error: buffer[%d] = %d but is expected to be %d\n", i, d_buf[i], i); } }
    if (flag == 0) printf("Run successful: received buffer has the right value\n");
    fflush(stdout);
}

rocshmem_free(d_buf);
rocshmem_finalize();
}
```

rocSHMEM Put Example

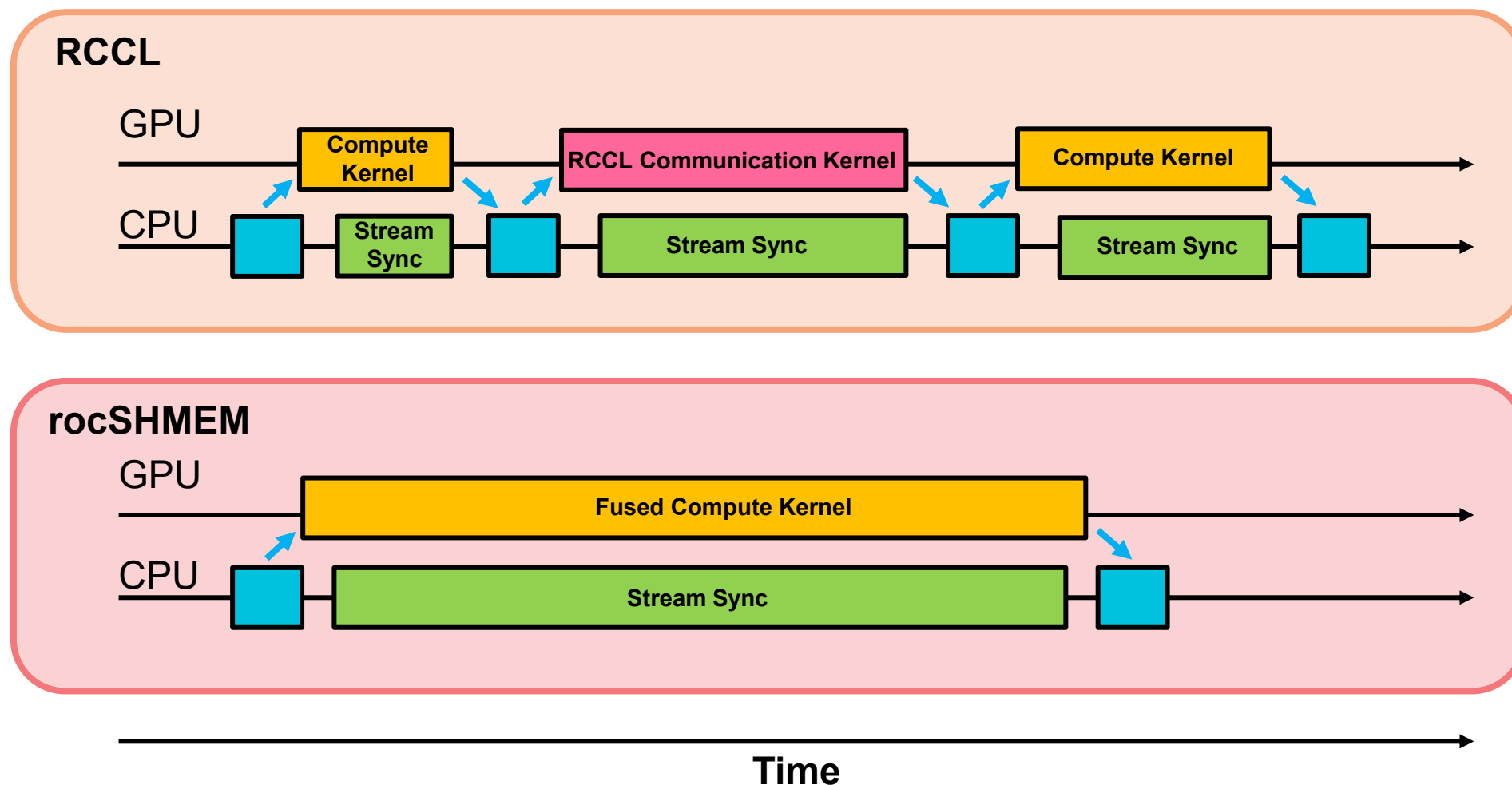
Note the difference between MPI
and RCCL point-to-point
communication.

Here we just have a one-sided
Put. Only PE 0 is putting data into
the buffer.

Validate results

Hot Interconnects: Understanding Communication
Performance on Emerging Heterogeneous Architectures

Maximizing GPU Utilization: RCCL vs rocSHMEM



Functional Overlap

	MPI	*CCL	*SHMEM
Programming Paradigm	Explicit message passing	Large message collectives (BW-bound)	PGAS one-sided
Point-to-Point	- Blocking - Non-blocking	Blocking on GPU stream/asynchronous from CPU perspective	- Put - Get
Collectives	- Standard - Non-blocking - CPU-based - GPU-based - Vector Ops	Highly optimized for throughput	Team-Based
One-Sided	RMA for CPUs	Starting to support RMA	Native
Initiation	CPU or GPU	CPU or GPU	GPU

* Using *CCL and *SHMEM here as this comparison is vendor-agnostic

What's on My System?

```
slockhar@aac6-fe1:~$ module load rocm/7.2.4
slockhar@aac6-fe1:~$ module avail
```

----- /nfsapps/ubuntu-24.04/modules/rocmplus-7.2.4 -----									
cupy/13.5.1	hypre/3.1.0	netcdf-fortran/4.6.2			pytorch/2.9.1	scorep/9.4			
elpa/master-548c64b	intellikit/main	openblas/0.5.53			pytorch/2.10.0_tunableop_enabled	scorep/11.0-dev (D)			
fftw/3.3.10	jax/0.8.0	openmpi/5.0.10-ucx1.6.0-ucx1.19.1-xpmem-2.7.4			pytorch/2.10.0	tau/dev			
ftorch/2.12.0	kokkos/4.7.04	petsc/3.24.1			pytorch/2.11.0_tunableop_enabled	tensorflow/2.20.0			
ftorch_amdflang/2.12.0	likwid/5.5.1	pnetcdf/1.14.1			pytorch/2.11.0	umpire/2025.12.0			
hdf5/2.1.1	magma/2.10.0	pytorch/2.7.1_tunableop_enabled			pytorch/2.12.0_tunableop_enabled	vllm/0.24.0			
hip-python/7.2.4	mdh/1.0.6	pytorch/2.7.1			pytorch/2.12.0 (D)				
hipblaslt/patched (L)	mpi4py/4.1.0	pytorch/2.8.0_tunableop_enabled			rccl-tests/7.2.4				
hipifly/dev	mpitrace/7.2.4	pytorch/2.8.0			roc-prim/0.5.0				
hpctoolkit/2025.1.2	netcdf-c/4.10.0	pytorch/2.9.1_tunableop_enabled			rocshmem/3.2.0-ro_ipc				

----- /nfsapps/ubuntu-24.04/modules/rocm-7.2.4 -----			
amdclang/22.0.0-7.2.4	hipfort/7.2.4	opencl/7.2.4	

----- /nfsapps/ubuntu-24.04/modules/base -----							
emacs/30.1	rocm/afar-23.1.0-7.12.0	rocm/6.3.2	rocm/6.4.1	rocm/7.0.1	rocm/7.2.0	rocm/7.2.4 (L,D)	roofline-extractor/dev
miniconda3/25.3.1	rocm/afar-23.2.1-7.13.0	rocm/6.3.3	rocm/6.4.2	rocm/7.0.2	rocm/7.2.1	rocm/7.12.0	tracelens/dev
miniforge3/24.9.0	rocm/6.3.0	rocm/6.3.4	rocm/6.4.3	rocm/7.1.0	rocm/7.2.2	rocm/7.13.0	turbovnc/3.0.3
rocbudai/dev	rocm/6.3.1	rocm/6.4.0	rocm/7.0.0	rocm/7.1.1	rocm/7.2.3	rocm/7.14.0	uprof/5.3-518

Where:
L: Module is loaded
D: Default Module

Verifying GPU Communication Library Support

```

slockhar@aac6-fe1:~$ module load openmpi
slockhar@aac6-fe1:~$ ompi_info | grep "Configure"
  Configure architecture: x86_64-pc-linux-gnu
    Configure by: admin
    Configure on: Sun Jul  5 19:25:42 UTC 2026
    Configure host: sh5-pl1-s12-15
  Configure command line: '--prefix=/nfsapps/ubuntu-24.04/opt/rocmplus-7.2.4/openmpi-5.0.10-ucc-1.6.0-ucx-1.19.1-xpmem-2.7.4' '--with-
h-rocm=/nfsapps/ubuntu-24.04/opt/rocm-7.2.4' '--with-ucx=/nfsapps/ubuntu-24.04/opt/rocmplus-7.2.4/ucx-1.19.1-xpmem-2.7.4' '--with-uc
c=/nfsapps/ubuntu-24.04/opt/rocmplus-7.2.4/ucc-1.6.0-ucx-1.19.1-xpmem-2.7.4' '--with-pmix=internal' '--with-prte=internal' '--enabl
e-mca-no-build=bt1-uct' '--enable-mpi' '--enable-mpi-fortran' '--disable-debug' 'CC=gcc' 'CXX=g++' 'FC=amdflang' '--with-xpmem=/nfsa
pps/ubuntu-24.04/opt/rocmplus-7.2.4/xpmem-2.7.4'
slockhar@aac6-fe1:~$

```

UCX is how we enable device support for OpenMPI

- OpenMPI's configure can be checked, the only *real* test is benchmarking
- RCCL is packaged with rocm
- rocSHMEM is usually not built by default and requires building

Hands-On Exercises

Log onto the system and probe the available modules

README

GPU Communication Micro-Benchmarks

This directory contains build and run scripts for three industry-standard benchmark suites that measure **point-to-point** and **collective communication latency and bandwidth** between AMD GPUs.

These benchmarks complement the application-level measurements in `CG-Tutorial/` by isolating communication performance from computation. Together they answer: *how fast can data actually move between GPUs on this node?*

Benchmark suites

Suite	What it measures	Communication layer
osu/	Point-to-point latency & bandwidth (GPU buffer → GPU buffer)	GPU-Aware MPI (UCX)
rccl-tests/	Send/recv latency & bandwidth; all-reduce throughput	RCCL (ROCm Collective Communications Library)
rocshmem/	One-sided put/get latency & bandwidth (intra-node)	rocSHMEM IPC backend

```
git clone https://github.com/Communication-Performance-Tutorial/micro-benchmarks.git  
  
cd micro-benchmarks/osu && bash build.sh  
  
cd ../micro-benchmarks/rccl-tests && bash build.sh  
  
cd ../micro-benchmarks/rocshmem && bash build.sh
```

Disclaimer

The information presented in this document is for informational purposes only and may contain technical inaccuracies, omissions, and typographical errors. The information contained herein is subject to change and may be rendered inaccurate for many reasons, including but not limited to product and roadmap changes, component and motherboard version changes, new model and/or product releases, product differences between differing manufacturers, software changes, BIOS flashes, firmware upgrades, or the like. Any computer system has risks of security vulnerabilities that cannot be completely prevented or mitigated. AMD assumes no obligation to update or otherwise correct or revise this information. However, AMD reserves the right to revise this information and to make changes from time to time to the content hereof without obligation of AMD to notify any person of such revisions or changes.

THIS INFORMATION IS PROVIDED 'AS IS.' AMD MAKES NO REPRESENTATIONS OR WARRANTIES WITH RESPECT TO THE CONTENTS HEREOF AND ASSUMES NO RESPONSIBILITY FOR ANY INACCURACIES, ERRORS, OR OMISSIONS THAT MAY APPEAR IN THIS INFORMATION. AMD SPECIFICALLY DISCLAIMS ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, OR FITNESS FOR ANY PARTICULAR PURPOSE. IN NO EVENT WILL AMD BE LIABLE TO ANY PERSON FOR ANY RELIANCE, DIRECT, INDIRECT, SPECIAL, OR OTHER CONSEQUENTIAL DAMAGES ARISING FROM THE USE OF ANY INFORMATION CONTAINED HEREIN, EVEN IF AMD IS EXPRESSLY ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Third-party content is licensed to you directly by the third party that owns the content and is not licensed to you by AMD. ALL LINKED THIRD-PARTY CONTENT IS PROVIDED "AS IS" WITHOUT A WARRANTY OF ANY KIND. USE OF SUCH THIRD-PARTY CONTENT IS DONE AT YOUR SOLE DISCRETION AND UNDER NO CIRCUMSTANCES WILL AMD BE LIABLE TO YOU FOR ANY THIRD-PARTY CONTENT. YOU ASSUME ALL RISK AND ARE SOLELY RESPONSIBLE FOR ANY DAMAGES THAT MAY ARISE FROM YOUR USE OF THIRD-PARTY CONTENT.

© 2026 Advanced Micro Devices, Inc. All rights reserved. AMD, the AMD Arrow logo, ROCm, Infinity Fabric, and combinations thereof are trademarks of Advanced Micro Devices, Inc. in the United States and/or other jurisdictions. Other names are for informational purposes only and may be trademarks of their respective owners.

Git and the Git logo are either registered trademarks or trademarks of Software Freedom Conservancy, Inc., corporate home of the Git Project, in the United States and/or other countries



Performance: Communication Frameworks and Libraries

Presenter: Amanda Bienz

Associate Professor

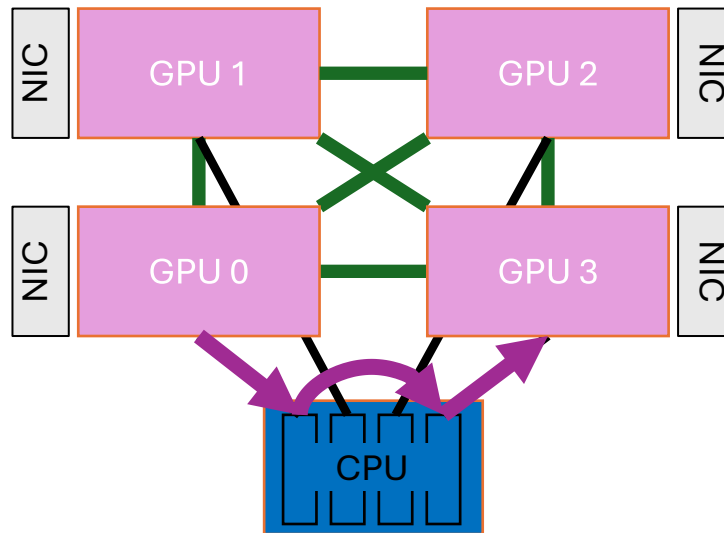


Agenda

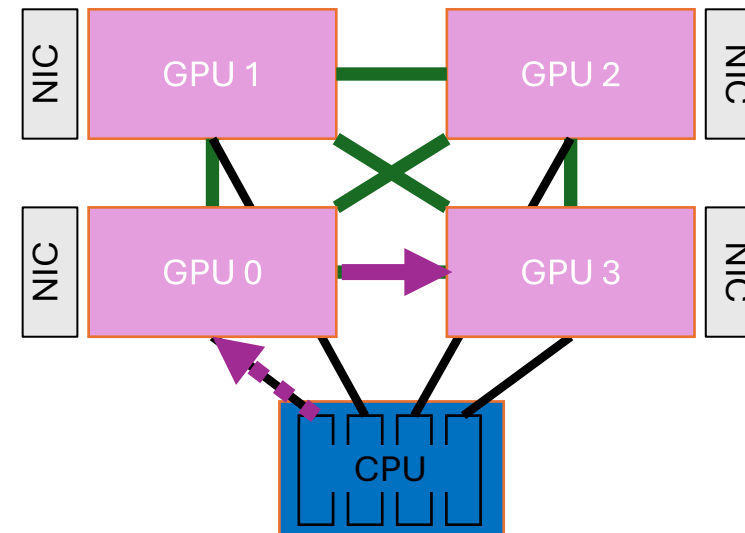
- Introduction
- Running GPU-Aware MPI examples
 - Point-to-Point Communication Example
 - Collective Communication Example
- Measuring GPU-Aware Communication BW and Latency
 - GPU Placement Consideration on AAC cloud
 - GPU Placement Consideration on LUMI
 - Communication Options
 - Measuring intra-node/inter-node communication bandwidth
 - Measuring collective communication performance
 - Communication performance on MI300A
- ROCm Collective Communication Library (RCCL)
- Summary
- GPU-Aware MPI Exercises
- MPI Ghost Exchange Example

GPU-Aware MPI

Copy to Host



Host-Initiated



Point-to-Point Communication

Data is exchanged directly between two processes using send() and recv().

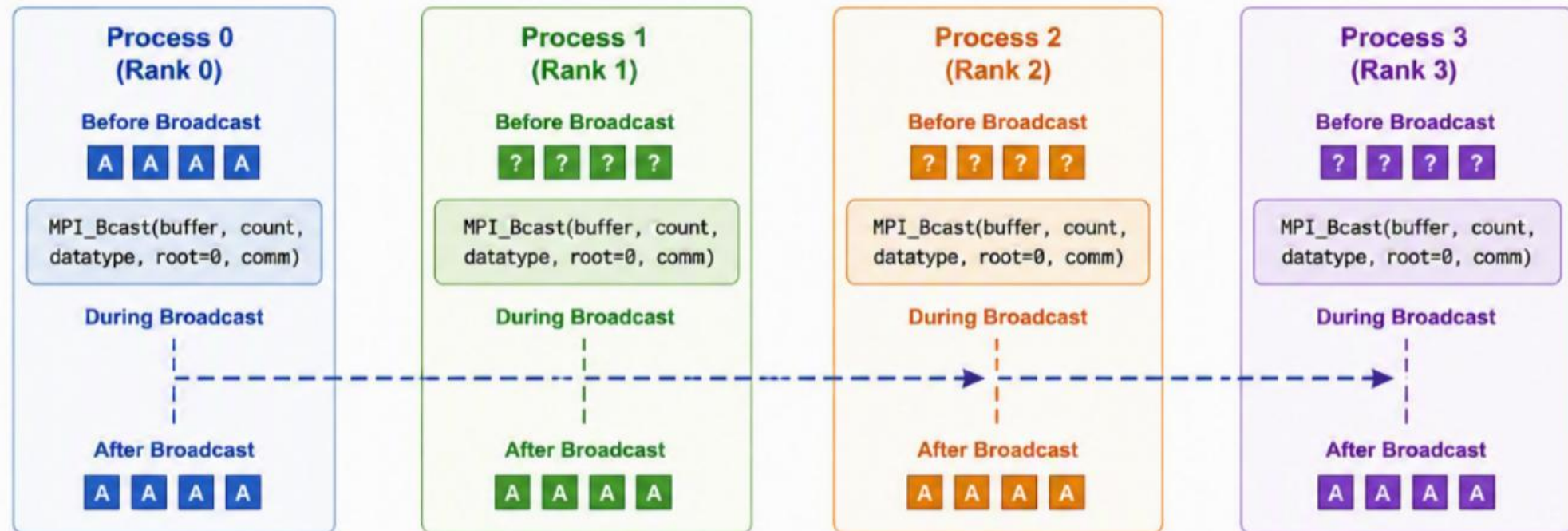


Collective Communication

The same operation is performed by a group of processes.

Example: MPI_Bcast (Broadcast)

Operation: Process 0 broadcasts data to all other processes in the communicator.

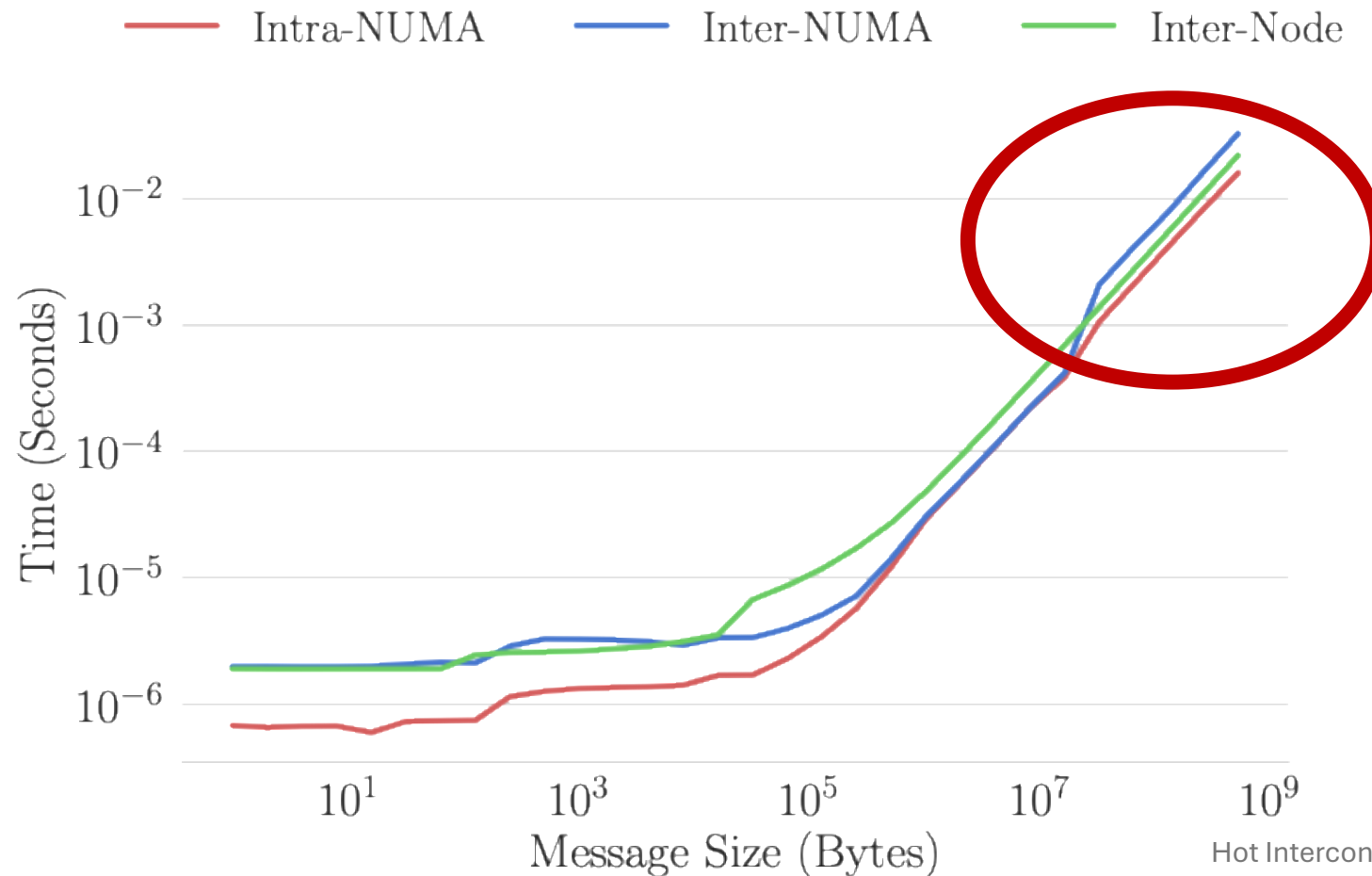


Expectations vs Reality

- Expectation: on-node communication is cheaper than off-node

Expectations vs Reality

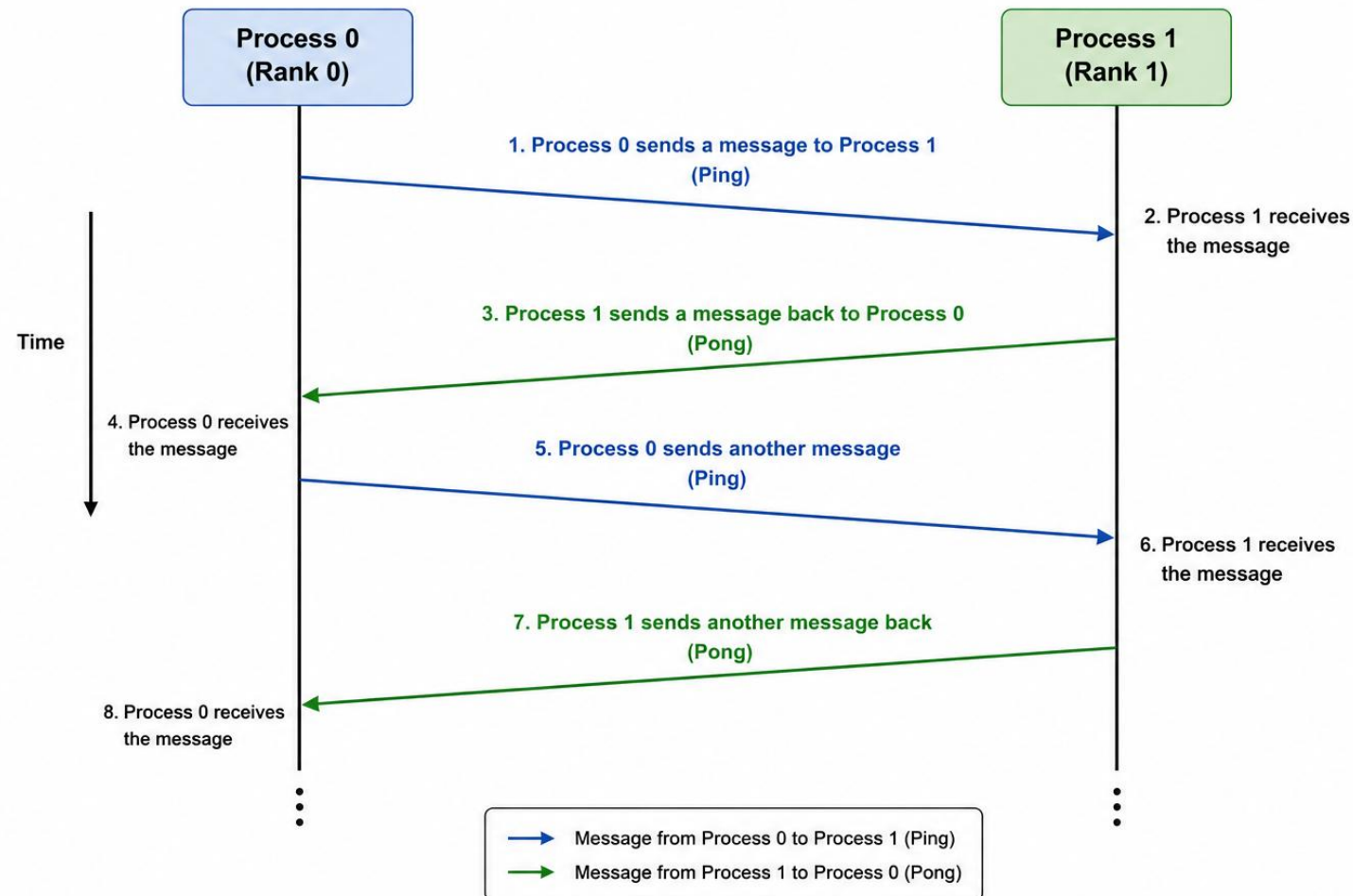
- Reality: large inter-NUMA messages are much more expensive than off-node



Microbenchmarks

- Targetted measurements to capture the cost of data movement
 - Latency: round-trip cost between two processes
 - Bandwidth: speed at which bytes are transported between two processes
 - Injection Bandwidth: speed at which the NIC can inject data into the network
- Measure different levels of locality:
 - Intra-NUMA messages vs inter-NUMA vs inter-node
- Can be used to measure baseline costs (latency, bandwidth), which can be plugged into performance models to predict and analyze costs of more complicated operations

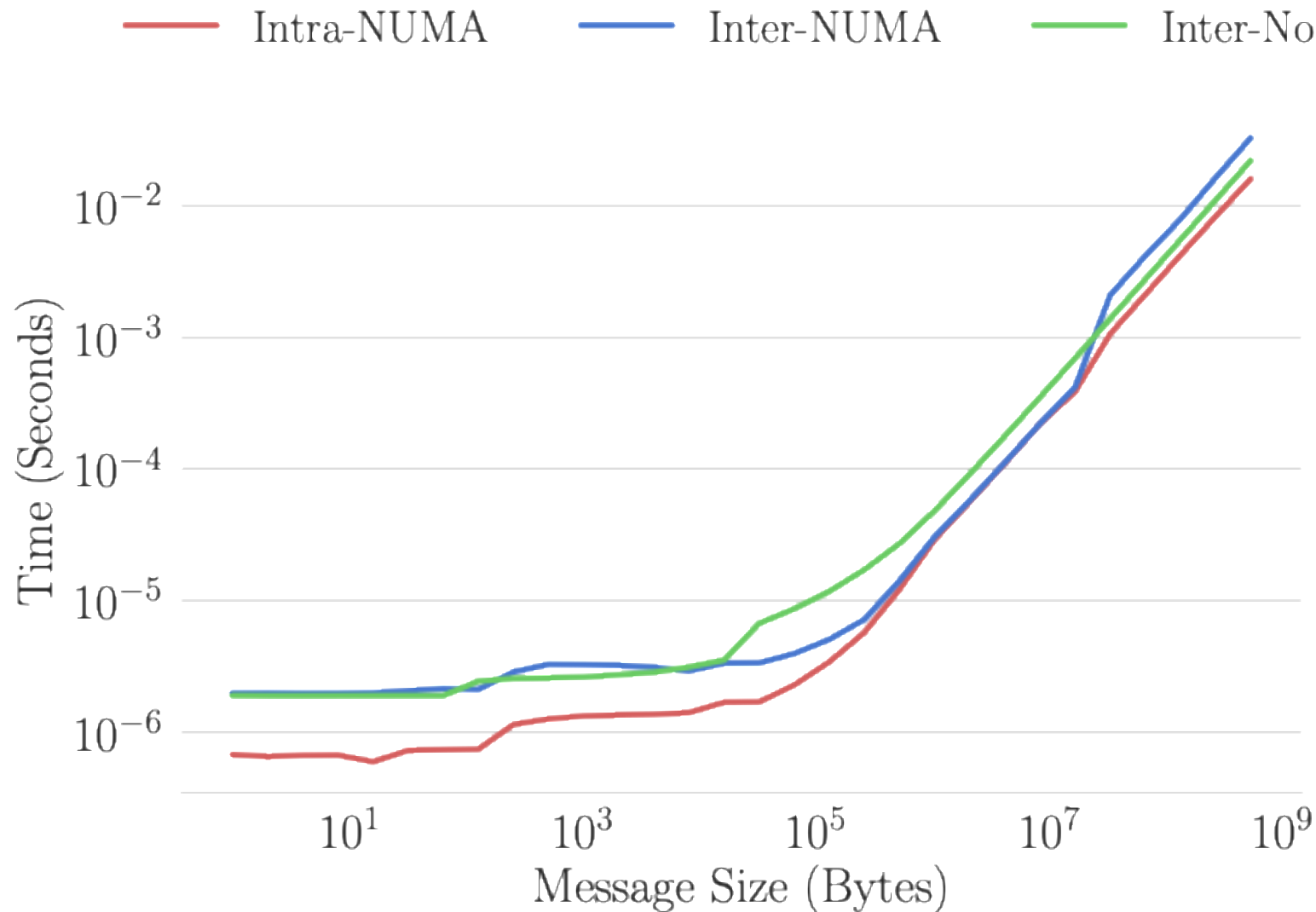
Measuring Latency : Ping-Pongs



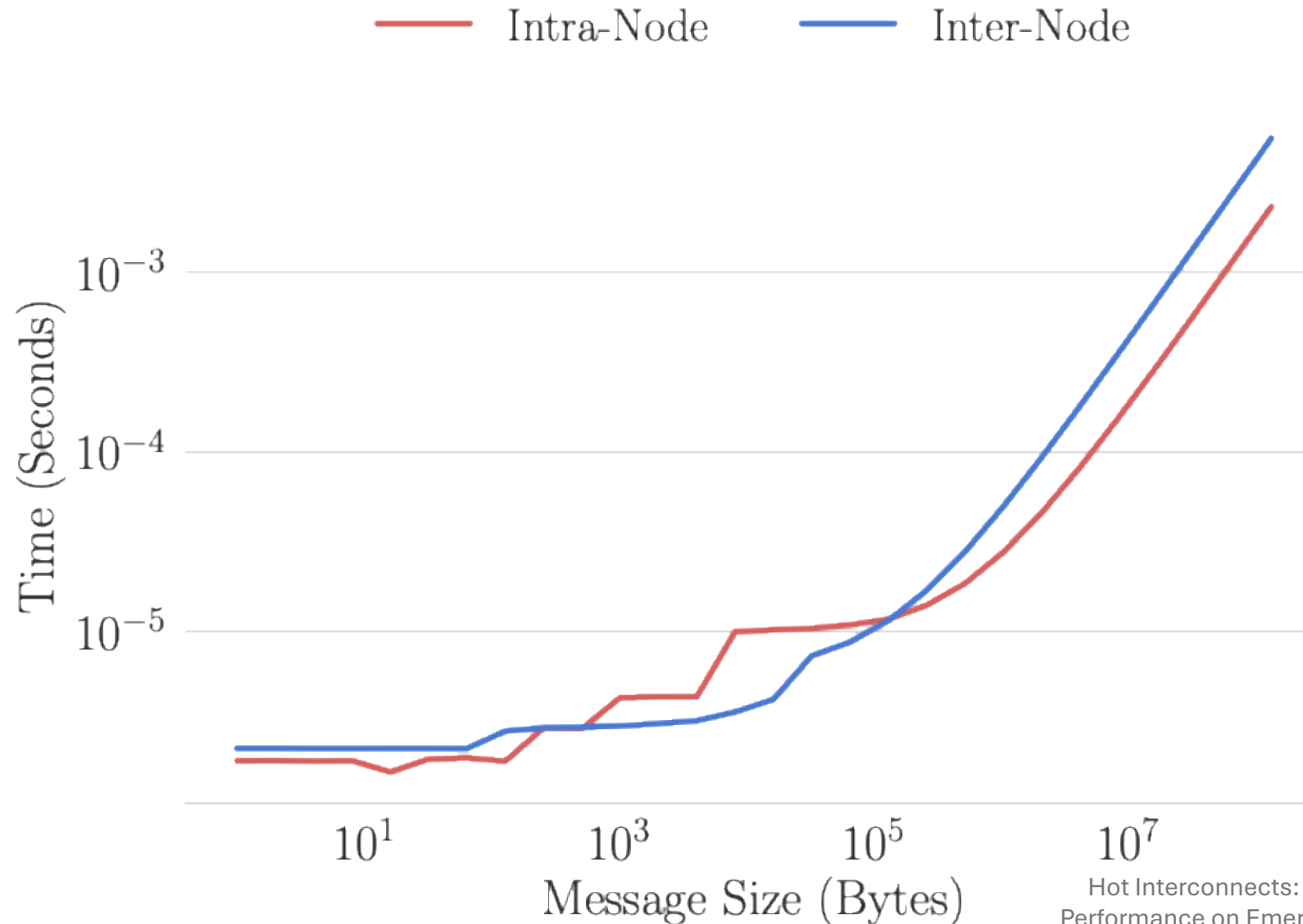
- Measure the cost of a single round-trip message

Aug 19-21, 2026

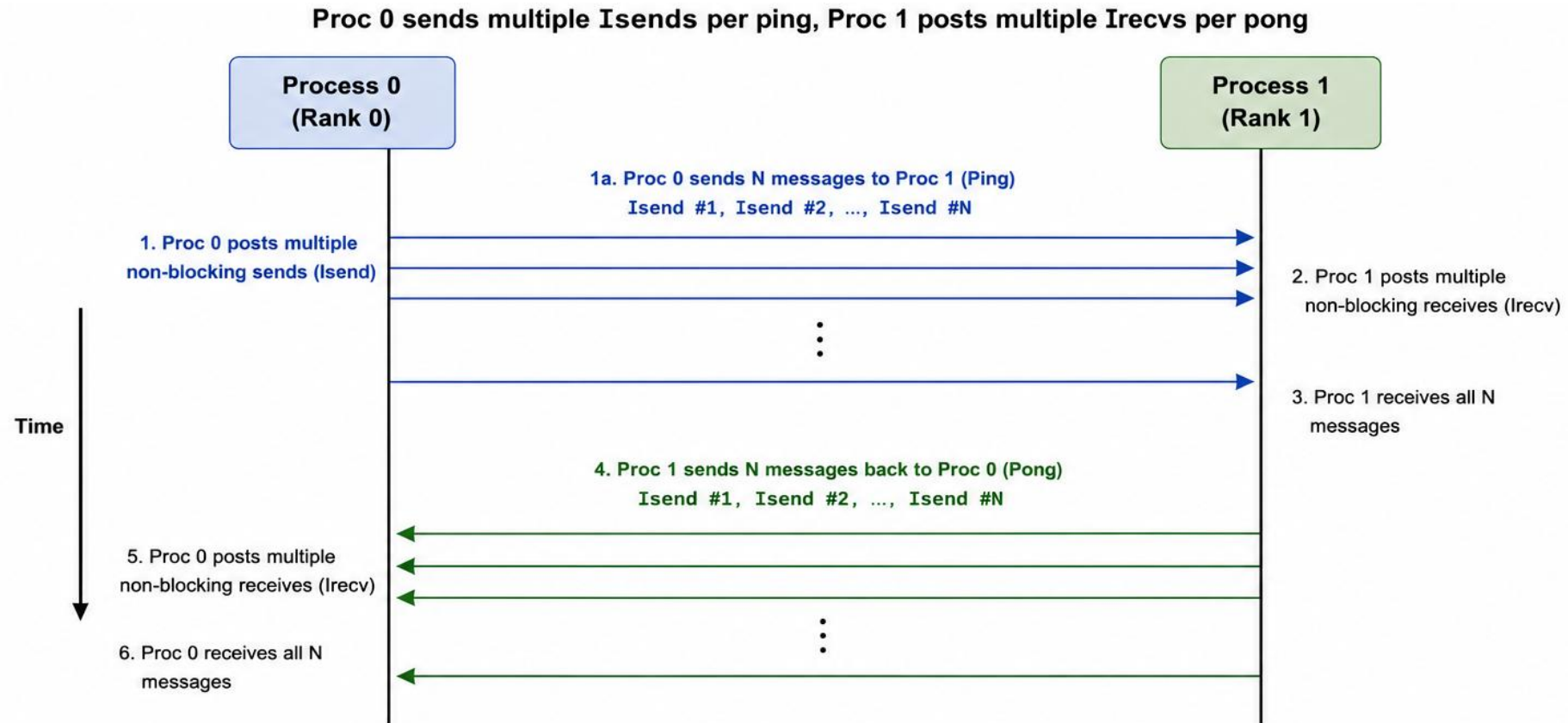
Measuring Latency : Inter-CPU Ping-Pongs



Measuring Latency : Inter-GPU Ping-Pongs

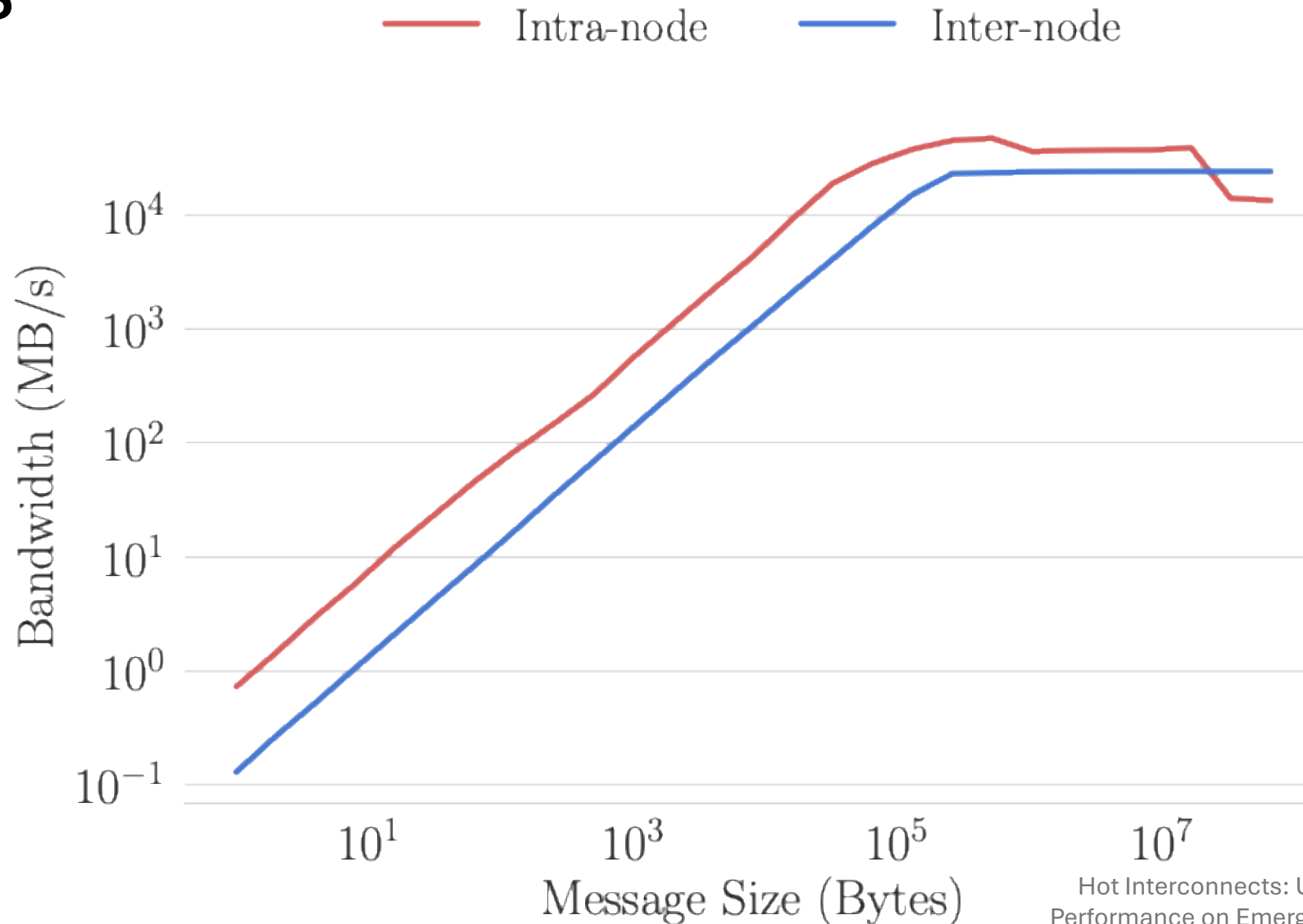


Measuring Bandwidth: Multi-Ping-Pongs



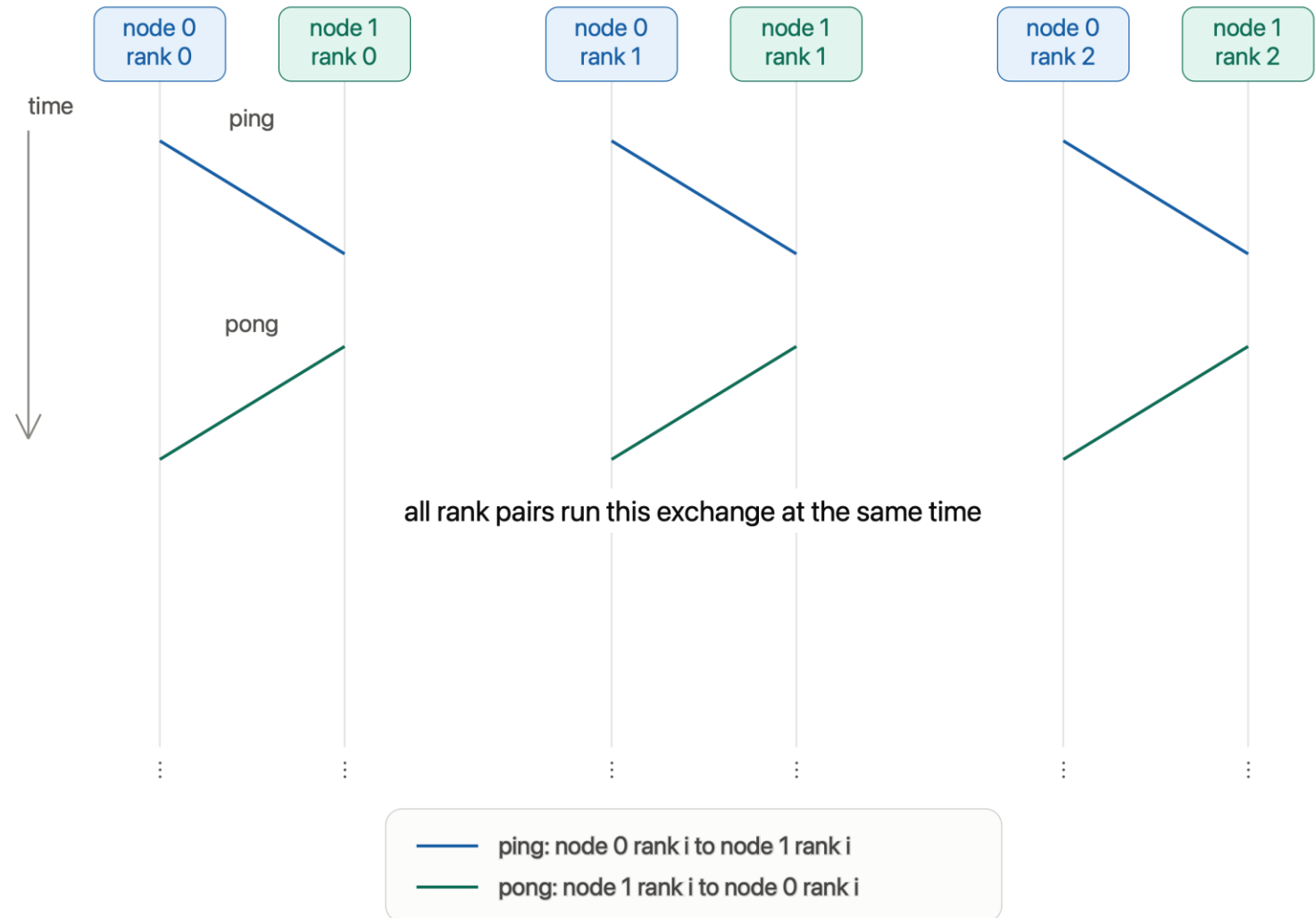
- Measure how well multiple messages can utilize available bandwidth

Measuring Bandwidth: Inter-GPU Multi-Ping-Pongs

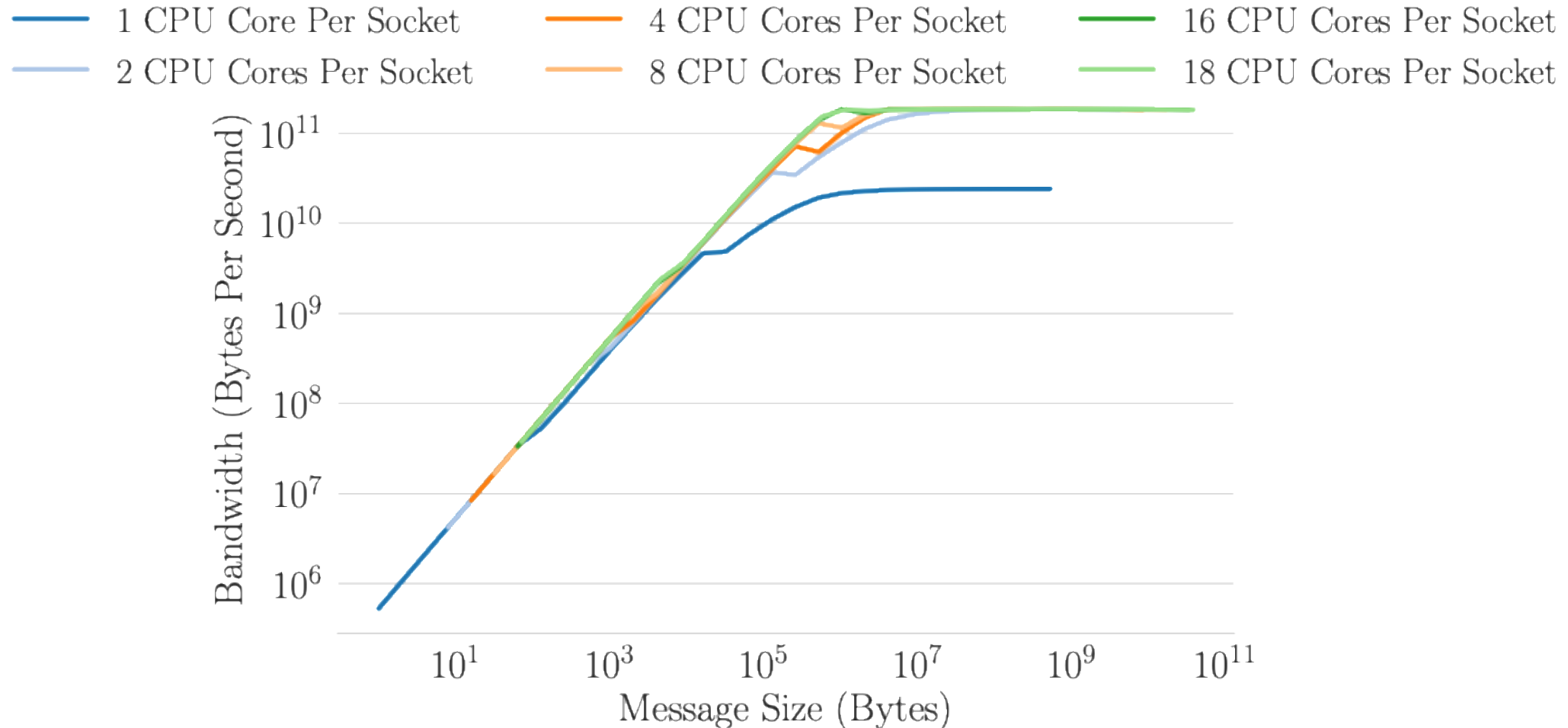


Measuring Bandwidth Limitations: Multi-Proc Ping-Pongs

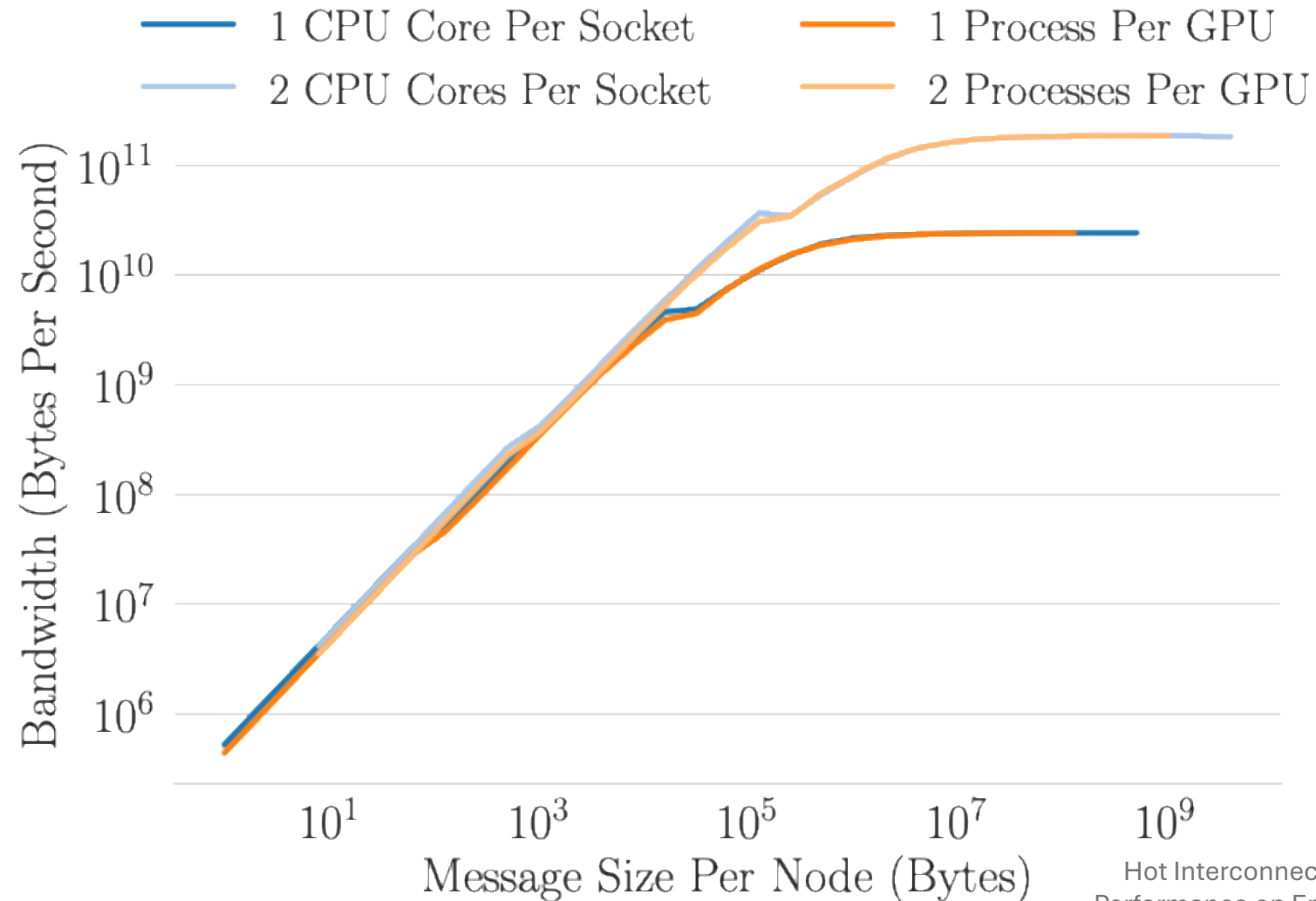
- Measures overheads and limits when multiple processes communication at same time
- Inter-node: measures injection bandwidth limits



Measuring Bandwidth Limitations: Inter-CPU Multi-Proc Ping-Pongs



Measuring Bandwidth Limitations: Inter-GPU Multi-Proc Ping-Pongs



GPU connectivity on AAC Cloud (MI210)

rocm-smi --showtopo

```
GPU[0] : (Topology) Numa Node: 0
GPU[0] : (Topology) Numa Affinity: 0
GPU[1] : (Topology) Numa Node: 0
GPU[1] : (Topology) Numa Affinity: 0
GPU[2] : (Topology) Numa Node: 0
GPU[2] : (Topology) Numa Affinity: 0
GPU[3] : (Topology) Numa Node: 0
GPU[3] : (Topology) Numa Affinity: 0
GPU[4] : (Topology) Numa Node: 1
GPU[4] : (Topology) Numa Affinity: 1
GPU[5] : (Topology) Numa Node: 1
GPU[5] : (Topology) Numa Affinity: 1
GPU[6] : (Topology) Numa Node: 1
GPU[6] : (Topology) Numa Affinity: 1
GPU[7] : (Topology) Numa Node: 1
GPU[7] : (Topology) Numa Affinity: 1
```

===== Link Type between two GPUs =====								
	GPU0	GPU1	GPU2	GPU3	GPU4	GPU5	GPU6	GPU7
GPU0	0	XGMI	XGMI	XGMI	PCIE	PCIE	PCIE	PCIE
GPU1	XGMI	0	XGMI	XGMI	PCIE	PCIE	PCIE	PCIE
GPU2	XGMI	XGMI	0	XGMI	PCIE	PCIE	PCIE	PCIE
GPU3	XGMI	XGMI	XGMI	0	PCIE	PCIE	PCIE	PCIE
GPU4	PCIE	PCIE	PCIE	PCIE	0	XGMI	XGMI	XGMI
GPU5	PCIE	PCIE	PCIE	PCIE	XGMI	0	XGMI	XGMI
GPU6	PCIE	PCIE	PCIE	PCIE	XGMI	XGMI	0	XGMI
GPU7	PCIE	PCIE	PCIE	PCIE	XGMI	XGMI	XGMI	0

===== Hops between two GPUs =====								
	GPU0	GPU1	GPU2	GPU3	GPU4	GPU5	GPU6	GPU7
GPU0	0	1	1	1	3	3	3	3
GPU1	1	0	1	1	3	3	3	3
GPU2	1	1	0	1	3	3	3	3
GPU3	1	1	1	0	3	3	3	3
GPU4	3	3	3	3	0	1	1	1
GPU5	3	3	3	3	1	0	1	1
GPU6	3	3	3	3	1	1	0	1
GPU7	3	3	3	3	1	1	1	0

Demo: Intra-node GPU-to-GPU Communication Bandwidth on AAC Cloud (MI210)

Use UCX for communication

```
$export HIP_VISIBLE_DEVICES=0,1
$mpirun -n 2 -mca pml ucx ../build/libexec/osu-micro-benchmarks/mpi/pt2pt/osu_bw -m $((16*1024*1024)):$((16*1024*1024)) D D
# OSU MPI-ROCM Bandwidth Test v7.2
# Send Buffer on DEVICE (D) and Receive Buffer on DEVICE (D)
# Size    Bandwidth (MB/s)
# Datatype: MPI_CHAR.
16777216    41454.31
```

GPU 0 & 1 → 41 GB/s

```
$export HIP_VISIBLE_DEVICES=0,4
$mpirun -n 2 -mca pml ucx ../build/libexec/osu-micro-benchmarks/mpi/pt2pt/osu_bw -m $((16*1024*1024)):$((16*1024*1024)) D D
# OSU MPI-ROCM Bandwidth Test v7.2
# Send Buffer on DEVICE (D) and Receive Buffer on DEVICE (D)
# Size    Bandwidth (MB/s)
# Datatype: MPI_CHAR.
16777216    25172.16
```

GPU 0 & 4 → 25 GB/s

Device to device communication

OSU Microbenchmark (OMB): Feature a series of MPI benchmarks that measure the performances of various MPI operations including point-to-point, collective, host-based and device-based communications

GPU-to-GPU Communication Options

There are two options for GPU-to-GPU communication

SDMA engine

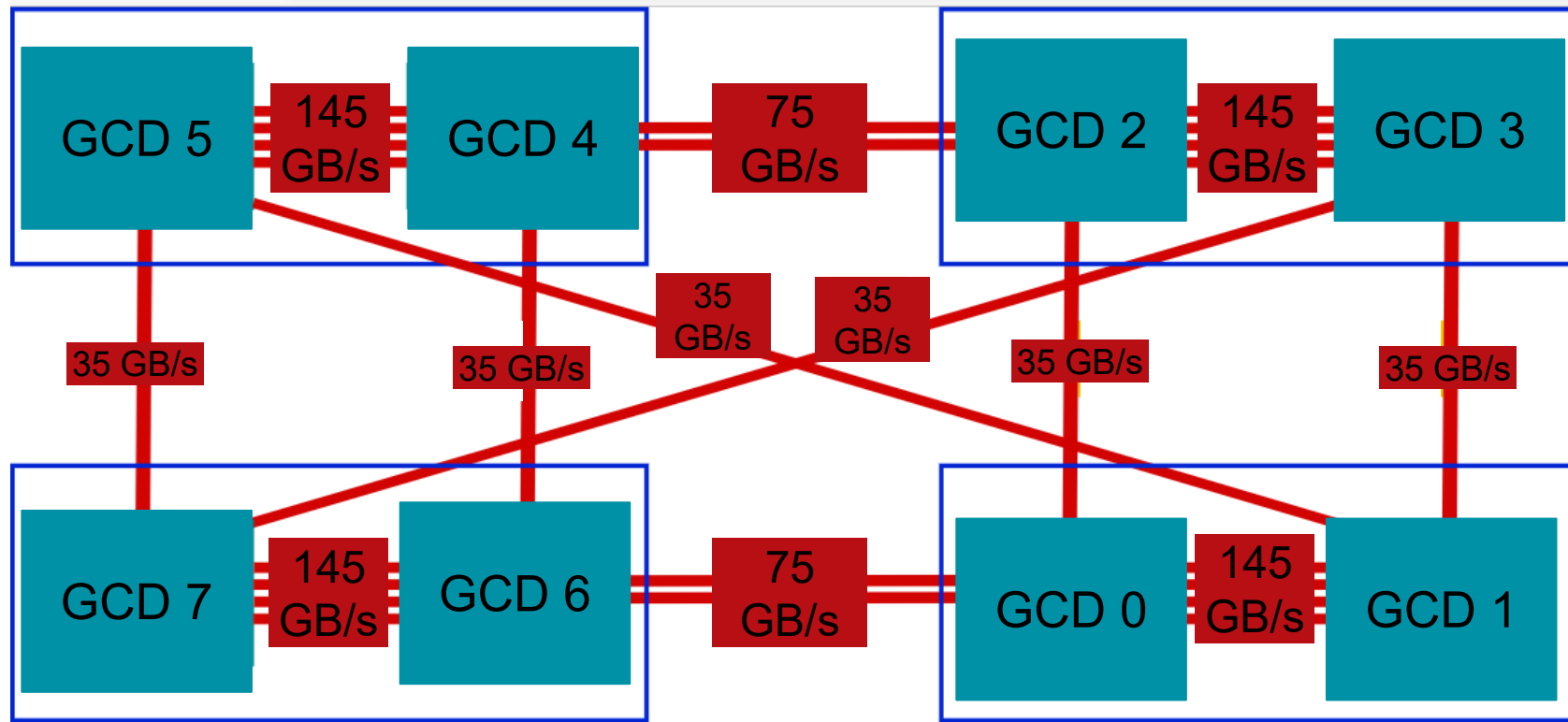
- Provides the opportunity to overlap communication with computation
- Each SDMA engine can provide maximum communication BW of 49 GB/s between GCDs

blit kernels

- Launch kernel to handle communication
- Pros: higher communication bandwidth
- Cons: cannot explicitly overlap communication with computation

GPU-to-GPU Communication Expectations

- Achievable GPU-to-GPU Communication Bandwidth using blit kernel
- Different number of Infinity Fabric™ links between GCDs
 - GCDs of the same GPU are connected with 4 Infinity Fabric™ links
- Different number of hops between GCDs



Aug 19-21, 2026

Hot Interconnects: Understanding Communication
Performance on Emerging Heterogeneous Architectures

²⁰ The numbers in this figure are calculated with (achievable bw ~ Theoretical bw * 0.7). Theoretical bw is obtained from https://docs.olcf.ornl.gov/systems/frontier_user_guide.html#frontier-compute-nodes

```

mghazimi@uan02:~/OMB/osu_benchmark> export HIP_VISIBLE_DEVICES=0,1
mghazimi@uan02:~/OMB/osu_benchmark> srun -N 1 -n 2 ./build/libexec/osu-micro-benchmarks/mpi/pt2pt/osu_bw -m $((16*1024*1024)):$((16*1024*1024)) D D
# OSU MPI-ROCM Bandwidth Test v7.0
# Send Buffer on DEVICE (D) and Receive Buffer on DEVICE (D)
# Size      Bandwidth (MB/s)
16777216    142341.39
mghazimi@uan02:~/OMB/osu_benchmark> export HIP_VISIBLE_DEVICES=0,2
mghazimi@uan02:~/OMB/osu_benchmark> srun -N 1 -n 2 ./build/libexec/osu-micro-benchmarks/mpi/pt2pt/osu_bw -m $((16*1024*1024)):$((16*1024*1024)) D D
# OSU MPI-ROCM Bandwidth Test v7.0
# Send Buffer on DEVICE (D) and Receive Buffer on DEVICE (D)
# Size      Bandwidth (MB/s)
16777216    38963.39
mghazimi@uan02:~/OMB/osu_benchmark> export HIP_VISIBLE_DEVICES=0,3
mghazimi@uan02:~/OMB/osu_benchmark> srun -N 1 -n 2 ./build/libexec/osu-micro-benchmarks/mpi/pt2pt/osu_bw -m $((16*1024*1024)):$((16*1024*1024)) D D
# OSU MPI-ROCM Bandwidth Test v7.0
# Send Buffer on DEVICE (D) and Receive Buffer on DEVICE (D)
# Size      Bandwidth (MB/s)
16777216    36903.69
mghazimi@uan02:~/OMB/osu_benchmark> export HIP_VISIBLE_DEVICES=0,4
mghazimi@uan02:~/OMB/osu_benchmark> srun -N 1 -n 2 ./build/libexec/osu-micro-benchmarks/mpi/pt2pt/osu_bw -m $((16*1024*1024)):$((16*1024*1024)) D D
# OSU MPI-ROCM Bandwidth Test v7.0
# Send Buffer on DEVICE (D) and Receive Buffer on DEVICE (D)
# Size      Bandwidth (MB/s)
16777216    36908.40
mghazimi@uan02:~/OMB/osu_benchmark> export HIP_VISIBLE_DEVICES=0,5
mghazimi@uan02:~/OMB/osu_benchmark> srun -N 1 -n 2 ./build/libexec/osu-micro-benchmarks/mpi/pt2pt/osu_bw -m $((16*1024*1024)):$((16*1024*1024)) D D
# OSU MPI-ROCM Bandwidth Test v7.0
# Send Buffer on DEVICE (D) and Receive Buffer on DEVICE (D)
# Size      Bandwidth (MB/s)
16777216    34986.18
mghazimi@uan02:~/OMB/osu_benchmark> export HIP_VISIBLE_DEVICES=0,6
mghazimi@uan02:~/OMB/osu_benchmark> srun -N 1 -n 2 ./build/libexec/osu-micro-benchmarks/mpi/pt2pt/osu_bw -m $((16*1024*1024)):$((16*1024*1024)) D D
# OSU MPI-ROCM Bandwidth Test v7.0
# Send Buffer on DEVICE (D) and Receive Buffer on DEVICE (D)
# Size      Bandwidth (MB/s)
16777216    76276.50
mghazimi@uan02:~/OMB/osu_benchmark> export HIP_VISIBLE_DEVICES=0,7
mghazimi@uan02:~/OMB/osu_benchmark> srun -N 1 -n 2 ./build/libexec/osu-micro-benchmarks/mpi/pt2pt/osu_bw -m $((16*1024*1024)):$((16*1024*1024)) D D
# OSU MPI-ROCM Bandwidth Test v7.0
# Send Buffer on DEVICE (D) and Receive Buffer on DEVICE (D)
# Size      Bandwidth (MB/s)
16777216    68778.59

```

GCD 0 & 1 → 142 GB/s

Device to device
communication

GCD 0 & 2 → 38 GB/s

GCD 0 & 3 → 36 GB/s

GCD 0 & 4 → 36 GB/s

GCD 0 & 5 → 34 GB/s

GCD 0 & 6 → 76 GB/s

GCD 0 & 7 → 68 GB/s

Demo: Intra-node GPU-to-GPU Communication Bandwidth on LUMI Using blit Kernels

\$module load rocm

\$module load cray-mpich/8.1.18

\$export MPICH_GPU_SUPPORT_ENABLED=1

\$export HSA_ENABLE_SDMA=0

Enable blit kernel

Aug 19-21, 2026

Hot Interconnects: Understanding Communication
Performance on Emerging Heterogeneous
Architectures

AMD
together we advance_

Demo: Intra-node GPU-to-GPU Communication Bandwidth on LUMI using SDMA

```
mghazimi@uan02:~/OMB/osu_benchmark> export HIP_VISIBLE_DEVICES=0,1
mghazimi@uan02:~/OMB/osu_benchmark> srun --jobid=2057636 -N 1 -n 2 ./build/libexec/osu-micro-benchmarks/mpi/pt2pt/osu_bw -m $((16*1024*1024)):$((16*1024*1024)) D D
# OSU MPI-ROCM Bandwidth Test v7.0
# Send Buffer on DEVICE (D) and Receive Buffer on DEVICE (D)
# Size      Bandwidth (MB/s)
16777216    49955.50
mghazimi@uan02:~/OMB/osu_benchmark> export HIP_VISIBLE_DEVICES=0,2
mghazimi@uan02:~/OMB/osu_benchmark> srun --jobid=2057636 -N 1 -n 2 ./build/libexec/osu-micro-benchmarks/mpi/pt2pt/osu_bw -m $((16*1024*1024)):$((16*1024*1024)) D D
# OSU MPI-ROCM Bandwidth Test v7.0
# Send Buffer on DEVICE (D) and Receive Buffer on DEVICE (D)
# Size      Bandwidth (MB/s)
16777216    36377.30
mghazimi@uan02:~/OMB/osu_benchmark> export HIP_VISIBLE_DEVICES=0,3
mghazimi@uan02:~/OMB/osu_benchmark> srun --jobid=2057636 -N 1 -n 2 ./build/libexec/osu-micro-benchmarks/mpi/pt2pt/osu_bw -m $((16*1024*1024)):$((16*1024*1024)) D D
# OSU MPI-ROCM Bandwidth Test v7.0
# Send Buffer on DEVICE (D) and Receive Buffer on DEVICE (D)
# Size      Bandwidth (MB/s)
16777216    36940.74
mghazimi@uan02:~/OMB/osu_benchmark> export HIP_VISIBLE_DEVICES=0,4
mghazimi@uan02:~/OMB/osu_benchmark> srun --jobid=2057636 -N 1 -n 2 ./build/libexec/osu-micro-benchmarks/mpi/pt2pt/osu_bw -m $((16*1024*1024)):$((16*1024*1024)) D D
# OSU MPI-ROCM Bandwidth Test v7.0
# Send Buffer on DEVICE (D) and Receive Buffer on DEVICE (D)
# Size      Bandwidth (MB/s)
16777216    36955.43
mghazimi@uan02:~/OMB/osu_benchmark> export HIP_VISIBLE_DEVICES=0,5
mghazimi@uan02:~/OMB/osu_benchmark> srun --jobid=2057636 -N 1 -n 2 ./build/libexec/osu-micro-benchmarks/mpi/pt2pt/osu_bw -m $((16*1024*1024)):$((16*1024*1024)) D D
# OSU MPI-ROCM Bandwidth Test v7.0
# Send Buffer on DEVICE (D) and Receive Buffer on DEVICE (D)
# Size      Bandwidth (MB/s)
16777216    36359.46
mghazimi@uan02:~/OMB/osu_benchmark> export HIP_VISIBLE_DEVICES=0,6
mghazimi@uan02:~/OMB/osu_benchmark> srun --jobid=2057636 -N 1 -n 2 ./build/libexec/osu-micro-benchmarks/mpi/pt2pt/osu_bw -m $((16*1024*1024)):$((16*1024*1024)) D D
# OSU MPI-ROCM Bandwidth Test v7.0
# Send Buffer on DEVICE (D) and Receive Buffer on DEVICE (D)
# Size      Bandwidth (MB/s)
16777216    49971.79
mghazimi@uan02:~/OMB/osu_benchmark> export HIP_VISIBLE_DEVICES=0,7
mghazimi@uan02:~/OMB/osu_benchmark> srun --jobid=2057636 -N 1 -n 2 ./build/libexec/osu-micro-benchmarks/mpi/pt2pt/osu_bw -m $((16*1024*1024)):$((16*1024*1024)) D D
# OSU MPI-ROCM Bandwidth Test v7.0
# Send Buffer on DEVICE (D) and Receive Buffer on DEVICE (D)
# Size      Bandwidth (MB/s)
16777216    49945.63
```

GCD 0 & 1 → 49 GB/s

GCD 0 & 2 → 36 GB/s

GCD 0 & 3 → 36 GB/s

GCD 0 & 4 → 36 GB/s

GCD 0 & 5 → 36 GB/s

GCD 0 & 6 → 49 GB/s

GCD 0 & 7 → 49 GB/s

\$module load rocm

\$module load cray-mpich/8.1.18

\$export
MPICH_GPU_SUPPORT_ENABLED=1

\$export HSA_ENABLE_SDMA=1

Enable SDMA

*\$export HSA_ENABLE_SDMA_GANG=1

Utilize all links to make SDMA transfers (like previous slide)...but will impact bidirectional traffic

Summary of the Achievable Bandwidth with blit kernel vs SDMA

- Achieve up to 49 GB/s using SDMA
- Achieve up to 142 GB/s using blit kernel
- The communication bandwidth between GCDs depends on
 - SDMA vs blit kernel
 - Number of Infinity Fabric™ links between GCDs
 - Number of hops between GCDs

Achieved Bandwidth on LUMI with blit kernel (GB/s)

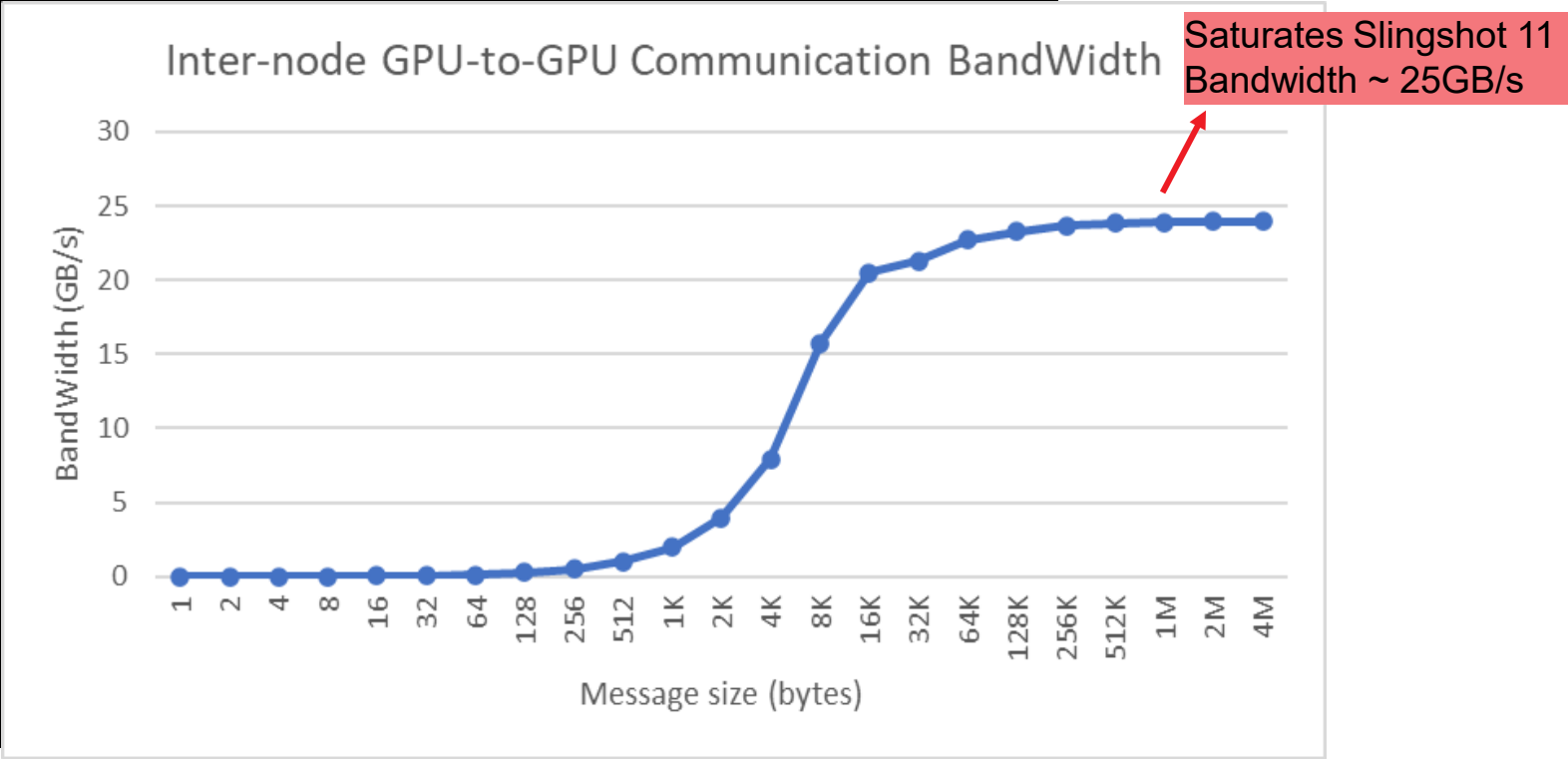
	GCD1	GCD2	GCD3	GCD4	GCD5	GCD6	GCD7
GCD0	142	38	36	36	34	76	68

Achieved Bandwidth on LUMI with SDMA (GB/s)

	GCD1	GCD2	GCD3	GCD4	GCD5	GCD6	GCD7
GCD0	49	36	36	36	34	49	49

Demo: Inter-node GPU-to-GPU Communication Bandwidth on LUMI

```
mghazimi@uan02:~/OMB/osu_benchmark> srun -N 2 -n 2 ./build/libexec/osu-micro-benchmarks/mpi/pt2pt/osu_bw D D
# OSU MPI-ROCM Bandwidth Test v7.0
# Send Buffer on DEVICE (D) and Receive Buffer on DEVICE (D)
# Size      Bandwidth (MB/s)
1           2.07
2           4.13
4           8.28
8           16.60
16          33.19
32          66.45
64          132.14
128         264.68
256         498.90
512         996.77
1024        1987.55
2048        3975.71
4096        7921.45
8192        15705.86
16384       20549.96
32768       21298.89
65536       22707.28
131072      23268.52
262144      23647.31
524288      23827.88
1048576     23903.00
2097152     23947.73
4194304     23968.83
```



Aug 19-21, 2026

Hot Interconnects: Understanding Communication
Performance on Emerging Heterogeneous Architectures

Demo: GPU-Aware Collective Communication

```
$srun -N 2 -n 8 --ntasks-per-node=4 ./build/libexec/osu-micro-benchmarks/mpi/collective/osu_allreduce -m 128 -d rocm
# OSU MPI-ROCM Allreduce Latency Test v7.0
# Size      Avg Latency(us)
4           5.23
8           5.22
16          5.23
32          5.22
64          5.26
128         5.57
```

4 ranks on node 0
4 ranks on node 1

```
srunk -N 1 -n 8 --ntasks-per-node=8 ./build/libexec/osu-micro-benchmarks/mpi/collective/osu_allreduce -m 128 -d rocm
# OSU MPI-ROCM Allreduce Latency Test v7.0
# Size      Avg Latency(us)
4           1.27
8           1.24
16          1.27
32          1.27
64          1.32
128         1.39
```

8 ranks on node 0

Aug 19-21, 2026

Hot Interconnects: Understanding Communication
Performance on Emerging Heterogeneous Architectures

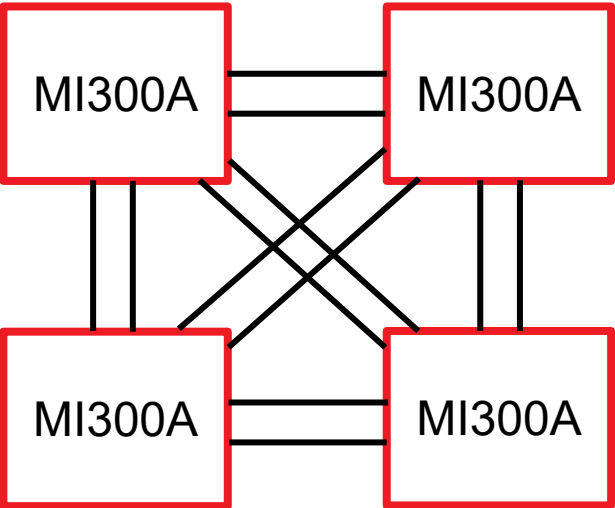
These tests have been executed as a demo (not performance claim) on LUMI pilot partition with MI250x GPUs, rocm 5.2.3 and cray-mpich/8.1.18 on May/03/2023

Network Topology on AAC Nodes with MI300A

rocm-smi --showtopo

```
===== Link Type between two GPUs =====
      GPU0      GPU1      GPU2      GPU3
GPU0    0        XGMI      XGMI      XGMI
GPU1  XGMI      0        XGMI      XGMI
GPU2  XGMI      XGMI      0        XGMI
GPU3  XGMI      XGMI      XGMI      0

===== Numa Nodes =====
GPU[0]      : (Topology) Numa Node: 0
GPU[0]      : (Topology) Numa Affinity: 0
GPU[1]      : (Topology) Numa Node: 1
GPU[1]      : (Topology) Numa Affinity: 1
GPU[2]      : (Topology) Numa Node: 2
GPU[2]      : (Topology) Numa Affinity: 2
GPU[3]      : (Topology) Numa Node: 3
GPU[3]      : (Topology) Numa Affinity: 3
```



Demo: Communication Example on MI300A System

```
$mpirun -n 2 --map-by package --mca pml ucx -x HIP_VISIBLE_DEVICES=1,2 ./install/libexec/osu-micro-benchmarks/mpi/pt2pt/osu_bw -m $((4194304*16)):$((4194304*16)) D D
```

```
# OSU MPI-ROCM Bandwidth Test v7.2
```

```
# Send Buffer on DEVICE (D) and Receive Buffer on DEVICE (D)
```

```
# Size    Bandwidth (MB/s)
```

```
# Datatype: MPI_CHAR.
```

```
67108864    90628.39
```

hipMalloc'ed buffer

```
$mpirun -n 2 --map-by package --mca pml ucx -x HIP_VISIBLE_DEVICES=1,2 ./install/libexec/osu-micro-benchmarks/mpi/pt2pt/osu_bw -m 4194304:4194304
```

```
# OSU MPI Bandwidth Test v7.2
```

```
# Size    Bandwidth (MB/s)
```

```
# Datatype: MPI_CHAR.
```

```
4194304    22986.99
```

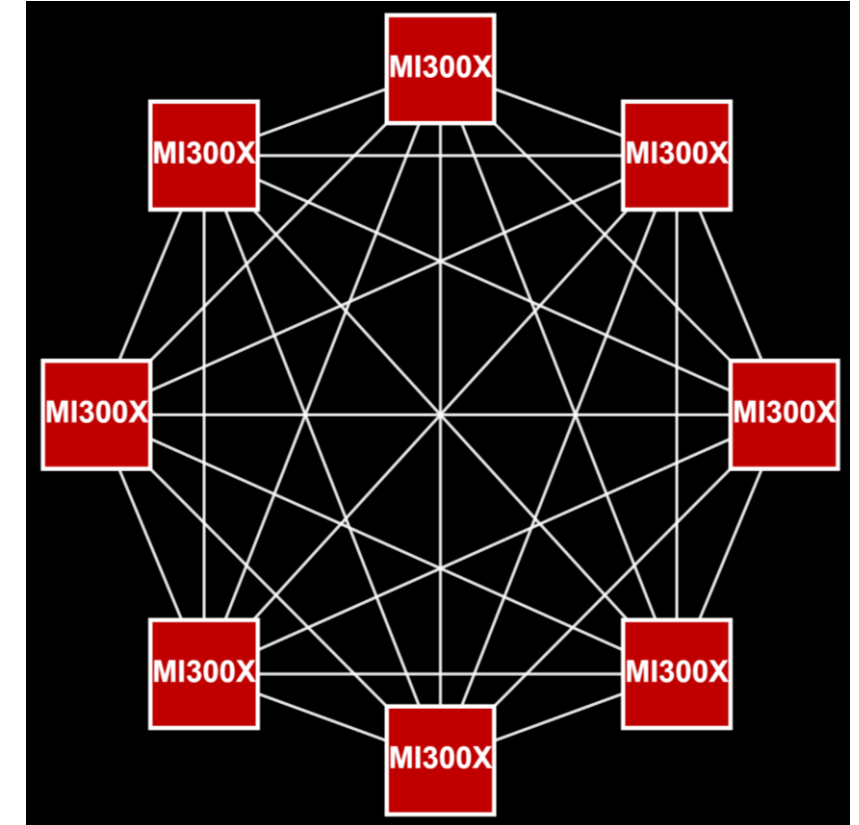
System allocated buffer e.g., malloc

Network Topology MI300X

rocm-smi --showtopo

```
===== Link Type between two GPUs =====
GPU0      GPU1      GPU2      GPU3      GPU4      GPU5      GPU6      GPU7
GPU0      0         XGMI      XGMI      XGMI      XGMI      XGMI      XGMI
GPU1      XGMI      0         XGMI      XGMI      XGMI      XGMI      XGMI
GPU2      XGMI      XGMI      0         XGMI      XGMI      XGMI      XGMI
GPU3      XGMI      XGMI      XGMI      0         XGMI      XGMI      XGMI
GPU4      XGMI      XGMI      XGMI      XGMI      0         XGMI      XGMI
GPU5      XGMI      XGMI      XGMI      XGMI      XGMI      0         XGMI
GPU6      XGMI      XGMI      XGMI      XGMI      XGMI      XGMI      0
GPU7      XGMI      XGMI      XGMI      XGMI      XGMI      XGMI      XGMI

===== Numa Nodes =====
GPU[0]      : (Topology) Numa Node: 0
GPU[0]      : (Topology) Numa Affinity: 0
GPU[1]      : (Topology) Numa Node: 0
GPU[1]      : (Topology) Numa Affinity: 0
GPU[2]      : (Topology) Numa Node: 0
GPU[2]      : (Topology) Numa Affinity: 0
GPU[3]      : (Topology) Numa Node: 0
GPU[3]      : (Topology) Numa Affinity: 0
GPU[4]      : (Topology) Numa Node: 1
GPU[4]      : (Topology) Numa Affinity: 1
GPU[5]      : (Topology) Numa Node: 1
GPU[5]      : (Topology) Numa Affinity: 1
GPU[6]      : (Topology) Numa Node: 1
GPU[6]      : (Topology) Numa Affinity: 1
GPU[7]      : (Topology) Numa Node: 1
GPU[7]      : (Topology) Numa Affinity: 1
===== End of ROCm SMI Log =====
```



Demo: Communication Example on MI300X System

```
$mpirun -n 2 --map-by package --mca pml ucx -x HIP_VISIBLE_DEVICES=1,2 ./install/libexec/osu-micro-benchmarks/mpi/pt2pt/osu_bw -m $((4194304*16)):$((4194304*16)) -d rocm D D
```

```
# OSU MPI-ROCM Bandwidth Test v7.2
# Send Buffer on DEVICE (D) and Receive Buffer on DEVICE (D)
# Size    Bandwidth (MB/s)
# Datatype: MPI_CHAR.
67108864    49082.78
```

hipMalloc'ed buffer – single XGMI link

```
$mpirun -n 2 --map-by package --mca pml ucx -x HIP_VISIBLE_DEVICES=1,2 ./install/libexec/osu-micro-benchmarks/mpi/pt2pt/osu_bw -m 4194304:4194304
```

```
# OSU MPI Bandwidth Test v7.2
# Size    Bandwidth (MB/s)
# Datatype: MPI_CHAR.
4194304    24183.57
```

System allocated buffer e.g., malloc

Demo: RCCL test on MI300A

`./build/all_reduce_perf -b 4M -e 128M -f 2 -g 4` → Run with 4 GPUs

Run for 4M to 128M messages

multiplication factor between sizes

```
[sg hazimi@d9dbb2d52d84:~/rccl/rccl-tests$ ./build/all_reduce_perf -b 4M -e 128M -f 2 -g 4
# nThread 1 nGpus 4 minBytes 4194304 maxBytes 134217728 step: 2(factor) warmup iters: 5 iters: 20 agg iters: 1 validation: 1 graph: 0
#
rccl-tests: Version develop:990f88c
# Using devices
# Rank 0 Pid 1004519 on d9dbb2d52d84 device 0 [0000:01:00.0] AMD Instinct MI300A
# Rank 1 Pid 1004519 on d9dbb2d52d84 device 1 [0001:01:00.0] AMD Instinct MI300A
# Rank 2 Pid 1004519 on d9dbb2d52d84 device 2 [0002:01:00.0] AMD Instinct MI300A
# Rank 3 Pid 1004519 on d9dbb2d52d84 device 3 [0003:01:00.0] AMD Instinct MI300A
#
#
# size count type redop root time out-of-place time in-place
# (B) (elements) (us) (GB/s) (GB/s) #wrong (us) (GB/s) (GB/s) #wrong
# 4194304 1048576 float sum -1 91.43 45.88 68.81 0 85.67 48.96 73.44 0
# 8388608 2097152 float sum -1 128.0 65.53 98.30 0 135.4 61.97 92.95 0
# 16777216 4194304 float sum -1 226.2 74.16 111.25 0 246.0 68.19 102.29 0
# 33554432 8388608 float sum -1 409.7 81.91 122.86 0 441.4 76.01 114.01 0
# 67108864 16777216 float sum -1 789.2 85.04 127.56 0 819.6 81.88 122.83 0
# 134217728 33554432 float sum -1 1567.0 85.65 128.48 0 1612.5 83.24 124.85 0
```

Demo: RCCL test on MI300X

`./build/all_reduce_perf -b 4M -e 128M -f 2 -g 8` → Run with 8 GPUs

Run for 4M to 128M messages

multiplication factor between sizes

```
rccl test: version: develop_20240417_1221
# Using devices
# Rank 0 Group 0 Pid 585049 on TheraC16 device 0 [0000:05:00] AMD Instinct MI300X
# Rank 1 Group 0 Pid 585049 on TheraC16 device 1 [0000:27:00] AMD Instinct MI300X
# Rank 2 Group 0 Pid 585049 on TheraC16 device 2 [0000:47:00] AMD Instinct MI300X
# Rank 3 Group 0 Pid 585049 on TheraC16 device 3 [0000:65:00] AMD Instinct MI300X
# Rank 4 Group 0 Pid 585049 on TheraC16 device 4 [0000:85:00] AMD Instinct MI300X
# Rank 5 Group 0 Pid 585049 on TheraC16 device 5 [0000:a6:00] AMD Instinct MI300X
# Rank 6 Group 0 Pid 585049 on TheraC16 device 6 [0000:c7:00] AMD Instinct MI300X
# Rank 7 Group 0 Pid 585049 on TheraC16 device 7 [0000:e5:00] AMD Instinct MI300X
#
#
#           size          count      type  redop  root      time      out-of-place      in-place
#           (B)      (elements)              (us)      algbw      busbw #wrong      time      algbw      busbw #wrong
#
#           4194304      1048576      float    sum    -1      158.1      26.52      46.42      0      136.0      30.85      53.98      0
#           8388608      2097152      float    sum    -1      181.5      46.21      80.86      0      184.8      45.39      79.43      0
#          16777216      4194304      float    sum    -1      232.6      72.13     126.23      0      237.9      70.53     123.43      0
#          33554432      8388608      float    sum    -1      330.8     101.42     177.49      0      341.4      98.28     171.99      0
#          67108864     16777216      float    sum    -1      515.5     130.17     227.80      0      524.7     127.90     223.82      0
#         134217728     33554432      float    sum    -1      890.1     150.78     263.87      0      914.2     146.82     256.94      0
```

Hands-On Exercises

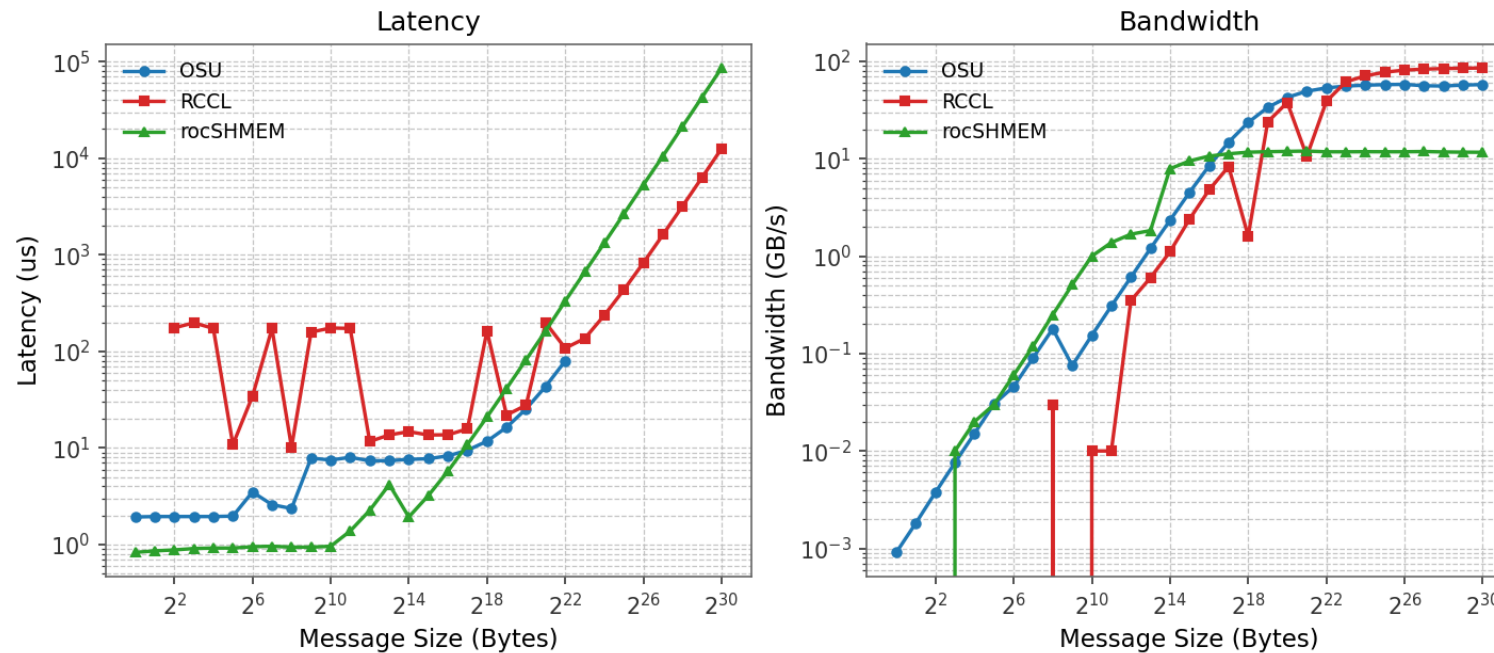
```
git clone https://github.com/Communication-Performance-Tutorial/micro-benchmarks.git

cd micro-benchmarks/osu && bash build.sh

cd ../micro-benchmarks/rccl-tests && bash build.sh

cd ../micro-benchmarks/rocshmem && bash build.sh
```

OSU vs. RCCL vs. rocSHMEM: Latency & Bandwidth (INTER_SOCKET)



Hands-On OSU

OSU Micro-Benchmarks — Point-to-Point (GPU and CPU buffers)

The [OSU Micro-Benchmarks \(OMB\)](#) are the standard tool for measuring MPI communication performance. This directory builds the OMB point-to-point tests and runs them two ways: with **GPU (device) buffers** (`run.sh`), the same path taken by the CG solver in `CG-Tutorial/CG-GPU/`, and with **CPU (host) buffers** (`run_host.sh`), a pure host-memory MPI baseline with no GPU involvement at all.

Test	What it measures
<code>osu_latency D D / H H</code>	Round-trip latency for a single message between two GPU (or host) buffers
<code>osu_bw D D / H H</code>	Unidirectional bandwidth from one GPU (or host) buffer to another

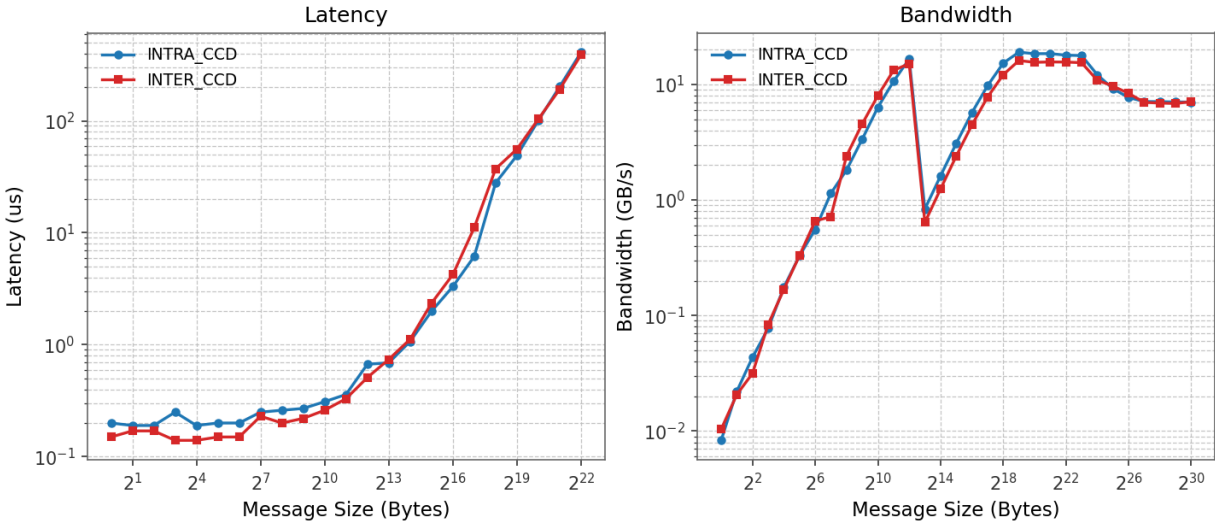
The `D D` flag tells OMB to allocate both the send and receive buffers in GPU device memory, exercising the GPU-Aware MPI path end to end. `H H` allocates both buffers in plain host memory with `malloc()`, exercising ordinary CPU-to-CPU MPI — RCCL (GPU-only collectives) and rocSHMEM (GPU symmetric-heap PGAS model) have no equivalent host-buffer mode, so this is the only host-memory communication test in the suite.

File layout

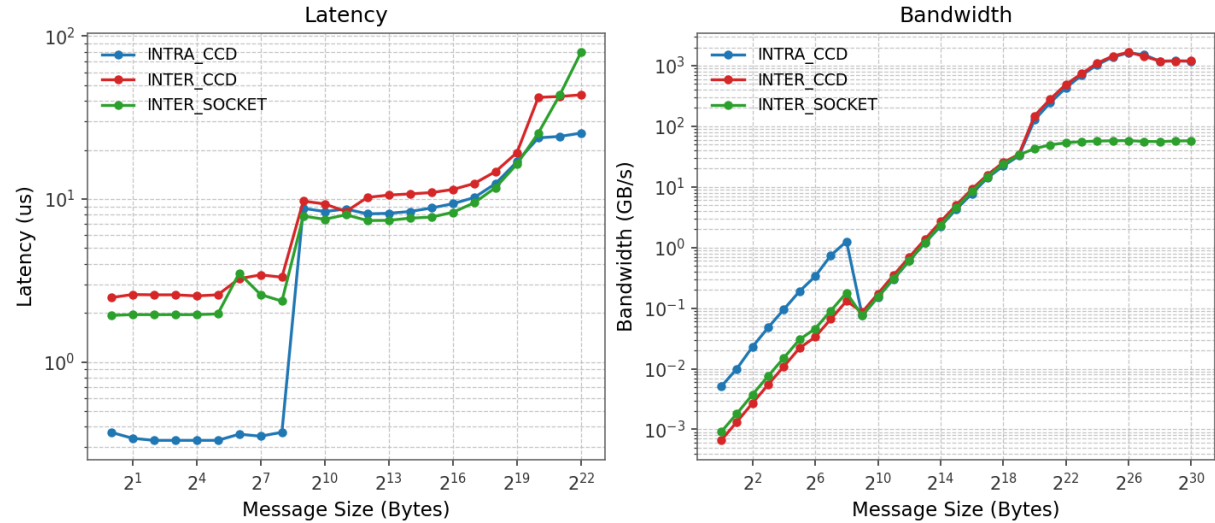
```
osu/  
├── build.sh  
├── micro-benchmarks/└── osu  
├── run.sh  
├── run_host.sh  
├── sbatch_host_test.sh  
├── sbatch_cpx_test.sh  
└── sbatch_test.sh
```

```
cd micro-benchmarks/osu && bash build.sh  
  
sbatch sbatch_host_test.sh  
  
sbatch sbatch_cpx_test.sh  
  
sbatch sbatch_test.sh
```

OSU Point-to-Point: CPU Affinity Sweep (CPU buffers, H H)



OSU Point-to-Point: CCD Affinity Sweep (GPU buffers, D D)



Hands-On rccl-tests

RCCL Tests — Collective Communication Benchmarks

[RCCL](#) (ROCm Collective Communications Library) is AMD's GPU-optimized implementation of the NCCL API. It provides collective operations (all-reduce, broadcast, send/recv, etc.) that exploit Infinity Fabric / xGMI links between GPU dies for transfers that bypass the CPU entirely.

This directory builds and runs two tests:

Test	What it measures
sendrecv_perf	Bidirectional point-to-point bandwidth and latency (2 GPUs)
all_reduce_perf	All-reduce throughput across all available GPUs

Together these answer: *how fast can RCCL move data between GPUs*, complementing the MPI-based measurements in `osu/`.

File layout

```
rccl-tests/
├─ build.sh           ← sparse-checkouts rccl-tests from ROCm/rocm-systems, patches, builds
├─ run.sh             ← runs sendrecv_perf (INTER_SOCKET only) + all_reduce_perf
├─ sbatch_test.sh     ← SLURM job: build once if needed, then run
└─ ./set_affinity_mi300a.sh ← shared per-rank CPU/GPU affinity wrapper
```

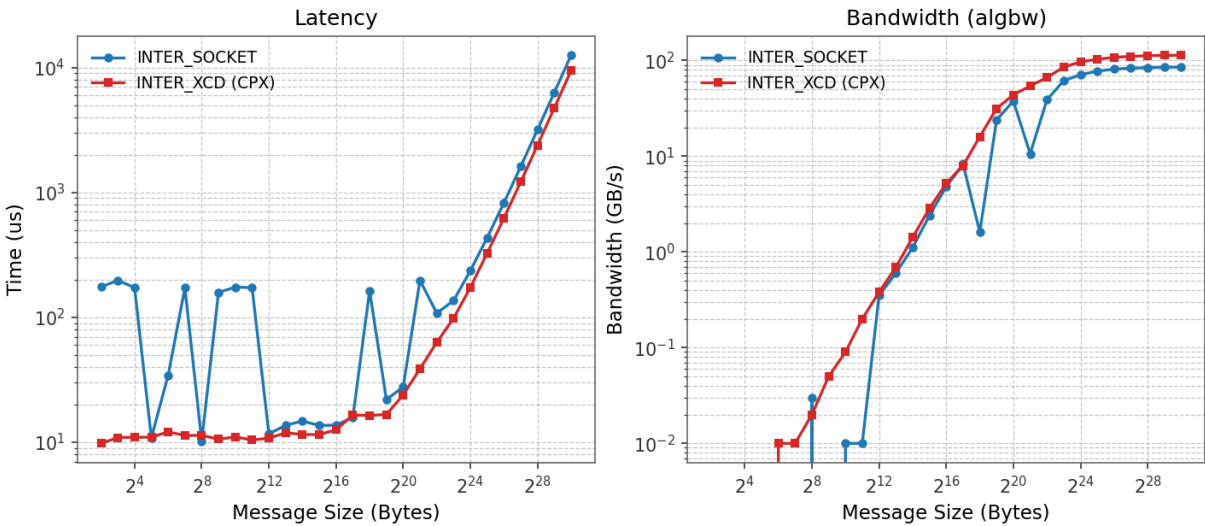
After a successful build, binaries live at:

```
rccl-tests/
└─ rocm-systems/
```

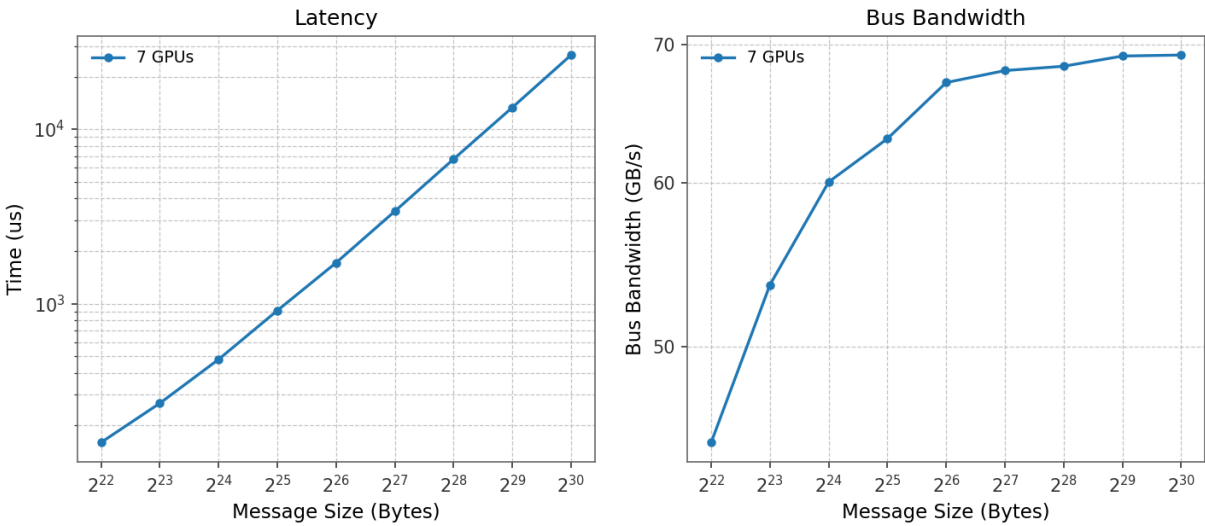
```
cd micro-benchmarks/rccl-tests && bash build.sh

sbatch sbatch_test.sh
```

RCCL sendrecv_perf: Latency & Bandwidth



RCCL all_reduce_perf: Latency & Bus Bandwidth



Hands-On rocSHMEM tests

rocSHMEM — One-Sided GPU Communication Benchmarks

[rocSHMEM](#) is AMD's implementation of the OpenSHMEM PGAS (Partitioned Global Address Space) model for GPUs. Instead of explicit send/receive pairs, rocSHMEM lets a GPU kernel *put* data directly into another GPU's memory or *get* data from it — the remote GPU never needs to call a matching receive.

This directory benchmarks three key operations using the **IPC backend**, which transfers data via ROCm IPC handles and delivers sub-microsecond latency for intra-node communication.

Test	What it measures
Ping-Pong (test ID 12)	One-sided put round-trip latency: 1 WG × 1 thread
WG Put (test ID 26)	Blocking put bandwidth, 1 WG × 64 threads, quiet included in timing
WG Get (test ID 24)	Blocking get bandwidth, 1 WG × 64 threads, quiet included in timing

Tests 26 and 24 use a single work-group (single context) to give a **single-stream point-to-point** measurement directly comparable to OSU's `bw` test and RCCL's `sendrecv_perf`. Both include `rocshmem_ctx_quiet` in the timing window, so the timer stops only after all in-flight data is committed — matching OSU's completion semantics.

File layout

```
rocshmem/
├── build.sh      + sparse-checkouts rocshmem from ROCm/roc-systems, builds (IPC backend)
├── run.sh        + runs ping-pong latency + flood put + WG get tests, 3 affinity modes
└── sbatch_test.sh + SLURM job: build once if needed, then run run.sh
```

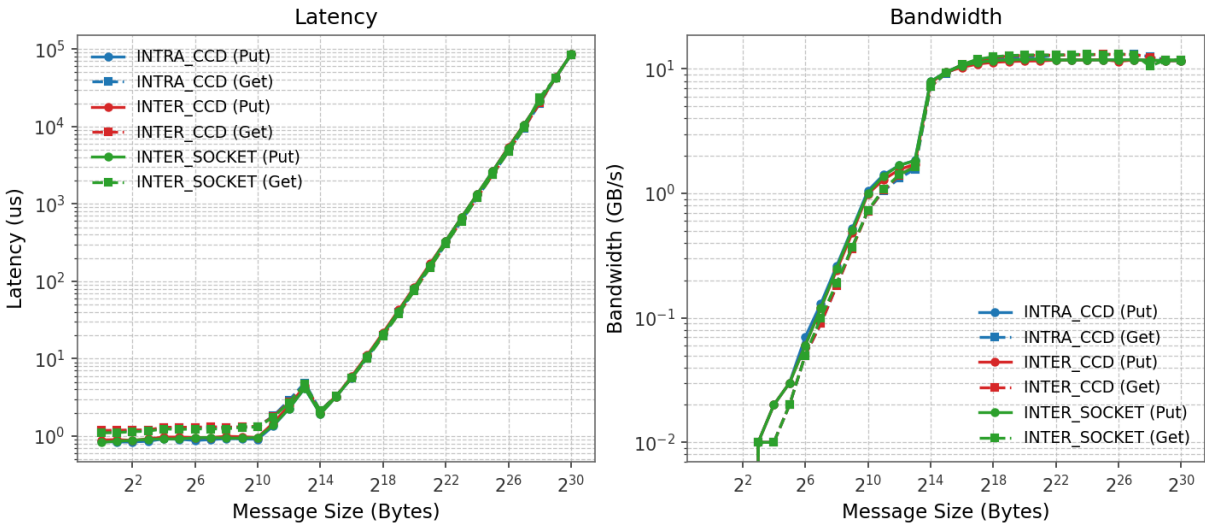
After a successful build, the installed binaries live at:

```
~/rocshmem/share/rocshmem/
├── rocshmem_functional_tests + used by run.sh
└── rocshmem_unit_tests
```

```
cd micro-benchmarks/rocshmem && bash build.sh

sbatch sbatch_test.sh
```

rocSHMEM Put/Get: Latency & Bandwidth (IPC backend)



Disclaimer

The information presented in this document is for informational purposes only and may contain technical inaccuracies, omissions, and typographical errors. The information contained herein is subject to change and may be rendered inaccurate for many reasons, including but not limited to product and roadmap changes, component and motherboard version changes, new model and/or product releases, product differences between differing manufacturers, software changes, BIOS flashes, firmware upgrades, or the like. Any computer system has risks of security vulnerabilities that cannot be completely prevented or mitigated. AMD assumes no obligation to update or otherwise correct or revise this information. However, AMD reserves the right to revise this information and to make changes from time to time to the content hereof without obligation of AMD to notify any person of such revisions or changes.

THIS INFORMATION IS PROVIDED 'AS IS.' AMD MAKES NO REPRESENTATIONS OR WARRANTIES WITH RESPECT TO THE CONTENTS HEREOF AND ASSUMES NO RESPONSIBILITY FOR ANY INACCURACIES, ERRORS, OR OMISSIONS THAT MAY APPEAR IN THIS INFORMATION. AMD SPECIFICALLY DISCLAIMS ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, OR FITNESS FOR ANY PARTICULAR PURPOSE. IN NO EVENT WILL AMD BE LIABLE TO ANY PERSON FOR ANY RELIANCE, DIRECT, INDIRECT, SPECIAL, OR OTHER CONSEQUENTIAL DAMAGES ARISING FROM THE USE OF ANY INFORMATION CONTAINED HEREIN, EVEN IF AMD IS EXPRESSLY ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Third-party content is licensed to you directly by the third party that owns the content and is not licensed to you by AMD. ALL LINKED THIRD-PARTY CONTENT IS PROVIDED "AS IS" WITHOUT A WARRANTY OF ANY KIND. USE OF SUCH THIRD-PARTY CONTENT IS DONE AT YOUR SOLE DISCRETION AND UNDER NO CIRCUMSTANCES WILL AMD BE LIABLE TO YOU FOR ANY THIRD-PARTY CONTENT. YOU ASSUME ALL RISK AND ARE SOLELY RESPONSIBLE FOR ANY DAMAGES THAT MAY ARISE FROM YOUR USE OF THIRD-PARTY CONTENT.

© 2026 Advanced Micro Devices, Inc. All rights reserved. AMD, the AMD Arrow logo, ROCm, Infinity Fabric, and combinations thereof are trademarks of Advanced Micro Devices, Inc. in the United States and/or other jurisdictions. Other names are for informational purposes only and may be trademarks of their respective owners.

Git and the Git logo are either registered trademarks or trademarks of Software Freedom Conservancy, Inc., corporate home of the Git Project, in the United States and/or other countries

HPE is a registered trademark of Hewlett Packard Enterprise Company and/or its affiliates.





TAU Performance System[®]

- Prof. Sameer Shende
- University of Oregon

- Hot Interconnects: Understanding Communication Performance on Emerging Heterogeneous Architectures
 - Aug 19-21, 2026



Aug 19-21, 2026

Hot Interconnects: Understanding Communication
Performance on Emerging Heterogeneous Architectures



Using TAU on AAC6

```
module load rocm openmpi pytorch tau
export TAU_PROFILE_FORMAT=merged # creates a single tauprofile.xml
wget https://tau.uoregon.edu/workshop.tgz; tar xf workshop.tgz; cat README
```

```
mpirun -np 4 ./a.out # Uninstrumented
```

```
mpirun -np 4 tau_exec -T ompt -rocm -ebs ./a.out
mpirun -np 4 tau_exec -T ompt -rocm_pc -ebs ./a.out. # For PC sampling on GPU and event-based sampling on CPU
export TAU_METRICS=ROCM_SQ_WAVES # see rocprofv3 -L for a complete list of counters
mpirun -np 4 tau_exec -T ompt -rocm -ebs ./a.out
```

```
paraprof tauprofile.xml
```

```
paraprof --dump tauprofile.xml
```

```
pprof -a | more
```

```
paraprof --pack foo.ppk
```

```
On your desktop: % paraprof foo.ppk
```

TAU Performance System®

- Instrumentation
 - Fortran, C++, C, UPC, Java®, Python™, Chapel, Spark
 - Automatic instrumentation
- Measurement and analysis support
 - MPI (MVAPICH2, gnu MPI), OpenSHMEM, ARMCI, PGAS, DMAPP
 - Supports gnu oneAPI compilers
 - pthreads, OpenMP®, OMPT interface, hybrid, other thread models
 - GPU: ROCm, OpenCL™, OpenACC, Kokkos, RAJA
 - Parallel profiling and tracing
- Analysis
 - Parallel profile analysis (ParaProf), data mining (PerfExplorer)
 - Performance database technology (TAUdb)
 - 3D profile browser

Instrumentation

Add hooks in the code to perform measurements

- **Source instrumentation using a preprocessor**
 - Add timer start/stop calls in a copy of the source code.
 - Use Program Database Toolkit (PDT) for parsing source code.
 - Requires recompiling the code using TAU shell scripts (tau_cc.sh, tau_f90.sh)
 - Selective instrumentation (filter file) can reduce runtime overhead and narrow instrumentation focus.
- **Compiler-based instrumentation**
 - Use system compiler to add a special flag to insert hooks at routine entry/exit.
 - Requires recompiling using TAU compiler scripts (tau_cc.sh, tau_f90.sh...)
- **Runtime preloading of TAU's Dynamic Shared Object (DSO)**
 - No need to recompile code! Use **mpiexec tau_exec ./app** with options.
 - It will preload TAU's DSO and intercept MPI calls and generate profile/trace data.

Using TAU for source instrumentation

- TAU supports several measurement and thread options

Phase profiling, profiling with hardware counters, MPI library, Python™...

Each measurement configuration of TAU corresponds to a unique stub makefile and library that is generated when you configure it

- To instrument source code automatically using PDT

Choose an appropriate TAU stub makefile in <arch>/lib:

```
% module load rocm openmpi pytorch tau
```

```
•% export TAU_MAKEFILE=/nfsapps/ubuntu-24.04/opt/rocmplus-7.2.4/tau/x86_64/lib/Makefile.tau-rocm-hip-rocprofsdk-rocmv6-clang-ompt-mpi-pthread-pdt-openmp
```

```
% export TAU_OPTIONS='-optVerbose ...' (see tau_compiler.sh )
```

```
% export PATH=$TAUDIR/x86_64/bin:$PATH
```

Use tau_f90.sh, tau_cxx.sh, tau_upc.sh, or tau_cc.sh as F90, C++, UPC, or C compilers respectively:

```
% mpif90 foo.f90          changes to
```

```
% tau_f90.sh foo.f90
```

- Set runtime environment variables, execute application and analyze performance data:

```
% pprof (for text based profile display)
```

```
% paraprof (for GUI)
```

TAU's Support for Runtime Systems

- *MPI*
 - PMPI profiling interface
 - MPI_T tools interface using performance and control variables
- *Pthread*
 - Captures time spent in routines per thread of execution
- *OpenMP®*
 - OMPT tools interface to track salient OpenMP® runtime events
 - Opari source rewriter
 - Preloading wrapper OpenMP® runtime library when OMPT is not supported
- *OpenACC*
 - OpenACC instrumentation API
 - Track data transfers between host and device (per-variable)
 - Track time spent in kernels

TAU's Support for Runtime Systems (contd.)

- *OpenCL™*
 - OpenCL™ profiling interface
 - Track timings of kernels
- *HIP (ROCm)*
 - Track time spent in kernels executing on GPU
 - Track time spent in HIP runtime and RCCL calls
- *Kokkos*
 - Kokkos profiling API
 - Push/pop interface for region, kernel execution interface
- *Python*
 - Python™ interpreter instrumentation API
 - Tracks Python routine transitions as well as Python to C transitions

Examples of Multi-Level Instrumentation

- *MPI + OpenMP®*
 - MPI_T + PMPI + OMPT may be used to track MPI and OpenMP®
- *MPI + pthread*
 - PMPI + pthread interfaces
- *MPI + HIP*
 - PMPI + ROCProfiler-SDK
- *OpenCL™ + Python™*
 - OpenCL™ + Python instrumentation interfaces
- *Kokkos + OpenMP*
 - Kokkos profiling API + OMPT to transparently track events
- *Kokkos + pthread + MPI*
 - Kokkos + pthread wrapper interposition library + PMPI layer
- *MPI + OpenCL™*
 - PMPI + OpenCL™ profiling interfaces

Binary instrumentation of libraries: Work in progress

- `% tau_run a.out -o a.inst`
instruments a binary. Other flags `-T <tags>`, `-f <selective instrumentation file>`
- `% tau_run -l /path/to/libhdf5.so.310 -o libhdf5.so.310`
instruments a DSO
- `% tau_exec ./a.out`
executes the uninstrumented application with the instrumented shared object.
- To use with DyninstAPI 13 on x86_64:
 - 1. Load spack: `source spack/share/spack/setup-env.sh`
 - 2. Install dyninst: `spack install dyninst@13 %gcc@11`
 - 3. Configure tau with dyninst:
 - 3.1 `spack find -p dyninst boost tbb elfutils`
 - 3.2 Copy the paths for each package into the configure line
 - 3.3 `./configure -bfd=download -dyninst=<dir> -tbb=<dir> -boost=<dir> -elf=<dir>; <set paths>; make install`

Binary instrumentation of libraries: HDF5

```
$ pprof
Reading Profile files in profile.*
```

```
NODE 0;CONTEXT 0;THREAD 0:
```

%Time	Exclusive msec	Inclusive total msec	#Call	#Subrs	Inclusive Name usec/call
100.0	0.272	68	1	1	68245 .TAU application
99.6	1	67	1	26	67973 taupreload_main
65.8	0.008	44	6	1	7484 H5open
65.8	6	44	2	14	22448 H5_init_library
36.0	4	24	1	12	24563 H5VL_init_phase2
27.8	1	18	1	319	18943 H5T_init
19.8	0.193	13	179	179	76 H5T__register_int
19.5	0.302	13	179	310	74 H5T__register
19.0	4	12	155	2555	84 H5T__path_find_real
13.0	2	8	1	79	8857 H5P_init_phase1
12.7	0.663	8	2	51	4349 H5F_open
11.2	0.348	7	1	6	7610 H5Fcreate
10.5	0.386	7	1	6	7138 H5F__create_api_common
9.8	0.406	6	1	2	6707 H5VL_file_create
9.2	0.005	6	1	1	6299 H5VL__native_file_create
7.1	1	4	488	976	10 H5T_copy
6.5	1	4	1	363	4452 H5E_init
5.6	0.013	3	4	12	956 H5I_dec_app_ref
5.6	0.013	3	2	10	1896 H5Fclose
5.5	0.009	3	2	4	1878 H5F__close_cb
5.5	0.01	3	2	6	1868 H5VL_file_close
5.4	0.013	3	2	4	1852 H5VL__native_file_close
5.4	0.019	3	4	8	924 H5F_try_close.localalias

Using TAU's Runtime Preloading Tool: `tau_exec`

- Preload a wrapper that intercepts the runtime system call and substitutes with another
 - MPI
 - OpenMP®
 - POSIX I/O
 - Memory allocation/deallocation routines
 - Wrapper library for an external package
- No modification to the binary executable!
- Enable other TAU options (communication matrix, OTF2, event-based sampling)

TAU Execution Command (tau_exec)

- Uninstrumented execution

- % mpirun -np 256 ./a.out

- Track GPU operations

- % mpirun -np 256 tau_exec -rocm ./a.out (-rocm_pc for PC sampling on GPU)
- % mpirun -np 256 tau_exec -cupti ./a.out
- % mpirun -np 256 tau_exec -cupti -um ./a.out (for Unified Memory) (-cupti_pc for PC sampling on GPU)
- % mpirun -np 256 tau_exec -l0 ./a.out
- % mpirun -np 256 tau_exec -opencl ./a.out
- % mpirun -np 256 tau_exec -openacc ./a.out

- Track MPI performance

- % mpirun -np 256 tau_exec ./a.out

- Track I/O, and MPI performance (MPI enabled by default)

- % mpirun -np 256 tau_exec -io ./a.out
- Track OpenMP® and MPI execution (using OMPT for gnu v19+ or Clang 8+)
 - % export TAU_OMPT_SUPPORT_LEVEL=full;
 - % mpirun -np 256 tau_exec -T ompt,mpi -ompt ./a.out

- Track memory operations

- % export TAU_TRACK_MEMORY_LEAKS=1
- % mpirun -np 256 tau_exec -memory_debug ./a.out (bounds check)

- Use event based sampling (compile with -g)

- % mpirun -np 256 tau_exec -ebs ./a.out
- Also export TAU_METRICS=TIME,PAPI_L1_DCM... -ebs_resolution=<file | function | line>

TAU Configurations

```
% ls /nfsapps/ubuntu-24.04/opt/rocmplus-7.2.4/tau/x86_64/lib/Makefile*
/nfsapps/ubuntu-24.04/opt/rocmplus-7.2.4/tau/x86_64/lib/Makefile.tau-rocm-hip-rocprofsdk-rocmv6-clang-mpi-pthread
/nfsapps/ubuntu-24.04/opt/rocmplus-7.2.4/tau/x86_64/lib/Makefile.tau-rocm-hip-rocprofsdk-rocmv6-clang-mpi-pthread-pdt
/nfsapps/ubuntu-24.04/opt/rocmplus-7.2.4/tau/x86_64/lib/Makefile.tau-rocm-hip-rocprofsdk-rocmv6-clang-ompt-mpi-pthread-openmp
/nfsapps/ubuntu-24.04/opt/rocmplus-7.2.4/tau/x86_64/lib/Makefile.tau-rocm-hip-rocprofsdk-rocmv6-clang-ompt-mpi-pthread-pdt-openmp
/nfsapps/ubuntu-24.04/opt/rocmplus-7.2.4/tau/x86_64/lib/Makefile.tau-rocm-hip-rocprofsdk-rocmv6-clang-ompt-pthread-openmp
/nfsapps/ubuntu-24.04/opt/rocmplus-7.2.4/tau/x86_64/lib/Makefile.tau-rocm-hip-rocprofsdk-rocmv6-clang-ompt-pthread-pdt-openmp
/nfsapps/ubuntu-24.04/opt/rocmplus-7.2.4/tau/x86_64/lib/Makefile.tau-rocm-hip-rocprofsdk-rocmv6-clang-pthread
/nfsapps/ubuntu-24.04/opt/rocmplus-7.2.4/tau/x86_64/lib/Makefile.tau-rocm-hip-rocprofsdk-rocmv6-clang-pthread-pdt
```

For an uninstrumented binary:

```
% salloc -N 1
% mpirun -np 16 tau_exec -T ompt,pdt,mpi
```

Picks the configuration represented by

```
/nfsapps/ubuntu-24.04/opt/rocmplus-7.2.4/tau/x86_64/lib/Makefile.tau-rocm-hip-rocprofsdk-rocmv6-clang-ompt-mpi-pthread-pdt-openmp
```

To use OpenMP® instrumentation:

```
% export TAU_OMPT_SUPPORT_LEVEL=full
% export OMP_NUM_THREADS=<N>
% srun -n 16 tau_exec -T ompt,mpi -ompt -ebs ./a.out
```

To use TAU's source code instrumentation:

```
% export TAU_MAKEFILE=/nfsapps/ubuntu-24.04/opt/rocmplus-7.2.4/tau/x86_64/lib/Makefile.tau-rocm-hip-rocprofsdk-rocmv6-clang-ompt-mpi-pthread-pdt-openmp
% make CC=tau_cc.sh F90=tau_f90.sh CXX=tau_cxx.sh
% % export TAU_PROFILE_FORMAT=merged; mpirun -np 16 ./a.out ; paraprof tauprofile.xml;
% paraprof --dump tauprofile.xml; pprof -a | more
% paraprof --pack foo.ppk tauprofile.xml
```

Tags in tau_exec and other tools

```
% cd $TAU; ls Makefile.*  
Makefile.tau-gnu-papi-mpi-pdt
```

```
% mpirun -np 4 ./matrix
```

```
•% tau_exec -T gnu,mpi,pdt ./a.out
```

•

Chooses Makefile.tau-gnu-mpi,pdt and associated libraries.

```
•% tau_exec -T serial,pdt ./a.out
```

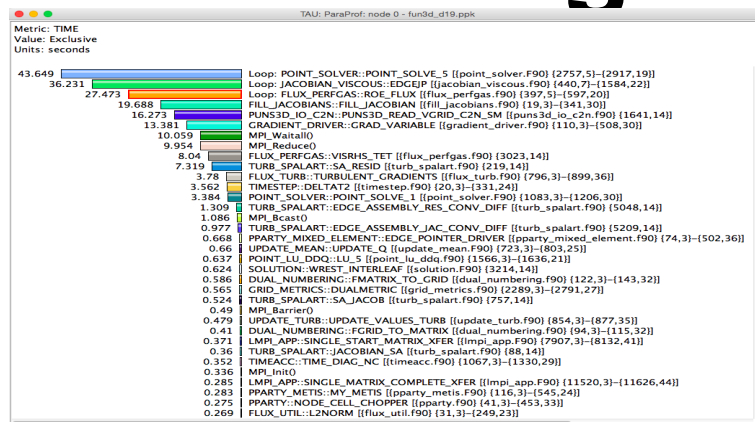
•Chooses Makefile.tau-pdt or the shortest Makefile name without -mpi.

•-T <list_of_tags> is used in several TAU tools:

- tau_run
- tau_python
- tau_rewrite
- tau_exec
- tau_gen_wrapper

Profiling and Tracing

Profiling

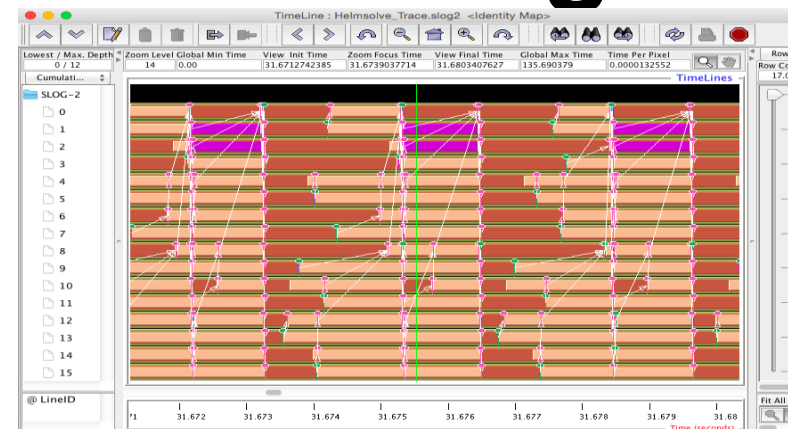


- **Profiling** shows you **how much** (total) time was spent in each routine
- Profiling and tracing

Profiling shows you **how much** (total) time was spent in each routine

Tracing shows you **when** the events take place on a timeline

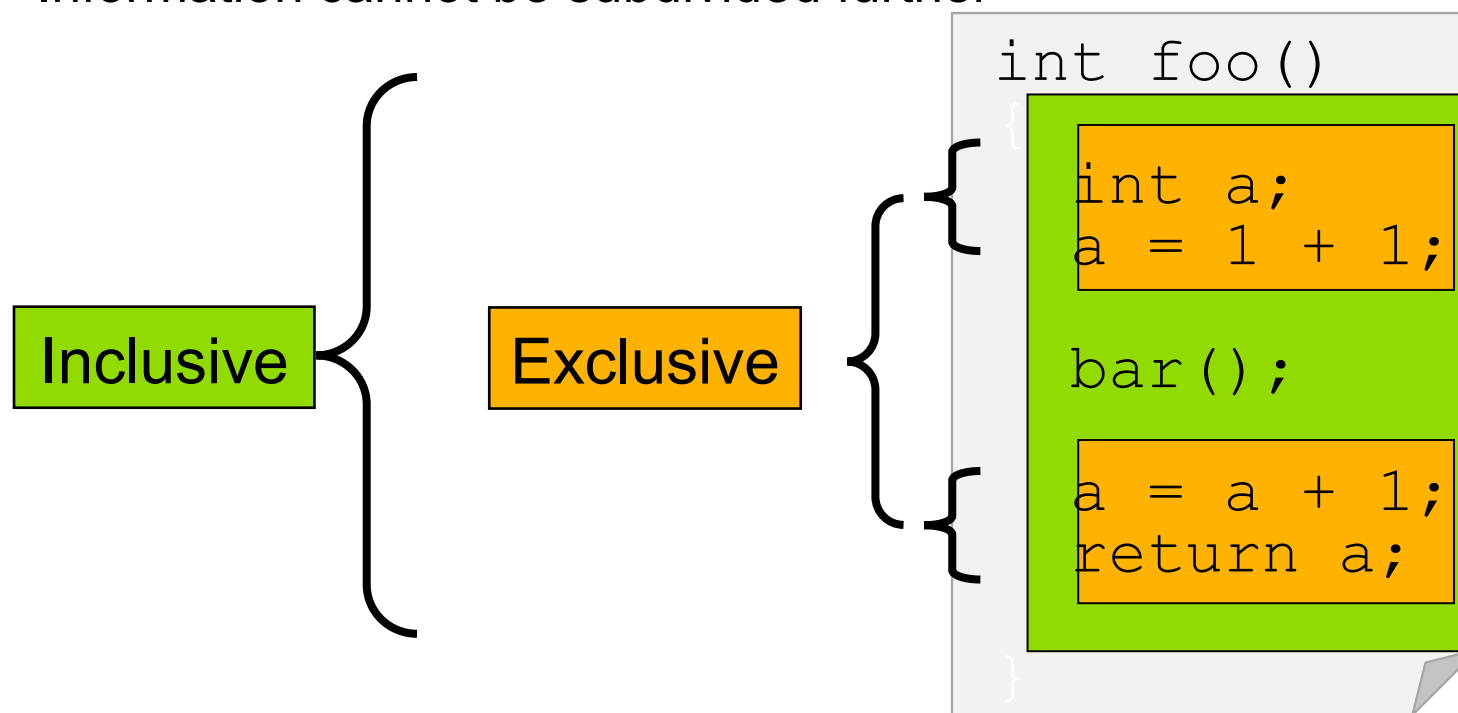
Tracing



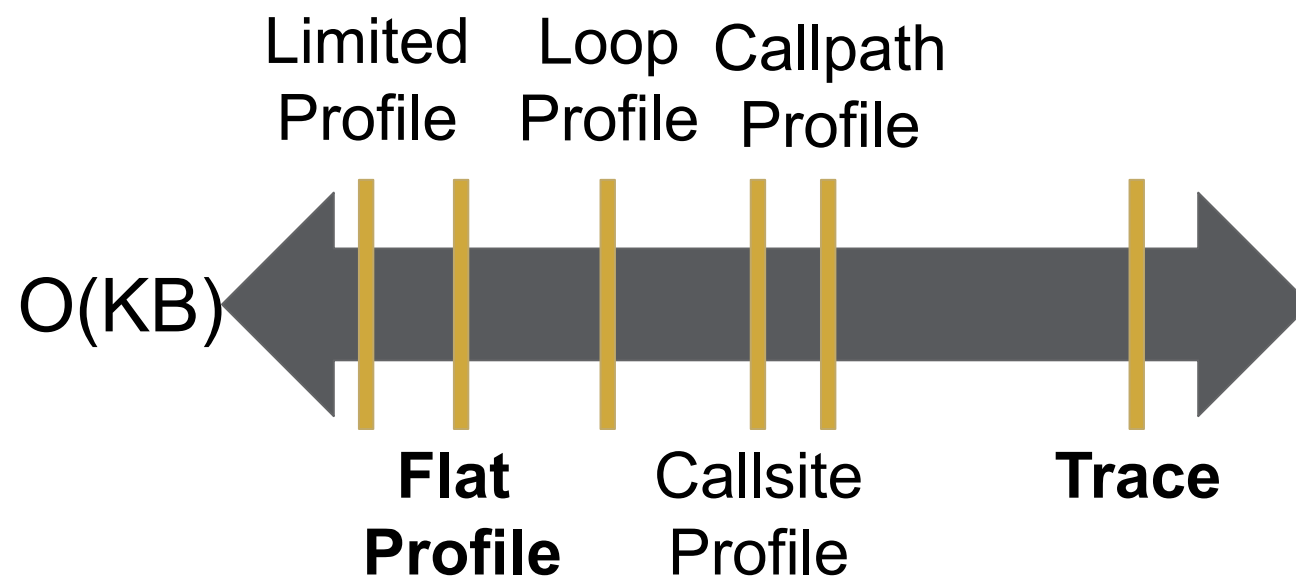
- Tracing shows you when the events take place on a timeline

Inclusive vs. Exclusive values

- Inclusive
 - Information of all sub-elements aggregated into single value
- Exclusive
 - Information cannot be subdivided further



How much data do you want?



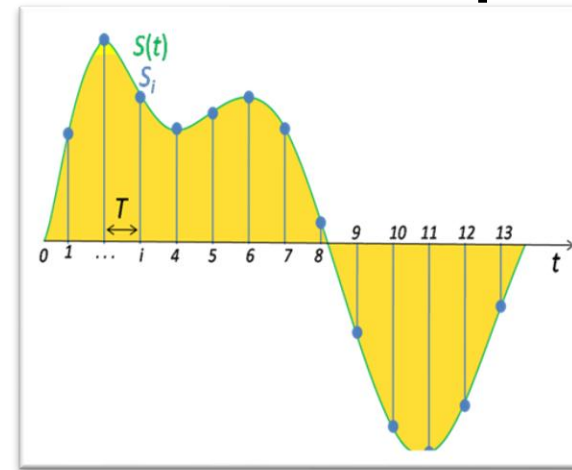
Performance Data Measurement

Direct via Probes

```
Call  
START('potential')  
// code  
Call  
STOP('potential')
```

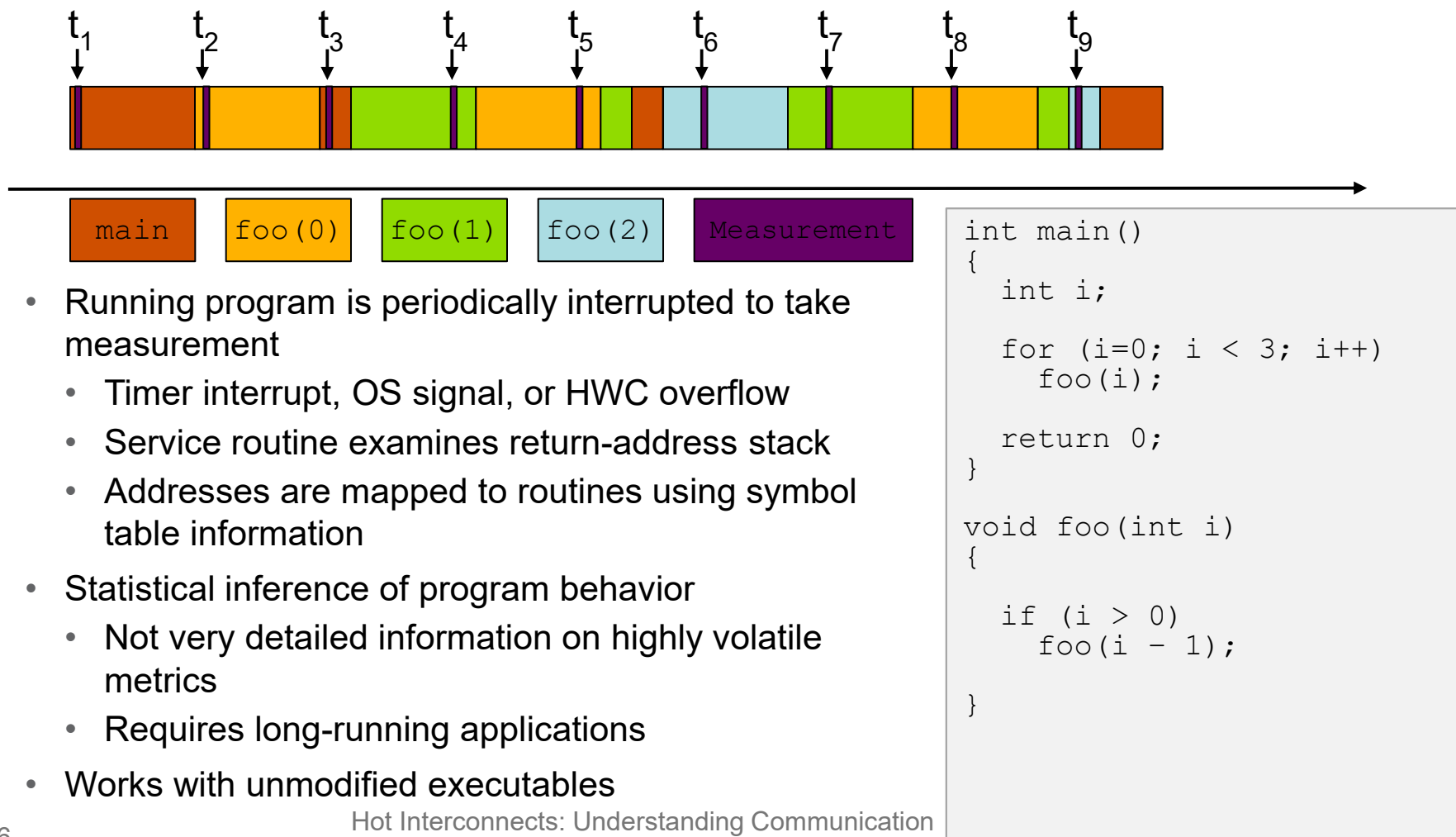
- Exact measurement
- Fine-grain control
- Calls inserted into code

Indirect via Sampling



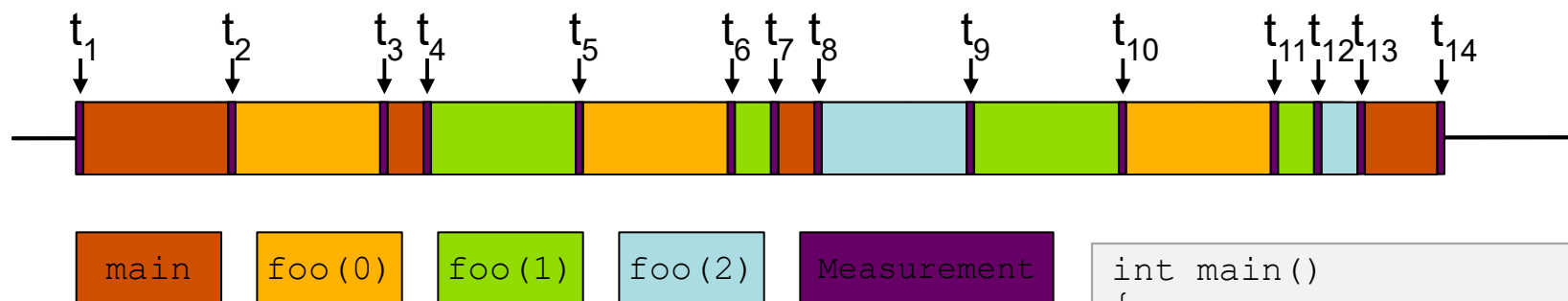
- No code modification
- Minimal effort
- Relies on debug symbols (-g)

Sampling



- Running program is periodically interrupted to take measurement
 - Timer interrupt, OS signal, or HWC overflow
 - Service routine examines return-address stack
 - Addresses are mapped to routines using symbol table information
- Statistical inference of program behavior
 - Not very detailed information on highly volatile metrics
 - Requires long-running applications
- Works with unmodified executables

Instrumentation

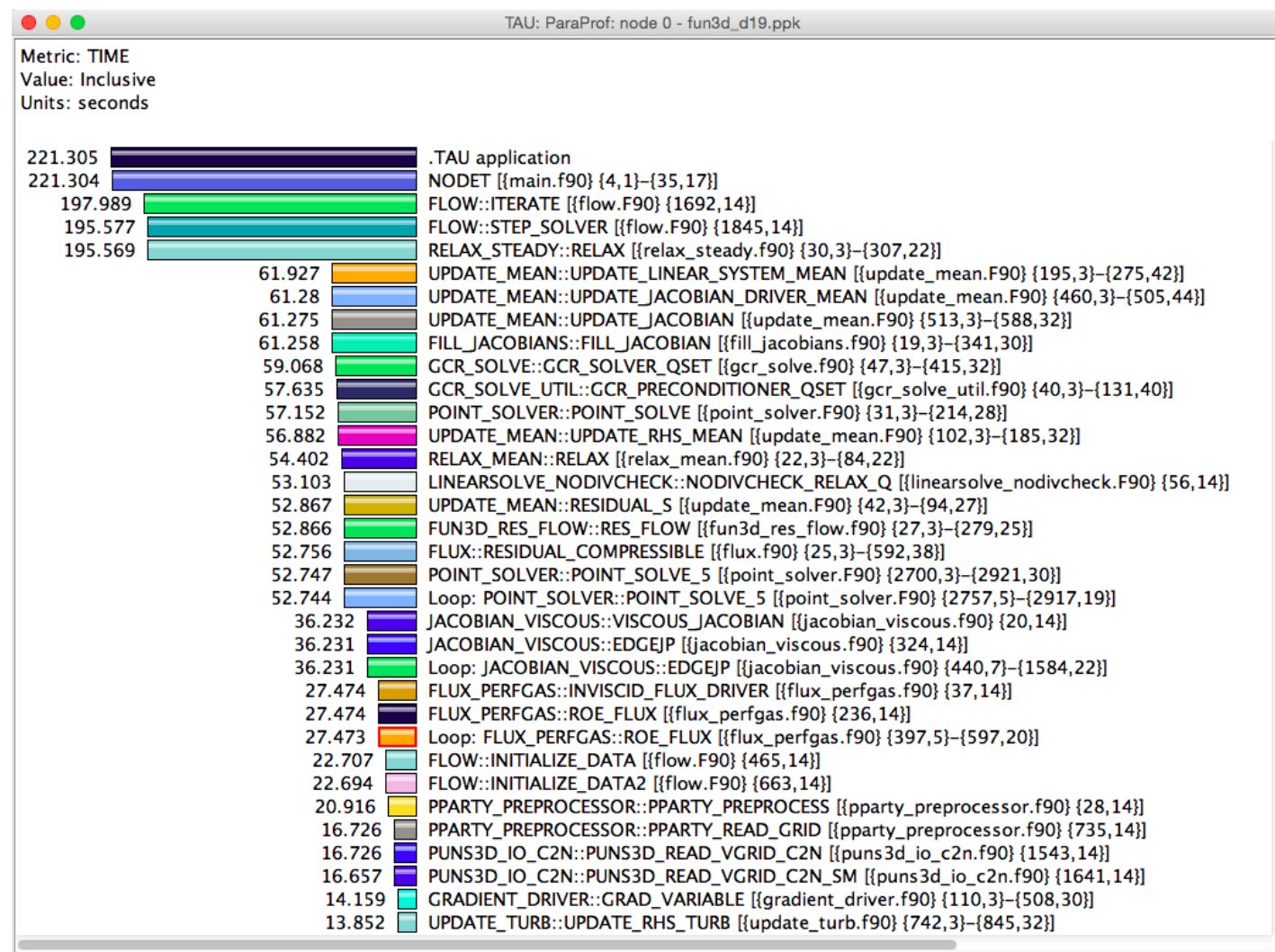


- Measurement code is inserted such that every event of interest is captured directly
 - Can be done in various ways
- Advantage:
 - Much more detailed information
- Disadvantage:
 - Processing of source-code / executable necessary
 - Large relative overheads for small functions

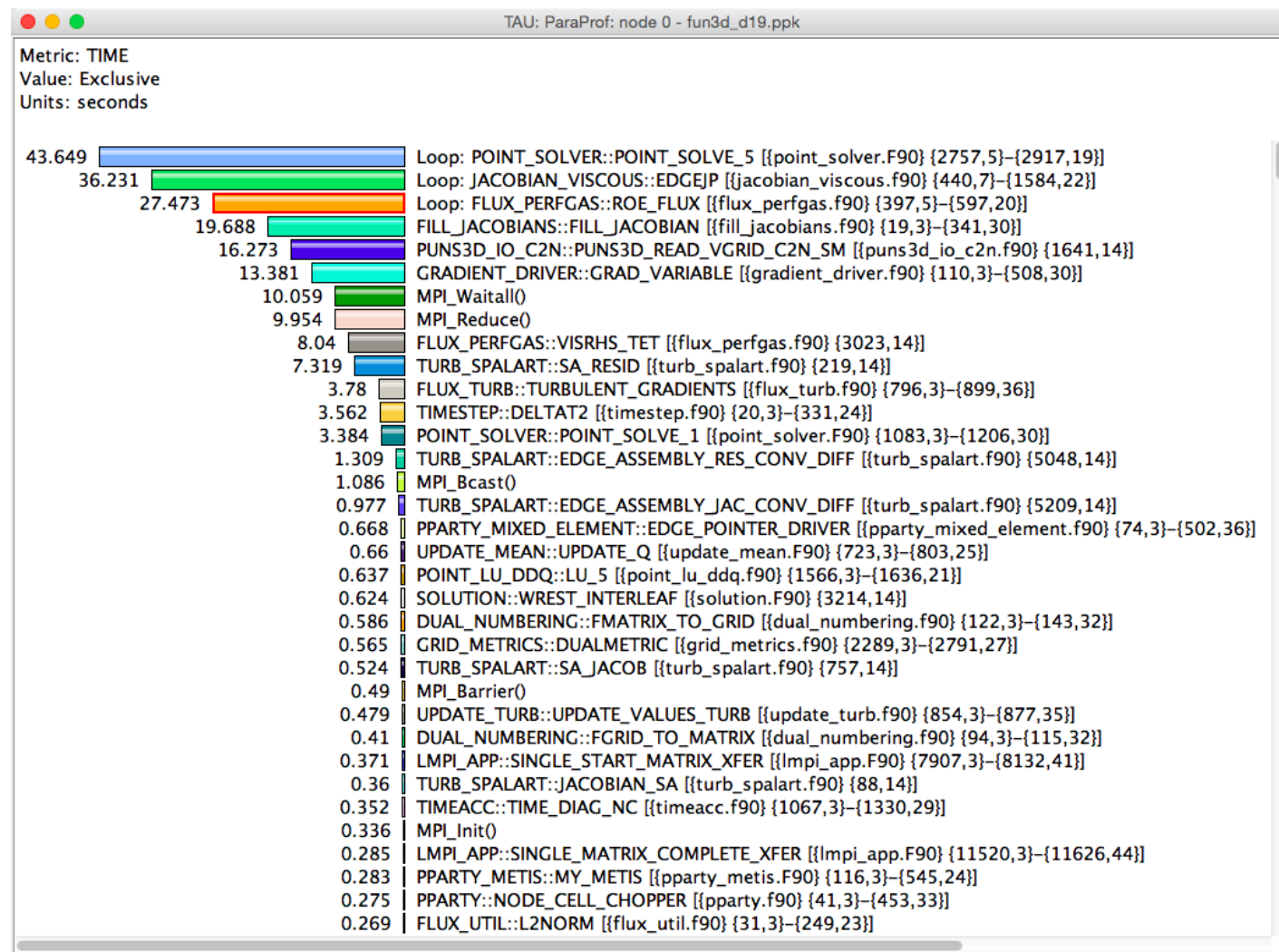
```
int main()
{
    int i;
    TAU_START("main");
    for (i=0; i < 3; i++)
        foo(i);
    TAU_STOP("main");
    return 0;
}

void foo(int i)
{
    TAU_START("foo");
    if (i > 0)
        foo(i - 1);
    TAU_STOP("foo");
}
```

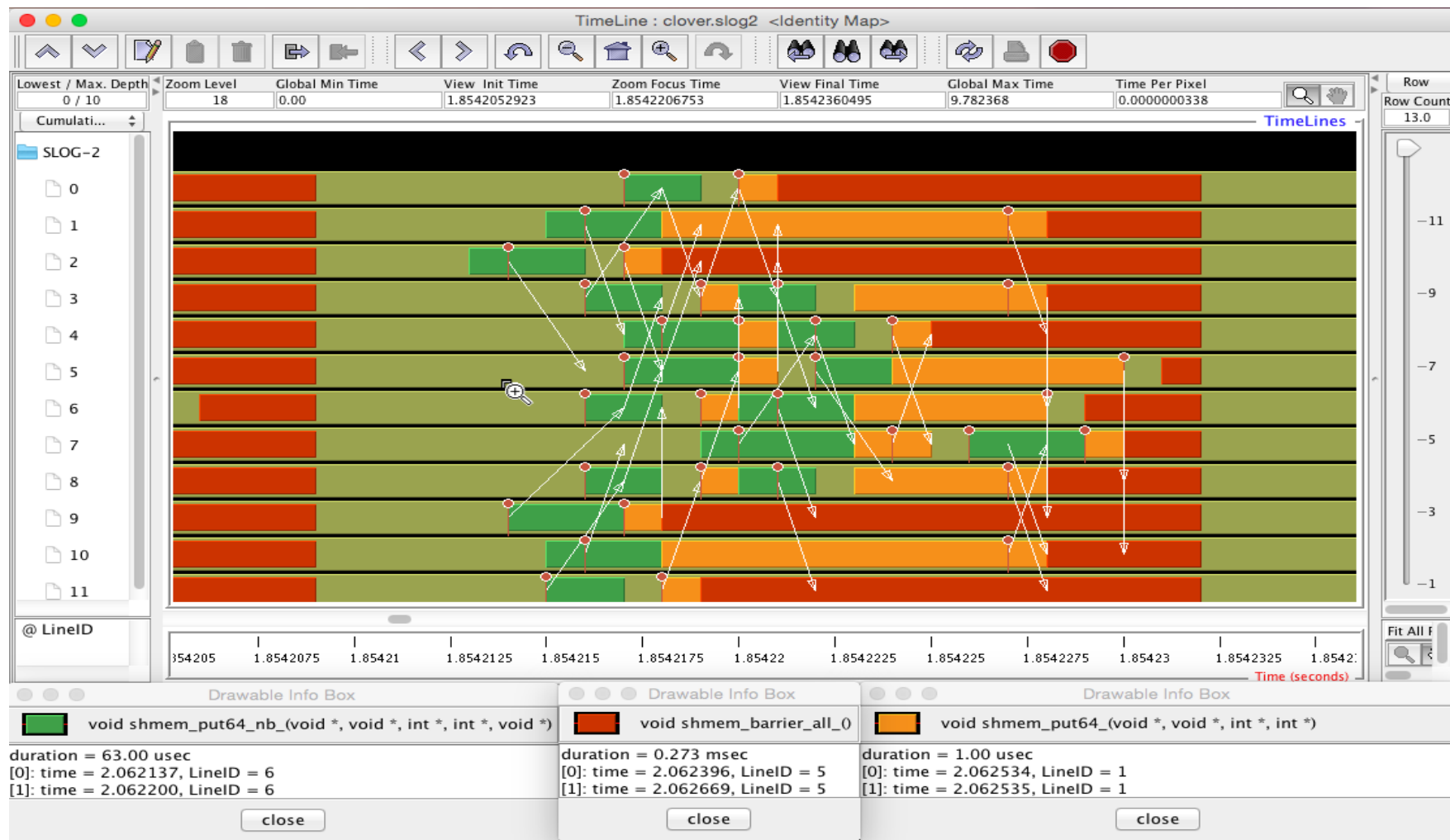
Inclusive Measurements



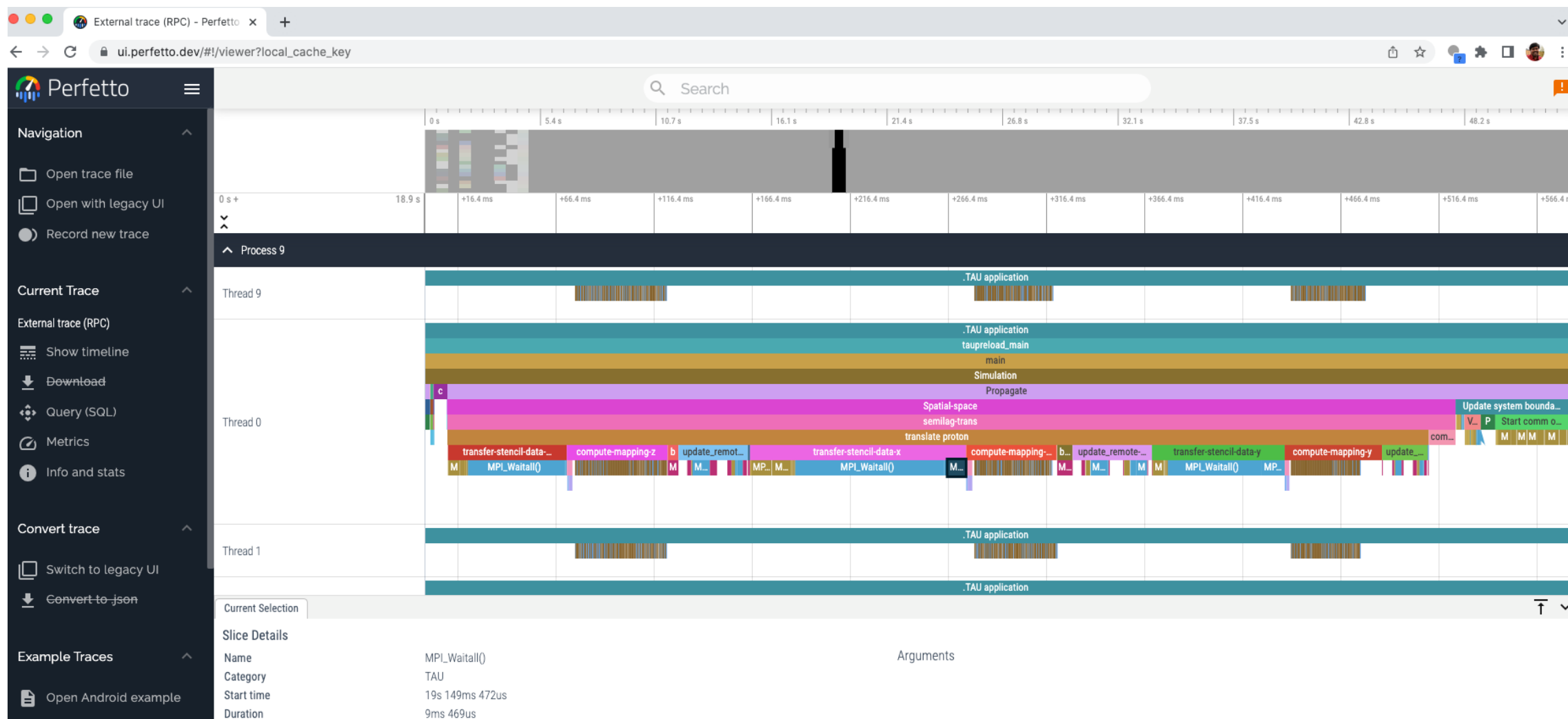
Exclusive Time



Tracing: Jumpshot [ANL] (ships with TAU)



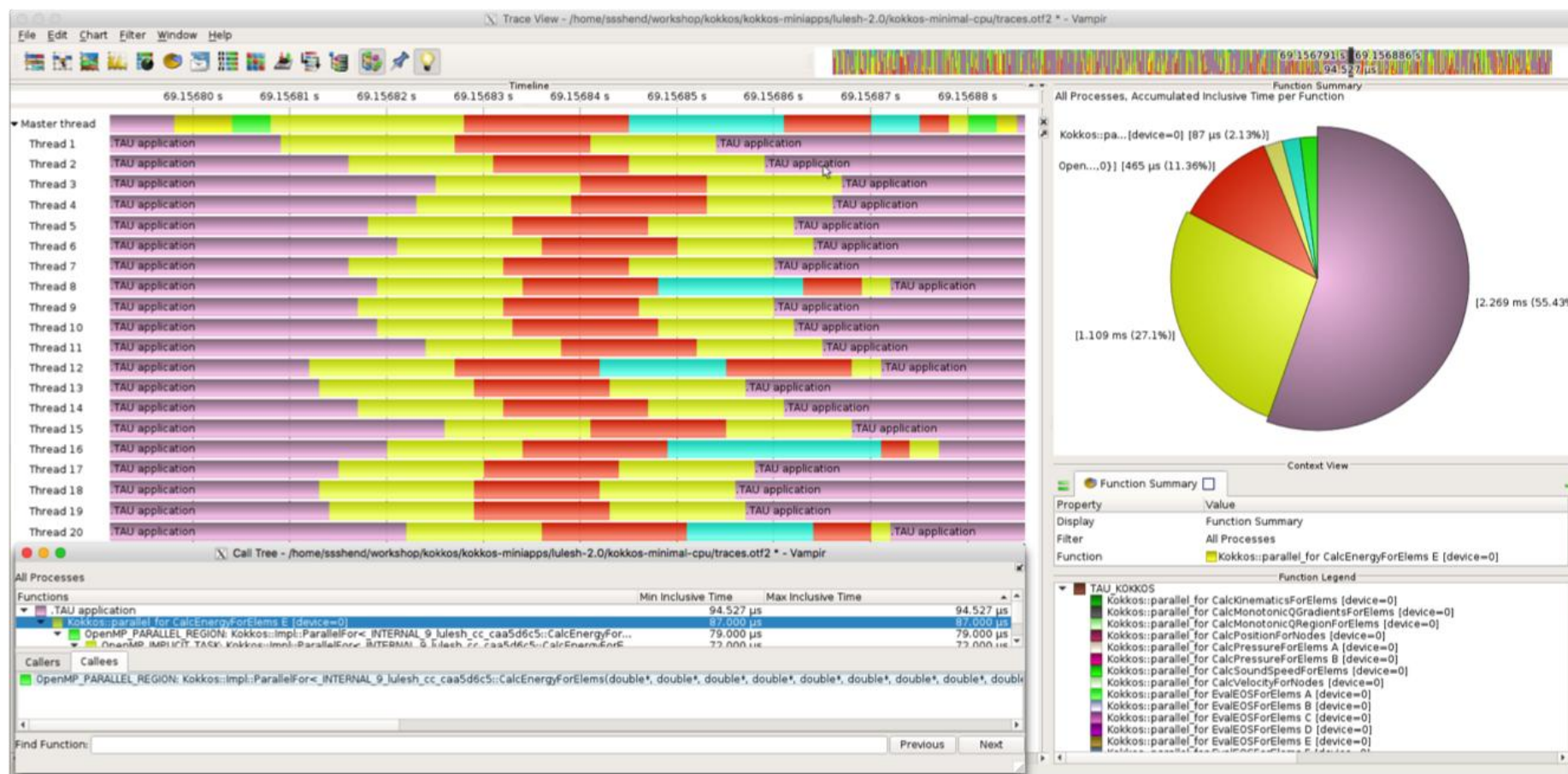
Perfetto.dev



Aug 19-21, 2026

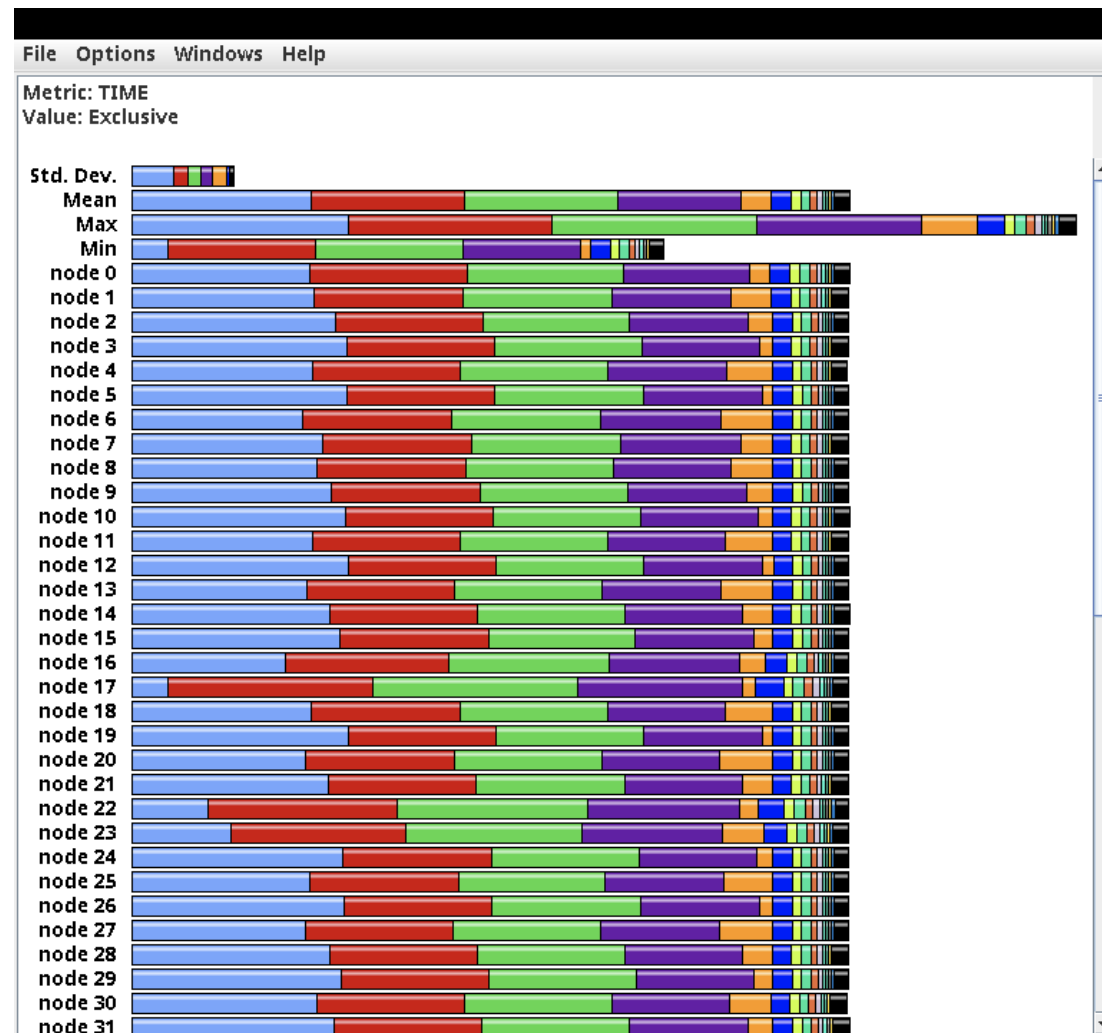
Hot Interconnects: Understanding Communication
Performance on Emerging Heterogeneous Architectures

Vampir [TU Dresden] Timeline: Kokkos



```
% export TAU_TRACE=1; export TAU_TRACE_FORMAT=otf2
% tau_exec -T ompt,serial -ompt ./a.out
% vampir traces.otf2 &
```

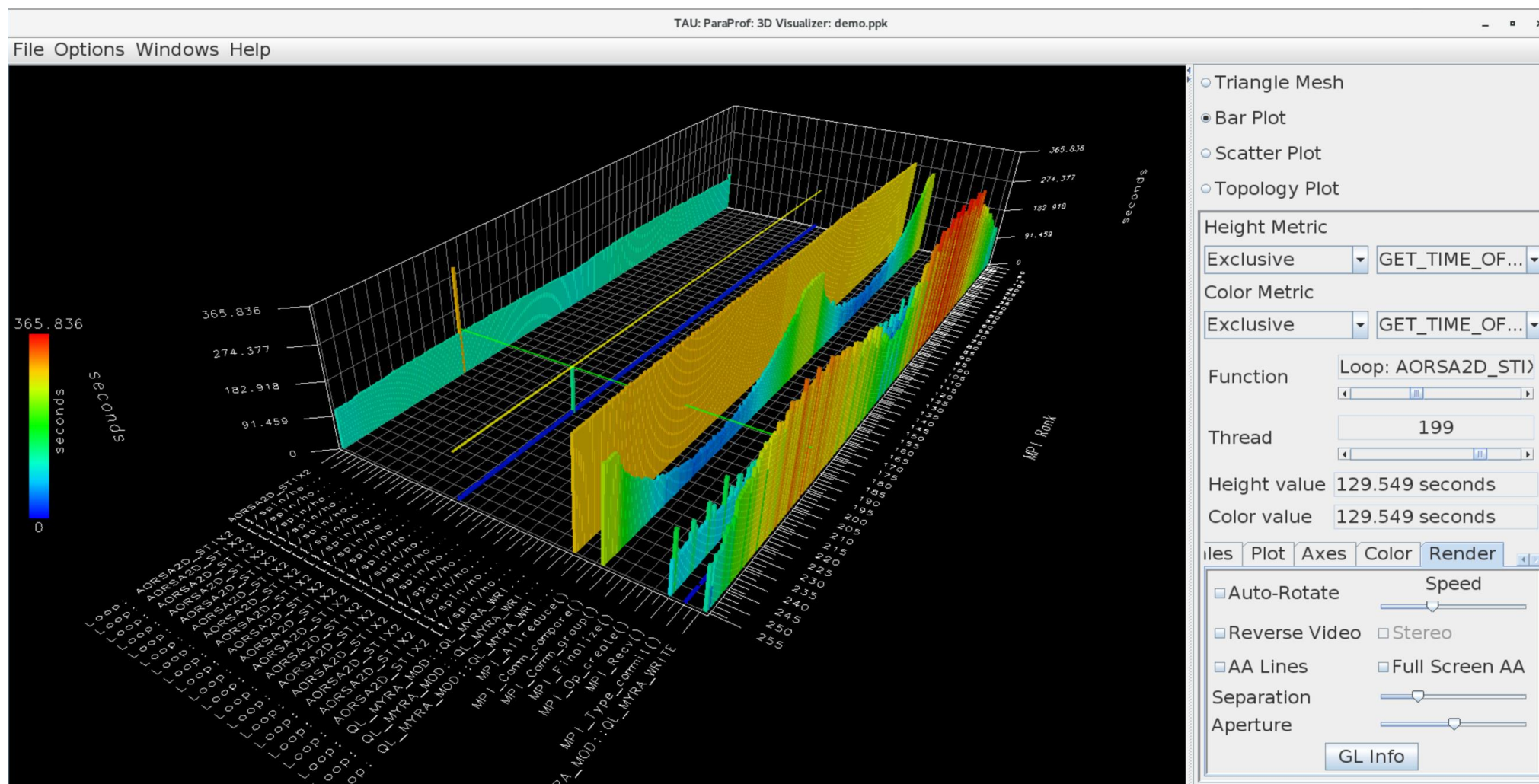
ParaProf Profile Browser



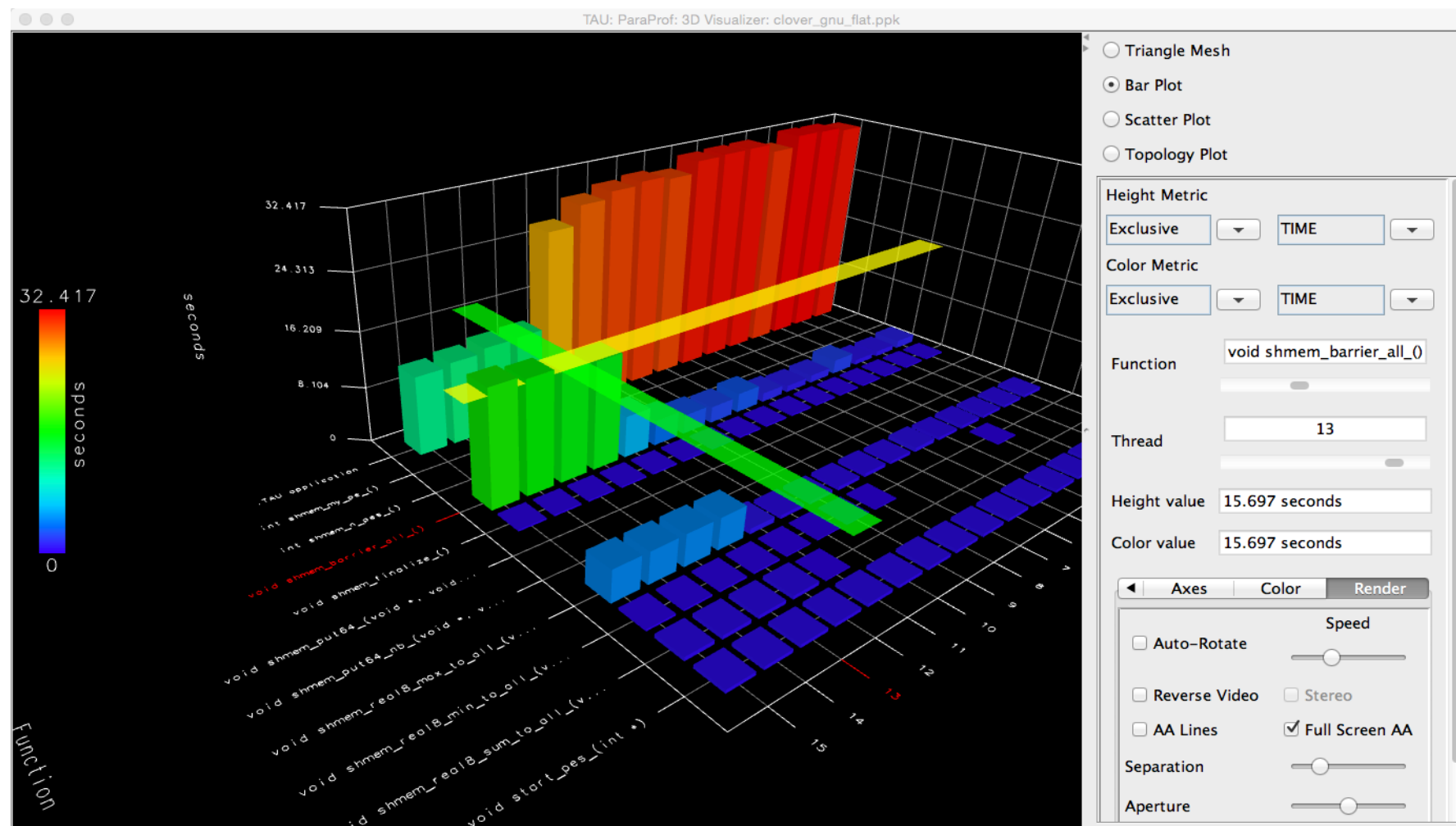
```
% paraprof
```

On a Mac if there are
black windows:
`% paraprof -fix-xquartz`

ParaProf 3D Profile Browser



TAU – ParaProf 3D Visualization

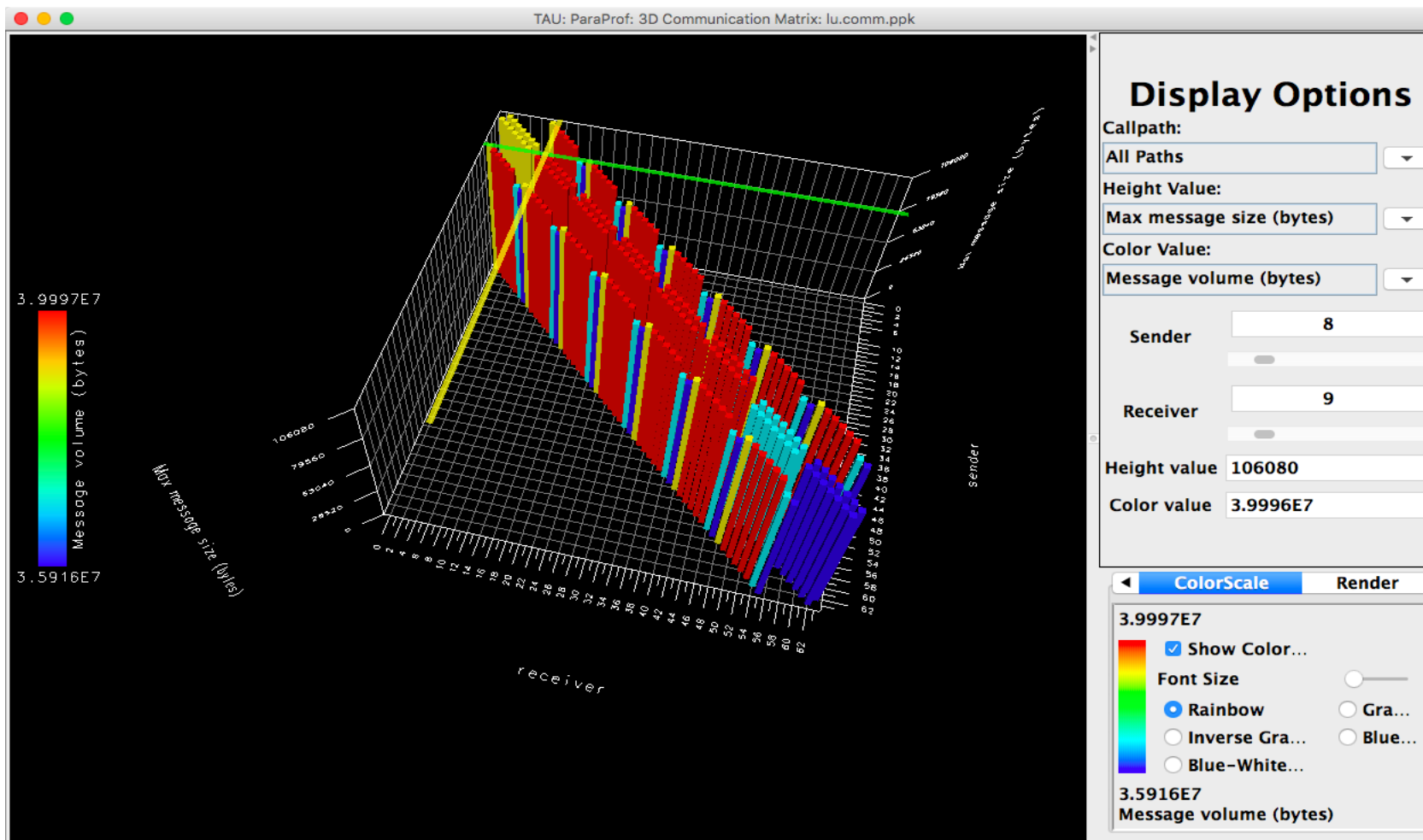


% paraprof app.ppk

Windows -> 3D Visualization -> Bar Plot (right pane)

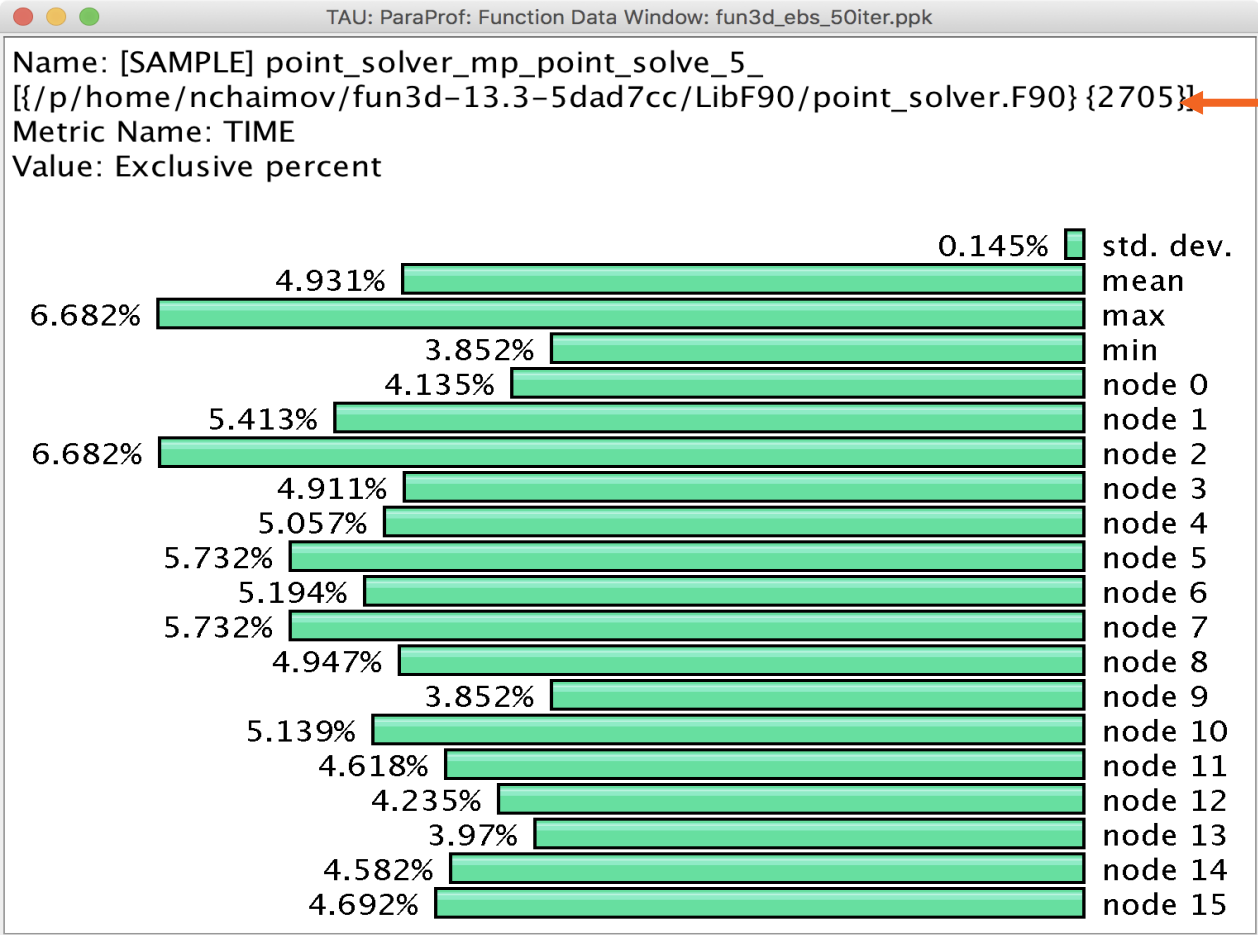
Hot Interconnects: Understanding Communication Performance on Emerging Heterogeneous Architectures

TAU – 3D Communication Window



```
% export TAU_COMM_MATRIX=1; mpirun ... tau_exec ./a.out
% paraprof ; Windows -> 3D Communication Matrix
```

Event Based Sampling (EBS)



File: point_solver.F90
Line: 2705

Uninstrumented!

```
% mpirun -n 16 tau_exec -ebs a.out
```

TAU's Runtime Environment Variables

Environment Variable	Default	Description
TAU_TRACE	0	Setting to 1 turns on tracing
TAU_CALLPATH	0	Setting to 1 turns on callpath profiling
TAU_TRACK_MEMORY_FOOTPRINT	0	Setting to 1 turns on tracking memory usage by sampling periodically the resident set size and high water mark of memory usage
TAU_TRACK_POWER	0	Tracks power usage by sampling periodically.
TAU_CALLPATH_DEPTH	2	Specifies depth of callpath. Setting to 0 generates no callpath or routine information, setting to 1 generates flat profile and context events have just parent information (e.g., Heap Entry: foo)
TAU_SAMPLING	1	Setting to 1 enables event-based sampling.
TAU_TRACK_SIGNALS	0	Setting to 1 generate debugging callstack info when a program crashes
TAU_COMM_MATRIX	0	Setting to 1 generates communication matrix display using context events
TAU_THROTTLE	1	Setting to 0 turns off throttling. Throttles instrumentation in lightweight routines that are called frequently
TAU_THROTTLE_NUMCALLS	100000	Specifies the number of calls before testing for throttling
TAU_THROTTLE_PERCALL	10	Specifies value in microseconds. Throttle a routine if it is called over 100000 times and takes less than 10 usec of inclusive time per call
TAU_CALLSITE	0	Setting to 1 enables callsite profiling that shows where an instrumented function was called. Also compatible with tracing.
TAU_PROFILE_FORMAT	Profile	Setting to “merged” generates a single file. “snapshot” generates xml format
TAU_METRICS	TIME	Setting to a comma separated list generates other metrics. (e.g., ENERGY,TIME,P_VIRTUAL_TIME,PAPI_FP_INS,PAPI_NATIVE_<event>:<subevent>)

Runtime Environment Variables

Environment Variable	Default	Description
TAU_TRACE	0	Setting to 1 turns on tracing
TAU_TRACE_FORMAT	Default	Setting to “otf2” turns on TAU’s native OTF2 trace generation (configure with –otf=download)
TAU_EBS_UNWIND	0	Setting to 1 turns on unwinding the callstack during sampling (use with tau_exec –ebs or TAU_SAMPLING=1)
TAU_EBS_RESOLUTION	line	Setting to “function” or “file” changes the sampling resolution to function or file level respectively.
TAU_TRACK_LOAD	0	Setting to 1 tracks system load on the node
TAU_SELECT_FILE	Default	Setting to a file name, enables selective instrumentation based on exclude/include lists specified in the file.
TAU_OMPT_SUPPORT_LEVEL	basic	Setting to “full” improves resolution of OMPT TR6 regions on threads 1.. N-1. Also, “lowoverhead” option is available.
TAU_OMPT_RESOLVE_ADDRESS_EAGERLY	1	Setting to 1 is necessary for event based sampling to resolve addresses with OMPT. Setting to 0 allows the user to do offline address translation.

Runtime Environment Variables

Environment Variable	Default	Description
TAU_TRACK_MEMORY_LEAKS	0	Tracks allocates that were not de-allocated (needs <code>-optMemDbg</code> or <code>tau_exec -memory</code>)
TAU_EBS_SOURCE	TIME	Allows using PAPI hardware counters for periodic interrupts for EBS (e.g., <code>TAU_EBS_SOURCE=PAPI_TOT_INS</code> when <code>TAU_SAMPLING=1</code>)
TAU_EBS_PERIOD	100000	Specifies the overflow count for interrupts
TAU_MEMDBG_ALLOC_MIN/MAX	0	Byte size minimum and maximum subject to bounds checking (used with <code>TAU_MEMDBG_PROTECT_*</code>)
TAU_MEMDBG_OVERHEAD	0	Specifies the number of bytes for TAU's memory overhead for memory debugging.
TAU_MEMDBG_PROTECT_BELOW/ABOVE	0	Setting to 1 enables tracking runtime bounds checking below or above the array bounds (requires <code>-optMemDbg</code> while building or <code>tau_exec -memory</code>)
TAU_MEMDBG_ZERO_MALLOC	0	Setting to 1 enables tracking zero byte allocations as invalid memory allocations.
TAU_MEMDBG_PROTECT_FREE	0	Setting to 1 detects invalid accesses to deallocated memory that should not be referenced until it is reallocated (requires <code>-optMemDbg</code> or <code>tau_exec -memory</code>)
TAU_MEMDBG_ATTEMPT_CONTINUE	0	Setting to 1 allows TAU to record and continue execution when a memory error occurs at runtime.
TAU_MEMDBG_FILL_GAP	Undefined	Initial value for gap bytes
TAU_MEMDBG_ALINGMENT	Sizeof(int)	Byte alignment for memory allocations
TAU_EVENT_THRESHOLD	0.5	Define a threshold value (e.g., .25 is 25%) to trigger marker events for min/max

Installing and Configuring TAU on aac6

- Installing TAU:

- `wget tau.uoregon.edu/tau.tgz; tar zxf tau.tgz; cd tau-2.<ver>`
- `wget http://tau.uoregon.edu/ext.tgz ; tar xf ext.tgz`
- `./configure -python -mpi -c++=mpicxx -cc=mpicc -fortran=mpif90 -bfd=download \`
`-unwind=download -iowrapper -dwarf=download -perfetto \`
`-rocm=/nfsapps/ubuntu-24.04/opt/rocm-7.2.4 -rocprofsdk -elfutils=download -otf=download`
- `make install`
- `export PATH=`pwd`/x86_64/bin:$PATH`
- `which paraprof`
- `which tau_exec`

Using TAU on Frontier@OLCF.ORNL.GOV

```
tar xf /sw/frontier/ums/ums002/paratools/tar/workshop.tgz
cd workshop; cat README
salloc -n 4 -A <ACCOUNT> -N 1 -t 0:55:00 -q debug -C nvme --gpus 2
module load ums ums002 tau
srun -n 4 ./a.out
srun -n 4 tau_exec -rocm -ebs ./a.out
srun -n 4 tau_exec -rocm_pc -ebs ./a.out. # For PC sampling on GPU and event-based sampling on CPU
export TAU_METRICS=ROCM_SQ_WAVES # see rocprofv3 -L for a complete list of counters
srun -n 4 tau_exec -rocm -ebs ./a.out

pprof -a | more
paraprof --pack foo.ppk
On your desktop: % paraprof foo.ppk
```

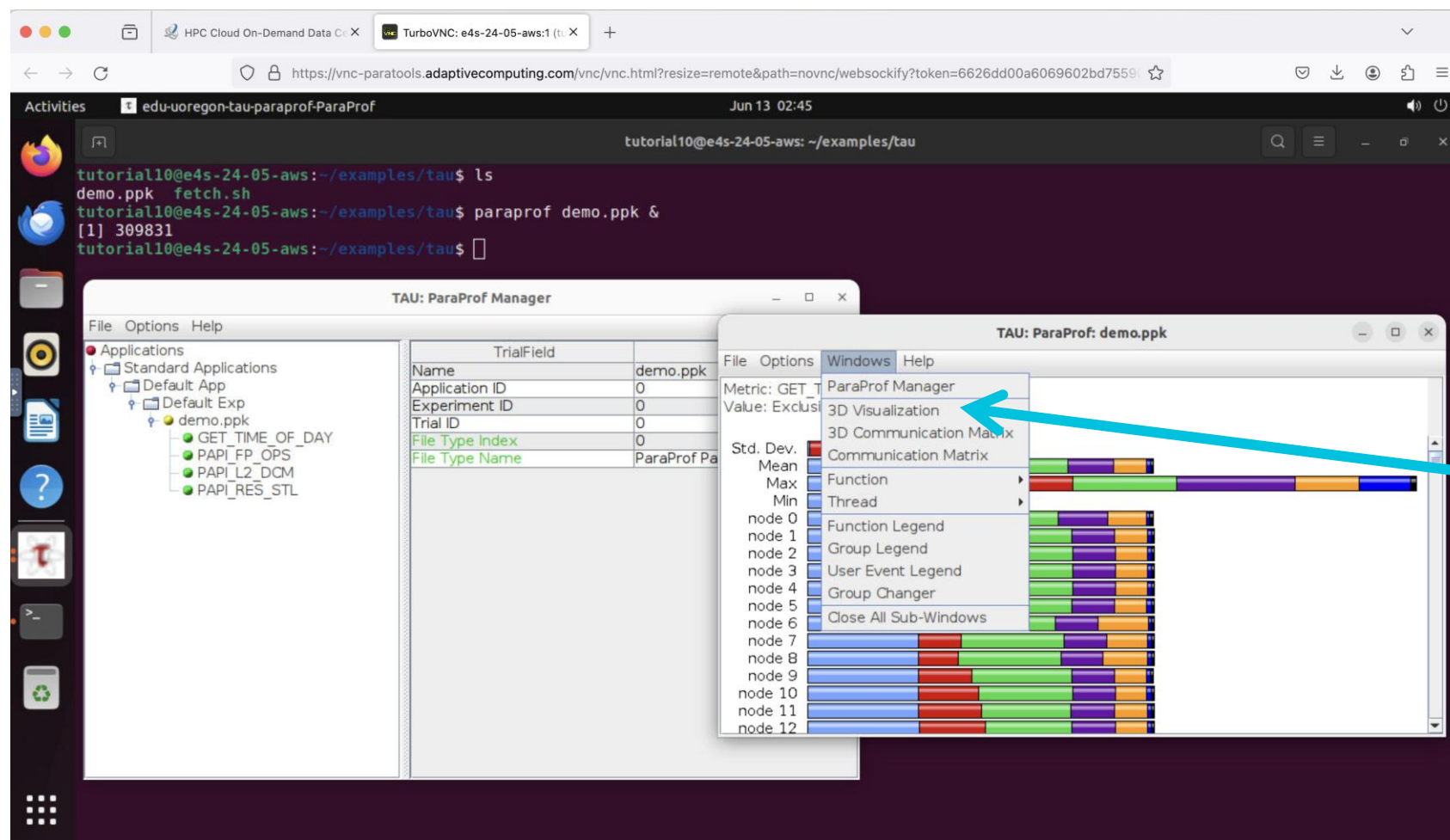
TAU's Runtime Environment Variables

Environment Variable	Default	Description
TAU_TRACE	0	Setting to 1 turns on tracing
TAU_CALLPATH	0	Setting to 1 turns on callpath profiling
TAU_TRACK_MEMORY_FOOTPRINT	0	Setting to 1 turns on tracking memory usage by sampling periodically the resident set size and high water mark of memory usage
TAU_TRACK_POWER	0	Tracks power usage by sampling periodically.
TAU_CALLPATH_DEPTH	2	Specifies depth of callpath. Setting to 0 generates no callpath or routine information, setting to 1 generates flat profile and context events have just parent information (e.g., Heap Entry: foo)
TAU_SAMPLING	1	Setting to 1 enables event-based sampling.
TAU_TRACK_SIGNALS	0	Setting to 1 generate debugging callstack info when a program crashes
TAU_COMM_MATRIX	0	Setting to 1 generates communication matrix display using context events
TAU_THROTTLE	1	Setting to 0 turns off throttling. Throttles instrumentation in lightweight routines that are called frequently
TAU_THROTTLE_NUMCALLS	100000	Specifies the number of calls before testing for throttling
TAU_THROTTLE_PERCALL	10	Specifies value in microseconds. Throttle a routine if it is called over 100000 times and takes less than 10 usec of inclusive time per call
TAU_CALLSITE	0	Setting to 1 enables callsite profiling that shows where an instrumented function was called. Also compatible with tracing.
TAU_PROFILE_FORMAT	Profile	Setting to "merged" generates a single file. "snapshot" generates xml format
TAU_METRICS	TIME	Setting to a comma separated list generates other metrics. (e.g., ENERGY,TIME,P_VIRTUAL_TIME,PAPI_FP_INS,PAPI_NATIVE_<event>:<subevent>)

Runtime Environment Variables

Environment Variable	Default	Description
TAU_TRACE	0	Setting to 1 turns on tracing
TAU_TRACE_FORMAT	Default	Setting to “otf2” turns on TAU’s native OTF2 trace generation (configure with –otf=download)
TAU_EBS_UNWIND	0	Setting to 1 turns on unwinding the callstack during sampling (use with tau_exec –ebs or TAU_SAMPLING=1)
TAU_EBS_RESOLUTION	line	Setting to “function” or “file” changes the sampling resolution to function or file level respectively.
TAU_TRACK_LOAD	0	Setting to 1 tracks system load on the node
TAU_SELECT_FILE	Default	Setting to a file name, enables selective instrumentation based on exclude/include lists specified in the file.
TAU_OMPT_SUPPORT_LEVEL	basic	Setting to “full” improves resolution of OMPT TR6 regions on threads 1.. N-1. Also, “lowoverhead” option is available.
TAU_OMPT_RESOLVE_ADDRESS_EAGERLY	1	Setting to 1 is necessary for event based sampling to resolve addresses with OMPT. Setting to 0 allows the user to do offline address translation.

TAU paraprof

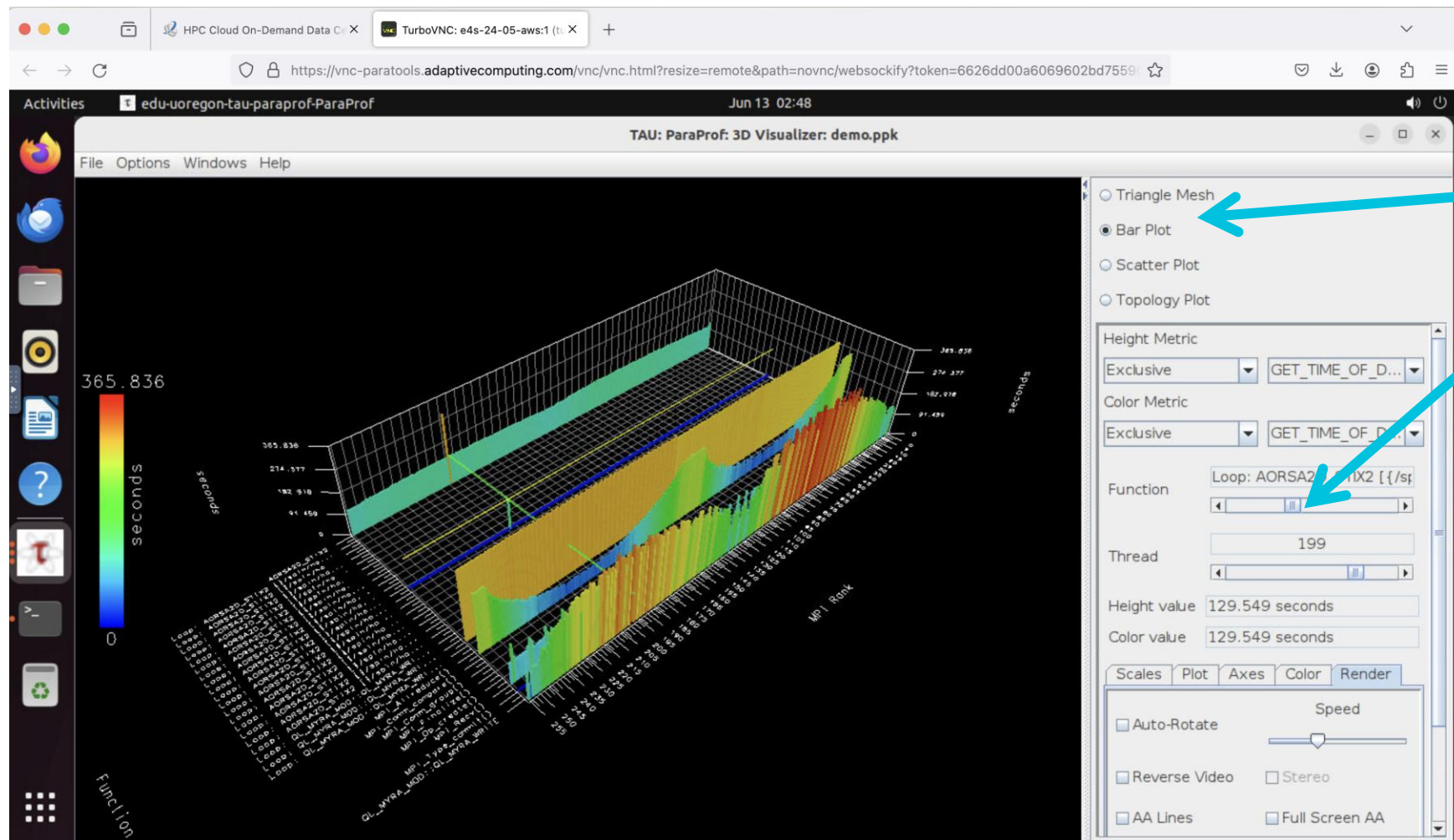


cd ~/examples/tau

paraprof demo.ppk &

Choose 3D Visualization

TAU paraprof 3D visualization



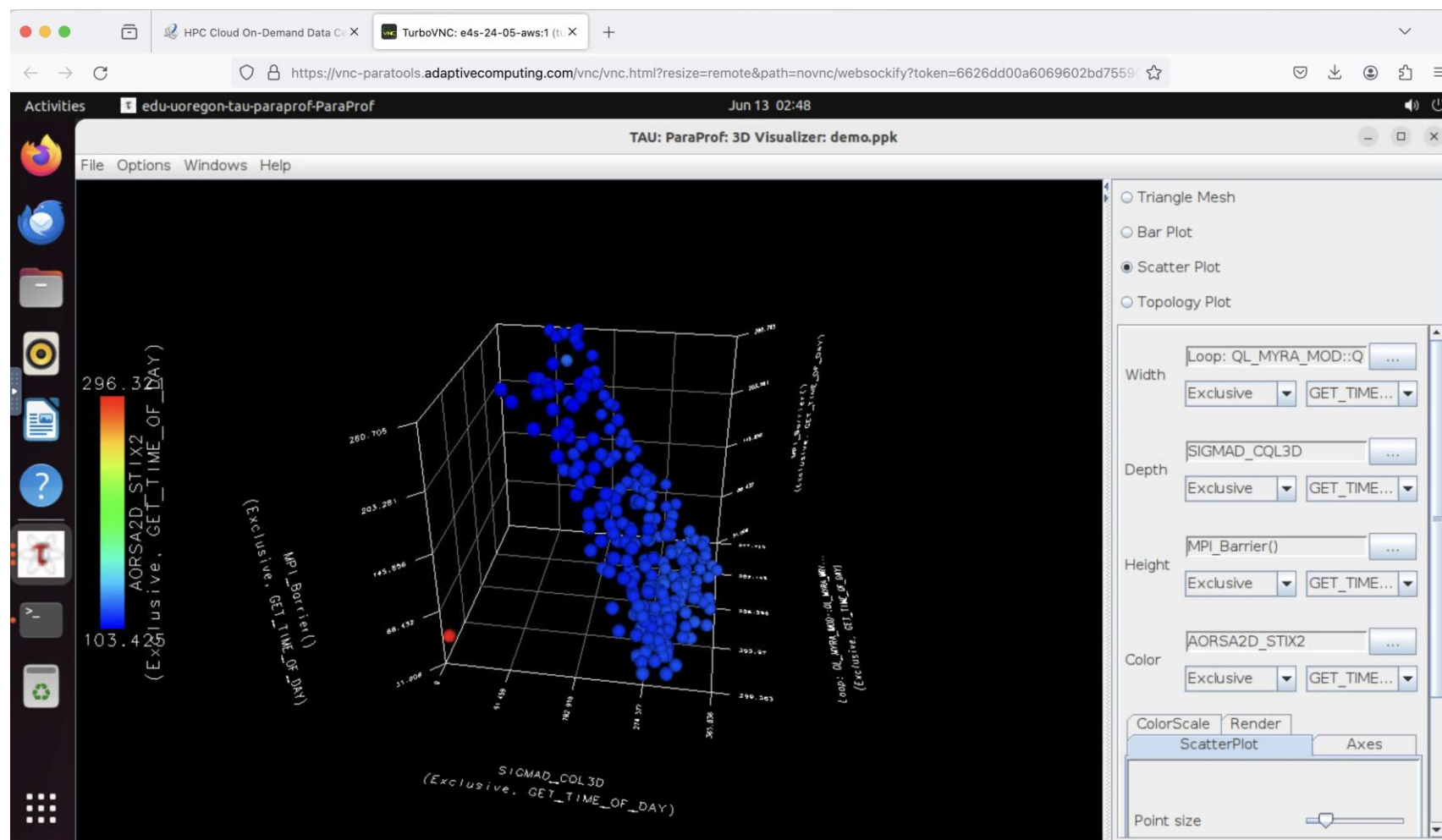
Choose Bar Plot and move

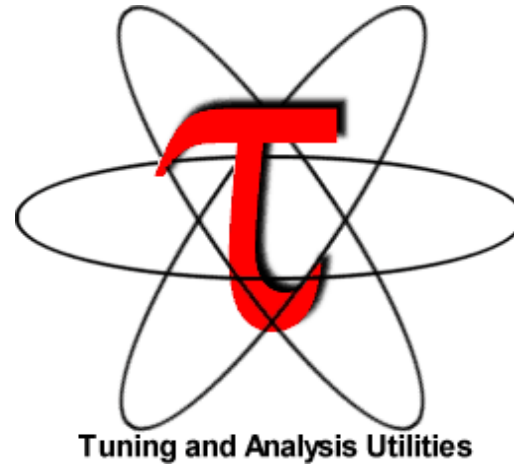
Function and Thread
Sliders

First mouse button to rotate
Second mouse button to
translate (left to right)
Scroll wheel (or +/- keys) to
zoom in.

Try Scatter plot next

TAU paraprof: 3D Scatter Plot





<https://tau.uoregon.edu>

<https://e4s.io> [TAU in Docker/Singularity containers]

for more information

Free download, open source, BSD license

Disclaimer

The information presented in this document is for informational purposes only and may contain technical inaccuracies, omissions, and typographical errors. The information contained herein is subject to change and may be rendered inaccurate for many reasons, including but not limited to product and roadmap changes, component and motherboard version changes, new model and/or product releases, product differences between differing manufacturers, software changes, BIOS flashes, firmware upgrades, or the like. Any computer system has risks of security vulnerabilities that cannot be completely prevented or mitigated. AMD assumes no obligation to update or otherwise correct or revise this information. However, AMD reserves the right to revise this information and to make changes from time to time to the content hereof without obligation of AMD to notify any person of such revisions or changes.

THIS INFORMATION IS PROVIDED 'AS IS.' AMD MAKES NO REPRESENTATIONS OR WARRANTIES WITH RESPECT TO THE CONTENTS HEREOF AND ASSUMES NO RESPONSIBILITY FOR ANY INACCURACIES, ERRORS, OR OMISSIONS THAT MAY APPEAR IN THIS INFORMATION. AMD SPECIFICALLY DISCLAIMS ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, OR FITNESS FOR ANY PARTICULAR PURPOSE. IN NO EVENT WILL AMD BE LIABLE TO ANY PERSON FOR ANY RELIANCE, DIRECT, INDIRECT, SPECIAL, OR OTHER CONSEQUENTIAL DAMAGES ARISING FROM THE USE OF ANY INFORMATION CONTAINED HEREIN, EVEN IF AMD IS EXPRESSLY ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

© 2026 Advanced Micro Devices, Inc. All rights reserved. AMD, the AMD Arrow logo, ROCm, Radeon, Instinct and combinations thereof are trademarks of Advanced Micro Devices, Inc. Other product names used in this publication are for identification purposes only and may be trademarks of their respective companies.

OpenCL™ is a trademark of Apple Inc. used by permission by Khronos Group, Inc.

The OpenMP name and the OpenMP logo are registered trademarks of the OpenMP Architecture Review Board.

OpenMPI is distributed by The Open MPI Project under the New BSD license and copyright 2004 - 2012.





Profiling Applications

IEEE Hot Interconnects Aug 19-21, 2026

Presenters: Shelby Lockhart, AMD

Giacomo Capodaglio, AMD

Bob Robey, AMD



PyTorch Example – RCCL communication

- PyTorch generally uses RCCL for communication between processes
- The examples we mention below are large enough to scale across multiple GPUs to profile and try out optimizations
- Example List (from <https://github.com/pytorch/examples>)
 - Imagenet – an image classification example using a large image database
 - Communication is dominated by allreduces
 - FSDP2 – Fully Sharded Data Parallel
 - FSDP can be considered a decomposition of Distributed Data Parallel (DDP)'s all-reduce into reduce-scatter and all-gather operations
 - minGPT-ddp – GPT like model with Distributed Data Parallel
- For the this tutorial, we will examine the **Imagenet** example:
<https://github.com/amd/HPCTrainingExamples/blob/main/MLExamples/Pytorch/imagenet/README.md>

Imagenet example

- Original source at: <https://github.com/pytorch/examples/tree/main/imagenet>

Name	Last commit message
..	
README.md	Add Accelerator Api to Imagenet Example
extract_ILSVRC.sh	typo
main.py	Fixed use_accel not defined issue (#1363)
requirements.txt	Add Accelerator Api to Imagenet Example
README.md	

ImageNet training in PyTorch

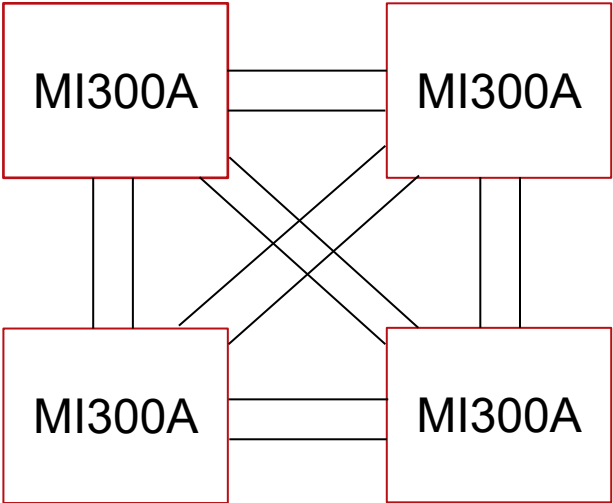
This implements training of popular model architectures, such as ResNet, AlexNet, and VGG on the ImageNet dataset.

This is the Python™ file we will be modifying during this tutorial

Imagenet – an image classification

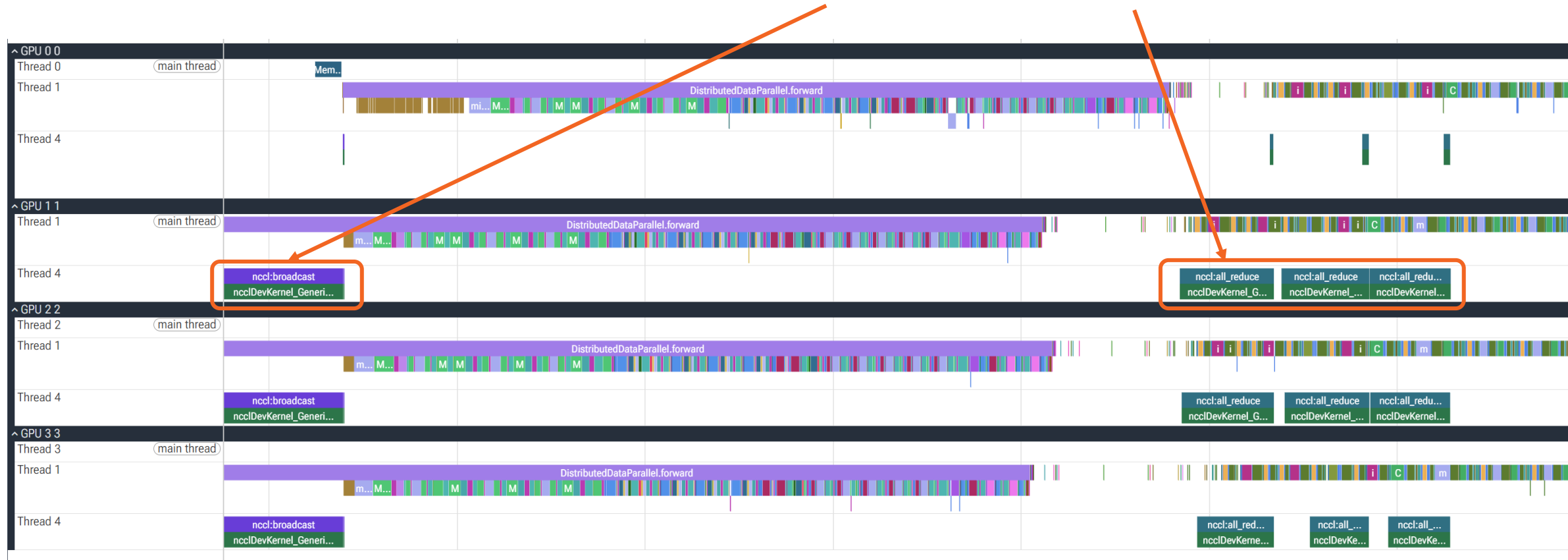
- This example has a --dummy flag that generates dummy data instead of downloading the large dataset
- For the exercises, we will use various configurations of the MI300A GPU.
- Concepts are applicable to other AMD GPUs and vendors such as Nvidia and Intel® as well.
- We'll begin with the standard workflow for the example and add simple profiling changes to support the optimization exercises.
- Diagram of PPAC MI300A node network
- `rocm-smi --showtopo`

===== Hops between two GPUs =====				
	GPU0	GPU1	GPU2	GPU3
GPU0	0	1	1	1
GPU1	1	0	1	1
GPU2	1	1	0	1
GPU3	1	1	1	0
===== Link Type between two GPUs =====				
	GPU0	GPU1	GPU2	GPU3
GPU0	0	XGMI	XGMI	XGMI
GPU1	XGMI	0	XGMI	XGMI
GPU2	XGMI	XGMI	0	XGMI
GPU3	XGMI	XGMI	XGMI	0



Imagenet Iteration

The RCCL communication for imagenet is mostly a **rccl_broadcast** and a **rccl_all_reduce**



The workflow for the Imagenet example is complex and unbalanced as shown in the snapshot of the timeline of one iteration.

Fig: Perfetto timeline from torch.profiler

Housekeeping – fix warning at exit

- Upstream main.py inits the NCCL process group but never destroys it, so PyTorch warns at exit ("`destroy\_process\_group()` was not called ... can leak resources").

```
[rank0]:[W724 08:40:08.467941441 ProcessGroupNCCL.cpp:1647] Warning: WARNING: destroy_process_group() was not called before program exit, which can leak resources. For more info, please see https://pytorch.org/docs/stable/distributed.html#shutdown (function operator())
```

- Register an atexit handler right after `init\_process\_group` so every worker (incl. mp.spawn children) cleans up on a normal exit.

```
dist.init_process_group(backend=args.dist_backend, init_method=args.dist_url,
                        world_size=args.world_size, rank=args.rank)
import atexit as _ax, torch.distributed as _d; _ax.register(lambda: _d.destroy_process_group() if _d.is_initialized() else None)
```

```
sed -i '/world_size=args.world_size, rank=args.rank)/a\         import atexit as _ax,
torch.distributed as _d; _ax.register(lambda: _d.destroy_process_group() if _d.is_initialized()
else None)' main.py
```

NOTE: Use copy-paste from the [README.md](#) rather than from this slides to avoid formatting issues

Shorten the demo

- Keep the demo (and the profiler trace) short:

```
data_time.update(time.time() - end)
if i >= 100: break
```

```
sed -i '/^      data_time.update(time.time() - end)/a\
      if i >= 100: break' main.py
```

Print peak GPU memory usage

- Print per-GPU peak memory once at the end of train():

```
torch.cuda.is_available() and getattr(args,"rank",0)<=0 and print(f"PEAK_MEM_MB {torch.cuda.max_memory_allocated()/1e6:.0f}")  
def validate(val_loader, model, criterion, args):
```

```
sed -i '/^def validate(/i\    torch.cuda.is_available() and getattr(args,"rank",0)<=0 and  
print(f"PEAK_MEM_MB {torch.cuda.max_memory_allocated()/1e6:.0f}")' main.py
```

Start torch.profiler to gather RCCL communication time

Start the profiler at the top of `train()`:

```
model.train()
import torch.profiler as _tp
_prof = _tp.profiler(activities=[_tp.ProfilerActivity.CPU, _tp.ProfilerActivity.CUDA], acc_events=True); _prof.start()

sed -i '/^    model.train()/a\
import torch.profiler as _tp\
_prof = _tp.profiler(activities=[_tp.ProfilerActivity.CPU,\
    _tp.ProfilerActivity.CUDA], acc_events=True); _prof.start()' main.py
```

Stop the profiler and print the total RCCL kernel time (at the end of `train()`, right before `def validate()`):

```
_prof.stop()
_rccl_ms = sum(e.self_device_time_total for e in _prof.key_averages() if "nccl" in e.key.lower())/1e3
_ws = getattr(args, "world_size", "?")
getattr(args, "rank", 0) <= 0 and print(f"RCCL_TOTAL_MS {_rccl_ms:.3f} gpus={_ws}")

def validate(val_loader, model, criterion, args):

sed -i '/^def validate(/i\
_prof.stop()\
_rccl_ms = sum(e.self_device_time_total for e in _prof.key_averages() \
    if "nccl" in e.key.lower())/1e3\
_ws = getattr(args, "world_size", "?")\
getattr(args, "rank", 0) <= 0 and print(f"RCCL_TOTAL_MS {_rccl_ms:.3f} gpus={_ws}")' main.py
```

Replacing PyTorch .to with custom .migrate

1 / 3

- On MI300A, you have unified shared memory in the physical hardware
- Data on the host can be made visible to the device **without having to copy it** there
- PyTorch does not natively support this

We do this to enable **copying the pointer instead of the data**

1. Run: `python main.py ... STAGE=migrate` (after it is modified as you will see in the next slides)
2. `main.py` reads `os.environ["STAGE"]` and sees it equal to `migrate`
3. `main.py` inserts [COMMON_DIR](#) into `sys.path` so zerocopy can be found
4. `from zerocopy import Stager` imports the Stager module
5. `main.py` instantiates `Stager(device, method="register")`, running `Stager.__init__`
6. `__init__` calls `unified_memory_available()`, which checks `HSA_XNACK=1 + ROCm`, then calls `_load_ext()`
7. `_load_ext()` calls `torch.utils.cpp_extension.load(...)`, which JIT-compiles [migrate_ext.cpp](#) into `migrate_ext.so` and loads it, caching the module as `_EXT`
8. The training loop calls `stager.to_device(images)`, which uses `_EXT.register_migrate(...)` to return a zero-copy aliased GPU tensor (or falls back to `.to(device)` if disabled)

Replacing PyTorch .to with custom .migrate

2 / 3

Point COMMON_DIR at the shared helpers and enable XNACK:

```
export COMMON_DIR="$(git rev-parse --show-toplevel)/MLExamples/Pytorch/common"
export HSA_XNACK=1
```

Set up the staging counters and (for STAGE=migrate) the zero-copy Stager at the top of train():

```
sed -i '/^    model.train()/a\
    _stage_ms = 0.0; _stage_n = 0; _stager = None\
    if os.environ.get("STAGE") == "migrate":\
        import sys; sys.path.insert(0, os.environ["COMMON_DIR"]); from zerocopy import Stager\
        _stager = Stager(device, method="register")' main.py
```

register-migrate needs **pageable** (non-pinned) host memory:

```
sed -i '/^from torch.utils.data import Subset/a\
def _zero_copy_active():\
    """pin_memory gate: pageable only when the STAGE=migrate register path is live."""\
    if os.environ.get("STAGE") != "migrate":\
        return False\
    try:\
        import sys; sys.path.insert(0, os.environ["COMMON_DIR"]); from zerocopy import unified_memory_available\
        return unified_memory_available()\
    except Exception:\
        return False' main.py
```

```
sed -i 's/pin_memory=True/pin_memory=not _zero_copy_active()/g' main.py
```

Replacing PyTorch .to with custom .migrate

3 / 3

Time the host->device staging, but only when the env variable STAGE is set (copy vs migrate):

```
sed -i '/^      images = images.to(device, non_blocking=True)/c\
      if os.environ.get("STAGE"):\
          _e0 = torch.cuda.Event(enable_timing=True); _e1 = torch.cuda.Event(enable_timing=True)\
          _e0.record()\
          images = _stager.to_device(images) if _stager is not None else images.to(device, non_blocking=True)\
          _e1.record(); torch.cuda.synchronize()\
          _stage_ms += _e0.elapsed_time(_e1); _stage_n += 1\
      else:\
          images = images.to(device, non_blocking=True)' main.py
```

Print the per-step staging time (inserted at the end of train(), right before def validate()):

```
sed -i '/^def validate(/i\
    _stg = os.environ.get("STAGE",""); _ws2 = getattr(args,"world_size","?")\
    getattr(args,"rank",0)<=0 and _stage_n and print(f"STAGE_MS_PER_STEP {_stage_ms/_stage_n:.4f} \
    gpus={_ws2} stage={_stg}")' main.py
```

Hands-on break

1. Ssh into AAC6: `ssh <your_username>@aac6.amd.com`
2. Clone the HCPTrainingExamples repo:
`git clone https://github.com/amd/HPCTrainingExamples.git`
3. Navigate to the Imagenet directory:
`cd HPCTrainingExamples/MLExamples/Pytorch/imagenet`
4. Follow the instructions in the [README.md](#) up to 4) (not included)
5. For the sake of time, you could also apply the code modifications all at once with `apply_basic_edits.sh` but we suggest you **keep the script for your future reference** and keep track of what each change does to the code



Running an interactive script and batch

To allocate a compute node interactively (release for others when done):

```
salloc -p PPAC_MI300A_SPX -N 1 --gpus=4 -t 00:40:00
```

From the compute node, you can check that the PyTorch module is imported correctly and that the GPUs are visible (**don't forget to re-activate your venv and load the necessary modules**):

```
python3 -c 'import torch; print(torch.cuda.is_available(), torch.cuda.device_count())'
```

From the interactive allocation, setting up and running can be done all at once with (this will also create the venv and re-clone the pytorch examples repo):

```
run_imagenet_uv.sh
```

Preferred method is to submit a batch job

```
sbatch run_imagenet_uv.sbatch
```

Additional batch jobs to explore CPX mode and larger number of GPUs

```
sbatch run_imagenet_uv_sh5_cpx.sbatch
```

```
sbatch run_imagenet_uv_ppac_cpx.sbatch
```

The MIOpen database

You should check that PyTorch is being actually imported from the location in module `show pytorch`

```
python3 -c 'import torch;print(torch.__file__)'
```

for rocm/7.2.4 and pytorch/2.12.0 it should show:

```
/nfsapps/ubuntu-24.04/opt/rocmplus-7.2.4/pytorch-v2.12.0/pytorch/lib/python3.12/site-packages/torch/__init__.py
```

Full MIOpen startup can take many minutes: set fast mode to shorten the startup time

```
export MIOPEN_FIND_MODE=FAST
```

Suppress some warning noise

```
export MIOPEN_LOG_LEVEL=3
```

```
export KINETO_LOG_LEVEL=3
```

Note: on AAC6, MIOPEN_USER_DB_PATH and MIOPEN_CUSTOM_CACHE_DIR are already set in the PyTorch module to a /tmp directory. Create the directory and set the automatic removal at end of job

```
mkdir -p "$MIOPEN_USER_DB_PATH"
```

```
trap 'rm -rf "$MIOPEN_USER_DB_PATH"' EXIT
```

The MIOpen cache warm-up

Then, warm up the MIOpen cache to get better performance numbers. Do it on one GPU (**HIP_VISIBLE_DEVICES=0**) to avoid lock collisions on database. Full runs will just read from the database and should not have lock collisions in FAST mode.

```
HIP_VISIBLE_DEVICES=0 python -c "import torch,torchvision.models as M; \
    d=torch.device('cuda'); \
    n=M.resnet50().to(d); \
    c=torch.nn.CrossEntropyLoss().to(d); \
    x=torch.randn(256,3,224,224,device=d); \
    y=torch.randint(0,1000,(256,),device=d); \
    [c(n(x),y).backward() for _ in range(3)]; \
    torch.cuda.synchronize(); \
    print('warm done')"
```

Run the scaling sweep (one line per GPU count)

Run the benchmark once per GPU count by changing HIP_VISIBLE_DEVICES:

```
HIP_VISIBLE_DEVICES=0      python main.py -a resnet50 --dummy --dist-url 'tcp://127.0.0.1:23456' \
  --dist-backend nccl --multiprocessing-distributed --world-size 1 --rank 0 -b 128 -p 20 --epochs 1 \
  |& tee run_1.log
HIP_VISIBLE_DEVICES=0,1    python main.py -a resnet50 --dummy --dist-url 'tcp://127.0.0.1:23456' \
  --dist-backend nccl --multiprocessing-distributed --world-size 1 --rank 0 -b 256 -p 20 --epochs 1 \
  |& tee run_2.log
HIP_VISIBLE_DEVICES=0,1,2,3 python main.py -a resnet50 --dummy --dist-url 'tcp://127.0.0.1:23456' \
  --dist-backend nccl --multiprocessing-distributed --world-size 1 --rank 0 -b 512 -p 20 --epochs 1 \
  |& tee run_4.log
```

APU programming model (MI300A)

.to() copy vs zero-copy migrate (single GPU) -- compare STAGE_MS_PER_STEP

```
STAGE=copy    HIP_VISIBLE_DEVICES=0 python main.py -a resnet50 --dummy \  
  --dist-url 'tcp://127.0.0.1:23456' --dist-backend nccl \  
  --multiprocessing-distributed --world-size 1 --rank 0 -b 128 -p 20 --epochs 1 |& tee stage_copy.log  
STAGE=migrate HIP_VISIBLE_DEVICES=0 python main.py -a resnet50 --dummy \  
  --dist-url 'tcp://127.0.0.1:23456' --dist-backend nccl \  
  --multiprocessing-distributed --world-size 1 --rank 0 -b 128 -p 20 --epochs 1 |& tee stage_migrate.log
```

The APU programming model works on most data center GPUs from many vendors using managed memory

Discrete GPUs will move the data for the programmer

The MI300A has unified shared memory and only needs to copy the pointer

Hands-on break

1. Proceed with step 4) from the [README.md](#)
 - This will warm up the MIOpen cache before running the code
2. Step 5) will run the scaling sweep for different numbers of GPUs acting on HIP_VISIBLE_DEVICES (note: it assumes and SPX partition)
3. Step 6) compares the performance between copying the data and just migrating it using the APU programming model



Calculating the performance – baseline

Run the performance summary script (make sure you are still in the imagenet_test directory)

```
../images_per_sec.sh
```

Sample output from script

```
=== RCCL total time (per GPU count) ===
RCCL_TOTAL_MS 0.000 gpus=1
RCCL_TOTAL_MS 2477.950 gpus=2
RCCL_TOTAL_MS 29238.757 gpus=4
=== Host->device staging: .to (copy) vs .migrate ===
STAGE_MS_PER_STEP 27.9058 gpus=4 stage=copy
STAGE_MS_PER_STEP 0.3033 gpus=4 stage=migrate
=== Calculating the performance ==
run_1.log  gpus=1  img/s=260  step=0.4930s  batch=128  peak_mem_mb=11592  speedup=1.00x
run_2.log  gpus=2  img/s=549  step=0.4660s  batch=256  peak_mem_mb=11592  speedup=2.12x
run_4.log  gpus=4  img/s=964  step=0.5310s  batch=512  peak_mem_mb=11592  speedup=3.71x
```

RCCL optimizations

1 / 2

There are some tips and tricks to try for improving RCCL performance, they are collected here:

https://github.com/amd/HPCTrainingExamples/blob/main/MLExamples/Pytorch/imagenet/README_rccl_optimization.md

Section 1 — Reduce the bytes on the wire

- bf16|fp16 gradient compression – should reduce communication by almost half
- find the DDP line in `main_worker()`:

```
model = torch.nn.parallel.DistributedDataParallel(model, device_ids=[args.gpu])
```

- add the hook right below it (same indentation):

```
model = torch.nn.parallel.DistributedDataParallel(model, device_ids=[args.gpu])  
from torch.distributed.algorithms.ddp_comm_hooks import default_hooks as ddp_hooks  
model.register_comm_hook(None, ddp_hooks.bf16_compress_hook)
```

RCCL optimizations

2 / 2

Section 2 — Tune the RCCL transport / algorithm

- Collective Algorithm: `NCCL_ALGO=[Tree|Ring]`
 - Tree reduces latency while Ring gives better bandwidth
- Wire Protocol: `NCCL_PROTO=[Simple|LL|LL128]`
 - LL – Low-latency, LL128 – Low-latency 128-byte, Simple – high bandwidth
- Channel Count: `NCCL_MIN_NCHANNELS=4; NCCL_MAX_NCHANNELS=32`
 - More channels increases bandwidth to a limit

Section 3 — Overlap and shrink the collectives (DDP knobs)

- Avoid a gradient copy — `gradient_as_bucket_view=True`
 - Sends from in-place instead of copying
- Bucket size — `bucket_cap_mb=[25|50|100]`
 - Larger buckets means fewer, more efficient all-reduces
- Static graph — `static_graph=True`
 - Communication graph is fixed each step, reducing overhead

Compute optimizations

There are some tips and tricks to try for improving compute performance, they are collected here:

https://github.com/amd/HPCTrainingExamples/blob/main/MLExamples/Pytorch/imagenet/README_compute_optimization.md

- **Section 1**
 - 1a) bf16 autocast
 - Reduces the precision for the forward pass and loss calculation – reduces memory usage and communication cost, but can change results
 - 1b) matmul precision
 - Uses the lower-precision matmul path
- **Section 2**
 - 2a) channels_last
 - Changes the memory layout for faster convolutions
 - 2b cudnn.benchmark
 - Searches for the fastest convolution algorithm and caches it
- **Section 3**
 - 3a) torch.compile
 - Fuses kernels and reduces launch overhead
 - 3b) fused optimizer
 - Fuses the whole parameter update into one kernel

Hands-on break

1. Proceed with step 7) from the [README.md](#)
2. Step 8) will run the scaling sweep for different numbers of GPUs acting on HIP_VISIBLE_DEVICES (note: it assumes and SPX partition)
3. Step 9) and 10) will be the RCCL optimizations and compute optimizations
 - You can also run these with the dedicated sbatch scripts



Profiling Imagenet – torch.profiler setup

- The profiling exercise uses the `torch.profiler` and the TAU profiler from University of Oregon to capture every GPU kernel. We'll break those down into compute and communication operations.

- Load the additional required module for TAU:

```
module load tau
```

- Install matplotlib in the venv

- `uv pip install matplotlib`

- Change number of iterations from 100 to 20

```
sed -i 's/if i >= 100: break/if i >= 20: break/' main.py
```

- Keep `torch.profiler` start as is

- Run this command to add the chrome trace export to the `torch.profiler` stop

```
sed -i '/^    _prof.stop()/a\  
    _rk = getattr(args, "rank", 0)\  
    _prof.export_chrome_trace(f"torch_trace_rank{_rk}.json")' main.py
```

Profiling Imagenet – torch.profiler capturing timelines

Capture: torch.profiler on 4 GPUs.

```
HIP_VISIBLE_DEVICES=0,1,2,3 python main.py -a resnet50 --dummy \  
  --dist-url 'tcp://127.0.0.1:23456' --dist-backend nccl \  
  --multiprocessing-distributed --world-size 1 --rank 0 -b 512 -p 20 --epochs 1 \  
  |& tee capture_run_torch.log
```

Merge the per-rank torch traces into ONE 4-lane Perfetto trace (Chrome JSON, gzipped).

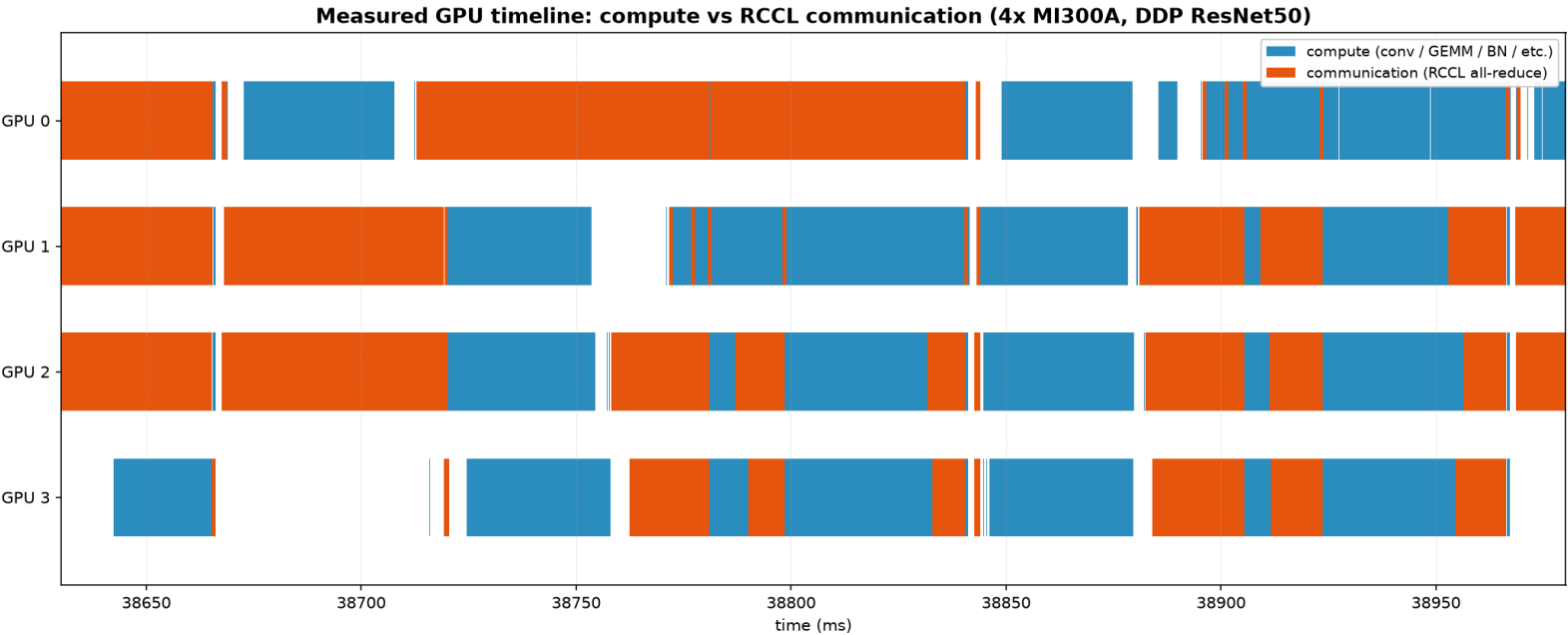
```
mkdir -p traces  
python3 ../profiling/merge_perfetto.py \  
  --glob 'torch_trace_rank*.json' \  
  --out traces/imagenet_4gpu.perfetto.json.gz
```

Profiling – torch.profiler rendering timelines

Render matplotlib timeline to produce the figure below (saved in \$PWD/figs/timeline_4gpu.png)

```
python3 "../profiling/render_timeline.py" \
  --out "$PWD/figs/timeline_4gpu.png" \
  --glob 'torch_trace_rank*.json'
```

Note: render_timeline.py is a couple of hundred lines



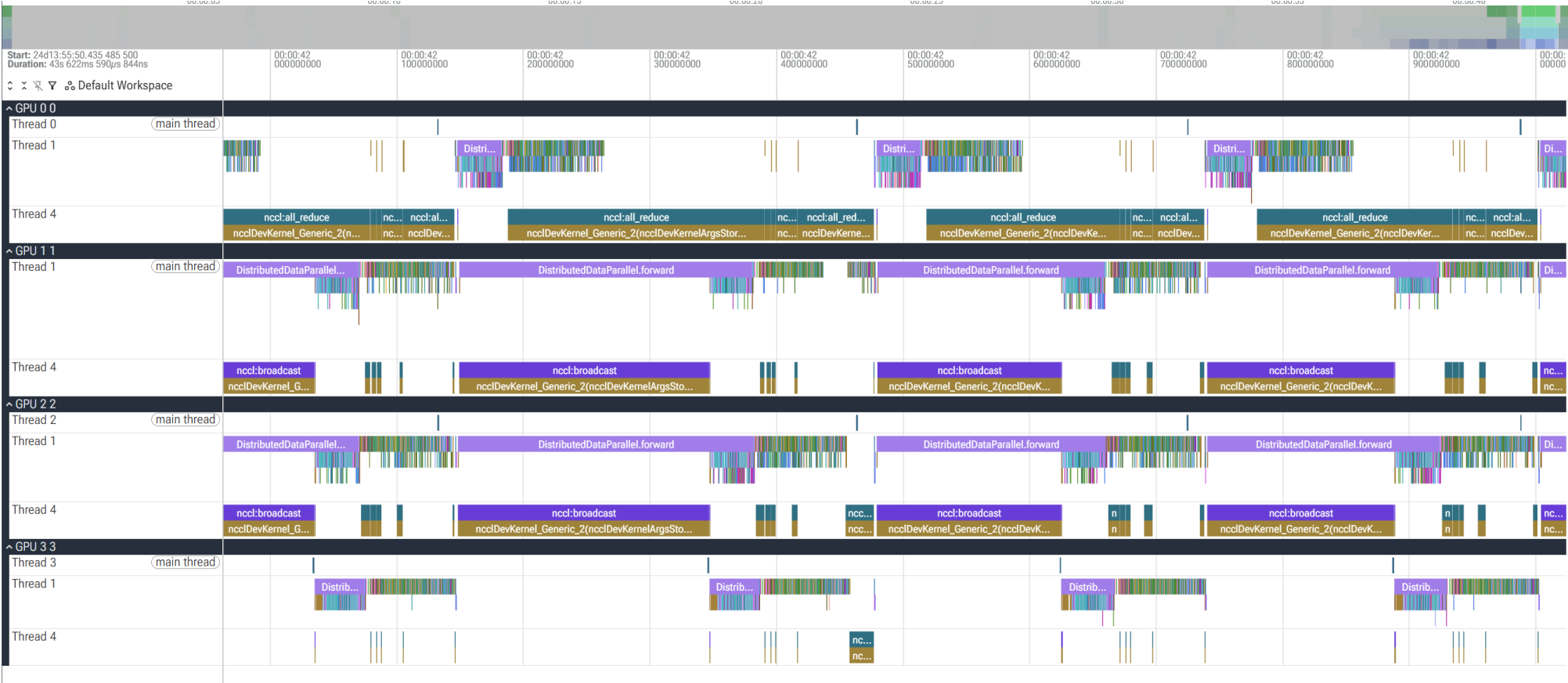
Compute is shown in blue

Communication in orange

Profiling – torch.profiler Perfetto traces

The Perfetto view is produced manually: download the merged trace below and open it in <https://ui.perfetto.dev/> using your preferred browser (also works offline):

`scp <user>@aac6.amd.com:HPCTrainingExamples/MLExamples/Pytorch/imagenet/profiling/traces/*.gz .`



Hands-on break

1. Start looking at the instructions in the [README.md](#) from Section 11)
2. Section 11.1) covers `torch.profiler`
 - Go through the commands we have shown and generate your own picture



Profiling – TAU setup

1 / 2

NOTE: TAU's rocprofsdk and torch.profiler (Kineto) both claim the ROCm profiler interface, so if `main.py` still has the `torch.profiler` edits, TAU attaches to nothing and writes no `profile.*` files

Re-fetch a clean upstream `main.py`:

```
[ -f pytorch_examples/imagenet/main.py ] || git clone --depth=1
https://github.com/pytorch/examples.git pytorch_examples
cp pytorch_examples/imagenet/main.py main_tau.py
sed -i '/world_size=args.world_size, rank=args.rank)/a\          import atexit as _ax,
torch.distributed as _d; _ax.register(lambda: _d.destroy_process_group() if _d.is_initialized()
else None)' main_tau.py
sed -i '/^          data_time.update(time.time() - end)/a\
          if i >= 20: break' main_tau.py
```

The `main_tau.py` will not have the code for the migration of memory without copy

Profiling – TAU setup

2 / 2

Make sure the TAU module is loaded

```
module list 2>&1 | grep tau
```

NOTE: we are using mpirun here instead of mp.spawn

Bind each MPI rank to a GPU.

```
unset TAU_PROFILE_FORMAT           # the tau module defaults this to "merged" (one tauprofile.xml);
                                   # unset it to get classic per-rank profile.<node>.<ctx>.<thread> files

export PROFILEDIR=$PWD/tau_trace
mkdir -p tau_trace
```

```
cat > tau_wrapper.sh <<'EOF'
#!/bin/bash
export HIP_VISIBLE_DEVICES=${OMPI_COMM_WORLD_LOCAL_RANK}
exec python main_tau.py -a resnet50 --dummy \
  --dist-url 'tcp://127.0.0.1:23457' --dist-backend nccl \
  --world-size ${OMPI_COMM_WORLD_SIZE} --rank ${OMPI_COMM_WORLD_RANK} \
  --gpu 0 -b 512 -p 20 --epochs 1
EOF
```

```
chmod +x tau_wrapper.sh
```

Collect the profile data

```
mpirun -n 4 --map-by numa --bind-to numa --report-bindings tau_exec -rocm ./tau_wrapper.sh
```

TAU specific part

Running TAU in the remote desktop

- Dump the per-rank profile as text, then let's produce a figure we can analyze:

```
ls tau_trace/profile.* # sanity: four per-rank profiles must exist before pprof
```

```
cd tau_trace
```

```
pprof -a > tau_pprof.txt      # per-rank profile as text
```

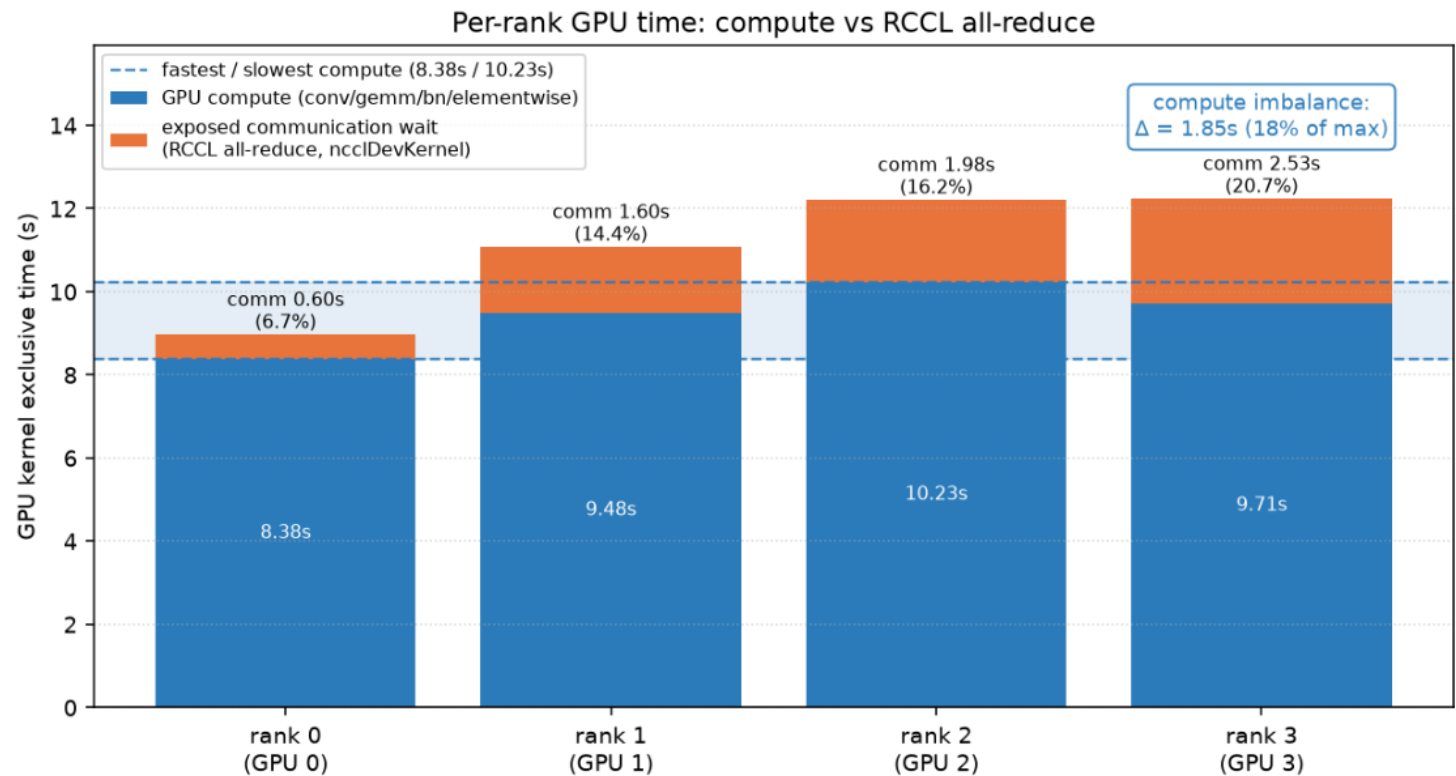
```
IMAGENET=$(git rev-parse --show-toplevel)/MExamples/Pytorch/imagenet
```

```
python3 "$IMAGENET/profiling/render_tau_profile.py" tau_pprof.txt \  
  --out "$IMAGENET/figs/tau_profile_4gpu.png"
```

Profiling – TAU performance plots

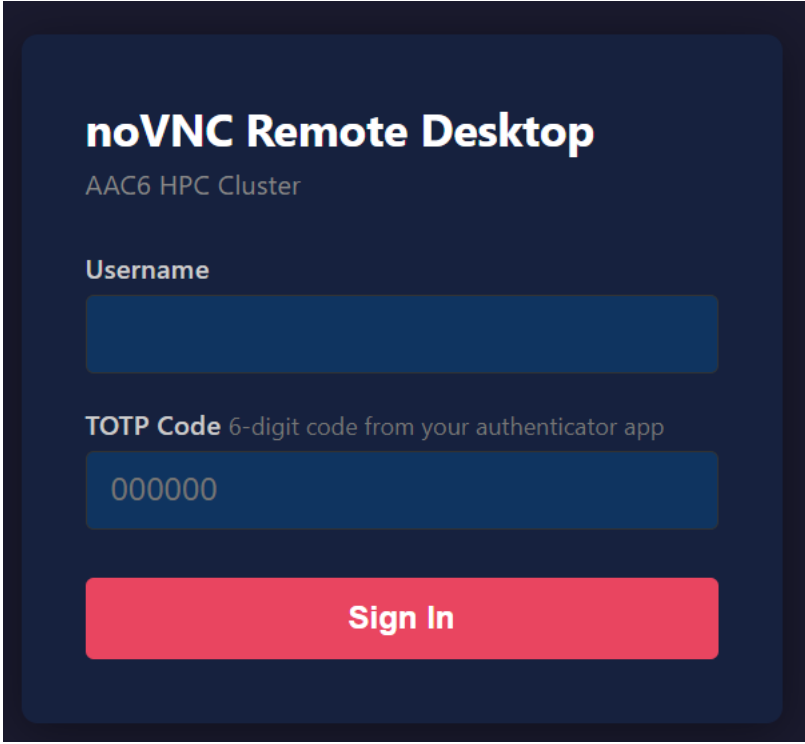
The pprof -a per-rank breakdown splits each rank's GPU-kernel exclusive time into two buckets: the RCCL all-reduce (ncc1DevKernel) and the ResNet-50 compute kernels (conv / GEMM / batch-norm / elementwise). We render this as one stacked bar per rank (blue = compute, orange = all-reduce), with a shaded band spanning the fastest to slowest rank's compute:

TAU (ParaProf profile): 4-rank ImageNet on MI300A -- per-rank GPU time (compute vs RCCL all-reduce)



Graphics desktop

- Create an ssh tunnel from your laptop/workstation
`ssh -L 6443:10.194.42.31:6443 <username>@aac6.amd.com`
- Enter your TOTP
- Get a node allocation
`salloc -p PPAC_MI300A_SPX -N1 --gpus=4 -t 08:00:00`
- Notice the node that you were allocated
`salloc: Nodes ppac-pl1-s24-16 are ready for job`
- Open a browser on your laptop/workstation, and enter in the navigation pane
https://localhost:6443/novnc/<node_name>/vnc.html
e.g. <https://localhost:6443/novnc/ppac-pl1-s24-16/vnc.html>
- A login screen should show up. You will likely get a warning and need to approve connecting. Enter your username and TOTP.
- shutdown order (close browser tab → release job → exit SSH, else SSH hangs)



noVNC Remote Desktop
AAC6 HPC Cluster

Username

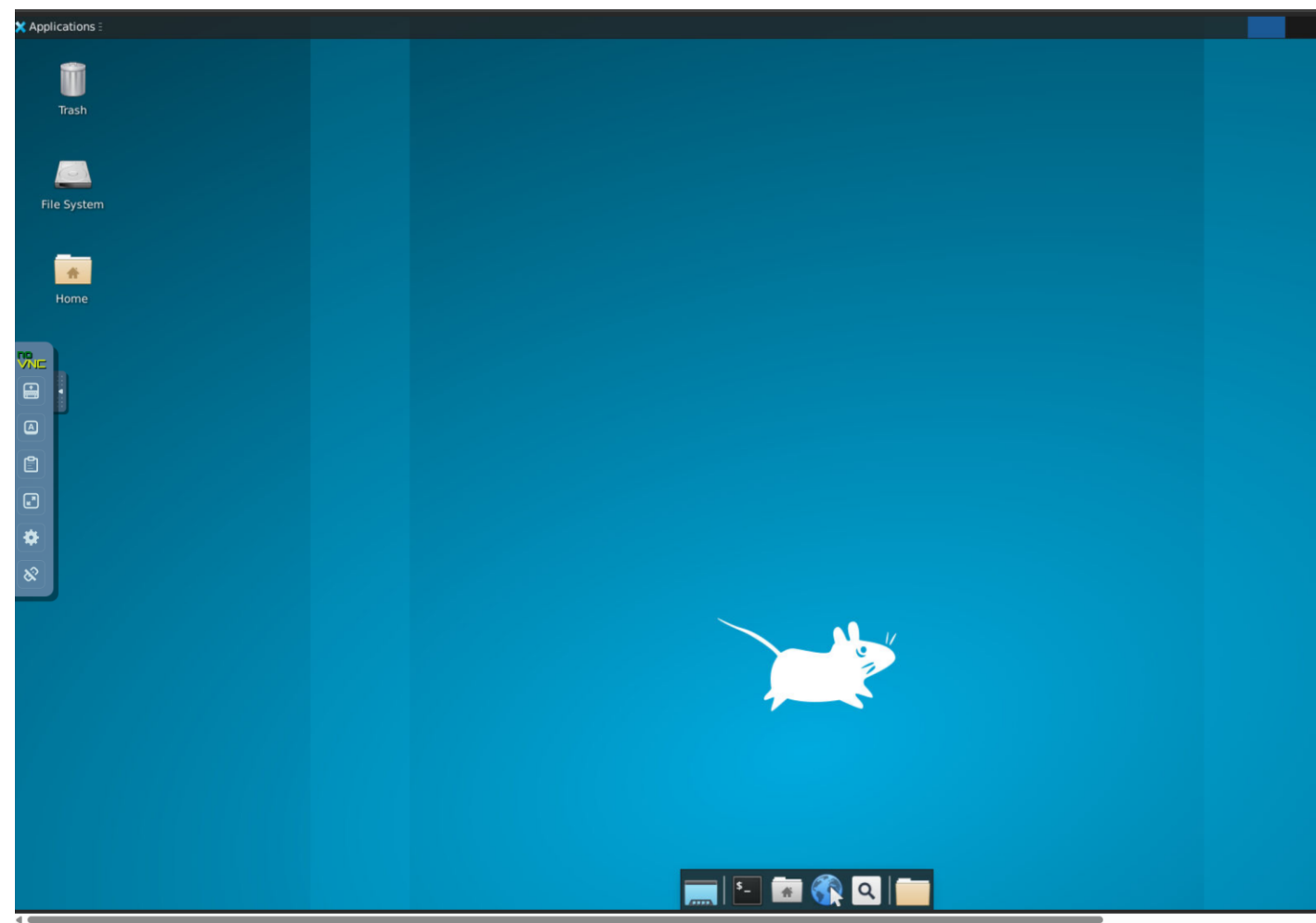
TOTP Code 6-digit code from your authenticator app

Sign In



The noVNC Desktop

- Applications menu is at the very top left of the desktop
- Right click the desktop to get a quick app menu
- Workplace switch is at the top right – having more virtual desktops helps to organize work
- Left edge of desktop has many setting options including a full-screen and a copy/paste in the clipboard panel



Roofline Extractor

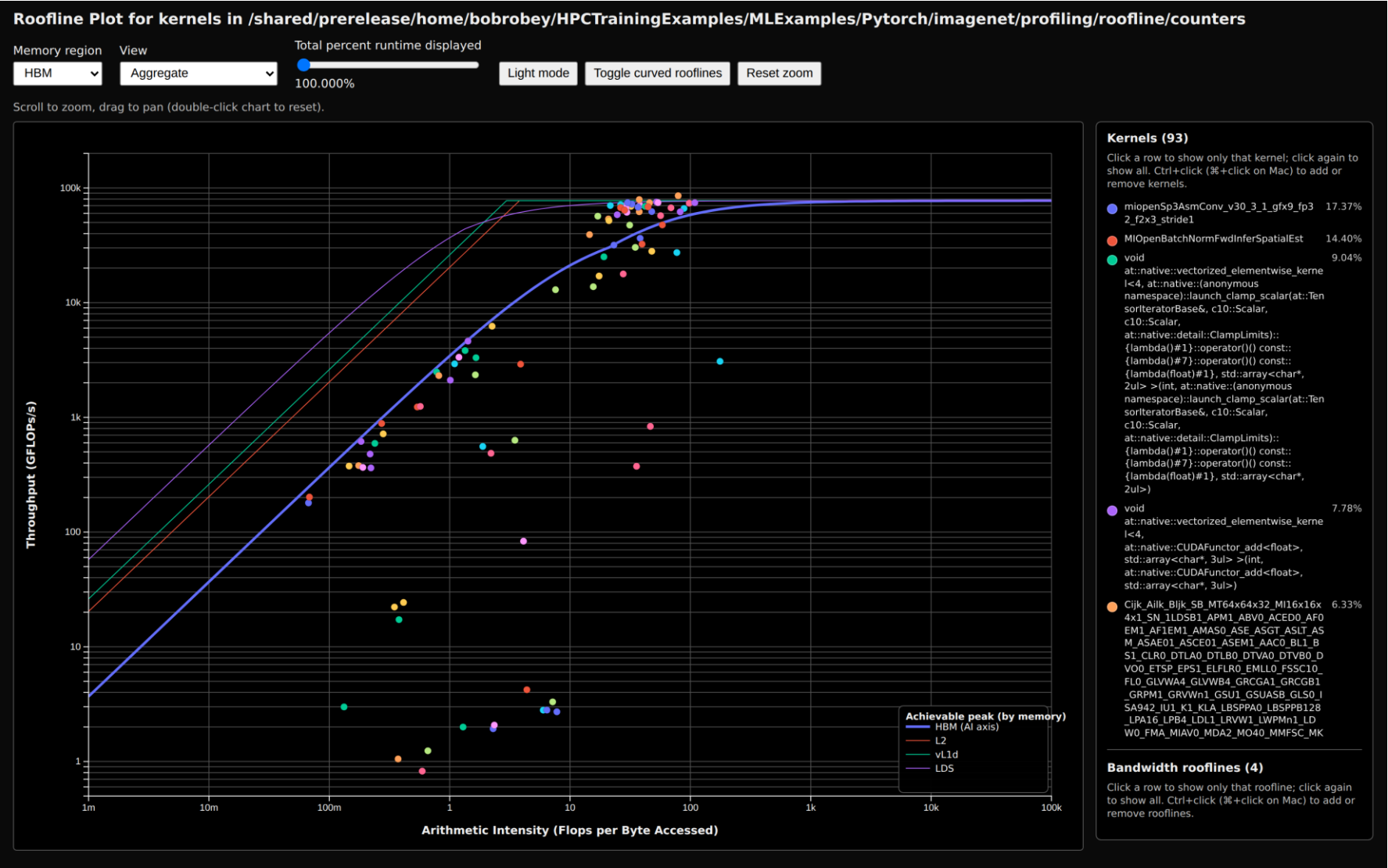
- [Roofline extractor](#) shows the performance of each GPU kernel plotted against the compute and bandwidth limits for the GPU. To run the batch script:

```
sbatch profiling/capture_roofline.sbatch
```

- The script basically loads the roofline extractor and reruns the imagenet example for 10 iterations on a single GPU

```
module load roofline-extractor          # puts roofline-extractor-* on $PATH
HIP_VISIBLE_DEVICES=0 roofline-extractor-profile -o profiling/roofline --arch MI300A -- \
    python main.py -a resnet50 --dummy --gpu 0 -b 256 -p 10 --epochs 1
```

Roofline Extractor



The roofline extractor window is an interactive html page. Open this page in a noVNC/XFCE desktop to explore more fully

Hands-on break

1. Keep looking at the instructions in the [README.md](#) from Section 11.2)
2. Section 11.2) covers TAU
 - Go through the commands we have shown and generate your own picture
3. Try running TAU in the remote desktop with `paraprof`
 - Follow the instructions in the [Appendix](#)
4. Section 12) covers the `roofline-extractor`
 - Produce a roofline plot
 - Open the generated html file with noVNC



Conjugate Gradient Example

Learning Objectives

1. Understanding communication patterns in Conjugate Gradient (CG)
 1. Unstructured point-to-point in SpMV
 2. Collective in global reduction
2. Look at the performance impact of different communication strategies
 1. Host-staging GPU buffers
 2. GPU-Aware MPI
 3. RCCL
3. Measure behavior with TAU and profile traces

Conjugate Gradient Iteration

$$\mathbf{r}_0 = \mathbf{b} - \mathbf{A} \mathbf{x}_0$$

$$\mathbf{p}_0 = \mathbf{r}_0$$

for $i = 0, 1, 2, \dots$:

$$\alpha = (\mathbf{r}_i, \mathbf{r}_i) / (\mathbf{A}\mathbf{p}_i, \mathbf{p}_i)$$

$$\mathbf{x} += \alpha \mathbf{p}_i$$

$$\mathbf{r}_{i+1} = \mathbf{r}_i - \alpha \mathbf{A}\mathbf{p}_i$$

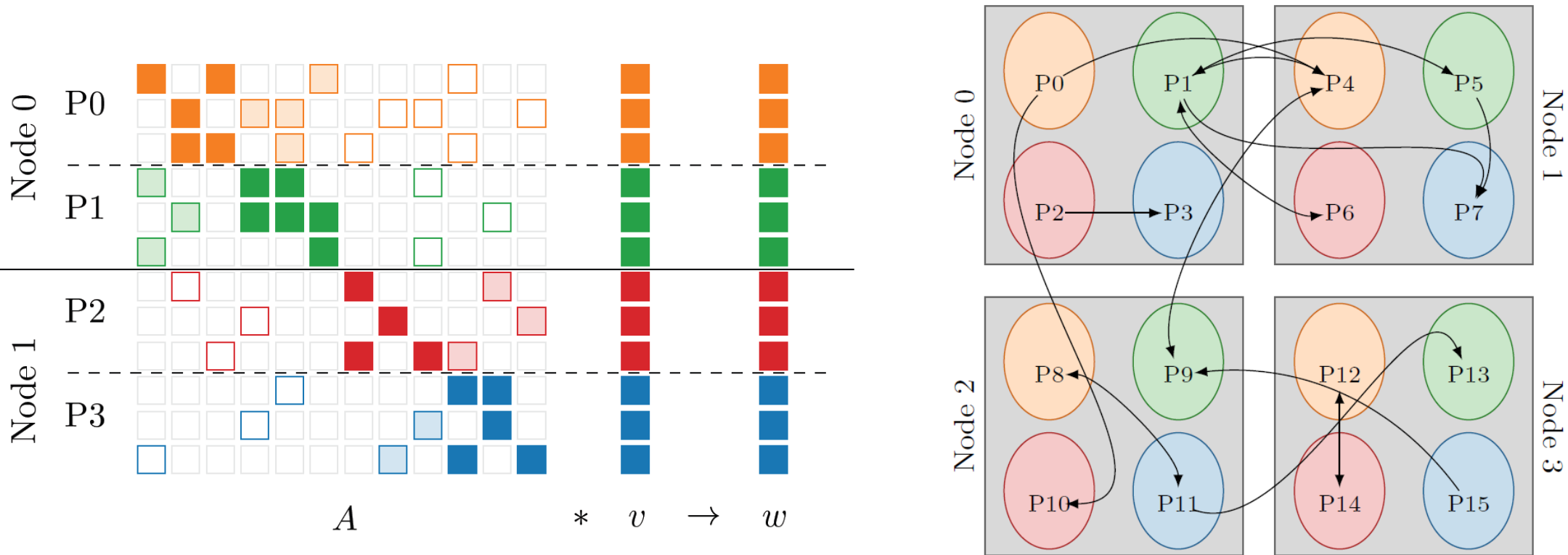
$$\beta = (\mathbf{r}_{i+1}, \mathbf{r}_{i+1}) / (\mathbf{r}_i, \mathbf{r}_i)$$

$$\mathbf{p}_{i+1} = \mathbf{r}_{i+1} + \beta \mathbf{p}_i$$

Single SpMV
per iteration

Inner Products

Conjugate Gradient SpMV Communication

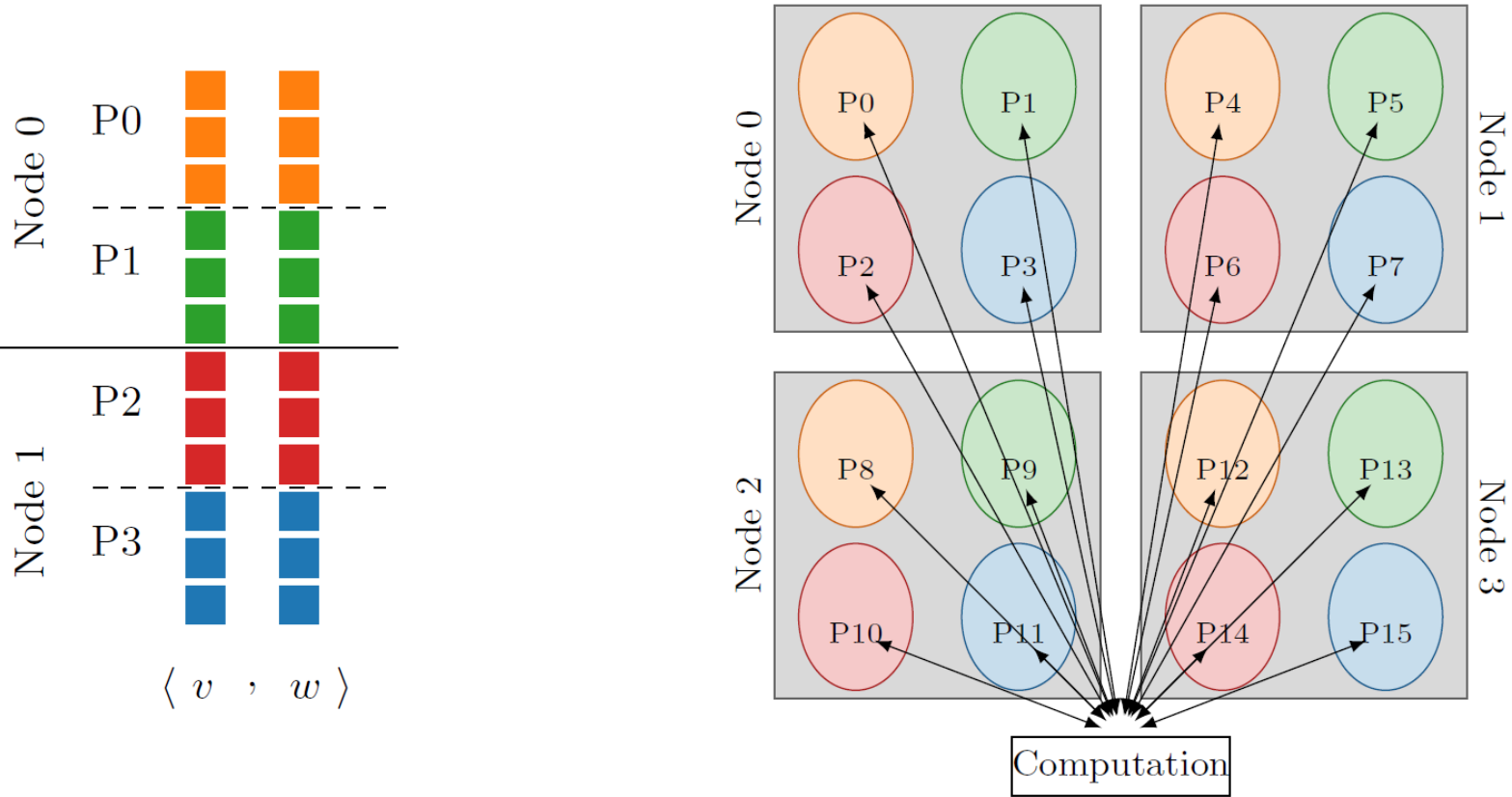


Sparse Matrix Vector Multiply



Unstructured Point-to-Point

Conjugate Gradient Inner Product Communication



Inner Product $\longrightarrow$ Collective Communication

```

static void spmv_isend_irecv(double alpha, GPUParMat& A, double* d_x,
                             double beta, double* d_b,
                             rocsparse_handle handle)
{
    int tag = 0;
    Comm& recv = *A.recv_comm;
    Comm& send = *A.send_comm;

    // Post receives into GPU memory
    for (int i = 0; i < recv.n_msgs; i++)
        MPI_Irecv(A.d_recvbuf + recv.ptr[i],
                  recv.ptr[i+1] - recv.ptr[i],
                  MPI_DOUBLE, recv.procs[i], tag,
                  MPI_COMM_WORLD, &recv.req[i]);

    // Gather send values on GPU: d_sendbuf[i] = d_x[ d_send_idx[i] ]
    if (send.size_msgs > 0) {
        ROCSPARSE_CHECK(rocsparse_dgthr(
            handle, send.size_msgs, d_x, A.d_sendbuf, A.d_send_idx,
            rocsparse_index_base_zero));
        HIP_CHECK(hipDeviceSynchronize()); // gather must finish before MPI reads d_sendbuf
    }

    // Post sends from GPU memory
    for (int i = 0; i < send.n_msgs; i++)
        MPI_Isend(A.d_sendbuf + send.ptr[i],
                  send.ptr[i+1] - send.ptr[i],
                  MPI_DOUBLE, send.procs[i], tag,
                  MPI_COMM_WORLD, &send.req[i]);

    // On-proc spmv overlaps with in-flight communication
    spmv(alpha, A.on_proc, d_x, beta, d_b, handle);

    if (recv.n_msgs)
        MPI_Waitall(recv.n_msgs, recv.req.data(), MPI_STATUSES_IGNORE);

    spmv(alpha, A.off_proc, A.d_recvbuf, 1.0, d_b, handle);

    if (send.n_msgs)
        MPI_Waitall(send.n_msgs, send.req.data(), MPI_STATUSES_IGNORE);
}

```

Common Distributed SpMV Implementation

Post MPI_Irecvs for required off-rank values of global vector

Post MPI_Isends for to partner ranks requiring on-rank values of global vector

Fully local computation

Wait for Irecvs

Computation requiring off-rank values

Wait for Isends

Alltoallv SpMV Implementation – Not Common

```
static void spmv_alltoallv(double alpha, GPUParMat& A, double* d_x,
                          double beta, double* d_b,
                          rocsparse_handle handle)
{
    Comm& send = *A.send_comm;

    // Gather send values on GPU
    if (send.size_msgs > 0) {
        ROCSPARSE_CHECK(rocsparse_dgthr(
            handle, send.size_msgs, d_x, A.d_sendbuf, A.d_send_idx,
            rocsparse_index_base_zero));
        HIP_CHECK(hipDeviceSynchronize());
    }

    // Submit on-proc SpMV to GPU (async); CPU continues to MPI_Alltoallv
    spmv(alpha, A.on_proc, d_x, beta, d_b, handle);

    // Exchange all ghost values in one collective (GPU-Aware MPI).
    // While the CPU is blocked here, the GPU runs on-proc SpMV in parallel.
    MPI_Alltoallv(A.d_sendbuf, A.a2a_sendcounts.data(), A.a2a_sdispls.data(),
        MPI_DOUBLE,
        A.d_recvbuf, A.a2a_recvcounts.data(), A.a2a_rdispls.data(),
        MPI_DOUBLE, MPI_COMM_WORLD);

    spmv(alpha, A.off_proc, A.d_recvbuf, 1.0, d_b, handle);
}
```

Pack the send buffer

Fully local computation

Exchange all the data at once

Computation requiring off-rank values

Distributed SpMV Communication Variants

1. GPU-Aware MPI with staged-through-host

1. Standard point-to-point overlapping
2. AlltoAllv before any computation

2. GPU-Aware MPI with device buffers

1. Standard point-to-point overlapping
 - HSA_ENABLE_SDMA=0
 - HSA_ENABLE_SDMA=1
2. AlltoAllv before any computation
 - HSA_ENABLE_SDMA=0
 - HSA_ENABLE_SDMA=1

3. RCCL ncclSend/ ncclRecv

```

=== staged ===
method=staged ranks=6 gpus_visible=1
Initial residual: 196.003
169 iterations to converge
2-norm of residual: 0.000195125
CG solve time:      0.0735 s (0.0004 s/iter)

=== alltoallv_staged ===
method=alltoallv_staged ranks=6 gpus_visible=1
Initial residual: 196.921
171 iterations to converge
2-norm of residual: 0.000187107
CG solve time:      0.0689 s (0.0004 s/iter)

=== rccl ===
method=rccl ranks=6 gpus_visible=1
Initial residual: 195.876
172 iterations to converge
2-norm of residual: 0.000191207
CG solve time:      0.1096 s (0.0006 s/iter)

=== isend HSA_ENABLE_SDMA=1 (sdma) ===
method=isend ranks=6 gpus_visible=1
Initial residual: 196.458
174 iterations to converge
2-norm of residual: 0.00018265
CG solve time:      0.0442 s (0.0003 s/iter)

```

Hands-On Exercises

Distributed CG Solver Example

A distributed (MPI) Conjugate Gradient (CG) solver used as a tutorial for studying **communication performance** on GPU clusters. The same sparse linear system is solved two ways — a plain CPU reference implementation and a GPU-accelerated port with **five interchangeable communication strategies** — so you can directly measure what GPU-Aware MPI, RCCL, and different collective/point-to-point patterns cost or save.

The test problem is `Dubcova2` (65 536 × 65 536, symmetric positive-definite), partitioned across MPI ranks and solved with the standard CG iteration:

```
r0 = b - A x0
p0 = r0
for i = 0, 1, 2, ...:
    α = (rᵢ, rᵢ) / (Apᵢ, pᵢ)
    x += α pᵢ
    r = b - A x      (recomputed every 8 iterations for numerical stability)
    β = (rᵢ₊₁, rᵢ₊₁) / (rᵢ, rᵢ)
    p = r + β p
```

Each iteration needs a distributed sparse matrix–vector product (SpMV), which requires exchanging “ghost” values between neighboring ranks — that halo exchange, plus the `MPI_Allreduce` for the dot products, is the communication this tutorial exists to study.

Directories

Directory	What it is
CG-CPU/	Reference implementation. Plain C++ + MPI, <code>std::vector</code> for vectors, hand-written SpMV/axpy/dot loops. No GPU, no external math libraries. Written by Amanda Bienz, UNM (source).
CG-GPU/	GPU port. Same algorithm, but vectors live on the GPU (<code>hipMalloc</code>), SpMV/dot/axpy use rocSPARSE/rocBLAS, and the halo exchange can run through any of 5 communication variants : CPU-staged <code>Isend/Irecv</code> , GPU-Aware <code>Isend/Irecv</code> , RCCL, and CPU-staged or GPU-Aware <code>MPI_Alltoallv</code> .

```
git clone https://github.com/Communication-
Performance-Tutorial/cg-solver-example.git

cd cg-solver-example/CG-CPU && make

cd ../cg-solver-example CG-GPU && make
```

Hands-On Exercises

Distributed Conjugate Gradient GPU-Based Example

A GPU-accelerated, MPI-distributed Conjugate Gradient (CG) solver.

This directory is a direct companion to `CG-CPU/` — it solves the same problem with the same algorithm but moves every num to the GPU and uses GPU-Aware MPI so communication buffers live in GPU memory the whole time.

What changed from `CG-CPU/`

CG-CPU/	CG-GPU/
<code>std::vector<double></code> for vectors	<code>hipMalloc / hipMemcpy</code> for device vectors
Hand-written SpMV loop	<code>rocsparse_spmv</code> (rocSPARSE generic API)
Hand-written gather (pack send buf)	<code>rocsparse_dgthr</code> (rocSPARSE gather)
Hand-written dot product	<code>rocbblas_ddot</code> (rocBLAS)
Hand-written axpy / scale	<code>rocbblas_daxpy / rocbblas_dscal</code> (rocBLAS)
<code>MPI_Irecv/Isend</code> with host buffers	<code>MPI_Irecv/Isend</code> with GPU pointers (GPU-Aware)
<code>MPI_Allreduce</code> on host scalar	Same (result copied from GPU → host, then allreduced)

The two header files (`sparse_mat.hpp`, `par_binary_IO.hpp`) are identical to the originals — file I/O and comm pattern setups

File layout

CG-Tutorial/	
├─ Makefile	
├─ README.md	
└─ src/	
├─ cg.cpp	← GPU solver (the interesting file)
├─ sparse_mat.hpp	← CPU structs + MPI comm setup (same as CG/)
├─ par_binary_IO.hpp	← parallel binary matrix reader (same as CG/)
└─ Dubcova2.pm	← test matrix (65 536 × 65 536, SPD)

```
cd cg-solver-example/CG-GPU

sbatch run_test.sh

# or interactively
mpirun -n 4 ./cg_gpu src/Dubcova2.pm staged

mpirun -n 4 ./cg_gpu src/Dubcova2.pm isend

mpirun -n 4 ./cg_gpu src/Dubcova2.pm rccl

mpirun -n 4 ./cg_gpu src/Dubcova2.pm alltoallv_staged

mpirun -n 4 ./cg_gpu src/Dubcova2.pm alltoallv
```

Play around with different environment variables and communication methods

Hands-On Exercises

Profiling with TAU

TAU intercepts MPI and, with `-rocm`, ROCm/HIP GPU calls via `LD_PRELOAD` — this directly measures the **communication** that GPU-only tools (rocpfv3, roofline, etc.) don't see: per-call MPI time, the per-rank point-to-point **communication matrix**, and (with `-rocm`) the ROCm runtime/kernel time in the same profile. No special instrumented build is needed — `tau_exec` wraps the plain `./cg_gpu` binary at run time.

```
module load rocm/6.4.3
module load openmpi/5.0.10-uccl1.6.0-ucx1.19.1-xpmem-2.7.4
module load tau/dev # layered on top of rocm - load it last

make
mpirun -n 4 --bind-to none bash set_affinity_mi300a.sh \
    tau_exec -T MPI,ROCM -rocm ./cg_gpu src/Dubcova2.pm rccl

pprof # text summary: per-call MPI + ROCm/HSA time, merged across ranks
```

`run_tau.sh` automates this end-to-end as an `sbatch` job (build → run under `tau_exec` → `pprof` summary), since the login node has no GPU:

```
sbatch run_tau.sh # profile 4 ranks, method=rccl
METHOD=isend sbatch run_tau.sh # profile a different comm. variant
TRACE=1 sbatch run_tau.sh # also write an OTF2 event trace
sbatch -p PPAC_MI300A_SPX --gpus=4 --ntasks=4 run_tau.sh # override partition/GPU count
```

Verified on AAC6/MI300A (ROCm 6.4.3, `tau/dev`): the profile below shows real per-call MPI time (`MPI_Init`, `MPI_Allreduce`, ...) merged with ROCm HSA runtime and kernel time (`rocparse::gthr_kernel`, `rocblas_dot_kernel_*`, ...) in one `pprof` summary:

%Time	Exclusive	Inclusive	#Call	#Subrs	Inclusive Name
100.0	34	8,976	1	1	8976004 .TAU application
67.3	6,038	6,038	1	0	6038301 MPI_Init()
0.2	7	20	306	306	69 MPI_Allreduce()
0.0	0.0688	0.0688	25.05	0	3 [ROCm Kernel] rocparse::gthr_kernel<...>

Known teardown quirk (ROCm 6.4.3 + `tau/dev`). The instrumented run reliably prints the CG solve result and writes every `prof` then the *process teardown* aborts with `corrupted size vs. prev_size in fastbins` (a rocprof sdk + glibc interaction, not a measu

```
cd cg-solver-example/CG-GPU
```

```
sbatch run_test.sh
```

```
# profile, method=rccl, 4 ranks
```

```
sbatch run_tau.sh
```

```
# profile a different comm. variant
```

```
METHOD=isend sbatch run_tau.sh
```

```
# also produce a perfetto trace
```

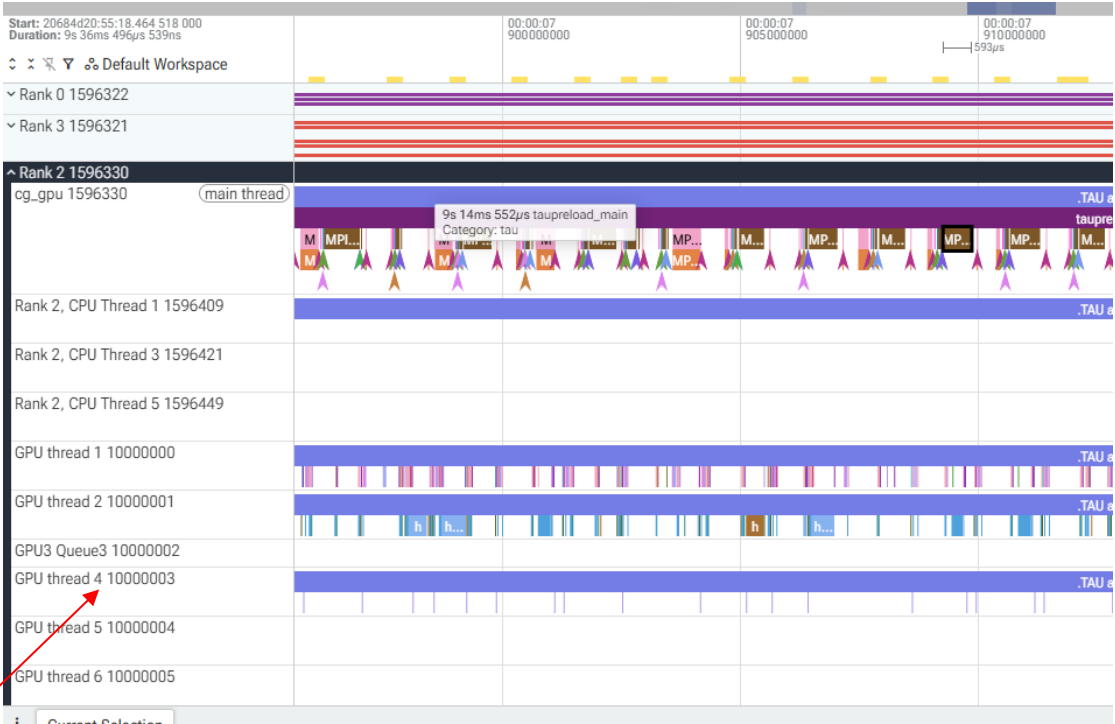
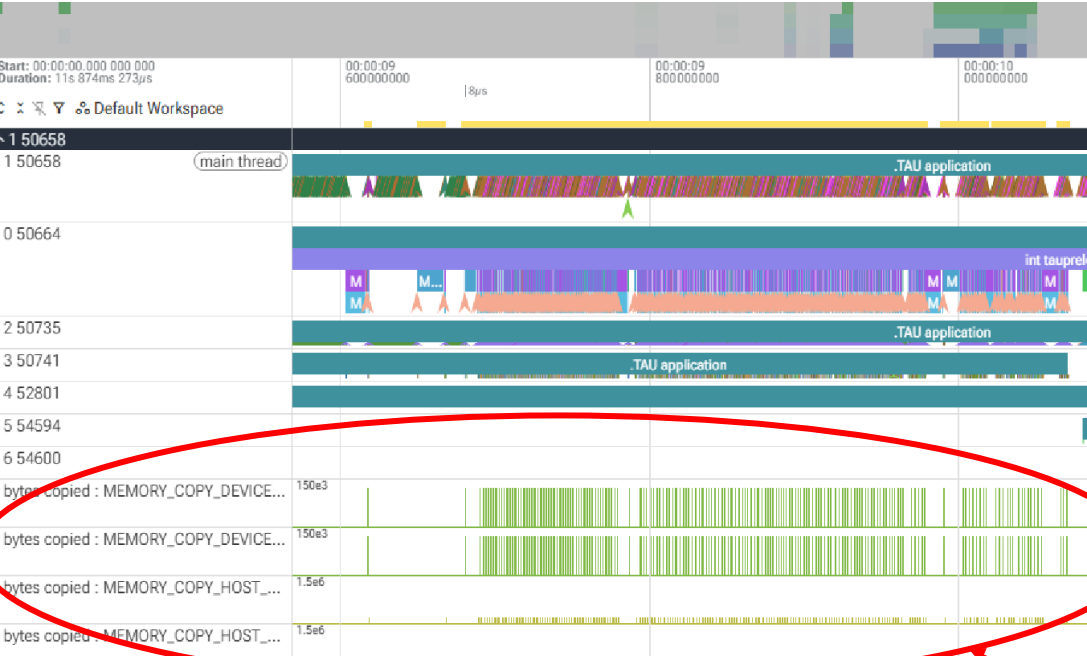
```
TRACE=1 sbatch run_tau.sh
```

Look at the tau breakdown for various implementations – where is time being spent?

TAU Traces for CG

```
TRACE=1 METHOD=staged sbatch run_tau.sh
```

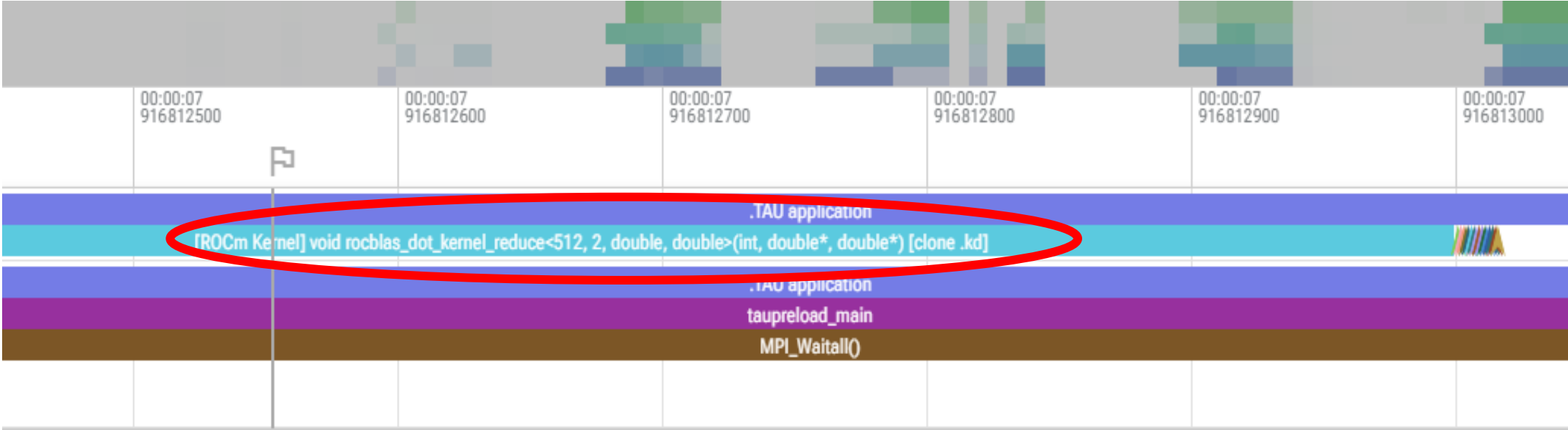
```
TRACE=1 METHOD=isend sbatch run_tau.sh
```



Notice the absence of data movement on the right

TAU Traces for CG

```
TRACE=1 METHOD=isend sbatch run_tau.sh
```



Look closely at the traces to see which knobs enable or disable communication and computation overlap

Hands-on break

1. Start with the instructions in the [README.md](#) from the cg-solver-example git repo
2. [CG-CPU](#) includes the reference CPU-based CG implementation
 - Try running this example first before looking at the GPU-based code
3. [CG-GPU](#) includes the GPU-based CG implementation with various communication options for the SpMV
 - Follow the instructions in the [README.md](#)
 - Run the example without TAU profiling, looking at the performance of the different communication strategies
 - Run with TAU profiling enabled
 - Do the generated profiles look like you expected?
 - What are the message sizes? Correlate the performance you see in the profiles with the micro-benchmark results you generated earlier. Do they match?



Disclaimer

The information presented in this document is for informational purposes only and may contain technical inaccuracies, omissions, and typographical errors. The information contained herein is subject to change and may be rendered inaccurate for many reasons, including but not limited to product and roadmap changes, component and motherboard version changes, new model and/or product releases, product differences between differing manufacturers, software changes, BIOS flashes, firmware upgrades, or the like. Any computer system has risks of security vulnerabilities that cannot be completely prevented or mitigated. AMD assumes no obligation to update or otherwise correct or revise this information. However, AMD reserves the right to revise this information and to make changes from time to time to the content hereof without obligation of AMD to notify any person of such revisions or changes.

THIS INFORMATION IS PROVIDED 'AS IS.' AMD MAKES NO REPRESENTATIONS OR WARRANTIES WITH RESPECT TO THE CONTENTS HEREOF AND ASSUMES NO RESPONSIBILITY FOR ANY INACCURACIES, ERRORS, OR OMISSIONS THAT MAY APPEAR IN THIS INFORMATION. AMD SPECIFICALLY DISCLAIMS ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, OR FITNESS FOR ANY PARTICULAR PURPOSE. IN NO EVENT WILL AMD BE LIABLE TO ANY PERSON FOR ANY RELIANCE, DIRECT, INDIRECT, SPECIAL, OR OTHER CONSEQUENTIAL DAMAGES ARISING FROM THE USE OF ANY INFORMATION CONTAINED HEREIN, EVEN IF AMD IS EXPRESSLY ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

© 2026 Advanced Micro Devices, Inc. All rights reserved. AMD, the AMD Arrow logo, ROCm, Radeon, Instinct and combinations thereof are trademarks of Advanced Micro Devices, Inc. Other product names used in this publication are for identification purposes only and may be trademarks of their respective companies.

Intel is a trademark of Intel Corporation or its subsidiaries.



Hot Interconnects 2026 - Understanding Communication Performance on Emerging Heterogeneous Architectures

<https://github.com/amd/HPCTrainingExamples>

*Shelby Lockhart (AMD) - Amanda Bienz (UNM) - Bob Robey (AMD)
Giacomo Capodaglio (AMD) - Sameer Shende (UO) - Patrick Bridges (UNM)*

Aug 19-21 2026

Contents

Distributed CG Solver Example	3
Directories	4
Quick start	4
What this tutorial is for	5
CPU Conjugate Gradient — Reference Implementation	5
File layout	5
Environment Setup	5
How to build	5
How to run	6
What's in <code>cg.cpp</code>	6
Distributed SpMV outline	6
CG algorithm	6
What changes in CG-GPU/	6
Distributed Conjugate Gradient GPU-Based Example	7
What changed from CG-CPU/	7
File layout	7
Environment Setup	7
How to build	8
How to run	8
GPU assignment	8
Communication variants	9
Method 1 — <code>staged</code> (CPU-buffered Isend/Irecv)	9
Method 2 — <code>isend</code> (GPU-Aware Isend/Irecv)	9
Method 3 — <code>rccl</code> (RCCL ncclSend / ncclRecv)	9
Method 4 — <code>alltoallv_staged</code> (CPU-buffered MPI_Alltoallv)	9
Method 5 — <code>alltoallv</code> (GPU-Aware MPI_Alltoallv)	9
Key concepts	9
1. One GPU per MPI rank	9
2. rocSPARSE SpMV instead of a CPU loop	10
3. GPU-Aware MPI — sending/receiving GPU buffers directly	10
4. rocBLAS for dot / axpy / scale	10
SDMA engines vs. blit kernels (HSA_ENABLE_SDMA)	10

When does each win?	11
Profiling with TAU	11
Perfetto / <code>chrome://tracing</code> timeline (TRACE=1)	12
Requirements	12
ImageNet DDP: measuring RCCL communication at scale	13
Contents	13
1. Get an allocation and load PyTorch	13
2. Get the examples	14
3. Optional modifications to the example source code	14
3a. Fix warning <code>destroy_process_group</code>	15
3b. Keep the demo (and the profiler trace) short.	15
3c. Add GPU peak memory instrumentation	15
3d. Enable the profiler and print total RCCL time	15
3e. Compare <code>.to</code> (copy) vs <code>.migrate</code> (zero-copy) staging	16
3f. Make <code>pin_memory</code> graceful for the fallback path	16
4. Warm the MIOpen cache (once per allocation)	17
5. Run the scaling sweep (assumes an SPX partition)	18
6. APU programming model (MI300A)	18
7. Read the two numbers (RCCL time, staging)	18
8. Calculating the performance	19
9. Featured RCCL optimization: tune the all-reduce with environment variables	19
10. Featured compute optimization: hands-on <code>main.py</code> edits	20
11. Featured profiling exercise: a measured timeline of compute vs communication	20
11.1 <code>torch.profiler</code> manual flow	20
11.2 <code>torch.profiler</code> automated flow	21
11.3 TAU manual flow	22
11.4 TAU automated flow	24
12. Featured profiling tool: roofline extractor (per-kernel compute vs. memory)	25
13. Cleanup	28
14. Run on CPX partitions (SH5_MI300A_CPX, PPAC_MI300A_CPX)	28
Next steps	28
Appendix A: noVNC desktop (view GUIs like ParaProf/Perfetto in the browser)	29
RCCL optimization exercises (hands-on <code>main.py</code> edits)	31
Setup	31
Baseline command (reuse this to assess the impact of each tip)	31
Section 1: Reduce the bytes on the wire	32
1a. bf16 gradient compression	32
Section 2: Tune the RCCL transport and algorithm	32
2a. Collective algorithm: <code>NCCL_ALGO</code>	32
2b. Wire protocol: <code>NCCL_PROTO</code>	32
2c. Channel count: <code>NCCL_MIN_NCHANNELS</code>	33
Section 3: Overlap and shrink the collectives (DDP knobs)	33
3a. Avoid a gradient copy: <code>gradient_as_bucket_view=True</code>	33
3b. Bucket size: <code>bucket_cap_mb</code>	33
3c. Static graph: <code>static_graph=True</code>	33
What moves what	34
Measured impact: batch study (NCCL algorithm / protocol / channels, 4 GPUs)	34
NCCL/RCCL variable defaults	35
Compute optimization exercises (hands-on <code>main.py</code> edits)	36
Setup	36
Baseline command (reuse this to assess the impact of each tip)	36

Section 1: Lower-precision math	36
1a. bfloat16 autocast (AMP)	36
1b. Reduced-precision float32 matmuls	37
Section 2: Memory layout and kernel selection	37
2a. <code>channels_last</code> (NHWC) memory format	37
2b. Autotune convolution kernels: <code>cuda.benchmark</code>	37
Section 3: Fuse kernels and cut launch overhead	37
3a. <code>torch.compile</code>	37
3b. Fused optimizer + cheaper <code>zero_grad</code>	38
Recommended stack	38
What moves what	38
Measured impact: batch study (<code>torch.compile</code> , 1 GPU)	39
ImageNet DDP benchmark: rigorous RCCL scaling study	40
Contents	40
Running on MI300A: required settings and measured optimization	40
The three fixes that make the test run	40
Recommended workflow	41
Runtime, version, and affinity	43
Measured results (MI300A, resnet50, per-GPU batch 128)	43
0. Get the example and an allocation	44
1. A single benchmark step (sanity check)	44
2. Confirm the RCCL topology	44
3. Scaling sweep (the RCCL measurement)	45
4. Pure RCCL bandwidth (compute-independent)	45
5. Attribute time to RCCL kernels (optional, precise)	46
6. Run it as a batch job	46
Useful RCCL environment variables	46
Alternative examples for studying RCCL scaling	47
<code>pytorch_examples/distributed/</code> : recommended companions	47
<code>MLEamples/TinyTransformer</code> : not suitable for RCCL scaling	48
Profiling the ImageNet DDP example: compute and RCCL communication	48
Which profiler for which job	48
Ground rules for trustworthy numbers	49
Quick reference	49

Distributed CG Solver Example

A distributed (MPI) Conjugate Gradient (CG) solver used as a tutorial for studying **communication performance** on GPU clusters. The same sparse linear system is solved two ways — a plain CPU reference implementation and a GPU-accelerated port with **five interchangeable communication strategies** — so you can directly measure what GPU-Aware MPI, RCCL, and different collective/point-to-point patterns cost or save.

The test problem is `Dubcova2` ($65\,536 \times 65\,536$, symmetric positive-definite), partitioned across MPI ranks and solved with the standard CG iteration:

```

r0 = b - A x0
p0 = r0
for i = 0, 1, 2, ...:
    α = (ri, ri) / (Api, pi)
    x += α pi
    r = b - A x          (recomputed every 8 iterations for numerical stability)
    β = (ri+1, ri+1) / (ri, ri)

```

$$\mathbf{p} = \mathbf{r} + \beta \mathbf{p}$$

Each iteration needs a distributed sparse matrix–vector product (SpMV), which requires exchanging “ghost” values between neighboring ranks — that halo exchange, plus the `MPI_Allreduce` for the dot products, is the communication this tutorial exists to study.

Directories

Directory	What it is
CG-CPU/	Reference implementation. Plain C++ + MPI, <code>std::vector</code> for vectors, hand-written SpMV/axpy/dot loops. No GPU, no external math libraries. Written by Amanda Bienz, UNM (source).
CG-GPU/	GPU port. Same algorithm, but vectors live on the GPU (<code>hipMalloc</code>), SpMV/dot/axpy use <code>rocSPARSE/rocBLAS</code> , and the halo exchange can run through any of 5 communication variants : CPU-staged <code>Isend/Irecv</code> , GPU-Aware <code>Isend/Irecv</code> , <code>RCCL</code> , and CPU-staged or GPU-Aware <code>MPI_Alltoallv</code> .

CG-GPU/ is built directly on top of CG-CPU/ — reading the two side by side shows exactly what changes when a solver moves from the CPU to the GPU, and then what changes again as GPU-Aware MPI and RCCL are layered on top of that.

Quick start

```
# CPU reference
cd CG-CPU
salloc -N 1 --ntasks=4 --gpus-per-task=1 -p SH5_MI300A_CPX
module load rocm/6.4.1
module load openmpi/5.0.10-ucc1.6.0-ucx1.19.1-xpmem-2.7.4
make
mpirun -n 4 ./cg_cpu src/Dubcova2.pm
exit

# GPU solver
cd ../CG-GPU
salloc -N 1 --ntasks=4 --gpus-per-task=1 -p SH5_MI300A_CPX
module load rocm/6.4.1
module load openmpi/5.0.10-ucc1.6.0-ucx1.19.1-xpmem-2.7.4
make
mpirun -n 4 ./cg_gpu src/Dubcova2.pm rccl
exit
```

See each directory’s README for the full build/run instructions, environment setup, and a breakdown of every communication variant.

What this tutorial is for

Comparing runs across CG-CPU/ and the different CG-GPU/ communication variants answers questions like:

- How much does GPU-Aware MPI save over staging through host (pinned) buffers?
- Is RCCL (GPU-native, Infinity-Fabric-aware) faster than MPI point-to-point for this halo pattern?
- Does replacing many small point-to-point messages with one `MPI_Alltoallv` collective help or hurt?
- Do SDMA engines or blit kernels move ghost data faster for this message size (`HSA_ENABLE_SDMA`)?

CG-GPU/ also has scripts for sweeping these variants automatically (`run_test.sh`) and for profiling the run with TAU — see `CG-GPU/README.md` for per-call MPI/ROCm timing and the communication matrix.

CPU Conjugate Gradient — Reference Implementation

A distributed, CPU-only Conjugate Gradient (CG) solver. This is the starting point for the tutorial: every operation is written in plain C++ with MPI, using only `std::vector` and hand-written loops. No GPU, no external math libraries.

Note: This implementation was written and provided by Amanda Bienz, UNM. Her main repo is located at: <https://github.com/bienz2/CG>.

CG-GPU/ is built directly on top of this implementation — comparing the two side by side shows exactly what changes when the solver is ported to the GPU.

File layout

```
CG-CPU/
--- Makefile
--- README.md
└-- src/
    --- cg.cpp           ← solver (the interesting file)
    --- sparse_mat.hpp   ← CPU structs + MPI comm setup
    --- par_binary_IO.hpp ← parallel binary matrix reader
    └-- Dubcova2.pm      ← test matrix (65 536 × 65 536, SPD)
```

Environment Setup

```
module load openmpi/5.0.10-ucc1.6.0-ucx1.19.1-xpmem-2.7.4
```

Any MPI build that provides `mpicxx` works; the solver has no GPU dependency.

How to build

```
cd CG-CPU
make
```

How to run

```
salloc -N 1 --ntasks=4 --gpus-per-task=1 -p SH5_MI300A_CPX
mpirun -n 4 ./cg_cpu src/Dubcova2.pm
exit
```

Expected output:

```
172 iterations to converge
2-norm of residual: 0.000190
CG solve time:      0.1588 s  (0.0009 s/iter)
```

The iteration count, residual, and solve time vary slightly between runs because the right-hand side vector $\mathbf{b}$ is generated from a random $\mathbf{x}_0$. The solve time covers the CG loop only and is reported as the maximum across all MPI ranks.

What's in cg.cpp

Function	Description
spmv(Mat&, ...)	Serial sparse matrix-vector product (CSR row loop)
spmv(ParMat&, ...)	Distributed SpMV: non-blocking Isend/Irecv, then on-proc + off-proc SpMV
inner_product	Local dot product reduced across ranks with MPI_Allreduce
axpy	$\mathbf{x} += \alpha * \mathbf{y}$ (CPU loop)
scale	$\mathbf{x} *= \alpha$ (CPU loop)
main	Reads the matrix, forms the comm pattern, runs CG

Distributed SpMV outline

1. Post non-blocking receives for ghost values (MPI_Irecv)
2. Pack and post non-blocking sends (CPU loop + MPI_Isend)
3. Compute on-proc SpMV (overlaps with in-flight messages)
4. Wait for receives to complete (MPI_Waitall)
5. Compute off-proc SpMV using received ghosts
6. Wait for sends to complete (MPI_Waitall)

CG algorithm

```
r0 = b - A x0
p0 = r0
for i = 0, 1, 2, ...:
    α = (ri, ri) / (A pi, pi)
    x += α pi
    r = b - A x      (recomputed every 8 iterations for numerical stability)
    β = (ri+1, ri+1) / (ri, ri)
    p = r + β p
```

What changes in CG-GPU/

This file (CPU)	CG-GPU/ (GPU)
<code>std::vector<double></code> for vectors	<code>hipMalloc</code> / <code>hipMemcpy</code> for device vectors
Hand-written SpMV loop	<code>rocsparse_spmv</code> (rocSPARSE generic API)
Hand-written pack loop (CPU)	<code>rocsparse_dgthr</code> (GPU gather)
Hand-written dot product	<code>rocblas_ddot</code> (rocBLAS)
Hand-written axpy / scale	<code>rocblas_daxpy</code> / <code>rocblas_dscal</code> (rocBLAS)
<code>MPI_Irecv/Isend</code> on host buffers	Choice of 5 GPU communication variants

Distributed Conjugate Gradient GPU-Based Example

A GPU-accelerated, MPI-distributed Conjugate Gradient (CG) solver.

This directory is a direct companion to `CG-CPU/` — it solves the same problem with the same algorithm but moves every numerical operation to the GPU and uses GPU-Aware MPI so communication buffers live in GPU memory the whole time.

What changed from `CG-CPU/`

CG-CPU/	CG-GPU/
<code>std::vector<double></code> for vectors	<code>hipMalloc</code> / <code>hipMemcpy</code> for device vectors
Hand-written SpMV loop	<code>rocsparse_spmv</code> (rocSPARSE generic API)
Hand-written gather (pack send buf)	<code>rocsparse_dgthr</code> (rocSPARSE gather)
Hand-written dot product	<code>rocblas_ddot</code> (rocBLAS)
Hand-written axpy / scale	<code>rocblas_daxpy</code> / <code>rocblas_dscal</code> (rocBLAS)
<code>MPI_Irecv/Isend</code> with host buffers	<code>MPI_Irecv/Isend</code> with GPU pointers (GPU-Aware)
<code>MPI_Allreduce</code> on host scalar	Same (result copied from GPU → host, then allreduced)

The two header files (`sparse_mat.hpp`, `par_binary_I0.hpp`) are **identical** to the originals — file I/O and comm pattern setup stay on the CPU.

File layout

```
CG-Tutorial/
--- Makefile
--- README.md
└-- src/
    --- cg.cpp           ← GPU solver (the interesting file)
    --- sparse_mat.hpp   ← CPU structs + MPI comm setup (same as CG/)
    --- par_binary_I0.hpp ← parallel binary matrix reader (same as CG/)
    └-- Dubcova2.pm      ← test matrix (65 536 × 65 536, SPD)
```

Environment Setup

```
module load rocm/6.4.1
module load openmpi/5.0.10-ucc1.6.0-ucx1.19.1-xpmem-2.7.4
```

ROCm provides hipcc, rocSPARSE, rocBLAS, and RCCL. The OpenMPI build must be compiled with ROCm/UCX support so that MPI can send and receive GPU pointers directly (GPU-Aware MPI).

How to build

```
cd CG-GPU
make
```

How to run

Pass the matrix file as the first argument and the communication method as the second (default: **staged**):

```
salloc -N 1 --ntasks=4 --gpus-per-task=1 -p SH5_MI300A_CPX
mpirun -n 4 ./cg_gpu src/Dubcova2.pm staged           # (default) Isend/Irecv through
↳ CPU host buffers
mpirun -n 4 ./cg_gpu src/Dubcova2.pm isend           # Isend/Irecv with GPU buffers
↳ (GPU-Aware)
mpirun -n 4 ./cg_gpu src/Dubcova2.pm rccl           # RCCL ncclSend/ncclRecv
mpirun -n 4 ./cg_gpu src/Dubcova2.pm alltoallv_staged # MPI_Alltoallv through CPU host
↳ buffers
mpirun -n 4 ./cg_gpu src/Dubcova2.pm alltoallv       # MPI_Alltoallv with GPU buffers
↳ (GPU-Aware)
exit
```

All five methods produce the same numerical result (the CG algorithm is identical; only the spmv data exchange differs).

Expected output (any method):

```
method=staged ranks=4 gpus_visible=24
Initial residual: 197.025
172 iterations to converge
2-norm of residual: 0.000196
CG solve time:      0.4724 s (0.0028 s/iter)
```

The iteration count, residual, and solve time vary slightly between runs because the right-hand side is seeded from a random initial guess. The solve time covers the CG loop only (matrix read, upload, and setup are excluded) and is reported as the maximum across all MPI ranks.

GPU assignment

The solver assigns one GPU per MPI rank using:

```
hipSetDevice(rank % num_gpus);
```

On a node with 4 MPI ranks and multiple visible GPUs, each rank gets a distinct GPU. To control which physical GPUs are used, set `ROCR_VISIBLE_DEVICES` before launching, or use a GPU affinity script with `mpirun --map-by NUMA`. See the Affinity part 2 blog post for guidance on matching MPI ranks to GPU NUMA domains.

Communication variants

Method 1 — staged (CPU-buffered Isend/Irecv)

The simplest baseline: all packing, communication, and unpacking go through the CPU. MPI operates on host pointers.

1. Copy the full local vector from GPU to host (`hipMemcpy D→H`)
2. Pack send values into a pinned host buffer (CPU loop)
3. `MPI_Irecv` / `MPI_Isend` on host pointers
4. After wait, copy received values back to GPU (`hipMemcpy H→D`)

Method 2 — isend (GPU-Aware Isend/Irecv)

Same point-to-point structure as `staged`, but ghost values never touch the CPU. `rocsparse_dgthr` packs the send buffer directly on the GPU, and `MPI_Isend` / `MPI_Irecv` operate on GPU pointers. On-proc SpMV overlaps with in-flight messages.

Comparing `staged` vs. `isend` shows the cost of the $D \rightarrow H + H \rightarrow D$ copies that GPU-Aware MPI eliminates.

Method 3 — rccl (RCCL ncclSend / ncclRecv)

RCCL (ROCm Collective Communications Library) provides GPU-native communication that can exploit Infinity Fabric / NVLink paths without any CPU involvement. Sends and receives are grouped with `ncclGroupStart` / `ncclGroupEnd` and launched on a dedicated `rccl_stream`. On-proc SpMV runs on the default GPU stream concurrently — true GPU-GPU overlap.

Method 4 — alltoallv_staged (CPU-buffered MPI_Alltoallv)

Replaces the many point-to-point messages with one collective, but uses explicit host staging:

1. `rocsparse_dgthr` packs into `d_sendbuf` on the GPU
2. `hipMemcpy D→H` → `h_sendbuf`
3. `MPI_Alltoallv` on host pointers
4. `hipMemcpy H→D` → `d_recvbuf`
5. On-proc SpMV overlaps with step 3 (CPU blocks in MPI, GPU computes)

Method 5 — alltoallv (GPU-Aware MPI_Alltoallv)

Same collective as `alltoallv_staged` but passes GPU pointers directly to `MPI_Alltoallv` — no host copies needed. On-proc SpMV is submitted to the GPU before the collective blocks the CPU, so both run in parallel.

Comparing `alltoallv_staged` vs. `alltoallv` isolates the PCIe round-trip cost ($D \rightarrow H + H \rightarrow D$) that GPU-Aware MPI avoids.

Key concepts

1. One GPU per MPI rank

```
hipSetDevice(rank % num_gpus);
```

Ranks round-robin across available GPUs. On a node with 4 MI300A partitions and 4 MPI ranks, each rank gets its own GPU.

2. rocSPARSE SpMV instead of a CPU loop

```
// CPU (original):
for (int i = 0; i < A.n_rows; i++)
    for (int j = A.rowptr[i]; j < A.rowptr[i+1]; j++)
        b[i] += alpha * A.data[j] * x[A.col_idx[j]];

// GPU (this file):
rocspmv(handle, rocspmv_operation_none,
        &alpha, A.descr, vec_x, &beta, vec_b, ...);
```

The rocSPARSE descriptor wraps the CSR arrays that were uploaded with `hipMemcpy`. rocSPARSE picks the best SpMV algorithm for the hardware.

3. GPU-Aware MPI — sending/receiving GPU buffers directly

```
// Pack the send buffer on the GPU (no host copy needed)
rocspmv_dgthr(handle, send_size, d_x, d_sendbuf, d_send_idx, ...);
hipDeviceSynchronize(); // gather must finish before MPI reads d_sendbuf

// Send and receive GPU pointers - the MPI runtime handles the rest
MPI_Isend(d_sendbuf + offset, count, MPI_DOUBLE, dest, ...);
MPI_Irecv(d_recvbuf + offset, count, MPI_DOUBLE, src, ...);
```

Without GPU-Aware MPI you would have to copy from GPU→host before sending and host→GPU after receiving. With GPU-Aware MPI you skip both copies.

4. rocBLAS for dot / axpy / scale

```
rocblas_ddot (blas_handle, n, d_a, 1, d_b, 1, &local_sum);
rocblas_daxpy(blas_handle, n, &alpha, d_y, 1, d_x, 1); // d_x += alpha*d_y
rocblas_dscal(blas_handle, n, &alpha, d_x, 1); // d_x *= alpha
```

After `rocblas_ddot`, a single `hipDeviceSynchronize()` makes the result visible on the CPU before `MPI_Allreduce` reduces it across ranks.

SDMA engines vs. blit kernels (HSA_ENABLE_SDMA)

When GPU-Aware MPI (`isend`, `alltoallv`) passes a GPU pointer to the MPI runtime, UCX must physically move the data out of device memory. ROCm gives it two mechanisms to do that:

Mechanism	HSA_ENABLE_SDMA	How it works
SDMA engines	1 (default)	Dedicated hardware DMA controllers transfer data between GPU memory and the fabric. They run independently of the shader engines and do not consume compute resources.

Mechanism	HSA_ENABLE_SDMA	How it works
Blit kernels	0	ROCm dispatches a small compute shader (a “blit”) to copy the data using the GPU’s shader engines. No dedicated DMA hardware is used.

Set the variable before `mpirun` to switch between them:

```
# SDMA engines (default)
HSA_ENABLE_SDMA=1 mpirun -n 8 --bind-to none bash set_affinity_mi300a.sh ./cg_gpu
↪ src/Dubcova2.pm isend

# Blit kernels
HSA_ENABLE_SDMA=0 mpirun -n 8 --bind-to none bash set_affinity_mi300a.sh ./cg_gpu
↪ src/Dubcova2.pm isend
```

`run_test_7.13.sh` sweeps both values automatically for `isend` and `alltoallv` and labels each run in the log output:

```
=== isend HSA_ENABLE_SDMA=1 (sdma) ===
=== isend HSA_ENABLE_SDMA=0 (blit_kernel) ===
=== alltoallv HSA_ENABLE_SDMA=1 (sdma) ===
=== alltoallv HSA_ENABLE_SDMA=0 (blit_kernel) ===
```

When does each win?

- **SDMA** tends to win for **large, infrequent transfers** where having the shader engines free to overlap computation matters more than raw copy throughput.
- **Blit kernels** tend to win for **small, latency-sensitive transfers** where the shader engines are otherwise idle and the lower software overhead of a compute dispatch beats the DMA engine’s start-up cost. On MI300A unified memory, blit kernels often outperform SDMA for the modest ghost-zone message sizes typical of sparse solvers.

The right choice is workload- and hardware-dependent; the sweep in `run_test_7.13.sh` surfaces the difference empirically.

Profiling with TAU

TAU intercepts MPI and, with `-rocm`, ROCm/HIP GPU calls via `LD_PRELOAD` — this directly measures the **communication** that GPU-only tools (`rocprofv3`, `roofline`, etc.) don’t see: per-call MPI time, the per-rank point-to-point **communication matrix**, and (with `-rocm`) the ROCm runtime/kernel time in the same profile. No special instrumented build is needed — `tau_exec` wraps the plain `./cg_gpu` binary at run time.

```
module load rocm/6.4.3
module load openmpi/5.0.10-ucc1.6.0-ucx1.19.1-xpmem-2.7.4
module load tau/dev # layered on top of rocm - load it last

make
mpirun -n 4 --bind-to none bash set_affinity_mi300a.sh \
    tau_exec -T MPI,ROCM -rocm ./cg_gpu src/Dubcova2.pm rccl

pprof # text summary: per-call MPI + ROCm/HSA time, merged across ranks
```

`run_tau.sh` automates this end-to-end as an `sbatch` job (build → run under `tau_exec` → `pprof` summary), since the login node has no GPU:

```

sbatch run_tau.sh                                # profile 4 ranks, method=rccl
METHOD=isend sbatch run_tau.sh                  # profile a different comm. variant
TRACE=1 sbatch run_tau.sh                       # also write an OTF2 trace + a
→ Perfetto/Chrome JSON trace
sbatch -p PPAC_MI300A_SPX --gpus=4 --ntasks=4 run_tau.sh # override partition/GPU count

```

Perfetto / chrome://tracing timeline (TRACE=1)

With `TRACE=1`, `run_tau.sh` merges the per-rank raw traces (`tau_treemerge.pl`) and converts the merged trace with `tau_trace2json ... -chrome` into `$EXPDIR/perfetto_<method>.json` — a standard Chrome Trace Event JSON file. Open it at `ui.perfetto.dev` (drag-and-drop the file) or `chrome://tracing` to see every MPI call, ROCm/HSA runtime call, and GPU kernel launch as a timeline per rank/thread, with flow arrows (s/f events) connecting each `MPI_Isend` to its matching `MPI_Irecv` — this is the easiest way to *see* overlap (or the lack of it) between communication and compute, rather than reading it out of aggregate call counts.

Not valid strict JSON — by design. The file is a bare `[ {...}, {...}, ... ,` with no closing `]` and a trailing comma; this is the documented legacy Chrome Trace Event Format convention (any trace tooling can crash mid-write and still leave a loadable file), and both Perfetto UI and `chrome://tracing` handle it natively. A strict `json.load()` will reject it — that’s expected, not corruption.

These traces are large. A 4-rank, ~170-iteration run produces a ~400 MB JSON (~2.9M events after `MPI_Init`, `hsa_*` runtime calls, and per-iteration kernels are all captured). Perfetto UI can still load it, but expect it to take a while; for a smaller trace, target fewer iterations/ranks or a smaller matrix.

Verified on AAC6/MI300A (ROCm 6.4.3, `tau/dev`): the profile below shows real per-call MPI time (`MPI_Init`, `MPI_Allreduce`, ...) merged with ROCm HSA runtime and kernel time (`rocspars::gthr_kernel`, `rocblas_dot_kernel_*`, ...) in one `pprof` summary:

%Time	Exclusive	Inclusive	#Call	#Subrs	Inclusive Name
100.0	34	8,976	1	1	8976004 .TAU application
67.3	6,038	6,038	1	0	6038301 MPI_Init()
0.2	7	20	306	306	69 MPI_Allreduce()
0.0	0.0688	0.0688	25.05	0	3 [ROCm Kernel] rocspars::gthr_kernel.

Known teardown quirk (ROCm 6.4.3 + tau/dev). The instrumented run reliably prints the CG solve result and writes every `profile.*` file, then the *process teardown* aborts with **corrupted size vs. prev_size in fastbins** (a `rocprofSDK` + `glibc` interaction, not a measurement failure) — `run_tau.sh` reads the profile anyway and does not treat that nonzero exit as a hard failure.

pprof/paraprof read \$PROFILEDIR themselves. Run them from CG-GPU/ with `PROFILEDIR` set to the experiment dir — *don’t* `cd` into that directory first, or they’ll look for a nonexistent nested `<dir>/<dir>/profile.*`. `tau_treemerge.pl/tau2otf2` (used for `TRACE=1`) are the opposite: they don’t consult `$TRACEDIR` at all and must be run from inside the experiment directory.

For the GUI (`paraprof`, per-call bar charts + the communication matrix), or to convert a `TRACE=1` run to OTF2 and view it in a trace viewer, open a remote graphical session (`man aac6_vnc` / `man aac6_novnc` / `man aac6_x11`) — `paraprof` needs a JRE, which may not be installed on every login node.

Requirements

- ROCm ≥ 6.3 (rocSPARSE, rocBLAS, hipcc)

- GPU-Aware MPI (OpenMPI/UCX built with ROCm support — see module list on the login banner)
- TAU (`module load tau/dev`, loaded after `rocm`) for MPI+ROCm profiling/tracing

ImageNet DDP: measuring RCCL communication at scale

README.md from `HPCTrainingExamples/MLExamples/Pytorch/imagenet` in the `HPCTrainingExamples` repository

This example sets up a large workload (ResNet on ImageNet-sized 224x224x3 images, 1000 classes) trained with true `DistributedDataParallel` (DDP), one process per GPU, using the RCCL (ROCm Collective Communication Library) backend. Recall that On ROCm, PyTorch’s `nccl` backend is provided by `librccl`, so all the `NCCL_*` environment variables are honored by RCCL.

Note that we use synthetic data so that no 150 GB ImageNet download is needed. The input pipeline is essentially free, so each training step is dominated by

1. GPU compute (the forward/backward of the CNN), and
2. the RCCL all-reduce of gradients across GPUs at the end of each step.

Synthetic data is enabled with the `--dummy` flag. By comparing step time across GPU counts we isolate and quantify the RCCL communication cost.

Contents

1. Get an allocation and load PyTorch
2. Get the examples
3. Optional modifications to the example source code
4. Warm the MIOpen cache (once per allocation)
5. Run the scaling sweep (assumes an SPX partition)
6. APU programming model (MI300A)
7. Read the two numbers (RCCL time, staging)
8. Calculating the performance
9. Featured RCCL optimization: tune the all-reduce with environment variables
10. Featured compute optimization: hands-on `main.py` edits
11. Featured profiling exercise: a measured timeline of compute vs communication (4×MI300A)
12. Featured profiling tool: roofline extractor (per-kernel compute vs. memory)
13. Cleanup
14. Run on CPX partitions (`SH5_MI300A_CPX`, `PPAC_MI300A_CPX`)
 - Next steps
 - Appendix A: noVNC desktop (view GUIs like ParaProf/Perfetto in the browser)

1. Get an allocation and load PyTorch

These instructions assume you are running this example on AMD’s AAC6 cluster, which has MI300A APUs. Pick whichever partition is free — the step-by-step example runs on all of them. SPX gives each rank a whole MI300A APU; CPX splits each APU into 6 smaller GPU partitions, so a CPX node exposes the single XCDs leaves more room for interactive experimentation. The `PPAC_MI300A_SPX` nodes are scarce, so the CPX partitions are good fallbacks.

```
# 4 full-APU GPUs on a PPAC SPX node (whole node; DefCpuPerGPU=48 -> 192 CPUs)
salloc -p PPAC_MI300A_SPX -N1 --gpus=4 -t 00:40:00

# 4 CPX partitions on the PPAC CPX node (24 GPUs/node; DefCpuPerGPU=8 -> 32 CPUs)
salloc -p PPAC_MI300A_CPX -N1 --gpus=4 -t 00:40:00

# 4 CPX partitions on an SH5 CPX node (6 GPUs/node; DefCpuPerGPU=8 -> 32 CPUs)
```

```
salloc -p SH5_MI300A_CPX -N1 --gpus=4 -t 00:40:00
```

CPX partitions carve one APU's 128 GB HBM into 6 (~21 GB each), so if you hit an OOM on CPX, drop the per-GPU batch size (e.g. `-b 128`).

Optional — CPU/GPU affinity. To bind each rank to the cores nearest its GPU, add task/affinity flags to the allocation (or to `srun`):

```
salloc -p PPAC_MI300A_SPX -N1 --gpus=4 --ntasks-per-node=4 \
      --cpus-per-task=48 --gpu-bind=closest --cpu-bind=cores -t 00:40:00
# On a CPX partition use --cpus-per-task=8 (matches DefCpuPerGPU=8, since the
# 6 CPX GPUs share one APU's 48 cores).
```

These matter most when you launch ranks with `srun`, where Slurm places and binds each task. The README's default path uses `mp.spawn` and the TAU path uses `mpirun` (Path B (TAU)), which do their own binding, so there the Slurm flags are only advisory.

Set up virtual environment to avoid scattering python packages across system and for more repeatability

Check `uv` is installed by doing `which uv`. If not, install it and then do:

```
uv init imagenet_test
cd imagenet_test
uv venv --system-site-packages
```

Use pre-installed module versions to avoid downloading large wheels. `uv pip install -r requirements.txt` installs nvidia packages, so we skip it.

The command below will load the default version of ROCm, make sure it matches the one you intend to use:

```
module load rocm openmpi pytorch
```

Activate the venv after `module load` (loading a module *prepends* its own `python` to `PATH`, so activating first would let the module's Python shadow the venv's). Then confirm `python3` really is the venv's:

```
source .venv/bin/activate
which python3 # must be ../imagenet_test/.venv/bin/python3, not a module path
```

Confirm the GPUs are visible:

```
python3 -c 'import torch; print(torch.cuda.is_available(), torch.cuda.device_count())'
```

You should also make sure that the PyTorch you are importing is the one that comes from the module you loaded:

```
python3 -c 'import torch; print(torch.__file__)'
```

check that the above path matches the path shown when doing `module show pytorch`.

2. Get the examples

```
git clone --depth=1 https://github.com/pytorch/examples.git pytorch_examples
cd pytorch_examples/imagenet/* .
```

3. Optional modifications to the example source code

Below there are some basic edits to the example to get the instrumentation in place for the optimizations exercises. We advise users going through this document to apply the changes one by one, to monitor what

each does to the original code. However, for the sake of time one could run the script below to apply all the changes at once. This is recommended for users that have gone through these instructions already at least once:

```
../apply_basic_edits.sh
```

3a. Fix warning `destroy_process_group`

Upstream `main.py` initiates the NCCL process group but never destroys it, so PyTorch warns at exit (“`destroy_process_group()` was not called ... can leak resources”). Register an `atexit` handler right after `init_process_group` so every worker (incl. `mp.spawn` children) cleans up on a normal exit.

```
sed -i '/world_size=args.world_size, rank=args.rank)/a\         import atexit as _ax,\n-> torch.distributed as _d; _ax.register(lambda: _d.destroy_process_group() if\n-> _d.is_initialized() else None)' main.py
```

3b. Keep the demo (and the profiler trace) short.

This shortens the demo to 100 iterations for quicker runs and to accommodate more users.

```
sed -i '/^         data_time.update(time.time() - end)/a\         if i >= 100: break' main.py
```

3c. Add GPU peak memory instrumentation

Understanding how much memory is being used relative to the available memory is important in optimizing a job.

Print per-GPU peak memory once at the end of `train()`:

```
sed -i '/^def validate(/i\     torch.cuda.is_available() and getattr(args,"rank",0)<=0 and\n-> print(f"PEAK_MEM_MB {torch.cuda.max_memory_allocated()/1e6:.0f}")' main.py
```

NOTE: the next two measurements below are independent, self-contained changes: 3d adds the profiler and the total-NCCL-time print; 3e adds the `.to` vs `.migrate` staging comparison. Each stands alone (neither references the other’s variables), so you can apply either one, and in either order.

3d. Enable the profiler and print total NCCL time

Start a `torch.profiler` at the top of `train()` and stop it just before `validate()`. The on-GPU time of the `nccl*` collective kernels is summed and printed as `NCCL_TOTAL_MS` (~0 at 1 GPU, growing with GPU count). The `break` from step 3b keeps the captured trace short.

Start the profiler at the top of `train()`:

```
sed -i '/^     model.train()/a\         import torch.profiler as _tp\         _prof = _tp.profile(activities=[_tp.ProfilerActivity.CPU, _tp.ProfilerActivity.CUDA],\n-> acc_events=True); _prof.start()' main.py
```

Stop the profiler and print the total NCCL kernel time (inserted at the end of `train()`, right before `def validate()`):

```
sed -i '/^def validate(/i\         _prof.stop()\         _nccl_ms = sum(e.self_device_time_total for e in _prof.key_averages() if "nccl" in\n-> e.key.lower())/1e3\         _ws = getattr(args,"world_size","?")\
```

```

    getattr(args,"rank",0)<=0 and print(f"RCCL_TOTAL_MS {_rccl_ms:.3f} gpus={_ws}")'
↪ main.py

```

3e. Compare `.to` (copy) vs `.migrate` (zero-copy) staging

The MI300A is a true APU with a single address space. Many other GPUs emulate the single address space with managed memory that copies arrays from host to device and back when needed.

The migrate path (`STAGE=migrate`) aliases the batch instead of copying it; it needs `COMMON_DIR` in the `sys.path` to find `zerocopy.Stager`, `HSA_XNACK=1`, and pageable (non-pinned) host memory. When `STAGE` is unset these edits are inert, so the plain scaling runs are unaffected.

Point `COMMON_DIR` at the shared helpers and enable `XNACK`. Anchoring to the repo root makes the export work from any directory (the manual flow above leaves you in `imagenet_test`, two levels below `common/`, so a bare `../common` would not resolve):

```

export COMMON_DIR="$(git rev-parse --show-toplevel)/MLExamples/Pytorch/common"
export HSA_XNACK=1

```

Set up the staging counters and (for `STAGE=migrate`) the zero-copy `Stager` at the top of `train()`:

```

sed -i '/^    model.train()/a\
    _stage_ms = 0.0; _stage_n = 0; _stager = None\
    if os.environ.get("STAGE") == "migrate":\
        import sys; sys.path.insert(0, os.environ["COMMON_DIR"]); from zerocopy import
↪ Stager\
    _stager = Stager(device, method="register")' main.py

```

Time the host-to-device staging, but only when `STAGE` is set (copy vs migrate):

```

sed -i '/^        images = images.to(device, non_blocking=True)/c\
        if os.environ.get("STAGE"):\
            _e0 = torch.cuda.Event(enable_timing=True); _e1 =
↪ torch.cuda.Event(enable_timing=True)\
            _e0.record()\
            images = _stager.to_device(images) if _stager is not None else
↪ images.to(device, non_blocking=True)\
            _e1.record(); torch.cuda.synchronize()\
            _stage_ms += _e0.elapsed_time(_e1); _stage_n += 1\
        else:\
            images = images.to(device, non_blocking=True)' main.py

```

Print the per-step staging time (inserted at the end of `train()`, right before `def validate()`):

```

sed -i '/^def validate(/i\
    _stg = os.environ.get("STAGE",""); _ws2 = getattr(args,"world_size","?")\
    getattr(args,"rank",0)<=0 and _stage_n and print(f"STAGE_MS_PER_STEP
↪ {_stage_ms/_stage_n:.4f} gpus={_ws2} stage={_stg}")' main.py

```

3f. Make `pin_memory` graceful for the fallback path

The `register` staging path needs pageable (non-pinned) host memory, because `hipHostRegister` fails on already-pinned buffers. But the plain `.to()` fallback (discrete GPU, `HSA_XNACK != 1`, or the extension failing to build) copies faster from pinned memory. So rather than forcing `pin_memory=False` unconditionally, gate it on whether the zero-copy `register` path is actually available – exactly mirroring `Stager`'s own graceful

fallback (`zerocopy.unified_memory_available()`). When `STAGE` is unset this is inert: `pin_memory` stays `True` (upstream behavior), so the plain scaling runs keep fast pinned copies.

Inject the gate helper right after the imports:

```
sed -i '/^from torch.utils.data import Subset/a\
def _zero_copy_active():\
    """pin_memory gate: pageable only when the STAGE=migrate register path is live."""\
    if os.environ.get("STAGE") != "migrate":\
        return False\
    try:\
        import sys; sys.path.insert(0, os.environ["COMMON_DIR"]); from zerocopy import
→ unified_memory_available\
        return unified_memory_available()\
    except Exception:\
        return False' main.py
```

Use pageable buffers only when zero-copy is active, pinned otherwise:

```
sed -i 's/pin_memory=True/pin_memory=not _zero_copy_active()/g' main.py
```

4. Warm the MIOpen cache (once per allocation)

MIOpen's default solver search can take >10 minutes cold for ResNet convolutions. Set fast selection, then warm the cache by running the warmup script or the 1-GPU `main.py` case (the same run the sweep uses) for a few steps: Warm once per allocation. A new `salloc/sbatch` gets a fresh job ID (and thus a fresh empty cache). Warming single-process first matters: inside one allocation all `N` ranks share that one cache dir, so a cold multi-rank run would contend on the SQLite db; after warming, ranks just read it (with `MIOpen_FIND_MODE=FAST`).

```
export MIOpen_FIND_MODE=FAST
```

The pytorch module on AAC6 already sets `MIOpen_USER_DB_PATH` / `MIOpen_CUSTOM_CACHE_DIR` at a stable per-allocation dir (e.g. `/tmp/$USER/miopen-cache/jobs/`), so DON'T override them – just inherit them. Create the directory

```
mkdir -p "$MIOpen_USER_DB_PATH"
```

Suppress some warning noise

```
export MIOpen_LOG_LEVEL=3
export KINETO_LOG_LEVEL=3
```

Then proceed to warm up the cache:

```
HIP_VISIBLE_DEVICES=0 python -c "import torch,torchvision.models as M; \
d=torch.device('cuda'); \
n=M.resnet50().to(d); \
c=torch.nn.CrossEntropyLoss().to(d); \
x=torch.randn(256,3,224,224,device=d); \
y=torch.randint(0,1000,(256,),device=d); \
[c(n(x),y).backward() for _ in range(3)]; \
torch.cuda.synchronize(); \
print('warm done')"
```

5. Run the scaling sweep (assumes an SPX partition)

Run the benchmark once per GPU count by changing HIP_VISIBLE_DEVICES:

```
HIP_VISIBLE_DEVICES=0      python main.py -a resnet50 --dummy --dist-url
↪ 'tcp://127.0.0.1:23456' \
    --dist-backend nccl --multiprocessing-distributed --world-size 1 --rank 0 -b 128
    ↪ -p 20 --epochs 1 |& tee run_1.log
HIP_VISIBLE_DEVICES=0,1    python main.py -a resnet50 --dummy --dist-url
↪ 'tcp://127.0.0.1:23456' \
    --dist-backend nccl --multiprocessing-distributed --world-size 1 --rank 0 -b 256
    ↪ -p 20 --epochs 1 |& tee run_2.log
HIP_VISIBLE_DEVICES=0,1,2,3 python main.py -a resnet50 --dummy --dist-url
↪ 'tcp://127.0.0.1:23456' \
    --dist-backend nccl --multiprocessing-distributed --world-size 1 --rank 0 -b 512
    ↪ -p 20 --epochs 1 |& tee run_4.log
```

6. APU programming model (MI300A)

The MI300A APU has a unified memory and does not need to copy the data, just the pointer. Other GPUS can emulate APU behavior leveraging the APU programming model. The APU programming model requires HSA_XNACK 1 to be set (you also need it on MI300A). We will compare `.to` (copy) vs `.migrate` staging looking at `STAGE_MS_PER_STEP` in the final report.

- Host-to-device staging time — the per-step `images.to(device)` copy is wrapped in CUDA events and printed as `STAGE_MS_PER_STEP`, but only when the `STAGE` environment variable is set, so the plain scaling runs are not perturbed. Setting `STAGE=migrate` swaps the `.to()` copy for the zero-copy `migrate()` path (MI300A unified memory), so you can compare the two directly:

```
# .to() copy vs zero-copy migrate (single GPU) -- compare STAGE_MS_PER_STEP
STAGE=copy      HIP_VISIBLE_DEVICES=0 python main.py -a resnet50 --dummy \
--dist-url 'tcp://127.0.0.1:23456' --dist-backend nccl \
--multiprocessing-distributed --world-size 1 --rank 0 -b 128 -p 20 --epochs 1 |& tee
↪ stage_copy.log
STAGE=migrate HIP_VISIBLE_DEVICES=0 python main.py -a resnet50 --dummy \
--dist-url 'tcp://127.0.0.1:23456' --dist-backend nccl \
--multiprocessing-distributed --world-size 1 --rank 0 -b 128 -p 20 --epochs 1 |& tee
↪ stage_migrate.log
```

The `.to()` path pays a `hipMemcpy` every step; `migrate` aliases the batch, so its `STAGE_MS_PER_STEP` should be much smaller. `migrate` requires `HSA_XNACK=1` and `COMMON_DIR` pointing at `../common` (both exported by `main.py`); if the `migrate` extension can't build, `Stager` falls back to a copy and the two numbers will match.

7. Read the two numbers (RCCL time, staging)

The scaling sweep and staging runs already wrote the logs; read back the two instrumented numbers — `RCCL_TOTAL_MS` (the profiler from the profiler edit) and `STAGE_MS_PER_STEP` (the staging timing from the staging edit).

RCCL total time per run, sorted by GPU count (`run_<N>.log`):

```
echo "=== RCCL total time (per GPU count) ==="
grep -h RCCL_TOTAL_MS run_*.log | sort -t= -k2 -n
```

`.to` (copy) vs `.migrate` staging (`stage_{copy,migrate}.log`):

```
echo "=== Host->device staging: .to (copy) vs .migrate ==="
grep -h STAGE_MS_PER_STEP stage_*.log
```

8. Calculating the performance

This sweep is a weak-scaling study: the per-GPU batch (`-b`) is held fixed at 128 images, so every GPU always does the same work as in the single-GPU run — it is the *total* (node-wide) batch that grows with GPU count (128 to 256 to 512), not the work per GPU. Because per-GPU compute is constant, the per-step **Time** should ideally stay flat as GPUs are added, and throughput should grow linearly ($\approx N \times$ the 1-GPU rate). The only new cost at ≥ 2 GPUs is the gradient all-reduce, so whatever the step time creeps up — and whatever throughput falls short of linear — is the (exposed) RCCL communication.

Contrast strong scaling, where the *total* problem size is fixed and per-GPU work shrinks as you add GPUs; there you'd expect the step time itself to drop. This demo does the opposite: more GPUs = more total work at (ideally) the same step time, so the payoff shows up as higher throughput, not a shorter step.

```
echo "=== Calculating the performance =="
../images_per_sec.sh
```

Stock upstream `main.py` prints periodic **Epoch:** progress lines that include the per-step **Time**. The `images_per_sec.sh` helper parses those logs (`run_1.log`, `run_2.log`, `run_4.log` written by `main.py`) and prints one line per GPU count:

```
run_1.log  img/s=968    step=0.1322s  batch=128  peak_mem_mb=...  speedup=1.00x
run_2.log  img/s=1901   step=0.1347s  batch=256  peak_mem_mb=...  speedup=1.96x
run_4.log  img/s=3720   step=0.1376s  batch=512  peak_mem_mb=...  speedup=3.84x
```

- **img/s** — global throughput: the global batch divided by the average per-step **Time**, i.e. images processed per second across the whole node.
 - *global batch* = per-GPU `-b` $\times$ number of GPUs, e.g. $128 \times 4 = 512$ images per step summed across the node.
 - *average per-step Time* = the running-average seconds-per-step that `main.py` prints in parentheses on its **Epoch:** lines (one full step: forward + backward + all-reduce + optimizer).

Per the weak-scaling note above, the ideal is linear growth with GPU count; the shortfall from linear is mostly the exposed (un-overlapped) all-reduce. Compare with `RCCL_TOTAL_MS` (profiler edit/two-numbers readout), the *total* RCCL time, to see how much is hidden behind compute.

- **speedup** — a throughput speedup, *not* a time speedup. It is the ratio of throughputs to the 1-GPU baseline, $\text{img/s}(N) / \text{img/s}(1)$ (so `run_1` reads 1.00x) — i.e. “how many times more images/sec than one GPU”. It is not the strong-scaling runtime speedup $T(1)/T(N)$ (how many times *faster* the same job finishes); in this weak-scaling demo the step time stays roughly flat, so a *time* speedup would be $\sim 1x$ and is not the point. Since each added GPU takes on an extra 128-image share, the ideal *throughput* speedup here is linear, $N \times$ (4 GPUs gives 4.00x). How close you get is the weak-scaling efficiency = $\text{speedup} / N$ (e.g. $3.84x / 4 \approx 96\%$).
- **step** — the average per-step time (the value **img/s** divides into). Flat across GPU counts = ideal weak scaling; any growth is the exposed all-reduce.
- **peak_mem_mb** — per-GPU peak allocated memory (`PEAK_MEM_MB`).

This is the simple, demo-friendly version. For the robust, per-step, per-rank instrumentation (with a fair pinned-memory baseline) use `ddp_resnet_bench.py`'s `--rccl-time`, `--host-copy`, and `--migrate` flags, documented in `benchmarks/README_benchmark.md`.

9. Featured RCCL optimization: tune the all-reduce with environment variables

Hands-on `main.py` edits for improving the RCCL all-reduce, together with a reproducible batch sweep and measured `RCCL_TOTAL_MS` numbers, are collected in `README_rccl_optimization.md`.

10. Featured compute optimization: hands-on main.py edits

Hands-on `main.py` edits for speeding up per-GPU compute (ResNet-50 forward/backward), together with a reproducible batch study and measured `torch.compile` throughput numbers, are collected in `README_compute_optimization.md`.

11. Featured profiling exercise: a measured timeline of compute vs communication

In this section, we will look at how to profile our workload to measure the time taken for communication and computation. Note that as before we are assuming we are on an SPX node with 4 MI300A APUs. We will be using both `torch.profiler` and TAU to record every GPU kernel with a timestamp, so the compute kernels (Conv/GEMM/batchnorm from forward & backward) and the RCCL all-reduce kernels (`ncclDevKernel`) fall into visually distinct bands — the same split that the two-numbers readout measures numerically as `RCCL_TOTAL_MS`, now laid out in time across all four ranks.

11.1 torch.profiler manual flow

We begin by considering the profiling output produced by `torch.profiler`: this will be displayed next as Chrome traces as well as Perfetto traces (using JSON files).

First, add `matplotlib` to your environment:

```
uv pip install matplotlib
```

Then, modify the `main.py` to make profiling faster by shortening the run to ~20 steps so the trace stays small:

```
sed -i 's/if i >= 100: break/if i >= 20: break/' main.py
```

and write per-rank Chrome/Perfetto traces when the profiler stops:

```
sed -i '/^    _prof.stop()/a\
    _rk = getattr(args, "rank", 0)\
    _prof.export_chrome_trace(f"torch_trace_rank{ _rk }.json")' main.py
```

Next, we run to produce the profiling output:

```
# run from imagenet_test (where main.py is); profiling/ is in the parent dir
HIP_VISIBLE_DEVICES=0,1,2,3 python main.py -a resnet50 --dummy \
  --dist-url 'tcp://127.0.0.1:23456' --dist-backend nccl \
  --multiprocessing-distributed --world-size 1 --rank 0 -b 512 -p 20 --epochs 1 \
  |& tee capture_run_torch.log
```

We can now create a figure with `matplotlib` to show the duration of communication and compute for each of the 4 GPUs involved in the run. `render_timeline.py` reads the per-rank traces directly (one lane per rank) — no merge step is needed here (`merge_perfetto.py` is only for the Perfetto UI):

```
# run from imagenet_test (where torch_trace_rank*.json were written)
IMAGENET=$(git rev-parse --show-toplevel)/MLExamples/Pytorch/imagenet
python3 "$IMAGENET/profiling/render_timeline.py" \
  --glob 'torch_trace_rank*.json' \
  --out "$IMAGENET/figs/timeline_4gpu.png"
```

An example of such a figure is reported below:

Regarding the legend: blue = compute, orange = the RCCL all-reduce, white = idle (data-loader / Python between steps). The data in the figure tells the following story:

- Per-step structure — each blue block is a step's forward+backward compute, ended by an orange RCCL all-reduce of the gradients.

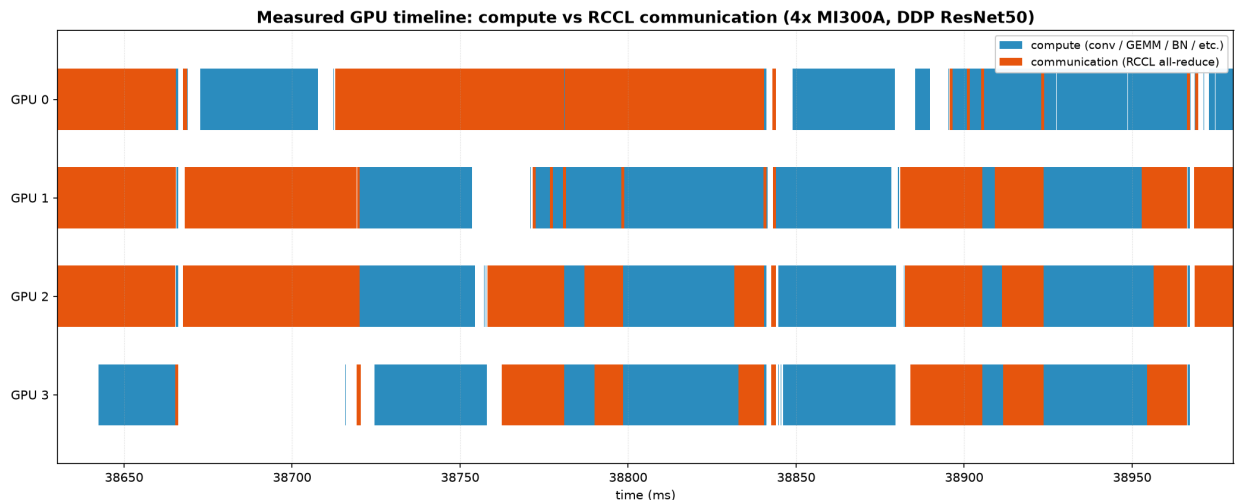


Figure 1: Measured 4×MI300A GPU timeline (torch.profiler): compute (blue) vs RCCL all-reduce (orange), ~700 ms steady-state window

- Exposed communication — the long orange blocks are a rank stuck inside `ncclDevKernel`, spin-waiting for the all-reduce to complete. In this run GPU 0 has the longest orange stretches — notably the ~125 ms block spanning ~38,720-38,840 ms — because it finishes its compute earliest and then waits on the slower ranks, which are still busy with compute (blue) or briefly idle (white). Which rank waits longest is set by the per-step compute imbalance, not by being rank 0. This exposed, imbalanced all-reduce is exactly what the RCCL optimizations (`NCCL_ALGO/channels`, `gradient_as_bucket_view`, `bucket_cap_mb`, `static_graph`) attack — the timeline makes the cost visible, not just the `RCCL_TOTAL_MS` number.
- Gaps between steps = per-step overhead (Python / launch / dummy data-loader), the target of the `torch.compile` compute optimization.

We can also open the traces using the Perfetto UI: this works from any browser and while disconnected from the internet. Before we do this, let's merge the per-rank torch traces into a single 4-lane Perfetto trace (Chrome JSON, gzipped):

```
# run from imagenet_test; traces were written here, merge script is in ../profiling
mkdir -p traces
python3 ../profiling/merge_perfetto.py \
  --glob 'torch_trace_rank*.json' \
  --out traces/imagenet_4gpu.perfetto.json.gz
```

From your browser of choice, open <https://ui.perfetto.dev> and load a `torch_trace_rank*.json` (one GPU), or the merged `imagenet_4gpu.perfetto.json.gz` (all four GPUs as lanes). The merged trace opens in the Perfetto UI as four GPU process lanes, see example picture below:

Within each GPU, the compute stream (Thread 1, running `DistributedDataParallel.forward` and the colored conv/GEMM kernels) is separate from the RCCL stream (Thread 4), where the `nccl:broadcast` at startup and the periodic `nccl:all_reduce`, which runs as `ncclDevKernel_Generic` slices, are the communication:

11.2 torch.profiler automated flow

The whole `torch.profiler` pipeline (capture, merge the per-rank traces, drop the raw traces, and render the `matplotlib` PNG) can be carried out by submitting an sbatch script from the `imagenet` directory:

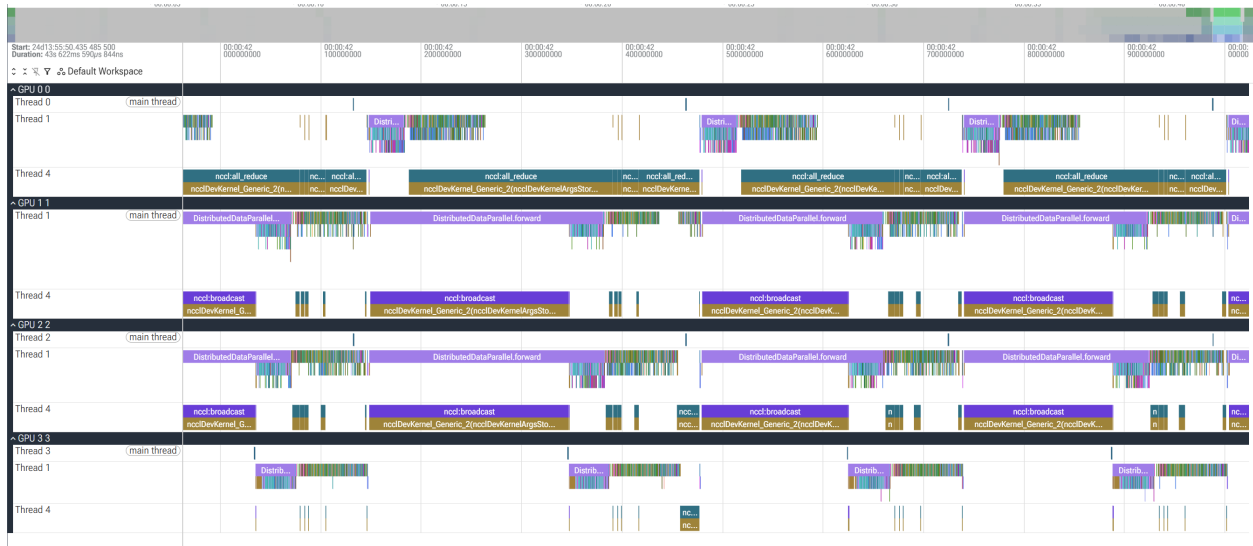


Figure 2: Merged 4-GPU torch.profiler trace in the Perfetto UI: per-GPU compute stream (Thread 1) vs the RCCL nccl:all_reduce / ncclDevKernel stream (Thread 4)

```
cd "$(git rev-parse --show-toplevel)/MLExamples/Pytorch/imagenet"
sbatch profiling/capture_torch.sbatch # it considers a 4-APU SPX node on AAC6,
↪ partition PPAC_MI300A_SPX
```

(TAU's ParaProf profile comes from the separate `capture_tau.sbatch`; see Path B (TAU).)

The job also drops the raw per-rank traces and the merged Perfetto trace under `profiling/traces/`, so with no local display you can `scp` them to your laptop and open the merged trace in `ui.perfetto.dev` exactly as in the manual flow above; for GUI viewers (ParaProf) or an on-node browser, use the noVNC desktop. The committed PNGs come from the headless `matplotlib` render, while the Perfetto view is a manual screenshot — optionally automated by `profiling/perfetto_shot.py` where a browser is available (Playwright + Chromium), which the capture job does not run.

Perfetto only ingests Perfetto protobuf (`.pftrace/.pb`) or Chrome JSON Trace Event format (`{"traceEvents": [...]}` or a bare event array). The `torch.profiler` exports above are Chrome JSON and load directly, so the Perfetto artifact comes from `torch.profiler` (Path A (`torch.profiler`)).

11.3 TAU manual flow

Let's begin setting up for TAU:

```
module load tau # adds tau_exec / pprof (on top of rocm openmpi pytorch)
unset TAU_PROFILE_FORMAT # tau module defaults to "merged" (one tauprofile.xml); unset -> per-
export PROFILEDIR=$PWD/tau_trace # directory the per-rank profile.* files are written to
mkdir -p tau_trace # create it before the run
```

The MPI-enabled TAU library assigns each rank from the launcher, so profiles are written only when the job runs under `mpirun`; a single, non-`mpirun` process produces none.

IMPORTANT: TAU's `rocprof sdk` and `torch.profiler` (Kineto) both claim the ROCm profiler interface, so if `main.py` still has the `torch.profiler` edits, TAU attaches to nothing and writes NO profile.* files. We will build a clean `main_tau.py` from the upstream clone with only the teardown + short-run edits (no `torch.profiler`):

```
# Re-fetch a pristine upstream main.py if the clone from setup isn't present, so this
# never depends on your edited ./main.py (and survives a Cleanup that removed it):
[ -f pytorch_examples/imagenet/main.py ] || git clone --depth=1
↳ https://github.com/pytorch/examples.git pytorch_examples
cp pytorch_examples/imagenet/main.py main_tau.py
sed -i '/world_size=args.world_size, rank=args.rank)/a\           import atexit as _ax,
↳ torch.distributed as _d; _ax.register(lambda: _d.destroy_process_group() if
↳ _d.is_initialized() else None)' main_tau.py
sed -i '/^           data_time.update(time.time() - end)/a\
           if i >= 20: break' main_tau.py
```

Unlike `torch.profiler`, TAU wants distinct MPI ranks, so we will run one process per GPU with `mpirun -n 4` to give TAU a proper 4-rank profile together with a tiny per-rank wrapper (`tau_wrapper.sh`) that pins one MI300A APU per rank, so each of the four ranks runs on its own APU:

```
cat > tau_wrapper.sh <<'EOF'
#!/bin/bash
export HIP_VISIBLE_DEVICES=${OMPI_COMM_WORLD_LOCAL_RANK}
exec python main_tau.py -a resnet50 --dummy \
    --dist-url 'tcp://127.0.0.1:23457' --dist-backend nccl \
    --world-size ${OMPI_COMM_WORLD_SIZE} --rank ${OMPI_COMM_WORLD_RANK} \
    --gpu 0 -b 512 -p 20 --epochs 1
EOF
chmod +x tau_wrapper.sh
```

Run the 4-rank capture with `tau_exec -rocm`. The `-rocm` flag turns on TAU's built-in rocprofiler-sdk GPU tracing. TAU writes `profile.<rank>.0.<thread>` files into `$PROFILEDIR (tau_trace)` — one per rank per thread, so expect several files per rank:

```
mpirun -n 4 --map-by numa --bind-to numa --report-bindings \
    tau_exec -rocm ./tau_wrapper.sh

ls tau trace/profile.*      # sanity: per-rank profiles must exist before pprof
```

Shortcut: the tau module also ships `tau_exec_launch`, which wraps the whole `mpirun ... tau_exec ...` capture and runs `pprof` for you (it keeps the module default merged format, so it emits a single `tauprofile.xml` rather than per-rank `profile.*`): `tau_exec_launch --np 4 --mode comm --outdir tau trace -- ./tau wrapper.sh`.

Binding note. The GPU is pinned by the wrapper (`HIP_VISIBLE_DEVICES=${OMPI_COMM_WORLD_LOCAL_RANK}`, so rank i maps to GPU i). The CPU/core placement is what `mpirun --map-by numa --bind-to numa` controls: each rank is bound to one NUMA domain's cores. Since each MI300A APU is its own NUMA node (its GPU + its cores), these two align so rank i runs on APU i 's cores *and* its GPU; add `--report-bindings` to print the per-rank core mask and confirm.

Dump the per-rank profile as text, then let's produce a figure we can analyze:

```
cd tau_trace
pprof -a > tau_pprof.txt           # per-rank profile as text; -a keeps the per-rank
    ↪ detail the renderer needs
IMAGENET=$(git rev-parse --show-toplevel)/MLExamples/Pytorch/imagenet
python3 "$IMAGENET/profiling/render_tau_profile.py" tau_pprof.txt \
    --out "$IMAGENET/figs/tau_profile_4gpu.png"
```

The `pprof -a` per-rank breakdown splits each rank’s GPU-kernel exclusive time into two buckets — the RCCL all-reduce (`ncclDevKernel`) and the ResNet-50 compute kernels (conv / GEMM / batch-norm /

elementwise). We render this as one stacked bar per rank (blue = compute, orange = all-reduce), with a shaded band spanning the fastest-to-slowest rank's compute:

TAU (ParaProf profile): 4-rank ImageNet on MI300A -- per-rank GPU time (compute vs RCCL all-reduce)

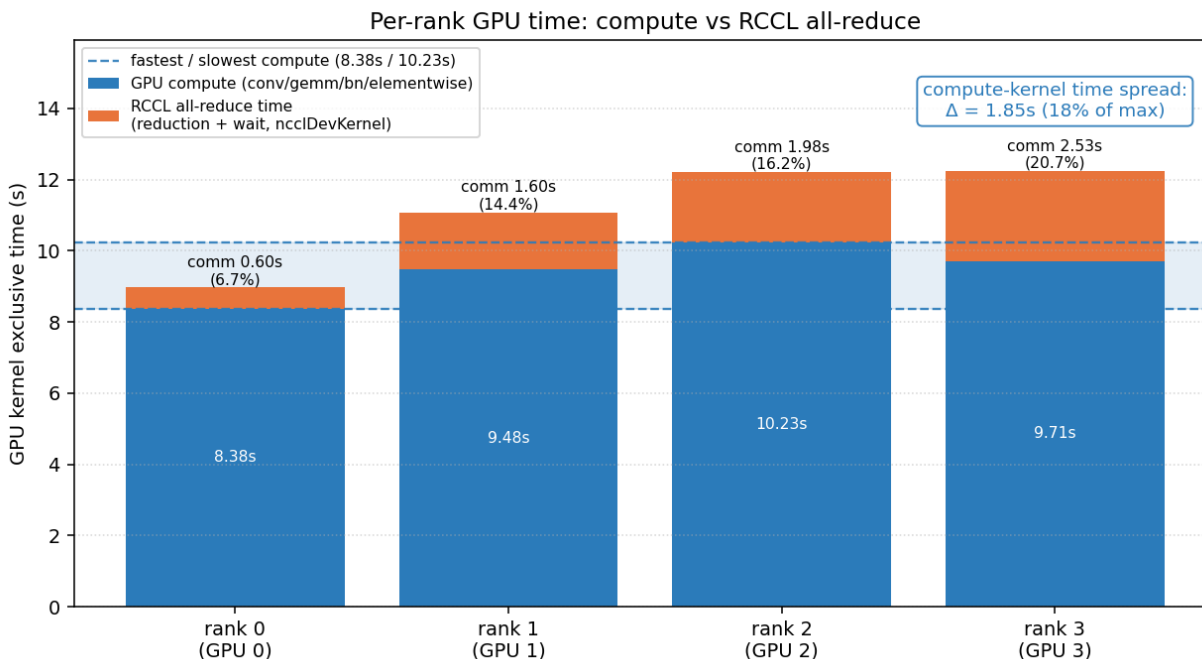


Figure 3: TAU ParaProf per-rank GPU-kernel breakdown on MI300A: one stacked bar per rank, blue ResNet-50 compute plus orange RCCL all-reduce, with a band marking the compute spread across ranks

How to read it:

- The blue bars are per-rank GPU compute time, ranging 8.38 s (rank 0) to 10.23 s (rank 2): a 1.86 s spread, 18 % of the max. All four ranks run the identical ResNet-50 on identical MI300A APUs with the same dummy batch, so this is not genuine load imbalance. Treat it as run-to-run scheduling variation plus profiler timing noise (this capture logged repeated rocprof sdk timestamp-ordering warnings), not an algorithmic straggler.
- The orange bars are the RCCL all-reduce (`ncclDevKernel`), 0.60–2.53 s, or 7 % to 21 % of each rank's GPU time. Because the collective synchronizes, this blends the actual reduction with time blocked waiting on peers; the two cannot be separated from this profile. The per-rank values also do not track the compute spread, so they are not “waiting for the slowest compute rank” times.

The headline is the all-reduce fraction, 7–21 % of per-rank GPU time, which is what the RCCL optimizations target. To quantify the true collective cost (as opposed to exposed wait), measure achieved bandwidth with `rccl-tests` or inspect compute/comm overlap in the Perfetto timeline.

11.4 TAU automated flow

The whole TAU pipeline (capture with `mpirun -n 4 tau_exec`, dump the per-rank `pprof` text, and render the `matplotlib` PNG headlessly — no Java, ParaProf GUI, or display needed) can be carried out by submitting an sbatch script from the imagenet directory:

```
cd "$(git rev-parse --show-toplevel)/MLExamples/Pytorch/imagenet"
sbatch profiling/capture_tau.sbatch # it considers a 4-APU SPX node on AAC6,
↪ partition PPAC_MI300A_SPX
```

Once the featured optimizations are applied, re-run either capture to see the effect: the RCCL all-reduce blocks shrink after the RCCL optimizations (channel tuning and the DDP overlap knobs), and the compute kernels fuse into fewer, longer ones after `torch.compile`. Reset the demo-length edit and the TAU variables when done:

```
sed -i 's/if i >= 20: break/if i >= 100: break/' main.py # restore the [demo-length
↪ break] (#sec-edit-break)
unset TAU_PROFILE PROFILEDIR
```

For a per-kernel roofline (compute- vs. memory-bound, % of MI300A peak) see the roofline-extractor walkthrough in roofline extractor. For the full menu of profilers on this example (torch.profiler, rocprofv3, rocprof-compute, rocprofiler-systems, Score-P, and TAU/HPCToolkit) see `profiling/PROFILING.md`.

To browse the `profile.*` in the ParaProf GUI, or run any other graphical tool on the node, see Appendix A: noVNC desktop.

12. Featured profiling tool: roofline extractor (per-kernel compute vs. memory)

The profiling exercise timeline shows where time goes (compute vs. the RCCL all-reduce). A roofline answers the next question — for each GPU kernel, is it compute-bound or memory-bound, and how close is it to the hardware peak? — so you know *which* kernel to optimize and *how*. The system `roofline-extractor` module (by Andrew Chisolm and Noah Wolfe) automates this: it drives `rocprofv3` to collect the FLOP and byte-movement counters per kernel, computes each kernel’s arithmetic intensity (FLOP per byte) and achieved throughput, and plots every kernel against the MI300A bandwidth/compute ceilings.

A roofline is a per-kernel, per-GPU compute analysis, so this profiles a single-GPU, short ResNet-50 `--dummy` run (not the 4-GPU DDP job). The `profiling/capture_roofline.sbatch` script packages the whole flow with the same self-contained pattern as the optimization batch studies (see `README_rccl_optimization.md` and `README_compute_optimization.md`), but loading `rocm pytorch roofline-extractor` and running the extractor:

```
# From the imagenet dir, on a node with a GPU:
sbatch profiling/capture_roofline.sbatch
```

Under the hood it just calls the module’s wrapper (five `rocprofv3` passes: four for counters, one for the kernel trace), which you can also run by hand inside an allocation:

```
module load rocm pytorch roofline-extractor # puts roofline-extractor-* on $PATH
HIP_VISIBLE_DEVICES=0 roofline-extractor-profile -o profiling/roofline --arch MI300A -- \
python main.py -a resnet50 --dummy --gpu 0 -b 256 -p 10 --epochs 1
```

Always invoke the tool through the `roofline-extractor-profile / roofline-extractor-extract` wrappers (not `python rooflineExtractor.py`): they set the `PYTHONPATH` for the module’s vendored dependencies. `--arch MI300A` skips auto-detection; `-f csv` needs ROCm 7+ (the default stack here) and is auto-dropped on older ROCm.

The measured roofline for this run (log-log: arithmetic intensity on x, throughput on y; the sloped lines are the HBM/L2/L1/LDS bandwidth roofs rising to the flat compute ceiling; each dot is a kernel):

The run covers 93 unique kernels / 52,001 dispatches / ~14.0 s of GPU time, achieving on average 68 % of the (linear) roofline. The kernels that dominate the time split cleanly into the two roofline regimes:

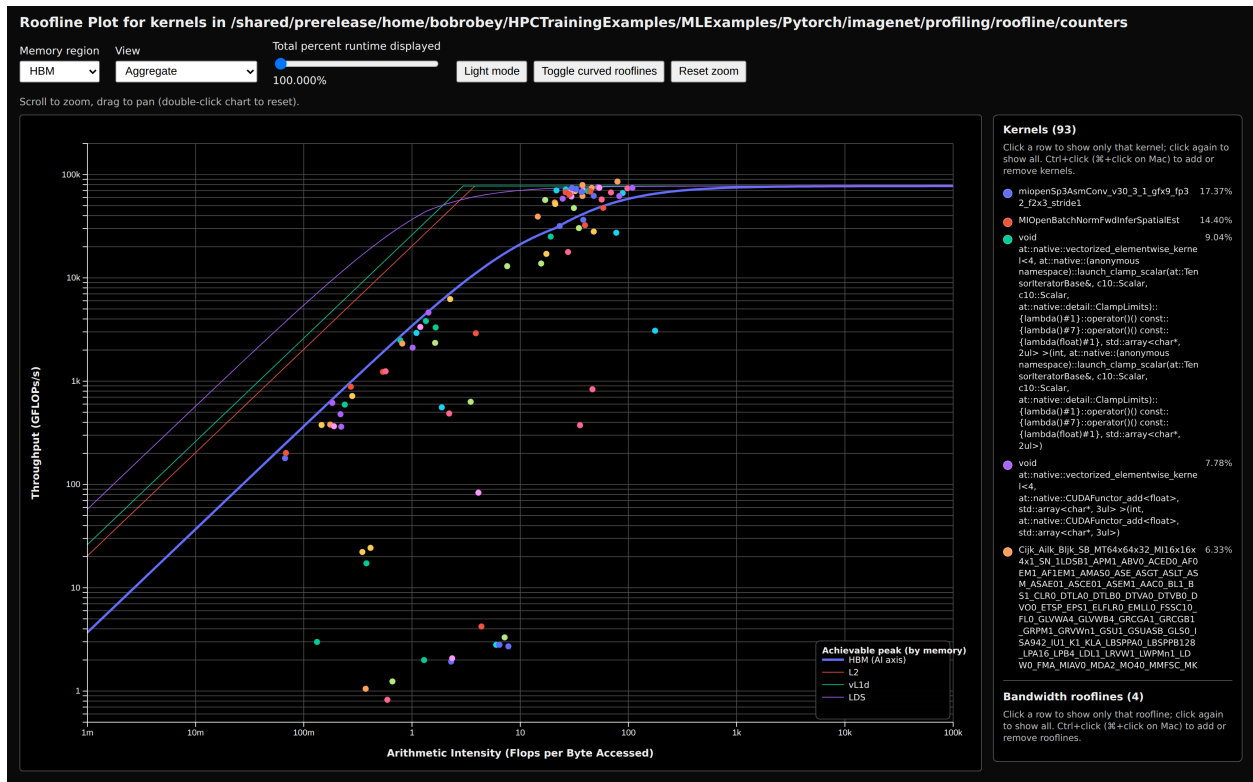


Figure 4: Measured single-MI300A ResNet-50 roofline: each dot is a GPU kernel, plotted by arithmetic intensity vs. achieved throughput against the MI300A bandwidth and compute ceilings

Kernel (ResNet-50 role)	% GPU time	Arith. intensity (FLOP/byte)	Achieved (TFLOP/s)	% of roofline
miopenSp3AsmConv...f2x3 (Winograd conv)	17.4 %	38.2	36.4	49 %
Cijk_...MT128x32x32... (rocBLAS GEMM)	4.8 %	69.0	67.2	65 %
Cijk_...MT64x64x32... (rocBLAS GEMM)	6.3 %	20.9	53.7	70 %
MIOpenBatchNormFwdInferSpatial1Est (batch norm)	1.1 %	0.54	1.23	62 %
vectorized_elementwise_kernel (clamp/ReLU)	9.0 %	0.78	2.49	87 %
vectorized_elementwise_kernel (add)	7.8 %	0.18	0.62	91 %

Read the roofline like this:

- High arithmetic intensity means compute-bound (right side, up against the flat ceiling). The conv and GEMM kernels (AI 21-69 FLOP/byte) sit near the FP32 compute roof (~74 TFLOP/s linear) and already reach tens of TFLOP/s. More HBM bandwidth would not help them; the levers are better matrix-engine utilization or lower precision — bf16 autocast (compute optimizations) moves them toward the far higher MFMA ceiling. (This run is FP32: main.py uses no AMP.)
- Low arithmetic intensity means memory-bound (left side, on the sloped roof). Batch norm and the elementwise ops (AI 0.18-0.78) run at ~1-2 TFLOP/s because they are limited by HBM bandwidth, not FLOPs. Their lever is fusion — fewer passes over HBM — which is exactly what `torch.compile` (compute optimizations) does by folding elementwise/BN work into the conv/GEMM epilogues.

Viewing the interactive plot. `roofline-extractor-profile` writes everything to `profiling/roofline/`:

Artifact	What it is
<code>counters.html</code>	interactive D3 roofline plot (hover a dot for the kernel; toggle HBM/LDS roofs)
<code>counters_EXTRACTED_AGG.csv</code>	per-kernel aggregated metrics (AI, throughput, % of roofline, limiter)
<code>counters_EXTRACTED.csv</code>	per-dispatch metrics
<code>profile_*.log</code>	the guided per-kernel text analysis (source of the table above)

The plot is an interactive HTML file, so the figure above is a screenshot of `counters.html` opened in a browser — the easiest way on a headless node is the noVNC/XFCE desktop (or `scp` the file and open it locally). For the terminal summary without any display, read the `profile_*.log` or the `*_AGG.csv`. To fold several runs/phases into one combined roofline, use the wrapper's `-D/--directory` mode; `--dump` writes the CSVs and `--send NAME` renames the HTML.

Where this figure comes from. `figs/roofline_1gpu.png` is a screenshot of the `profiling/roofline/counters.html` produced by one measured single-MI300A run of `profiling/capture_roofline.sbatch`; the numbers in the table are quoted verbatim from that run's `profile_*.log`.

13. Cleanup

```
deactivate
cd ..
rm -rf imagenet_test
```

14. Run on CPX partitions (SH5_MI300A_CPX, PPAC_MI300A_CPX)

The sweep above assumes SPX mode, where each MI300A APU is one HIP device (so PPAC_MI300A_SPX --gpus=4 = 4 devices). The same study runs on CPX partitions with just some small changes to mimic running on larger systems with multiple nodes. The CPX compute mode subdivides the MI300A node where each of an APU's 6 XCDs is exposed as its own HIP device:

Partition	Physical APUs	HIP devices	What one device is
SH5_MI300A_CPX	1	6	one XCD (~1/6 of an APU)
PPAC_MI300A_CPX	4	24	one XCD (~1/6 of an APU)

Two ready-to-submit batch scripts drive the CPX sweeps:

```
sbatch run_imagenet_uv_sh5_cpx.sbatch      # 1 APU, GPU_LIST="1 2 4 6"
sbatch run_imagenet_uv_ppac_cpx.sbatch     # 4 APUs, GPU_LIST="1 2 4 6 12 24"
```

Both are like the SPX driver but with the sweep written as a loop over a GPU-count list, and two CPX-specific adjustments:

- Smaller per-GPU batch. A CPX partition has ~1/6 the compute and memory of a full APU, so the SPX per-GPU batch of 128 can OOM or crawl. The scripts default to PERGPU_BATCH=32; the global batch is $N * \text{PERGPU_BATCH}$ (weak scaling). Both GPU_LIST and PERGPU_BATCH are overridable at submit time, e.g. `sbatch --export=ALL,PERGPU_BATCH=64 run_imagenet_uv_sh5_cpx.sbatch`. Tune it up until `peak_mem_mb` approaches the partition's memory limit (which depends on whether the node uses shared (NPS1) or split (NPS4) memory — check `rocm-smi`).
- Extended GPU-count list. SH5 goes up to 6 (one XCD up to the whole chip); PPAC continues to 12 and 24.

`images_per_sec.sh` handles either sweep: it derives the GPU count N from each `run_<N>.log` name and reads PERGPU_BATCH from the environment, printing the same `img/s` / `speedup` lines (speedup is relative to the smallest N).

Interpreting CPX results. The RCCL story differs sharply between the two:

- On SH5_MI300A_CPX all ranks live on one APU, so the gradient all-reduce travels over the on-package Infinity Fabric. It is extremely fast, so `RCCL_TOTAL_MS / comm` cost stays near zero even at 6 GPUs — communication looks almost free.
- On PPAC_MI300A_CPX up to 6 ranks stay intra-APU (cheap), but at $N=12$ and $N=24$ the collective crosses physical APUs (socket-to-socket links), so RCCL cost should visibly rise. This is the CPX sweep that best reproduces the real communication behavior the example is built to expose.

So CPX is a fine substitute for running the *scaling mechanics*, but the intra-chip CPX scaling understates RCCL cost — the interesting communication behavior only appears once the rank count crosses physical APUs (i.e. on the 24-GPU PPAC_MI300A_CPX node).

Next steps

- `benchmarks/` — the rigorous study: automated sweep drivers (`benchmarks/ddp_bench_sweep.sh`), optimization levers (`--channels-last`, `--amp`, `--compile`), the required MI300A/RCCL settings,

measured results, batch jobs, the pure-RCCL bandwidth micro-benchmark, and how this compares to the other distributed examples.

- Self-contained runs — `run_imagenet_uv.sh` (and `submit_imagenet_uv.batch`) run the whole scaling sweep end-to-end with the same self-contained pattern as the optimization batch studies, with automatic cleanup. See `benchmarks/README_benchmark.md`.

Appendix A: noVNC desktop (view GUIs like ParaProf/Perfetto in the browser)

Some tools (ParaProf, ParaView, or just a browser for `ui.perfetto.dev`) need a graphical desktop. noVNC gives you the compute node's XFCE desktop in your local browser — no VNC client install — proxied through the control node's nginx (TLS + TOTP). Full details are in `man aac6_novnc` and `man aac6_vnc`; the short version:

1. Open an SSH tunnel from your laptop/workstation to the nginx proxy (port 6443), and enter your TOTP when prompted:

```
ssh -L 6443:10.194.42.31:6443 <username>@aac6.amd.com
```

2. Get a node allocation and note the node name from the output:

```
salloc -p PPAC_MI300A_SPX -N1 --gpus=4 -t 08:00:00
# salloc: Nodes ppac-pl1-s24-16 are ready for job
```

The Slurm prolog automatically starts a TurboVNC desktop (display :1, 1920×1080) and the websockify bridge on that node — no manual `vncserver` needed.

3. Open the noVNC URL in your local browser, substituting your node name:

```
https://localhost:6443/novnc/<node name>/vnc.html
# e.g. https://localhost:6443/novnc/ppac-pl1-s24-16/vnc.html
```

The proxy uses a self-signed certificate, so your browser shows a one-time security warning — approve it to continue (the connection is still TLS-encrypted; see `man aac6_novnc` to import the cert and silence the warning).

4. Log in with your cluster username and current 6-digit TOTP code (this is the *only* login — there is no separate VNC password; the desktop is started with security type `None` and is protected by the TOTP proxy):
5. Click Connect to open the XFCE desktop:

A signed session cookie is issued, so you only log in once per 24-hour browser session. From the desktop you can launch `paraprof &` on the TAU `profile.*`, open a browser on `ui.perfetto.dev`, or run any other GUI tool.

Navigating the XFCE desktop (a few hints):

- Launch apps: the Applications menu is at the top-left of the panel — *Terminal Emulator*, *File Manager*, a web browser, and *Settings* live there. You can also right-click the desktop for a quick app menu and *Open Terminal Here*.
- Panel & clock: the top panel lists open windows and holds the clock and the workspace switcher; XFCE has several virtual desktops — click a square to switch, or drag a window onto one to move it.
- Windows: drag the title bar to move, double-click it to maximize, and use `Alt+drag` to move a window that's larger than the screen.
- Copy/paste & fullscreen (noVNC): hover the small tab on the left edge of the browser window to open the noVNC control bar. It has a Clipboard panel (paste text between your laptop and the remote desktop), a Fullscreen button, and Settings, then Scaling Mode, then Local Scaling, so the 1920×1080 desktop fits your browser window.
- Files: your `$HOME` on the cluster is shared, so `profiling/tau/`, `figs/`, etc. are the same paths you see over SSH.

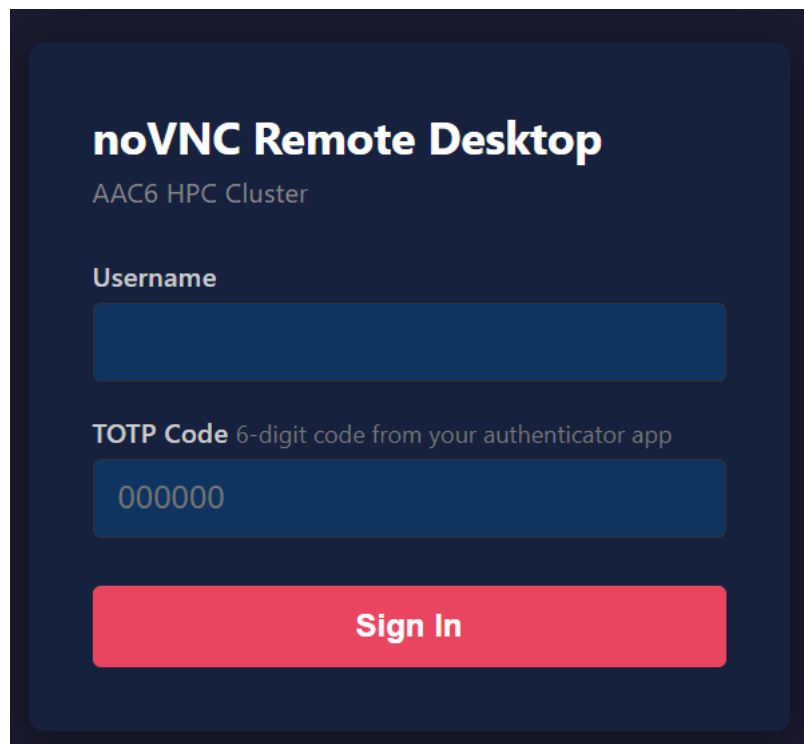


Figure 5: noVNC login page: enter your cluster username and 6-digit TOTP code



Figure 6: noVNC connect screen

Open a TAU profile in the desktop. In a desktop terminal (Applications menu, then *Terminal Emulator*, or right-click the desktop and select *Open Terminal Here*), capture or reuse a profile as in the TAU manual flow / automated flow, then from its `profile.*` directory (`profiling/tau/`) launch the GUI browser:

```
cd profiling/tau
paraprof &          # GUI profile browser (needs the desktop); `pprof` gives the text
→ summary
```

ParaProf shows one bar per rank; double-click a rank to drill into the per-kernel breakdown (RCCL all-reduce vs. ResNet-50 compute). It explores interactively the same per-rank data rendered headless in `figs/tau_profile_4gpu.png` — see the TAU manual flow for what that split means.

Shut down in order (`man aac6_vnc`): close the noVNC browser tab first, then release the Slurm job (`exit/scancel`), then exit the SSH session. The WebSocket is forwarded through the tunnel, so leaving the tab open makes the SSH logout hang. If it hangs, close the tab, or press **Enter** ~ . to force-close SSH.

Only one TOTP prompt is expected. If a *second* page asks for a password, the node's VNC server is advertising PAM auth types instead of `None`; restart it with `vncserver -kill :1 && vncserver :1 -geometry 1920x1080 -securitytypes None -localhost` on the node, or report it to the admins.

RCCL optimization exercises (hands-on `main.py` edits)

Here we present a set of small, self-contained tips for optimizing the RCCL gradient all-reduce in the upstream PyTorch imagenet example. We apply every change by hand in `main.py`, so we can see exactly where each optimization lives and reuse the same pattern in our own workloads.

These build on the scaling study in `README.md`. We assume the nodes all have `iommu=pt` set (as they do on AAC6), so RCCL already uses the direct xGMI / Infinity Fabric path: we do not need to touch the transport for correctness, only to tune it.

Setup

These exercises assume §1-8 of `README.md` have already been completed. If not, work through them first.

Baseline command (reuse this to assess the impact of each tip)

```
HIP_VISIBLE_DEVICES=0,1,2,3 python main.py -a resnet50 --dummy \
--dist-url 'tcp://127.0.0.1:23456' --dist-backend nccl \
--multiprocessing-distributed --world-size 1 --rank 0 -b 512 -p 20 --epochs 1
```

We keep it simple: do one edit, rerun the same baseline command, compare the numbers, then move to the next. Undo an edit before the next one unless a section says to stack them, so each contribution stays isolated.

We record two numbers each time:

- `RCCL_TOTAL_MS`: total RCCL communication time (lower is better).
- the `Time` value in the `Epoch:` lines: the average per-step time (lower is better; this is what improves when communication is hidden behind compute).

The RCCL signal is small on a single MI300A (on-package fabric is nearly free). For a stronger signal, rerun with more GPUs: especially the `PPAC_MI300A_CPX` 12- and 24-GPU cases, where the all-reduce crosses physical APUs.

Section 1: Reduce the bytes on the wire

The fewer bytes RCCL moves, the cheaper the all-reduce. This is usually the biggest single win.

1a. bf16 gradient compression

--amp casts compute to bf16, but DDP still all-reduces fp32 gradients. A communication hook halves the bytes by compressing gradients to bf16 for the all-reduce only.

Find the DDP line in `main_worker()`:

```
        model = torch.nn.parallel.DistributedDataParallel(model,
↪   device_ids=args.gpu)
```

Add the hook right below it (same indentation):

```
        model = torch.nn.parallel.DistributedDataParallel(model,
↪   device_ids=args.gpu)
        from torch.distributed.algorithms.ddp_comm_hooks import default_hooks as
↪   ddp_hooks
        model.register_comm_hook(None, ddp_hooks.bf16_compress_hook)
```

Expect: `NCCL_TOTAL_MS` drops toward half, and per-step Time improves when the run is communication-bound (more GPUs, or across APUs). Try `fp16_compress_hook` for comparison.

Section 2: Tune the RCCL transport and algorithm

RCCL reads its `NCCL_*` settings when the communicator is built, at `dist.init_process_group(...)`. So these must be set in `main.py` before that call. That ordering is the whole point of doing it here rather than exporting a shell variable after the fact.

Find this block in `main_worker()`:

```
        dist.init_process_group(backend=args.dist_backend, init_method=args.dist_url,
                               world_size=args.world_size, rank=args.rank)
```

Add the setting(s) on the line above it (8-space indent), for example:

```
        os.environ["NCCL_ALGO"] = "Tree"
        dist.init_process_group(backend=args.dist_backend, init_method=args.dist_url,
                               world_size=args.world_size, rank=args.rank)
```

2a. Collective algorithm: NCCL_ALGO

```
        os.environ["NCCL_ALGO"] = "Tree"    # try "Ring" (default) vs "Tree"
```

Ring maximizes bandwidth for ResNet50's large gradient buckets; Tree cuts latency and often wins once the all-reduce crosses APUs (12/24-GPU CPX).

2b. Wire protocol: NCCL_PROTO

```
        os.environ["NCCL_PROTO"] = "LL128"    # try "Simple", "LL", "LL128"
```

LL128 is usually the sweet spot for medium messages on high-bandwidth coherent links; Simple favors the largest messages.

2c. Channel count: NCCL_MIN_NCHANNELS

```
os.environ["NCCL_MIN_NCHANNELS"] = "8"    # more channels = more parallelism
```

More channels drive the copy with more compute units, raising effective bandwidth on big all-reduces until it saturates. (NCCL_MAX_NCHANNELS caps it.)

Expect (2a-2c): RCCL_TOTAL_MS shifts up or down; the best choice depends on message size and whether ranks span APUs. Change one variable at a time.

Section 3: Overlap and shrink the collectives (DDP knobs)

These don't change the bytes; they change how well the all-reduce is hidden behind the backward pass and how many separate collectives are launched. Watch the per-step Time here more than RCCL_TOTAL_MS.

All three edit the same DDP line in `main_worker()`:

```
model = torch.nn.parallel.DistributedDataParallel(model,
↪ device_ids=[args.gpu])
```

3a. Avoid a gradient copy: `gradient_as_bucket_view=True`

```
model = torch.nn.parallel.DistributedDataParallel(
    model, device_ids=[args.gpu], gradient_as_bucket_view=True)
```

Lets DDP reduce gradients in place instead of copying them into buckets.

3b. Bucket size: `bucket_cap_mb`

```
model = torch.nn.parallel.DistributedDataParallel(
    model, device_ids=[args.gpu], bucket_cap_mb=100)
```

Default is 25 MB. On fast fabric, larger buckets mean fewer, larger, more bandwidth-efficient all-reduces (trade-off: slightly less compute/comm overlap). Sweep 25, then 50, then 100.

3c. Static graph: `static_graph=True`

```
model = torch.nn.parallel.DistributedDataParallel(
    model, device_ids=[args.gpu], static_graph=True)
```

Tells DDP the graph is fixed each step, cutting per-iteration bookkeeping (and enabling better overlap). Do not set `find_unused_parameters=True`: it adds a synchronization.

You can stack all three:

```
model = torch.nn.parallel.DistributedDataParallel(
    model, device_ids=[args.gpu],
    gradient_as_bucket_view=True, bucket_cap_mb=100, static_graph=True)
```

Expect: per-step Time drops as the same all-reduce overlaps better with compute; RCCL_TOTAL_MS (the bytes) stays about the same.

What moves what

Exercise	Edit location in <code>main.py</code>	Watch
1a bfl16 hook	after <code>DistributedDataParallel(...)</code>	<code>RCCL_TOTAL_MS</code> lower (~half)
2a <code>NCCL_ALGO</code>	before <code>init_process_group</code>	<code>RCCL_TOTAL_MS</code>
2b <code>NCCL_PROTO</code>	before <code>init_process_group</code>	<code>RCCL_TOTAL_MS</code>
2c <code>NCCL_MIN_NCHANNELS</code>	before <code>init_process_group</code>	<code>RCCL_TOTAL_MS</code>
3a <code>gradient_as_bucket_view</code>	<code>DistributedDataParallel(...)</code> args	step Time
3b <code>bucket_cap_mb</code>	<code>DistributedDataParallel(...)</code> args	step Time
3c <code>static_graph</code>	<code>DistributedDataParallel(...)</code> args	step Time

Apply to your own workload: the two lines that matter everywhere are the communication hook (Section 1, right after we wrap the model in DDP) and the DDP constructor arguments (Section 3). The `NCCL_*` settings (Section 2) are workload- and topology-dependent, so always re-measure to be safe.

Measured impact: batch study (NCCL algorithm / protocol / channels, 4 GPUs)

The `NCCL_*` levers above are packaged as a self-contained SLURM sweep, `run_imagenet_uv_rccl_opt_sweep.sbatch`. It builds a disposable `uv` venv, clones the upstream example, applies the source-code instrumentation, warms the MIOpen cache, then runs one phase per swept value on 4 SPX GPUs at batch 512, holding the other two knobs at their NCCL defaults.

```
# NCCL algorithm / protocol / channel-count sweep (4 GPUs, SPX, b=512)
sbatch run_imagenet_uv_rccl_opt_sweep.sbatch

# read the summary the job appends to its .out file
sed -n '/=== RCCL total time by NCCL algorithm/, $p' run_imagenet_uv_spx-<jobid>.out
```

On PPAC_MI300A_SPX the 11-phase sweep (2 algorithm + 3 protocol + 6 channel) takes about 44 minutes end to end (`--time=08:00:00` is generous headroom); each phase re-pays MIOpen/allocator warmup.

Each table reports `RCCL_TOTAL_MS` (defined above; lower is better) with only the swept knob set:

Algorithm (protocol + channels = default):

NCCL_ALGO	RCCL_TOTAL_MS
Tree	42630
Ring	45848

Protocol (algorithm + channels = default):

NCCL_PROTO	RCCL_TOTAL_MS
LL	4111
Simple	10819

NCCL_PROTO	RCCL_TOTAL_MS
LL128	37944

Channel count (algorithm + protocol = default):

NCCL_MIN/MAX_NCHANNELS	RCCL_TOTAL_MS
1	30821
8	37725
16	40134
2	41681
32	44286
4	45877

Takeaway: NCCL_PROTO=LL is the dominant lever here, ~9-10x less collective time than the LL128/auto baseline for this ~102 MB gradient all-reduce; **Tree** edges out **Ring** (~7 %); and on a single APU fewer channels win (1 is best, extra channels add CUDA-block overhead without a bandwidth payoff).

Single-APU caveat. On one SPX APU the all-reduce stays on the on-package Infinity Fabric, so these are relative signals on a small, fast collective. The ranking (especially the protocol effect) is far more pronounced once the collective crosses physical APUs: rerun the sweep on the 12-/24-GPU PPAC_MI300A_CPX node (see the CPX section of README.md) to see the large-message behavior.

NCCL/RCCL variable defaults

On ROCm, `librccl` honors the NCCL_* names. Unless they are set, RCCL auto-selects; the sweep’s “default” columns above reflect those auto choices:

Variable	Default	Behavior when unset
NCCL_ALGO	auto	Internal performance model picks per collective/size & topology (typically Ring or Tree).
NCCL_PROTO	auto	Chooses per message size from the allowed set: <code>LL</code> , <code>LL128</code> , <code>Simple</code> on supported platforms (<code>LL</code> , <code>Simple</code> otherwise). <code>LL</code> for small, <code>LL128</code> for medium, <code>Simple</code> for large messages.
NCCL_MIN_NCHANNELS	platform-dependent	Auto-tuned lower bound on channels (CUDA blocks) from topology.
NCCL_MAX_NCHANNELS	platform-dependent (max capped at 32 in recent NCCL/RCCL)	Auto-tuned upper bound; RCCL picks the actual count within <code>[min,max]</code> .

Upstream guidance is that these are best left unset: manual values only win for a specific message size / topology, which is exactly what this sweep demonstrates (NCCL_PROTO=LL beats the auto choice for this particular all-reduce size).

Compute optimization exercises (hands-on `main.py` edits)

Here we present a set of small, self-contained tips for speeding up the per-GPU compute of the upstream PyTorch imagenet example (ResNet50 forward/backward). We apply every change by hand in `main.py`, so we can see exactly where each optimization lives and reuse the same pattern in our own workloads.

These build on the scaling study in `README.md` but target *compute*, not communication, so the exercises use a single-GPU baseline: that isolates kernel/precision/overhead effects from the RCCL all-reduce. Once we have picked the fast compute settings, we rerun the multi-GPU scaling sweep in `README.md` to see the combined effect.

Setup

These exercises assume §1-8 of `README.md` have already been completed. If not, work through them first.

Baseline command (reuse this to assess the impact of each tip)

```
HIP_VISIBLE_DEVICES=0 python main.py -a resnet50 --dummy \
  --dist-url 'tcp://127.0.0.1:23456' --dist-backend nccl \
  --multiprocessing-distributed --world-size 1 --rank 0 -b 128 -p 20 --epochs 1
```

We keep it simple: do one edit, rerun the same baseline command, compare the per-step time, then move to the next. Undo an edit before the next one unless a section says to stack them, so each contribution stays isolated.

We watch the `Time` value in the `Epoch: lines:` the average per-step time (lower is better). Throughput is roughly `img/s = 128 / Time`. We can also watch memory in another shell with `rocm-smi` (bf16 and `channels_last` also reduce memory, which leaves room for a bigger batch later).

Section 1: Lower-precision math

MI300A has fast bf16; using it for the matmul/conv-heavy work is the single biggest compute win.

1a. bf16 autocast (AMP)

Casts the forward pass and loss to bf16. bf16 has enough dynamic range that no `GradScaler` is needed.

Find, in the training loop of `train()`:

```
# compute output
output = model(images)
loss = criterion(output, target)
```

Wrap it in an autocast context:

```
# compute output
with torch.autocast(device_type="cuda", dtype=torch.bfloat16):
    output = model(images)
    loss = criterion(output, target)
```

Expect: per-step `Time` drops substantially, and peak memory drops too.

1b. Reduced-precision float32 matmuls

For any ops left in fp32, allow the lower-precision (bf16-accumulate) matmul path. Add once near the top of `main_worker()`, just after `args.gpu = gpu`:

```
def main_worker(gpu, ngpus_per_node, args):
    global best_acc1
    args.gpu = gpu
    torch.set_float32_matmul_precision("high")
```

Expect: a smaller gain than 1a on its own, mainly for runs that stay in fp32. Harmless to leave on together with 1a.

Section 2: Memory layout and kernel selection

2a. channels_last (NHWC) memory format

CDNA convolutions prefer NHWC. Converting the model and the input batch lets MIOpen pick faster kernels. This is two edits.

Edit 1: right after the model is created in `main_worker()`:

```
print("=> creating model '{}'.format(args.arch))
model = models.__dict__[args.arch]()
model = model.to(memory_format=torch.channels_last)
```

Edit 2: in the `train()` loop, convert the input batch. Find:

```
images = images.to(device, non_blocking=True)
```

and change it to:

```
images = images.to(device,
↪ non_blocking=True).to(memory_format=torch.channels_last)
```

Expect: per-step Time drops, and it stacks well with bf16 (1a): that combination is the recommended default.

2b. Autotune convolution kernels: `cuda.cudnn.benchmark`

Lets the backend (MIOpen on ROCm) search for the fastest conv algorithm for the fixed input shape and cache it. Add near the top of `main_worker()`:

```
def main_worker(gpu, ngpus_per_node, args):
    global best_acc1
    args.gpu = gpu
    cudnn.benchmark = True
```

Expect: a one-time search cost on the first steps, then a faster steady-state Time. (`MIOpen_FIND_MODE=FAST` and the warmed cache from README.md §3 keep the search cheap.) Note this is incompatible with the deterministic `--seed` path.

Section 3: Fuse kernels and cut launch overhead

3a. `torch.compile`

Captures the model into a graph and fuses kernels, cutting Python/launch overhead.

Find the DDP line in `main_worker()`:

```
        model = torch.nn.parallel.DistributedDataParallel(model,
↪    device_ids=[args.gpu])
```

Compile the model right after it (same indentation):

```
        model = torch.nn.parallel.DistributedDataParallel(model,
↪    device_ids=[args.gpu])
        model = torch.compile(model)
```

Expect: the first step is much slower (one-time compile), then per-step Time improves. Try `torch.compile(model, mode="max-autotune")` for more aggressive tuning at a longer compile cost.

3b. Fused optimizer + cheaper zero_grad

A fused optimizer does the whole parameter update in one kernel instead of one per tensor, cutting launch overhead.

Edit 1: the optimizer in `main_worker()`. Find:

```
optimizer = torch.optim.SGD(model.parameters(), args.lr,
                             momentum=args.momentum,
                             weight_decay=args.weight_decay)
```

Add `fused=True`:

```
optimizer = torch.optim.SGD(model.parameters(), args.lr,
                             momentum=args.momentum,
                             weight_decay=args.weight_decay, fused=True)
```

Edit 2: in the `train()` loop, make the gradient clear cheaper. Find `optimizer.zero_grad()` and change it to:

```
optimizer.zero_grad(set_to_none=True)
```

Expect: a small Time improvement, biggest when steps are short (small model or after the other optimizations shrink compute).

Recommended stack

For ResNet50 on MI300A, the high-value combination is `bf16 autocast` (1a), `channels_last` (2a), and `torch.compile` (3a). We apply all three, then rerun the multi-GPU sweep from `README.md`: compute gets faster, which also makes the RCCL all-reduce a larger share of each (now shorter) step. That is why the RCCL exercises matter more once compute is optimized.

What moves what

Exercise	Edit location in <code>main.py</code>	Watch
1a <code>bf16 autocast</code>	forward/loss in <code>train()</code> loop	Time much lower, memory lower
1b <code>matmul precision</code>	top of <code>main_worker()</code>	Time (fp32 ops)
2a <code>channels_last</code>	after model create + input in loop	Time lower (with 1a)
2b <code>cuda.benchmark</code>	top of <code>main_worker()</code>	Time lower after warm-up
3a <code>torch.compile</code>	after <code>DistributedDataParallel(...)</code>	Time lower (slow 1st step)

Exercise	Edit location in <code>main.py</code>	Watch
3b fused optimizer	SGD args + <code>zero_grad</code> in loop	Time (small)

Apply to your own workload: the portable patterns are the autocast context around the forward/loss (1a), the `channels_last` conversion of model and inputs (2a), and wrapping the model in `torch.compile` (3a). Always re-measure: gains depend on model shape and how compute- or memory-bound the kernels are.

Measured impact: batch study (`torch.compile`, 1 GPU)

The edits above are packaged as a self-contained SLURM script, `run_imagenet_uv_compute_opt.sbatch`, so the throughput impact is reproducible with a single submission. The script builds a disposable `uv` venv, clones the upstream example, applies the source-code instrumentation, warms the MIOpen cache, then runs the model twice on one SPX GPU at batch 512: once eager and once through `torch.compile`.

```
# eager vs torch.compile (1 GPU, SPX, b=512)
sbatch run_imagenet_uv_compute_opt.sbatch

# read the summary the job appends to its .out file
sed -n '/=== compute optimization/, $p' run_imagenet_uv_spx-<jobid>.out
```

On `PPAC_MI300A_SPX` the job takes about 14 minutes end to end (`--time=02:00:00` is generous headroom); the wall time is dominated by the venv build, the upstream clone, MIOpen warmup, and the one-time `torch.compile` graph build rather than the ~100 timed steps.

We report throughput as a steady-state median, not a running average. The script's `summarize()` collects the instantaneous per-step `Time`, drops the first (warmup / one-time graph-compile) step, and takes the median; `img/s` is the global batch divided by that median. Averaging `Time` over the full 100-iteration cap instead would fold the tens-of-seconds first compile step into the mean and make `torch.compile` look like a regression.

The script prints (steady-state median over its 100-iteration capped run, ~9 samples after the warmup step is dropped):

Metric	eager	<code>torch.compile</code>	Impact
Steady-state median step	0.416 s	0.384 s	-7.7 % (faster)
Steady-state img/s	1232	1335	+8.3 %
Peak memory (<code>PEAK_MEM_MB</code>)	44874 MB	42329 MB	-5.7 %

A 100-iteration window samples only the fast early steps, so it reads slightly optimistic. Re-measuring the same median over a full epoch (2503 steps, ~116 samples per phase) gives the sustained figures eager 0.445 s / 1151 img/s and compile 0.421 s / 1216 img/s, a steadier +5.4 %. Either way the ranking is the same.

Takeaway: once the one-time compile is amortized, `torch.compile` gives a solid throughput gain (~5-8 %) and lower peak memory for ResNet-50 on a single MI300A. Try `torch.compile(model, mode="max-autotune")` for a larger gain at a longer compile cost.

When resources allow, prefer longer runs. The study is capped at 100 iterations so it finishes quickly and stays a polite neighbor on a shared system. The steady-state median (first step dropped) already keeps the one-time graph build out of the headline number, but 100 iterations still read slightly optimistic versus the sustained rate. With more compute and fewer users, raise the iteration cap (`if i >= 100: break` in `main.py`) or run full epochs so the compile cost amortizes and the median approaches the true sustained throughput, which is the number to quote for real training.

ImageNet DDP benchmark: rigorous RCCL scaling study

README_benchmark.md from HPCTrainingExamples/MLExamples/Pytorch/imagenet/benchmarks in the Training Examples repository.

This is the in-depth companion to the quick-start README.md. Start there for the simple, manual step-by-step sweep. This document covers the scaling-sweep drivers, the required MI300A environment, the optimization levers, measured results, the pure-RCCL bandwidth micro-benchmark, profiling, and how this example relates to the other distributed examples.

On ROCm, PyTorch's `nccl` backend is provided by `librccl`. All the `NCCL_*` environment variables below are honored by RCCL.

Contents

File	Purpose
<code>ddp_resnet_bench.py</code>	Recommended torchrun DDP ResNet benchmark; synthetic data, <code>no_sync()</code> RCCL isolation, <code>--channels-last/--amp</code>
<code>ddp_bench_sweep.sh</code>	Scaling driver for <code>ddp_resnet_bench.py</code> (pre-warms MIOpen, prints <code>comm%/speedup/efficiency</code>); <code>OPT_SETS=</code> enables baseline-vs-knob comparison
<code>submit_ddp_bench.batch</code>	Slurm job: baseline + optimized (<code>channels_last/AMP</code>) sweeps
<code>run_imagenet_uv.sh</code>	Self-contained upstream <code>main.py --dummy</code> sweep in a disposable uv venv (auto clone + install + warm + cleanup)
<code>submit_imagenet_uv.batch</code>	Slurm job wrapper for <code>run_imagenet_uv.sh</code> (per-job MIOpen cache, auto cleanup)
<code>warm_miopen.py</code>	One-shot single-process MIOpen cache warm-up helper
<code>rccl_allreduce_bench.py</code>	Standalone all-reduce bandwidth benchmark (pure RCCL, no model)
<code>rccl_scaling_sweep.sh</code>	Sweep driver for the upstream <code>imagenet/main.py --dummy</code> (see caveat below)
<code>pytorch_imagenet_ddp_venv.batch</code>	Slurm job: venv install + full scaling sweep
<code>pytorch_imagenet_ddp_module.batch</code>	Slurm job: <code>module load</code> variant of the sweep
<code>submit_rccl_allreduce.batch</code>	Slurm job: all-reduce bandwidth vs. GPU count

This example intentionally does not copy `main.py`; it drives the upstream `pytorch/examples` copy so it always tracks the canonical version.

MI300A / AAC6 settings and a caveat about `main.py`. The PPAC MI300A nodes are now booted with `iommu=pt` (required, see fix 1 below), so RCCL uses direct xGMI P2P and no P2P override is needed. With the default MIOpen tuning, though, the naive test can still take many minutes to start. The MI300A settings section documents the required environment and shows that the upstream `main.py mp.spawn` path can hang on this build (it initializes CUDA before spawning). The `ddp_resnet_bench.py` + `ddp_bench_sweep.sh` pair is the reliable, measured path and is recommended for the scaling study here.

Running on MI300A: required settings and measured optimization

This section captures what it actually takes to get the test running well on an AMD MI300A node (measured on AAC6, PPAC_MI300A_SPX, ROCm 6.4.3, PyTorch 2.12), and the optimization result.

The three fixes that make the test run

1. RCCL P2P hang: nodes booted without `iommu=pt` emit `NCCL WARN Missing "iommu=pt" ... can lead to ... hang`, and RCCL then hangs on the first collective (it can even mark the node down). The

cause is that intra-node GPU-to-GPU peer-to-peer uses IPC handles (`hipIpcGetMemHandle`), which need IOMMU passthrough to map the DMA.

Real fix (preferred, and now in place on PPAC): boot the node with `iommu=pt`. The PPAC MI300A nodes are now booted with `amd_iommu=on iommu=pt` on the kernel command line, and this setting must remain for the collectives to run. Verify on any node with `grep -o 'iommu=pt' /proc/cmdline`. With passthrough on, RCCL uses direct xGMI/Infinity Fabric and we set nothing below.

Fallback (node not yet fixed): force collectives off P2P so they fall back to host-staged shared memory. This clears the hang but at a large bandwidth cost (no direct xGMI), so use it only until `iommu=pt` is in place:

```
export NCCL_P2P_DISABLE=1
```

NCCL_P2P_LEVEL=SYS is not the right knob: per the NCCL/RCCL docs SYS is the *most* permissive P2P level (it enables P2P across NUMA), not a way to disable it. The host-staging workaround is `NCCL_P2P_DISABLE=1` (equivalently `NCCL_P2P_LEVEL=LOC`). Confirm the transport RCCL actually chose with `NCCL_DEBUG=INFO NCCL_DEBUG_SUBSYS=INIT,GRAPH` (via P2P/xGMI vs via SHM).

2. MIOpen startup: the default MIOpen solver search can take >10 minutes cold for ResNet convolutions, and the site module points the cache at a *per-job* directory so every run recompiles. Fix with fast selection + a persistent shared cache:

```
export MIOPEN_FIND_MODE=FAST
export MIOPEN_USER_DB_PATH=/tmp/$USER/miopen-shared
export MIOPEN_CUSTOM_CACHE_DIR=$MIOPEN_USER_DB_PATH
```

3. `mp.spawn` hang: the upstream `main.py --multiprocessing-distributed` calls `torch.accelerator.device_count()` (initializing CUDA) in the parent *before* `mp.spawn`, which poisons the child CUDA contexts and hangs RCCL on this build. Use `torchrun` instead (independent processes). That is exactly what `ddp_resnet_bench.py` does.

`ddp_bench_sweep.sh` sets no P2P override (the PPAC nodes have `iommu=pt`, so RCCL uses direct xGMI), applies fix-2, and pre-warms the MIOpen cache single-process (so ranks don't contend on the cold cache SQLite db).

If you ever land on a node without `iommu=pt`, export `NCCL_P2P_DISABLE=1` before the sweep; the driver scripts pass that through to the ranks. That is a host-staged fallback only: the correct fix is to keep `iommu=pt` on the node.

Recommended workflow

```
salloc -p PPAC_MI300A_SPX -N1 --gpus=4 -t 00:20:00
module load rocm openmpi pytorch      # add version pins to sample a combo, e.g.
→ rocm/7.2.3 pytorch/2.12.0

# Baseline weak scaling (per-GPU batch held constant):
GPUS="1 2 4" ARCH=resnet50 BATCH=128 ./ddp_bench_sweep.sh

# Optimized: NHWC memory format + bf16 autocast
GPUS="1 2 4" ARCH=resnet50 BATCH=128 OPTS="--channels-last --amp" ./ddp_bench_sweep.sh

# Add torch.compile (graph capture + kernel fusion) on top:
GPUS="1 2 4" ARCH=resnet50 BATCH=128 OPTS="--channels-last --amp --compile"
→ ./ddp_bench_sweep.sh
```

```
# Knob comparison: baseline vs each optimization at each GPU count (OPT_SETS is a
# ';' separated "label=flags" list; the first entry is the baseline for vs_base):
GPUS="2 4" OPT_SETS="baseline= ; amp=--amp ; ch+amp=--channels-last --amp ; \
bf16comm=--channels-last --amp --bf16-comm" ./ddp_bench_sweep.sh
```

For the simple, by-hand version of this sweep (one warm-up command + one line per GPU count), see the quick-start README.md.

The optimization levers exposed by `ddp_resnet_bench.py` (pass via `OPTS`, or as the `flags` of an `OPT_SETS` entry). These mirror the by-hand exercises in `../README_compute_optimization.md` and `../README_rccl_optimization.md`.

Compute:

Flag	Effect
<code>--channels-last</code>	NHWC memory format: matches CDNA conv layout
<code>--amp</code>	bf16 autocast: the single biggest throughput win
<code>--compile</code> [<code>--compile-mode</code> default\ <code>reduce-overhead</code> \ <code>max-autotune</code>]	<code>torch.compile</code> graph capture + kernel fusion; first (warm-up) step pays a one-time compile cost
<code>--matmul-precision</code> high\ medium	<code>set_float32_matmul_precision</code> : lower-precision path for fp32 matmuls
<code>--cudnn-benchmark</code>	MIOpen fastest-kernel autotuning for the fixed input shape
<code>--fused-optimizer</code>	Fused SGD step (one kernel instead of one per tensor)

RCCL communication:

Flag	Effect
<code>--bf16-comm</code>	bf16 gradient-compression DDP comm hook: halves the all-reduce bytes
<code>--nccl-algo</code> Ring\ Tree	Sets <code>NCCL_ALGO</code> before init (Ring = bandwidth, Tree = latency/cross-APU)
<code>--nccl-proto</code> Simple\ LL\ LL128	Sets <code>NCCL_PROTO</code> before init
<code>--nccl-min-nchannels</code> N	Sets <code>NCCL_MIN_NCHANNELS</code> before init (more parallel channels)

DDP bucketing / overlap:

Flag	Effect
<code>--grad-as-bucket-view</code>	<code>gradient_as_bucket_view=True</code> : skip a gradient copy
<code>--bucket-cap-mb</code> N	All-reduce bucket size (default 25); larger = fewer, bigger collectives
<code>--static-graph</code>	<code>static_graph=True</code> ; incompatible with the <code>no_sync()</code> comm split, so <code>comm_pct</code> is reported as -1; add <code>--rccl-time</code> for the RCCL number

Input staging (MI300A):

Flag	Effect
<code>--migrate</code>	Stage each host input batch to the GPU with zero-copy <code>migrate()</code> (MI300A unified memory; needs <code>HSA_XNACK=1</code>). See <code>../../../../common/README.md</code>
<code>--migrate-method managed\ register</code>	Zero-copy method: <code>managed</code> aliases a <code>hipMallocManaged</code> buffer (default); <code>register</code> <code>hipHostRegisters</code> any pageable tensor (e.g. a <code>DataLoader</code> batch)
<code>--host-copy</code>	Stage each host input batch with a <code>.to()</code> copy: the baseline to compare <code>--migrate</code> against

hipBLASLt note. ResNet `--amp` is conv-bound (MIOpen), so hipBLASLt has little effect here: the `hipblaslt/patched` module makes no measurable difference (244 vs 245 img/s). Unlike the transformer examples, ResNet `--amp` does not hit the ROCm 7.2.x bf16 hipBLASLt stall. See `../../../../common/hipblaslt-notes.md`.

Zero-copy input staging (MI300A). By default the input batch is pre-resident on the GPU. `--host-copy/--migrate` instead produce the batch on the host each step and move it to the GPU: a `hipMemcpy` copy vs. an aliased unified-memory pointer. The raw transfer is $\sim 30\times$ cheaper with `migrate` (see `../../../../common/README.md` for the micro-benchmark), but for this compute-bound step the reused-buffer copy overlaps compute, so end-to-end throughput is within noise. The win shows up for large, per-step-fresh host batches and in memory footprint: the copy path keeps a second device-resident copy of every batch, while `migrate` keeps one (measured 100% of the device duplicate eliminated). Requires `HSA_XNACK=1`.

Runtime, version, and affinity

See `../../../../common/PERFORMANCE_NOTES.md` for the cross-cutting, measured results on MI300A: ROCm/PyTorch version selection (ROCm 6.4 vs 7.2.3, TunableOp, and why the pip wheel vs. site module is a wash at matched ROCm), and NUMA affinity/placement (negligible for these GPU-resident runs; enable with `AFFINITY=1 ./ddp_bench_sweep.sh` for a CPU-heavy or host-staged path).

Measured results (MI300A, resnet50, per-GPU batch 128)

These numbers were measured earlier with the host-staged P2P workaround (nodes without `iommu=pt`, busbw ~ 80 GB/s). The PPAC nodes are now booted with `iommu=pt`, so RCCL uses direct xGMI and the all-reduce is several-fold faster, so `comm%` should drop below what is shown here. Re-run the sweep to refresh these.

Baseline (fp32, NCHW):

GPUs	step_s	comm%	img_per_s	speedup	eff
1	0.1297	2.4	987	1.00	100%
2	0.1321	2.9	1938	1.96	98%
4	0.1338	3.0	3826	3.88	97%

Optimized (`--channels-last --amp, bf16`):

GPUs	step_s	comm%	img_per_s	speedup	eff
1	0.0780	4.0	1641	1.00	100%
2	0.0798	4.8	3208	1.95	98%
4	0.0797	4.1	6424	3.91	98%

Takeaways:

- About 1.67x throughput from AMP bf16 (987 to 1641 img/s at 1 GPU; 3826 to 6424 at 4 GPU). `channels_last` alone was neutral for fp32 convs on this MIOpen build; its value shows up together with AMP.

- Weak-scaling efficiency stays 97-98%: the RCCL all-reduce of ResNet50's ~102 MB of gradients is small and overlaps backward compute.
- `comm%` rises (2.4% to ~4%) after optimization: making compute faster does not change the bytes communicated, so RCCL becomes a larger share of a shorter step. This is the general rule, that the faster the compute, the more communication matters, which is why the RCCL study becomes more important as we optimize.

0. Get the example and an allocation

The `--dummy` ImageNet trainer lives in the upstream `pytorch/examples` repo:

```
git clone --depth=1 https://github.com/pytorch/examples.git ~/pytorch_examples
```

Grab a multi-GPU allocation (8 GPUs shown; adapt the partition to your site):

```
salloc --gpus=8 --ntasks=1 --time=01:00:00
```

Set up PyTorch exactly as in the mnist README (venv, container, or module). For a venv:

```
python3 -m venv rocm-pytorch && source rocm-pytorch/bin/activate
pip3 install torch torchvision --index-url https://download.pytorch.org/whl/rocm6.4
pip3 install -r ~/pytorch_examples/imagenet/requirements.txt
```

Confirm the GPUs are visible:

```
python3 -c 'import torch; print(torch.cuda.is_available(), torch.cuda.device_count())'
amd-smi monitor -g # or: rocm-smi
```

1. A single benchmark step (sanity check)

The example uses `torch.multiprocessing.spawn` internally, so a single command launches one process per visible GPU. Start with a quick `--dummy` run on all GPUs of the node:

```
cd ~/pytorch_examples/imagenet
python main.py -a resnet50 --dummy \
  --dist-url 'tcp://127.0.0.1:23456' --dist-backend nccl \
  --multiprocessing-distributed --world-size 1 --rank 0 \
  -b 2048 -p 5 --epochs 1
```

You will see progress lines like:

```
Epoch: [0] [ 5/625] Time 0.312 (0.318) Data 0.001 (0.002) Loss ... Acc@1 ...
```

The `Time x.xxx (y.yyy)` field is per-step wall time: current (running avg). Read the running average after ~20 steps of warm-up: that is the steady-state step time we use for scaling. Loss/accuracy are meaningless with `--dummy`; that is expected. We can stop the run with Ctrl-C once the average settles.

`--world-size` here is the number of *nodes*. With `--multiprocessing-distributed` the effective world size becomes `nodes x GPUs_per_node`, so `--world-size 1` on an 8-GPU node gives 8 ranks.

2. Confirm the RCCL topology

Before measuring, dump how RCCL wired the GPUs (rings/trees, xGMI peers, whether it fell back to PCIe/SHM). This is the single most useful diagnostic:

```
NCCL_DEBUG=INFO NCCL_DEBUG_SUBSYS=INIT,COLL \
python main.py -a resnet50 --dummy \
```

```
--dist-url 'tcp://127.0.0.1:23456' --dist-backend nccl \
--multiprocessing-distributed --world-size 1 --rank 0 \
-b 2048 -p 50 --epochs 1 2>&1 | grep -E 'NCCL|Ring|Channel|Tree' | head -40
```

Look for lines showing `Channel .. via ... [xGMI] / P2P` (good, on-fabric) vs. `via SHM` or `via PCI` (host-staged, slower). On an MI300A node with a healthy Infinity Fabric mesh you want peer transfers, not SHM.

3. Scaling sweep (the RCCL measurement)

`rccl_scaling_sweep.sh` automates runs at 1, 2, 4, 8 GPUs by masking visible devices (the trainer uses `device_count()` to pick the rank count), extracts the steady-state step time, and prints throughput, speedup, and efficiency.

Two complementary modes:

Weak scaling holds *per-GPU* batch constant (global batch grows with GPUs). With a perfect interconnect the step time stays flat, so any increase is RCCL all-reduce overhead:

```
MODE=weak ARCH=resnet50 GPUS="1 2 4 8" PERGPU_BATCH=256 ./rccl_scaling_sweep.sh
```

Strong scaling holds the *global* batch constant. Speedup below N reflects RCCL cost plus shrinking per-GPU work:

```
MODE=strong ARCH=resnet50 GPUS="1 2 4 8" GLOBAL_BATCH=2048 ./rccl_scaling_sweep.sh
```

Example output shape (numbers are illustrative):

GPUs	step_time_s	img_per_s	speedup	efficiency
1	0.320	800.0	1.00	100%
2	0.331	1547.0	1.93	97%
4	0.345	2968.0	3.71	93%
8	0.370	5535.0	6.92	86%

The drop from 100% is the fraction of time spent in (or stalled behind) the RCCL gradient all-reduce. `NCCL_DEBUG=INFO ./rccl_scaling_sweep.sh` preserves the full RCCL logs per run under `rccl_scaling_logs/`.

Amplify the communication signal to make RCCL cost obvious:

```
# Bigger model => bigger gradient tensors => more bytes all-reduced per step
ARCH=resnet152 MODE=weak GPUS="1 2 4 8" ./rccl_scaling_sweep.sh

# Disable DDP's comp/comm overlap to see the *unhidden* all-reduce cost
NCCL_DEBUG=WARN DDP_OVERLAP=0 ... # (see step 5 profiler for the clean split)
```

4. Pure RCCL bandwidth (compute-independent)

Training throughput mixes compute and communication. To measure RCCL by itself, run the all-reduce micro-benchmark. It launches one rank per GPU with `torchrun` and reports algorithm and bus bandwidth per message size:

```
# 8 ranks on one node
torchrun --standalone --nproc_per_node=8 rccl_allreduce_bench.py

# sweep rank counts by masking GPUs
HIP_VISIBLE_DEVICES=0,1 torchrun --standalone --nproc_per_node=2 rccl_allreduce_bench.py
```

Interpretation:

- $\text{algbw} = \text{size} / \text{time}$: what the application “sees”.
- $\text{busbw} = \text{algbw} \times 2(N-1)/N$: normalizes for the ring all-reduce data movement so it is comparable across GPU counts and should approach the hardware link bandwidth (xGMI/Infinity Fabric).
- busbw that stays flat as N grows means the interconnect scales; busbw that falls means RCCL is the scaling bottleneck for that message size.

The batch script `submit_rccl_allreduce.batch` runs this for 2/4/8 GPUs. With the ROCm `rccl-tests` installed, `all_reduce_perf -b 1M -e 1G -f 2 -g <N>` is the equivalent lower-level reference.

5. Attribute time to RCCL kernels (optional, precise)

To split a real training step into compute vs. communication, wrap the trainer with a profiler. The `ddp_resnet_bench.py` benchmark exposes this directly:

```
# torch.profiler: per-op/kernel + RCCL table, trace per rank under ./torch_prof
torchrun --standalone --nproc_per_node=2 ddp_resnet_bench.py \
  -a resnet50 -b 128 --profile --profile-dir ./torch_prof

# DeepSpeed FlopsProfiler: FLOPs / MACs / params (compute ceiling)
torchrun --standalone --nproc_per_node=1 ddp_resnet_bench.py -a resnet50 -b 64 --flops
```

Quick pointers:

- `torch.profiler`: look for `nccl:all_reduce / ncclDevKernel_AllReduce_*` entries in the key-averages table; their total is the RCCL time.
- `rocprow3` (kernel trace): the RCCL collectives show up as `ncclDevKernel_* / all-reduce kernels`.
- `rocprow-sys` (timeline): captures host+device activity so we can *see* the all-reduce overlapping (or not) with backward compute.

See `./profiling/PROFILING.md` for the full, MI300A-verified guide covering `torch.profiler`, the DeepSpeed FlopsProfiler, TensorBoard, `rocprow3`, `rocprow-compute` (roofline), `rocprower-systems` (timeline), and multi-node TAU/HPCToolkit.

6. Run it as a batch job

```
sbatch submit_ddp_bench.batch           # recommended: module + ddp_bench_sweep.sh
↪ (comm% metrics)
sbatch submit_imagenet_uv.batch         # self-contained uv venv + upstream main.py
↪ sweep
sbatch pytorch_imagenet_ddp_venv.batch  # venv install + weak & strong sweep
sbatch pytorch_imagenet_ddp_module.batch # module load variant
sbatch submit_rccl_allreduce.batch      # pure all-reduce bandwidth sweep
```

Tune RCCL from the environment without touching code, e.g.:

```
export NCCL_DEBUG=INFO           # topology + collective logging
export NCCL_P2P_DISABLE=1       # force off peer-to-peer (compare vs. on)
export NCCL_MIN_NCHANNELS=8     # more channels for large messages
```

Useful RCCL environment variables

Variable	Effect
<code>NCCL_DEBUG=INFO</code>	Print init topology, rings/trees, transport used
<code>NCCL_DEBUG_SUBSYS=INIT,COLL</code>	Restrict debug output to init + collectives

Variable	Effect
NCCL_P2P_DISABLE=1	Disable GPU peer-to-peer (isolate xGMI vs. staged copies)
NCCL_SHM_DISABLE=1	Disable shared-memory transport
NCCL_MIN_NCHANNELS / NCCL_MAX_NCHANNELS	Tune channel count / parallelism
HIP_VISIBLE_DEVICES / ROCr_VISIBLE_DEVICES	Select which GPUs participate

Alternative examples for studying RCCL scaling

The two families below were evaluated as alternatives. Summary:

Example	Collective / API	Dataset & model	Best for	Multi-node
imagenet --dummy (this dir)	DDP all-reduce	FakeData ImageNet, ResNet	First stop: easy, no data, clean weak/strong sweep	yes
distributed/ ddp-tutorial-series	DDP all-reduce	tiny synthetic	Learning the DDP/torchrun launch mechanics	yes
distributed/ minGPT-ddp	DDP all-reduce	char-level text	LLM-shaped gradients (large all-reduce)	yes
distributed/FSDP, FSDP2	all-gather + reduce-scatter	T5 + wikipoh	Sharded (ZeRO-3) comm pattern, big models	yes
distributed/tensor_ parallelism	all-reduce/all-gather in-layer	Llama2 block	Intra-layer TP + sequence parallel comm	yes
MExamples/ TinyTransformer	none (single GPU)	synthetic tokens	Single-GPU kernel profiling only	no

pytorch_examples/distributed/: recommended companions

Located at ~/pytorch_examples/distributed/. These are purpose-built for distributed training and are the best complements to this example:

- ddp-tutorial-series/: single_gpu.py, then multigpu.py, then multigpu_torchrun.py, then multinode.py. Each step adds one piece of the DDP stack. Ideal for teaching the launch mechanics (process groups, torchrun, DistributedSampler, multi-node rendezvous, the included slurm/sbatch_run.sh). The model/data are trivial, so it shows *how to scale*, not *how much*; use it before this imagenet example. Launch:

```
cd ~/pytorch_examples/distributed/ddp-tutorial-series
torchrun --standalone --nproc_per_node=8 multigpu_torchrun.py 50 10
```

- minGPT-ddp/: a GPT-style transformer trained with DDP. Gradient tensors are large and transformer-shaped, so the all-reduce volume per step is much bigger than ResNet, a more LLM-representative RCCL workload than imagenet. Uses the same DDP all-reduce collective, so the measurement methodology here transfers directly. Config-driven (gpt2_train_cfg.yaml) and ships a Slurm script. Good “step 2” once imagenet scaling is understood.

```
cd ~/pytorch_examples/distributed/minGPT-ddp/mingpt
torchrun --standalone --nproc_per_node=8 main.py
```

- **FSDP/ and FSDP2/:** Fully Sharded Data Parallel (ZeRO-style). The collective pattern is different from DDP: parameters are `all_gather`-ed before use and gradients `reduce_scatter`-ed, so there is more, and more frequent, communication. Use these to study the comm pattern of large, memory-sharded models (T5 on the wikipedi summarization dataset). Higher setup cost (real dataset download via `download_dataset.sh`).
- **tensor_parallelism/:** tensor/sequence parallelism on a Llama2 block. Communication happens inside each layer (activation all-reduce/all-gather) rather than once per step. Best for studying TP/SP collectives, but the most involved to set up.

Recommendation: use `ddp-tutorial-series` to learn the launch, this `imagenet --dummy` example as the primary DDP all-reduce scaling study (no data needed), and `minGPT-ddp` for an LLM-shaped all-reduce. Move to FSDP/ `tensor_parallelism` only when we specifically need sharded or intra-layer collective patterns.

MExamples/TinyTransformer: not suitable for RCCL scaling

The TinyTransformer workshop is an excellent single-GPU profiling progression (PyTorch profiler, rocprofv3, rocprof-sys, rocprof-compute; kernel fusion and Triton optimization of a Tiny-LLaMA). However, it contains no distributed code: no `init_process_group`, `DistributedDataParallel`, `torchrun`, or collective calls (verified across all four `version*` implementations). Every run is confined to one GPU, so there is no RCCL communication to measure.

TinyTransformer is the right tool for *per-GPU kernel* optimization and roofline analysis; it is complementary to, not a substitute for, the DDP examples above. To study RCCL with a transformer, use `minGPT-ddp` instead. (One could in principle wrap the Tiny-LLaMA model in DDP to create a scaling exercise, but that work does not exist in the repo today.)

Profiling the ImageNet DDP example: compute and RCCL communication

The benchmark (`ddp_resnet_bench.py`) already prints a coarse compute vs. communication split from CUDA-event timers: it differences a normal step against a `no_sync()` step so `comm_s/comm_pct` is the gradient all-reduce cost (see README §3 and `RESULT ... comm_pct=`). This guide shows how to go deeper and attribute time to individual kernels (conv/GEMM/batchnorm), the RCCL collectives (`ncclDevKernel_*` / `nccl:all_reduce`), and hardware limits (HBM bandwidth, FLOP/s).

The per-tool steps live in the shared per-profiler guides under `../common/profilers/`: each page carries the exact `torchrun` line for this example. Read the ground rules below first.

Which profiler for which job

Profiler	Load / install	Scope	Measures here
torch.profiler (Kineto)	in <code>pytorch/2.12.0</code>	host + GPU	Per-op/kernel table; RCCL <code>all_reduce</code> vs. conv/bn; Chrome/Perfetto trace; memory
DeepSpeed FlopsProfiler	in <code>pytorch/2.12.0</code>	model (1 GPU)	FLOPs / MACs / params / per-module latency: the compute ceiling

Profiler	Load / install	Scope	Measures here
TensorBoard	<code>pip install (venv)</code>	host + GPU	GUI for the torch.profiler trace: timeline, kernel view, comm/compute overlap
rocpv3	<code>module load rocm</code>	GPU	Per-kernel time (MIOpen conv, GEMM) + RCCL kernel/API trace
rocpv3-compute	<code>module load rocm</code>	GPU	Roofline: achieved HBM BW / FLOP/s vs. peak for the conv kernels
rocpv3-systems	<code>module load rocpv3-systems</code>	GPU + host	Perfetto timeline: see all-reduce overlap (or not) with backward
Score-P	<code>module load scorep + venv</code>	Python regions	Per-rank training-loop regions, written to Cube/OTF2
TAU / HPCToolkit	<code>module load tau / hpctoolkit</code>	MPI + GPU	Whole-application, multi-node MPI + GPU (CG guide)

`torch.profiler` is the first stop for an ML workload: it speaks the framework’s language (ops, `nccl:all_reduce`, module names) and needs no extra tooling. We drop to `rocpv3/rocpv3-compute` when we need kernel-level or roofline detail, and to `rocpv3-systems` when we need to *see* the overlap on a timeline.

Verified on PPAC / MI300A (ROCm 6.4.3, pytorch/2.12.0, 2×MI300A, resnet50). `torch.profiler` and the DeepSpeed FLOPsProfiler were run against `ddp_resnet_bench.py` via the `--profile` and `--flops` flags added for this purpose. See the shared pages for the verified output.

Ground rules for trustworthy numbers

Everything in README §3–5 still applies under a profiler, more so because profiling adds overhead:

- Profile inside an affinity-bound, `--exclusive` allocation. Bad GPU/NUMA affinity dominates every measurement.
- Nodes must be booted with `iommu=pt` so RCCL uses direct xGMI P2P (`grep -o iommu=pt /proc/cmdline`). Otherwise the all-reduce is host-staged and the “comm” number measures the wrong thing.
- Warm up MIOpen first (`export MIOPEN_FIND_MODE=FAST`, or run `../benchmarks/warm_miopen.py`) so conv autotuning does not land inside the profiled window.
- Profile a fixed, short window (the `--profile` path uses a `wait=1/warmup=3/active=6` schedule): a full epoch produces an unusable trace.
- Keep the per-rank traces separate (the flag writes `rank0/`, `rank1/`, ...).

Quick reference

Goal	Guide	Command
Op/kernel + RCCL table, trace	<code>torch.profiler</code>	<code>torchrun/benchmarks/ddp_resnet_bench.py --profile --profile-dir ./torch_prof</code>
Model FLOPs / MACs / params	DeepSpeed	<code>torchrun/benchmarks/ddp_resnet_bench.py --flops</code>

Goal	Guide	Command
TensorBoard GUI	TensorBoard	<code>tensorboard --logdir ./torch_prof</code>
Kernel / RCCL trace	rocpv3	<code>rocpv3 --kernel-trace --stats -- torchrun ...</code>
Roofline	rocpv3-compute	<code>rocpv3-compute profile -n r -- torchrun ...</code>
Overlap timeline	rocpv3-systems	<code>rocpv3-sys-run -- torchrun ...</code>
Per-rank Python regions	Score-P	<code>NPROC=2 ../common/scorep_launch.sh ../benchmarks/ddp_resnet_bench.py ...</code>



Summary and Look Ahead

IEEE Hot Interconnects Aug 19-21, 2026

Presenter: Shelby Lockhart, AMD

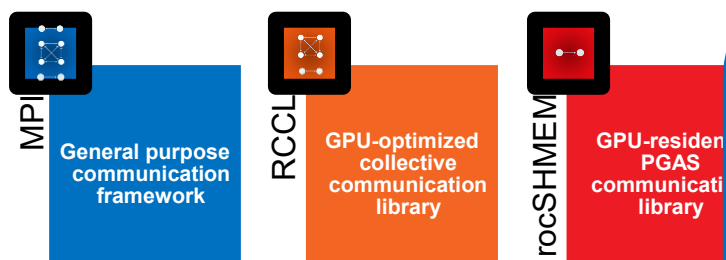
AMD 
together we advance_

Tutorial Goals

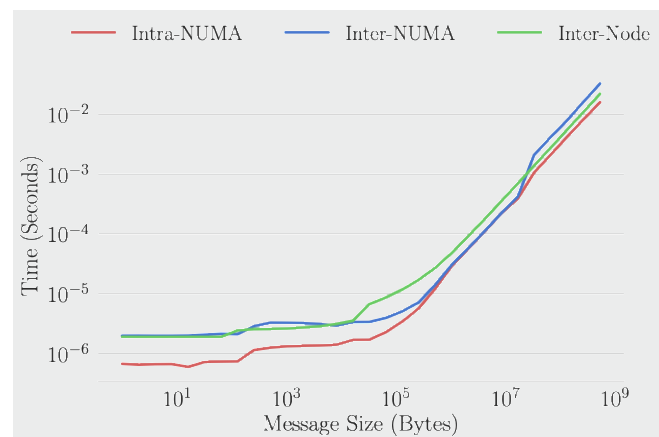
```
Initial residual: 195.472
181 iterations to converge
2-norm of residual: 0.000191484
CG solve time:      0.0614 s (0.0003 s/iter)
```

Is this good performance?

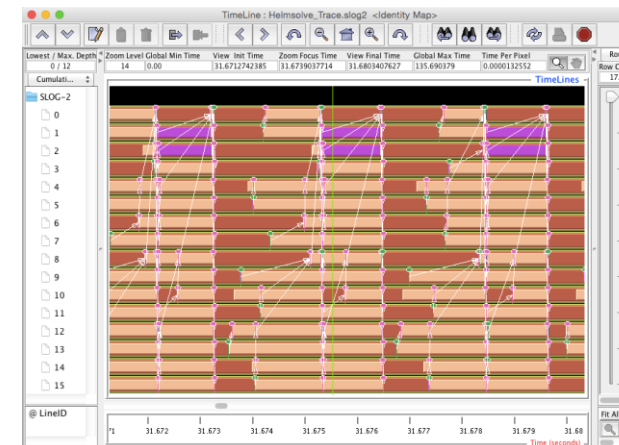
System Stack



Microbenchmarking



Profiling and Tracing



Starter Workflow

1

Identify what's on your system

2

Microbenchmark the relevant communication libraries and communication patterns

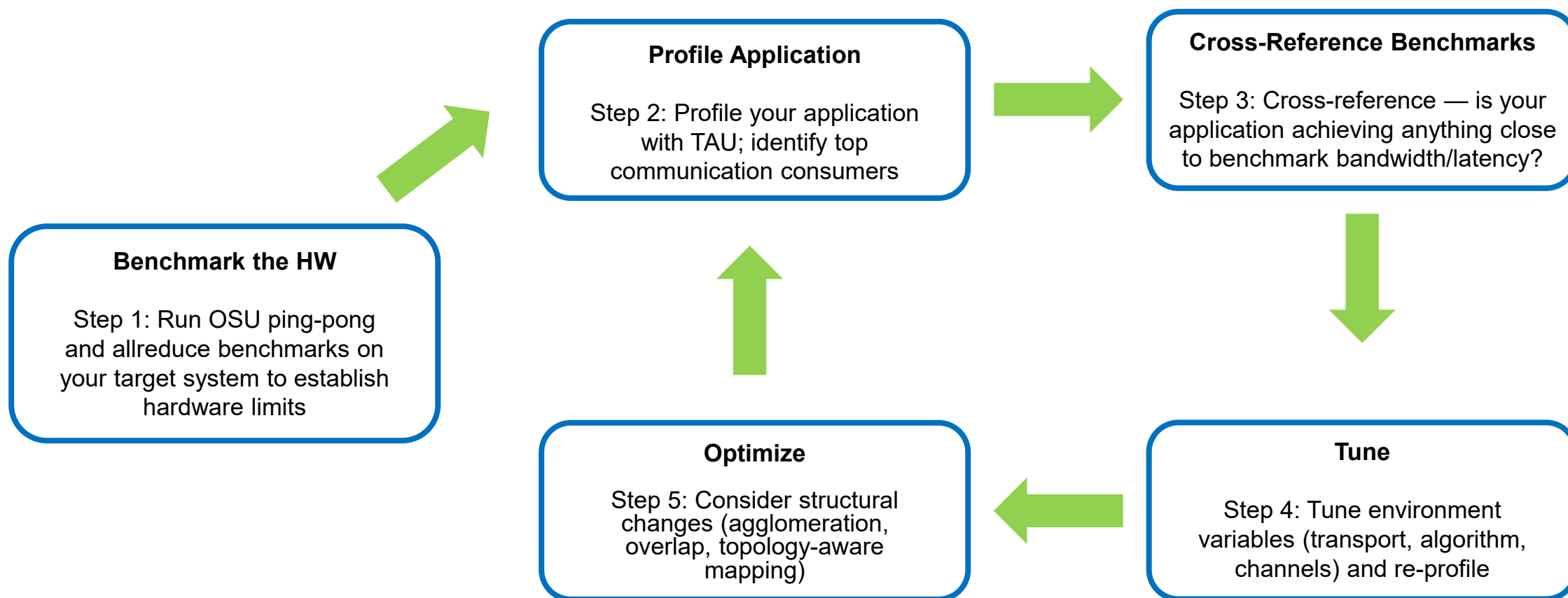
3

Profile and trace your application to ensure you're meeting performance expectations

Summary of the Tools and Tuning Knobs Covered

- Key Benchmarks Covered:
 - OSU Microbenchmarks
 - rccl-tests
 - rocSHMEM tests
- Key Profiling and Tracing Tools Covered
 - TAU
 - ParaProf
 - Perfetto
- Key levers covered:
 - GPU-Aware MPI
 - SDMA vs blit kernel
 - RCCL channels/protocols/algorithms

Applying the Workflow to Your Own Applications



Additional Resources

- Tutorial GitHub repository: <https://github.com/Communication-Performance-Tutorial>
- AMD HPCTrainingExamples GitHub repository: <https://github.com/amd/HPCTrainingExamples/>
- ROCm documentation: <https://docs.amd.com>
- RCCL GitHub: [rocm-systems/projects/rccl](https://github.com/rocm-systems/projects/rccl) at develop · ROCm/rocm-systems
- rocSHMEM GitHub: [rocm-systems/projects/rocshmem](https://github.com/rocm-systems/projects/rocshmem) at develop · ROCm/rocm-systems
- Slingshot plugin for RCCL: [HewlettPackard/shs-ccl-docs](https://github.com/HewlettPackard/shs-ccl-docs)
- MPI Advance library (open-source MPI optimization layer): <https://github.com/mpi-advance>
- TAU documentation: tau.uoregon.edu
- OSU Microbenchmarks: mvapich.cse.ohio-state.edu

Appendix: Environment Variable Quick Reference

Variable	Purpose	Values
HSA_ENABLE_SDMA	Select copy mechanism	0 = blit kernel (high BW), 1 = SDMA (overlap)
HSA_ENABLE_SDMA_GANG	Use all SDMA links	1 = enabled (impacts bidirectional)
MPICH_GPU_SUPPORT_ENABLED	Enable GPU-Aware MPI in Cray-MPICH	1
HIP_VISIBLE_DEVICES	Restrict visible GPUs per process	comma-separated device IDs
NCCL_MAX_NCHANNELS	Max RCCL channels	integer
NCCL_PROTO	RCCL protocol	Simple, LL, LL128
NCCL_ALGO	RCCL algorithm	Ring, Tree, CollNet
HSA_XNACK	Enable managed/unified memory	1 (required for OpenMP/HIP Ver1)
TAU_PROFILE	Enable TAU profiling	1
TAU_TRACE	Enable TAU tracing	1
TAU_COMM_MATRIX	Capture MPI comm matrix	1

Disclaimer

The information presented in this document is for informational purposes only and may contain technical inaccuracies, omissions, and typographical errors. The information contained herein is subject to change and may be rendered inaccurate for many reasons, including but not limited to product and roadmap changes, component and motherboard version changes, new model and/or product releases, product differences between differing manufacturers, software changes, BIOS flashes, firmware upgrades, or the like. Any computer system has risks of security vulnerabilities that cannot be completely prevented or mitigated. AMD assumes no obligation to update or otherwise correct or revise this information. However, AMD reserves the right to revise this information and to make changes from time to time to the content hereof without obligation of AMD to notify any person of such revisions or changes.

THIS INFORMATION IS PROVIDED 'AS IS.' AMD MAKES NO REPRESENTATIONS OR WARRANTIES WITH RESPECT TO THE CONTENTS HEREOF AND ASSUMES NO RESPONSIBILITY FOR ANY INACCURACIES, ERRORS, OR OMISSIONS THAT MAY APPEAR IN THIS INFORMATION. AMD SPECIFICALLY DISCLAIMS ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, OR FITNESS FOR ANY PARTICULAR PURPOSE. IN NO EVENT WILL AMD BE LIABLE TO ANY PERSON FOR ANY RELIANCE, DIRECT, INDIRECT, SPECIAL, OR OTHER CONSEQUENTIAL DAMAGES ARISING FROM THE USE OF ANY INFORMATION CONTAINED HEREIN, EVEN IF AMD IS EXPRESSLY ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Third-party content is licensed to you directly by the third party that owns the content and is not licensed to you by AMD. ALL LINKED THIRD-PARTY CONTENT IS PROVIDED "AS IS" WITHOUT A WARRANTY OF ANY KIND. USE OF SUCH THIRD-PARTY CONTENT IS DONE AT YOUR SOLE DISCRETION AND UNDER NO CIRCUMSTANCES WILL AMD BE LIABLE TO YOU FOR ANY THIRD-PARTY CONTENT. YOU ASSUME ALL RISK AND ARE SOLELY RESPONSIBLE FOR ANY DAMAGES THAT MAY ARISE FROM YOUR USE OF THIRD-PARTY CONTENT.

© 2026 Advanced Micro Devices, Inc. All rights reserved. AMD, the AMD Arrow logo, ROCm, Infinity Fabric, and combinations thereof are trademarks of Advanced Micro Devices, Inc. in the United States and/or other jurisdictions. Other names are for informational purposes only and may be trademarks of their respective owners.

